



# Mobil Pentesting Qo'llanmasi



## **MUNDARIJA**

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>I-BOB. Mobil ilovalar xavfsizligi.....</b>       | <b>6</b>  |
| <b>Smartfonlarni bozoridagi ulushi .....</b>        | <b>6</b>  |
| Android operatsion tizimi .....                     | 7         |
| iOS operatsion tizimi.....                          | 7         |
| <b>Mobil ilovalarning turlari.....</b>              | <b>8</b>  |
| Native ilovalar .....                               | 9         |
| Mobil web ilovalar .....                            | 9         |
| Gibrid ilovalar .....                               | 9         |
| <b>Ommaviy Android va iOS zaifliklari.....</b>      | <b>10</b> |
| <b>OWASP Mobil Top 10 .....</b>                     | <b>11</b> |
| Mobil ilovalarni buzish va teskari muhandislik..... | 18        |
| Android ilovalardagi zaifliklari.....               | 19        |
| iOS ilovalardagi zaifliklari.....                   | 27        |
| <b>II-BOB. Android va iOS arxitekturasini.....</b>  | <b>36</b> |
| <b>Android arxitekturasini.....</b>                 | <b>36</b> |
| Linux kernel .....                                  | 38        |
| Hardware Abstraction Layer .....                    | 39        |
| Android Runtime (Dalvik va ART).....                | 41        |
| Application Framework .....                         | 42        |
| Applications (Ilovalar) .....                       | 45        |
| Java virtual mashina.....                           | 48        |
| Dalvik virtual mashina .....                        | 48        |
| Android ruxsatlari .....                            | 49        |
| <b>Ios arxitekturasini.....</b>                     | <b>52</b> |
| Cocoa Touch.....                                    | 54        |
| Media.....                                          | 54        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| Core services .....                                    | 54        |
| Core Os.....                                           | 54        |
| <b>iOS SDK va Xcode.....</b>                           | <b>55</b> |
| <b>iOS ruxsatlari.....</b>                             | <b>55</b> |
| <b>III-BOB. Mobil ilovalar pentestingi .....</b>       | <b>57</b> |
| <b>Android Studio va SDK .....</b>                     | <b>57</b> |
| <b>Emulator, simulatorlar, va real qurilmalar.....</b> | <b>60</b> |
| Emulatorlar.....                                       | 61        |
| Simulatorlar .....                                     | 61        |
| Real qurilmalar .....                                  | 61        |
| <b>IV-BOB. Mobil Pentesting Vositalari .....</b>       | <b>62</b> |
| <b>Android xavfsizlik vositalari.....</b>              | <b>62</b> |
| <b>Android Debug Bridge .....</b>                      | <b>62</b> |
| Adbni o‘rnatish.....                                   | 62        |
| Qurilmaga ulanish .....                                | 67        |
| Qurilmaga ilova o‘rnatish .....                        | 68        |
| Qurilmadan fayllarni olish .....                       | 70        |
| Fayllarni qurilmaga saqlash .....                      | 70        |
| Log ma’lumotlarini ko‘rish.....                        | 71        |
| Adb buyruqlari .....                                   | 74        |
| APKAnalyser .....                                      | 77        |
| APKTool.....                                           | 78        |
| Androguard .....                                       | 80        |
| MobSF .....                                            | 81        |
| APKleaks.....                                          | 83        |
| Frida .....                                            | 84        |
| MAVS.....                                              | 88        |

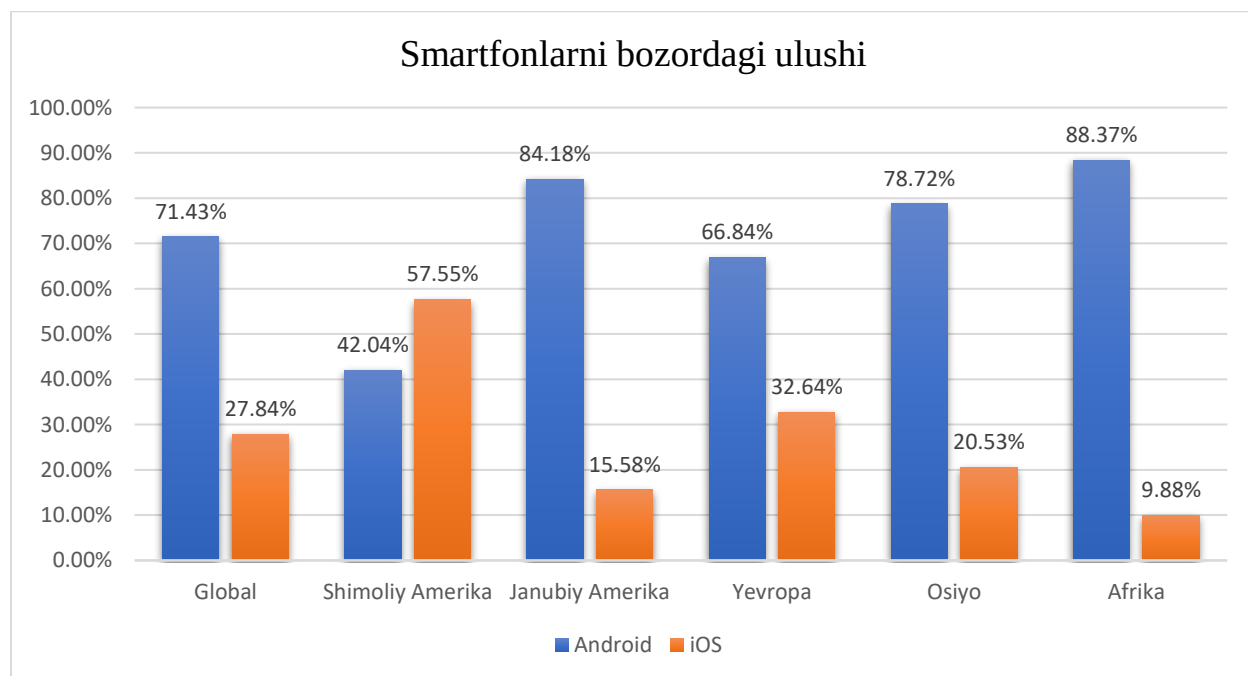
|                                                           |            |
|-----------------------------------------------------------|------------|
| APK Hunt.....                                             | 90         |
| Tapjacker .....                                           | 91         |
| Logcat.....                                               | 92         |
| APK DeepLens.....                                         | 94         |
| Reqable.....                                              | 95         |
| Objection .....                                           | 96         |
| HTTP Toolkit.....                                         | 98         |
| Burpsuite .....                                           | 99         |
| Drozer.....                                               | 101        |
| Genymotion.....                                           | 103        |
| Nmap .....                                                | 104        |
| Dirb / Dirbuster / FFUF .....                             | 106        |
| WAF W00F .....                                            | 107        |
| Janus Vulnerability Tester .....                          | 109        |
| <b>iOS xavfsizlik vositalari .....</b>                    | <b>110</b> |
| oTool.....                                                | 110        |
| SSL Kill Switch.....                                      | 112        |
| Keychain dumper .....                                     | 114        |
| LLDB .....                                                | 116        |
| Clutch .....                                              | 116        |
| Class-dump-z.....                                         | 118        |
| <b>V-BOB. Mobil Ilovalar Pentesting Bosqichlari .....</b> | <b>120</b> |
| <b>Statik tahlil .....</b>                                | <b>120</b> |
| Ilovani virustotal orqali tekshirish .....                | 121        |
| Ilovani dekompiyatsiya qilish (apktool, jadx-gui).....    | 122        |
| Obfusikatsiya.....                                        | 128        |
| Ilova imzosini tekshirish. ....                           | 131        |

|                                                                                                 |            |
|-------------------------------------------------------------------------------------------------|------------|
| Ilova kodini tahlil qilish va o‘zgartirish.....                                                 | 132        |
| Eskirgan va zaif kutubxonalarni aniqlash .....                                                  | 137        |
| AndroidManifest.xml faylini tahlil qilish .....                                                 | 138        |
| Hard kod qilib yozilgan malumotlar .....                                                        | 138        |
| Resurs fayllarni ko‘rish (res/, assets/ ).....                                                  | 139        |
| Log malumotlarni tekshirish .....                                                               | 139        |
| Debug modni tekshirish .....                                                                    | 140        |
| Zaif WebViewdan foydalanish.....                                                                | 140        |
| Zaif hashlash va shifrlash algorithmhlari.....                                                  | 141        |
| Backup olishga ruxsat berish oqibatida .....                                                    | 141        |
| Statik zaifliklar .....                                                                         | 142        |
| <b>Dinamik tahlil .....</b>                                                                     | <b>143</b> |
| Ilovani qurilmaga yoki emulyatorda o‘rnatish.....                                               | 144        |
| Tarmoq trafikni kuzatish (burp suite, mitmproxy).....                                           | 147        |
| Xavfsizlik cheklovlarini sinash (root, emulator, vpn, zararli kod deteksiya, SSL pinning) ..... | 151        |
| Faoliyatlarni sinash (Activity, Service, BroadcastReceiver) .....                               | 154        |
| Ma’lumotlar oqimini kuzatish (runtime debugging) .....                                          | 156        |
| Umumiy xotira malumotlarini ko‘rish .....                                                       | 157        |
| API so‘rovlarini kuzatish .....                                                                 | 160        |
| Tapjacking hujumiga tekshirish .....                                                            | 163        |
| Dinamik zaifliklar .....                                                                        | 165        |
| <b>VI-BOB. Hisobot shakllantirish .....</b>                                                     | <b>167</b> |
| <b>Android.....</b>                                                                             | <b>167</b> |
| <b>iOS.....</b>                                                                                 | <b>167</b> |
| <b>API .....</b>                                                                                | <b>168</b> |

## Mobil ilovalar xavfsizligi

### Smartfonlarni bozoridagi ulushi

Bozor ulushini tushunish orqali biz kiber jinoyatchilarning nimalarni nishonga olayotganini va potentsial himoya nuqtalarini bilib olamiz. Mobil ilova ishlab chiquvchilari o'z ilovasini Play market yoki App Storega joylashtirib, sotishdan tushgan daromadning bir qismi bilan mukofotlanishadi. Juda keng tarqalgan operatsion tizimlar esa jinoyatchilarning e'tiborini tortadi, chunki ular ko'proq qurbon va daromad imkoniyati mavjud bo'lgan auditoriyani o'z ichiga oladi. designveloper.com saytidagi ma'lumotlarga qaraganda smartfonlar, operatsion tizimiga qarab quydagicha mintaqalarda tarqalgan.



### **Android operatsion tizimi**

Android bu mobil qurilmalar (smartfonlar va planshetlar) uchun yaratilgan Linux asosidagi ochiq manbali operatsion tizimdir. U Google hamda boshqa kompaniyalar rahbarligidagi Open Handset Alliance tomonidan ishlab chiqilgan. Android OS Linuxga asoslangan bo‘lib, C/C++ tilida dasturlash mumkin, ammo ilova yaratishda asosan Java ishlatiladi. Java orqali C kutubxonalariga JNI (Java Native Interface) yordamida murojaat qilinadi. Android

smartfonlar, planshetlar, elektron kitoblar, raqamli pleerlar, qo‘l soatlari, fitnes brasletlar, o‘yin pristavkalari, noutbuklar, netbuklar, smartbuklar, Google Glass ko‘zoynaklari, televizorlar, proyektorlar va boshqa qurilmalar uchun mo‘ljallangan operatsion tizim (2015-yilda avtomobilning ko‘ngilochar tizimlari va maishiy robotlar uchun ham qo‘llab-quvvatlash paydo bo‘lgan). Android Linux yadrosi asosida yaratilgan va Google tomonidan ishlab chiqilgan Java virtual mashinasi asosidagi texnologiyalardan foydalanadi. Keyinchalik Google Open Handset Alliance ochiq platformani qo‘llab-quvvatlovchi va rivojlantiruvchi alyansni tashkil etdi. 2014-yilning ikkinchi choragida dunyo bo‘yicha sotilgan smartfonlarning 86 foizida Android operatsion tizimi o‘rnatilgan edi. 2017-yil may oyida bo‘lib o‘tgan ishlab chiquvchilar konferensiyasida Google Android tizimida ishga tushirilgan qurilmalar soni 2 milliarddan oshganini e‘lon qilgan. StatCounter platformasi ma‘lumotlariga ko‘ra, 2024-yil noyabr oyida Androidning global smartfon bozoridagi ulushi 72% ni tashkil qilgan.



### **iOS operatsion tizimi**

iOS (avval iPhone OS deb atalgan) bu Apple tomonidan o‘zining mobil qurilmalari uchun maxsus ishlab chiqilgan mobil operatsion tizimdir. U ilk bor 2007-yil yanvar oyida birinchi avlod iPhone uchun e‘lon qilingan va 2007-yil iyun oyida ishga tushirilgan. IPhonedan tashqari, iOS Apple tomonidan ishlab chiqilgan yana uchta operatsion



tizimning asosi hisoblanadi: iPadOS, tvOS va watchOS. iOS ilgari iPadlarni ham boshqarardi, biroq 2019-yilda iPadOS taqdim etilgach, bu vazifani iPadOS o'z zimmasiga oldi. Shuningdek, iPod Touch qurilmalari ham iOS asosida ishlagan, biroq ular ishlab chiqarishdan to'xtatilgan. iOS dunyodagi ikkinchi eng keng tarqalgan mobil operatsion tizim bo'lib, Androiddan keyin turadi. 2023-yil dekabr holatiga ko'ra, Applening App Store do'konida 3,8 milliondan ortiq iOS ilovalari mavjud. iOS macOS tizimiga asoslangan. Bu Unixga o'xshash operatsion tizim hisoblanadi. iOSning ayrim qismlari Apple Public Source License va boshqa litsenziyalar asosida ochiq manba sifatida taqdim etilgan bo'lsada, iOSning o'zi xususiy (proprietary) dasturiy ta'minot hisoblanadi. iOS Applening mobil qurilmalar (iPhone, iPad, iPod Touch) uchun ishlab chiqilgan Unixga yaqin operatsion tizimidir. U XNU yadrosi (Darwin) asosida qurilgan. iOS 1.0 2007-yilgi birinchi iPhone bilan chiqib, dastlab faqat o'z ichki ilovalari (Safari, Mail, Photos) va ko'p tegizishli interfeysni ta'minladi. Keyingi yillar davomida har bir yirik yangilanish yangi funksiyalar bilan birga xavfsizlik kuchayishlarini ham olib keldi. Masalan, iOS 2.0 (2008) App Store, push xabarnoma va ma'lumotlarni to'liq shifrlashni joriy etdi. iOS 4 (2010) multitasking va batafsil sandbox qoidalari, iOS 5 (2011) ob-havo va iCloud integratsiyasini kiritdi. iOS 7 (2013) interfeysni yangiladi, 64-bit arxitektura va manzil fazosini tasodifiylashtirish qo'llab-quvvatlashni boshladi. So'nggi versiyalarda xavfsizlikni mustahkamlashga e'tibor qaratilib, masalan, iOS 9 da App Transport Security (ATS) standarti joriy etilib, barcha tarmoq aloqalari uchun kamida TLS 1.2 talab qilindi. iOS 12–15 da xotirani himoya qilish, Secure Enclave (maxfiy ma'lumotlarni ushlab turuvchi maxsus protsessor) va ikki faktorli autentifikatsiya kabi texnologiyalar ishlab chiqildi.

### **Mobil ilovalarning turlari**

Hozirda turli ehtiyojlar uchun minglab ilovalar mavjud: chat, videokonferensiyalar, o'yinlar, sog'liqni tekshirish, qimor o'yinlari, ijtimoiy tarmoqlar, savdo va moliyaviy xizmatlar va boshqalar.

Kelajakdagi qiziqarli texnologiyalardan biri iOS va Android qurilmalarida ishlovchi ilovalar bo'lib, ular atrofdagi beaconlardan signal qabul qilib, unga mos javob beradi. Bunday ilovalar iBeacon deb ataladi. Ilovalar quyidagi turlarga ajratiladi:

- Native ilovalar – har bir platformaga mos yozilgan dasturlar



- Mobil veb-illovalar – brauzerda ishlaydigan saytlarga o‘xshashlar
- Gibril illovalar – veb va native texnologiyalarni birlashtirganlar

### **Native illovalar**

Native illovalar mobil operatsion tizim ichida o‘rnatiladi va tegishli ilova do‘konlari orqali (App Store, Google Play) tarqatiladi. Bunday illovalar odatda platformaga mos rivojlantirish vositalari va dasturlash tillarida yaratiladi masalan, iOS illovalari uchun Xcode + Objective-C yoki Swift, Android illovalari uchun esa Android Studio + Java yoki Kotlin ishlatiladi. Ular aynan shu platforma uchun mo‘ljallangan bo‘lib, qurilmadagi barcha imkoniyatlardan kamera, GPS, telefon kontaktlari va boshqa funktsiyalardan to‘liq foydalana oladi.

### **Mobil web illovalar**

Mobil veb-illovalar bu *native* (ya’ni qurilmaga o‘rnatiladigan haqiqiy mobil ilova) bo‘lmagan illovalardir. Ularning aksariyati HTML5, JavaScript va CSS texnologiyalari asosida yaratilgan bo‘lib, foydalanuvchiga mobil illovalardagidek ko‘rinish va ishlash uslubini taqdim etadi. Foydalanuvchilar bu illovalarga oddiy veb-sahifaga kirgandek tashrif buyurishadi, ya’ni ular brauzer orqali ochiladi. Ular aynan mobil qurilmalar uchun moslashtirilgan veb-sahifalardir. Bu turdagi illovalar HTML5 paydo bo‘lgach ommalasha boshladi, chunki shu orqali odamlar brauzer orqali oddiy *native* illovalardagidek funktsiyalarni ishlata olishdi. Ularni ishlab chiqish va test qilish oson, chunki ular uchun ko‘plab ishlab chiqish vositalari mavjud.

### **Gibril illovalar**

Gibril illovalar deganda ikki xil ma’no tushuniladi. Birinchidan, bu veb-kontent va qurilmadagi *native* (ya’ni o‘rnatilgan) komponentlarning birlashmasidan tashkil topgan illovalardir. Ular mobil qurilmaning xizmatlaridan, ayniqsa, ma’lumotlarni saqlash (storage) imkoniyatidan foydalanadi. Ikkinchidan, bu mijoz-server (client-server) arxitekturasiga asoslangan mobil illovalardir. Masalan, korxona uchun mo‘ljallangan mobil illovalar (enterprise apps) bunga kiradi. Gibril illovalar aslida veb-texnologiyalar (HTML5, CSS, JavaScript) asosida yaratilgan bo‘lib, ular *native* mobil ilova shakliga o‘raladi va barcha platformalarda ishlay oladigan xususiyatga ega bo‘ladi. Ya’ni, bu illovalar veb illovalarning moslashuvchanligi va *native* illovalarning kuchli jihatlarini birlashtiradi. Masalan, quyidagi rasmda mashhur

yangiliklar ilovasining ekrandan olingan tasviri berilgan bo‘lib, bu aynan gibril ilovaga misol bo‘ladi.

### **Ommaviy Android va iOS zaifliklari**

Android va iOSdagi turli xil zaifliklarni ko‘rib chiqishdan oldin, ushbu bo‘lim sizni Android va iOSni operatsion tizim sifatida tanishtiradi va mobil ilovalar xavfsizligi bo‘yicha tajriba oltirish uchun tushunilishi kerak bo‘lgan turli fundamental tushunchalarni qamrab oladi.

| <b>Yil</b> | <b>Android</b>                   | <b>Android API</b> | <b>iOS</b>   |
|------------|----------------------------------|--------------------|--------------|
| 2007/2008  | 1.0                              | 1                  | iPhone OS 1  |
|            |                                  |                    | iPhone OS 2  |
| 2009       | 1.1                              | 2                  | iPhone OS 3  |
|            | 1.5(Cupcake)                     | 3                  |              |
|            | 1.6(Donut)                       | 4                  |              |
|            | 2.0(Eclair)                      | 5                  |              |
|            | 2.0.1(Eclair)                    | 6                  |              |
| 2010       | 2.1(Eclair)                      | 7                  | iOS 4        |
|            | 2.2(Froyo)                       | 8                  |              |
|            | 2.3-2.3.2(Gingerbread)           | 9                  |              |
| 2011       | 2.3.4-2.3.7 (Gingerbread)        | 10                 | iOS 5        |
|            | 3.0 (HoneyComb)                  | 11                 |              |
|            | 3.1 (HoneyComb)                  | 12                 |              |
|            | 3.2 (HoneyComb)                  | 13                 |              |
|            | 4.0-4.0.2 (Ice Cream Sandwich)   | 14                 |              |
|            | 4.0.3-4.0.4 (Ice Cream Sandwich) | 15                 |              |
| 2012       | 4.1 (Jelly Bean)                 | 16                 | iOS 6        |
|            | 4.2 (Jelly Bean)                 | 17                 |              |
| 2013       | 4.3 (Jelly bean)                 | 18                 | iOS 7        |
|            | 4.4 (KitKat)                     | 19-20              |              |
| 2014       | 5.0 (Lollipop)                   | 21                 | iOS 8        |
|            | 5.1 (Lollipop)                   | 22                 |              |
| 2015       | 6.0 (Marshmallow)                | 23                 | iOS 9 (beta) |
| 2016       | 7.0 (Nougat)                     | 24-25              | iOS 10       |
| 2017       | 8.0 (Oreo)                       | 26-27              | iOS 11       |
| 2018       | 9.0 (Pie)                        | 28                 | iOS 12       |
| 2019       | 10                               | 29                 | iOS 13       |
| 2020       | 11                               | 30                 | iOS 14       |
| 2021       | 12                               | 31-32              | iOS 15       |
| 2022       | 13                               | 33                 | iOS 16       |
| 2023       | 14                               | 34                 | iOS 17       |
| 2024       | 15                               | 35                 | iOS 18       |
| 2025       | 16(beta)                         | 36                 | iOS 26(beta) |

Hewlett Packard (HP) tomonidan olib borilgan qiziqarli tadqiqotda bu yirik dasturiy ta'minot kompaniyasi 600 dan ortiq kompaniyalarga tegishli 2000 dan ortiq mobil ilovani sinovdan o'tkazgan quyidagi statistika aniqlangan.

- Sinovdan o'tgan ilovalarning 97% i hech bo'lmaganda bitta shaxsiy ma'lumot manbaiga murojaat qiladi.
- 86% ilovalar zamonaviy hujumlarga qarshi oddiy *binary hardening* (binar xavfsizlik mustahkamlash) himoya vositalaridan foydalanmagan.
- 75% ilovalar mobil qurilmada ma'lumot saqlayotganda to'g'ri shifrlash usullaridan foydalanmagan.
- Aniqlangan zaifliklarning 71% i veb-serverlarda joylashgan.
- 18% ilovalar foydalanuvchi nomi va parollarni HTTP orqali yuborgan, yana 18% esa SSL/HTTPSni noto'g'ri joriy qilgan.

Shunday qilib, mobil ilovalarda uchraydigan asosiy zaifliklar quyidagilardan kelib chiqadi: xavfsizlik bo'yicha yetarli tushunchaning yo'qligi, ishlab chiquvchilarning foydalanish qulayligi bilan xavfsizlik o'rtasidagi muvozanatni topa olmasligi, ilovalarning ortiqcha ruxsatlar talab qilishi va shaxsiy hayotga bo'lgan e'tiborsizlik. Bunga qo'shimcha ravishda, ilovalarga oid hujjatlarning yetarli emasligi natijasida ishlab chiquvchilar ushbu zaifliklardan bexabar bo'lib qolishadi.

### **OWASP Mobil Top 10**

*OWASP Mobile Top 10* – bu mobil ilovalardagi eng xavfli va keng tarqalgan xavfsizlik xatoliklari ro'yxati bo'lib, uni OWASP (Open Worldwide Application Security Project) tashkiloti tomonidan tuziladi. OWASP Mobile Top 10 2024 autentifikatsiyadan tortib xavfsiz ma'lumotlarni saqlashgacha bo'lgan 10 ta zaiflikni o'z ichiga oladi va mobil xavfsizlik stsenariysi haqida umumiy ma'lumot beradi. Ushbu ro'yxat:

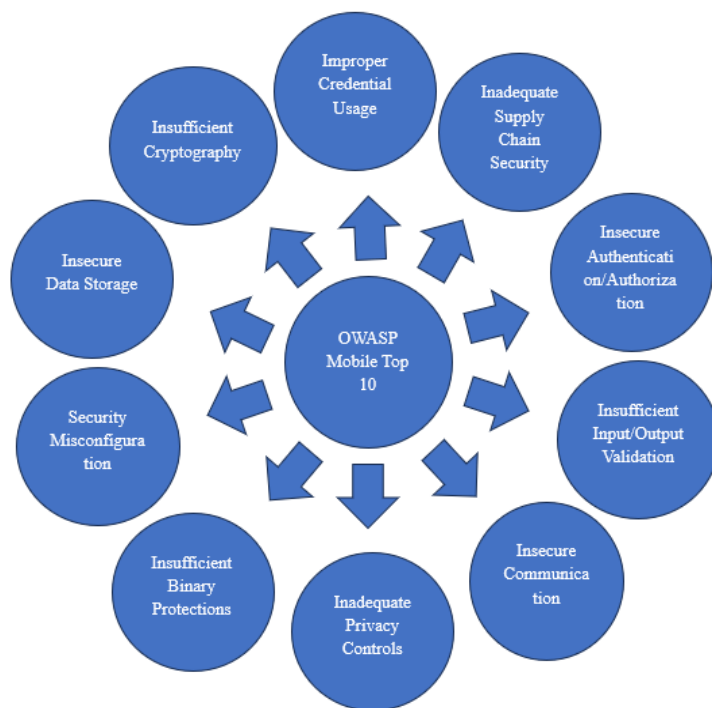
- mobil ilova ishlab chiquvchilarga,

- test qiluvchilarga
- xavfsizlik mutaxassislariga

mobil xavfsizlik bo'yicha yo'l-yo'riq beradi va eng muhim zaifliklarga qarshi qanday choralar ko'rish kerakligini ko'rsatadi.

OWASPning so'nggi hisobotida eng xavfli va keng tarqalgan 10 ta zaifliklar ro'yxati keltirilgan.

1. Improper Credential Usage - Hisob ma'lumotlaridan noto'g'ri foydalanish.
2. Inadequate Supply Chain Security – Ta'minot zanjiri xavfsizligining yetarli emasligi.
3. Insecure Authentication/Authorization - Ishonchsiz autentifikatsiya/avtorizatsiya.
4. Insufficient Input/Output Validation - Kirish/chiqishni tekshirish yetarli emas.
5. Insecure Communication - Himoyalanmagan ma'lumot almashinuvi.
6. Inadequate Privacy Controls - Maxfiylik nazoratini yuqligi.
7. Insufficient Binary Protections - Ikkilik (binary) fayllar uchun yetarli himoya choralarning yo'qligi.
8. Security Misconfiguration - Xavfsizlik konfiguratsiyasidagi xatolik.
9. Insecure Data Storage – Ma'lumotlarni himoyalanmagan tarzda saqlash.
10. Insufficient Cryptography – Kriptografik kamchiliklar.



1-rasm. OWASP Mobile Top 10

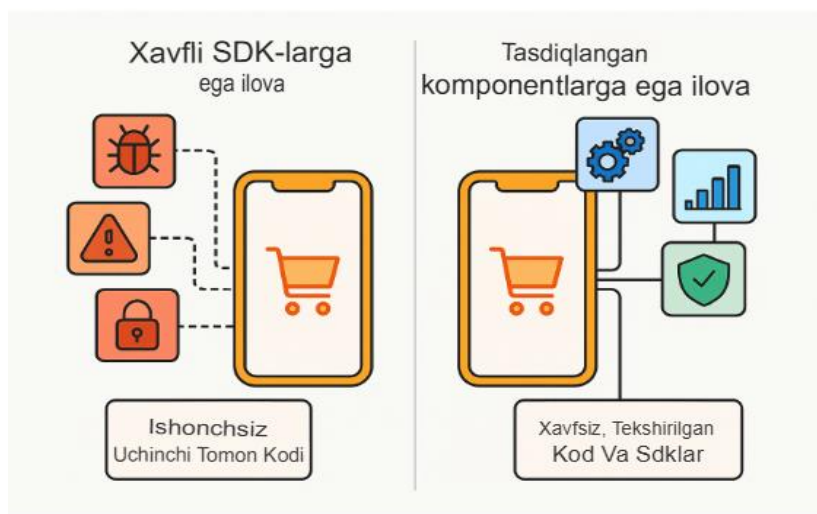
### **Improper Credential Usage**(Hisob ma'lumotlaridan noto'g'ri foydalanish.)

– Aksariyat mobil ilovalar hali ham hisob ma'lumotlaridan noto'g'ri foydalanish xavfiga duch kelishadi. Ushbu zaiflik ilovalar foydalanuvchi hisob ma'lumotlarini to'g'ri ishlatilmaganligi yoki xavfsiz bo'lmagan joylarda saqlanmaganligi orqali paydo bo'ladi. Hisob ma'lumotlarini xavfsiz saqlamaganligi tufayli yomon niyatli foydalanuvchilarni ruxsatsiz kirishi va foydalanuvchi hisoblari va maxfiy ma'lumotlardan noto'g'ri maqsadlarda foydalanishga olib kelishi mumkin.



Hisob ma'lumotlari ya'ni foydalanuvchi login va parollarini noto'g'ri ishlatilishi ilovaning xavfsizligini jiddiy zaiflashtiradi. Agar hisob ma'lumotlari to'g'ri himoyalansa yoki noto'g'ri saqlansa, hujumchilar ularni osongina qo'lga kiritib, ilova yoki uning serveriga ruxsatsiz kira oladi.

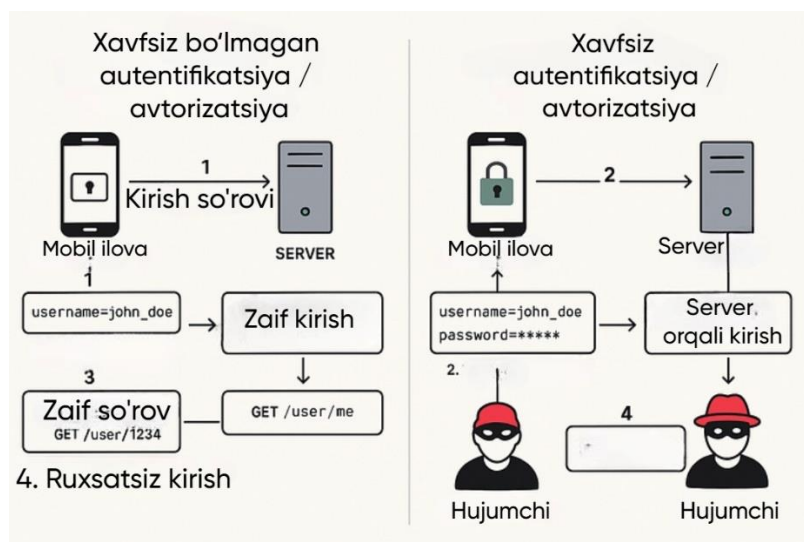
**Inadequate Supply Chain Security**(Ta'minot zanjiri xavfsizligining yetarli emasligi) – Ishlab chiquvchilar mobil ilovalarni yaratishda ko'pincha boshqa kompaniyalar yoki ochiq manbali jamoalar tomonidan yaratilgan kodlar, kutubxonalar yoki vositalardan foydalanadilar. Bu ularga ilovalarni tezroq yaratishga va noldan boshlamasdan funksiyalar qo'shishga yordam beradi. Biroq, agar ushbu uchinchi tomon komponentlaridan birortasi eskirgan yoki zaif bo'lsa, ular ilovaga jiddiy zaifliklarni kiritishi mumkin.



Zaif ta'minot zanjiri bu uchinchi tomon komponentlari va kutubxonalar orqali tarqaladigan zaifliklarning mavjudligi, mobil ilovalarni yaratishda ruxsat etilgan joylarda (kamroq vaqt ichida) ishlab chiqaruvchilar foydalanadilar.

**Insecure Authentication /Authorization**(Ishonchsiz autentifikatsiya/avtorizatsiya) – Yana bir keng tarqalgan OWASP Mobile Top 10 zaifligi mobil ilovalarda autentifikatsiya avtorizatsiyaning zaifligi bo'lib, muhim funksiyalar va

maxfiy ma'lumotlarga ruxsatsiz kirishga ruxsat berishi mumkin. Bu shuni anglatadiki, hujumchilar foydalanuvchi identifikatorlarini tekshirishning dastlabki bosqichini chetlab o'tishlari mumkin (masalan: bitta login ostida bir nechta odam autentifikatsiya qilinganda).



Zaif parol siyosati, seansni noto'g'ri boshqarish, kirishni noto'g'ri boshqarish va ko'p faktorli autentifikatsiyaning yo'qligi ba'zi bir keng tarqalgan muammolarning bir nechta misolidir. Buzg'unchilar bunday zaifliklardan autentifikatsiyani chetlab o'tish, o'z imtiyozlarini oshirish yoki haqiqiy foydalanuvchi nomini o'zgartirish uchun foydalanishi mumkin (IDOR va imtiyozlarni kuchaytirish).

**Insufficient Input/Output Validation**(Kirish/chiqishni tekshirish yetarli emas) – Mobil ilovada foydalanuvchi tomonidan kiritilgan ma'lumotlari yetarli darajada tekshirilmaganligi va sanitarizatsiya qilinmaganligi jiddiy xavfsizlik muammolarini keltirib chiqarishi mumkin. Bunday ma'lumotlarni to'g'ri tekshira olmaydigan va zararsizlantira olmaydigan mobil ilovalar mobil muhitlarga xos hujumlar, jumladan SQL in'ektsiyasi, buyruqlarni kiritish va saytlararo skript (XSS) hujumlari tufayli foydalanish xavfi ortadi.

Ushbu zaifliklar zararli oqibatlarga olib kelishi mumkin, jumladan, maxfiy ma'lumotlarga ruxsatsiz kirish, ilova funksiyalarini manipulyatsiya qilish va butun mobil tizimning potentsial buzilishi.

**Insecure Communication**(Himoyalalmagan ma'lumot almashinuvi) – Ko'pgina zamonaviy mobil ilovalar bir yoki bir nechta uzoq serverlar bilan ma'lumot almashadi. Ma'lumot uzatish sodir bo'lganda, u odatda mobil qurilmaning tashuvchisi tarmog'i va internet orqali o'tadi, simni tinglayotgan tahdid agenti, agar u ochiq matnda yoki eskirgan shifrlash protokoli yordamida uzatilgan bo'lsa, ma'lumotlarni ushlab turushi va o'zgartirishi mumkin. Tahdid agentlari nozik ma'lumotlarni o'g'irlash, josuslik qilish, shaxsni o'g'irlash va boshqalar kabi turli maqsadlarga ega bo'lishi mumkin.

**Inadequate Privacy Controls**(Maxfiylik nazoratini yuqligi) – Ilovalar foydalanuvchi manzili, yoshi yoki kredit karta ma'lumotlari kabi shaxsiy ma'lumotlarni to'plaganda va ma'lumotlarni xavfsiz saqlamasa, bunday muammolarga duch kelishi mumkin. Asosiy to'siqlar ma'lumotlarni haddan tashqari saqlash, foydalanuvchi roziligini ta'minlash imkoniyatlarini yo'qotish va samarasiz himoya choralaridir. Buzg'unchilar shaxsiy ma'lumotlarga kirish yoki hatto ma'lumotlarni himoya qilish qonunlarini buzish uchun ushbu zaifliklardan foydalanishlari mumkin.

Ishlab chiquvchilar ushbu xavfni shaxsiy hayotga yangi yovuzlik sifatida qarash, ma'lumotlar yig'ishni minimallashtirish va Dizayn bo'yicha maxfiylik tamoyilini qabul qilish orqali tuzatishi mumkin. Ular barcha maxfiy shaxsiy ma'lumotlarni kuchli shifrlash bilan saqlashi va foydalanuvchi boshqaruvini ta'minlashi, foydalanuvchilar o'z almashish yoki o'chirishni nazorat qilishlarini ta'minlashi kerak.

**Insufficient Binary Protections**(Ikkilik (binary) fayllar uchun yetarli himoya choralarining yo'qligi) – Ilovadagi binary fayllarni nishonga olgan tajovuzkorlar turli sabablarga ko'ra turtki bo'ladi. Binary faylda tajovuzkor noto'g'ri foydalanishi mumkin bo'lgan tijorat API kalitlari yoki qattiq kodlangan kriptografik sirlar kabi qimmatli sirlar bo'lishi mumkin. Bunga qo'shimcha ravishda, ikkilik kod o'z-o'zidan qimmatli bo'lishi mumkin, masalan, u muhim biznes mantig'ini yoki



oldindan o'rgatilgan AI modellarini o'z ichiga oladi. Ba'zi yomon niyatli foydalanuvchilar ilovaning o'ziga ham mo'ljallanmagan bo'lishi mumkin, lekin undan hujumga tayyorgarlik ko'rish uchun tegishli backendning potentsial zaif tomonlarini o'rganish uchun foydalanishi mumkin.

**Security Misconfiguration**(Xavfsizlik konfiguratsiyasidagi xatolik) – Mobil ilovalardagi xavfsizlik noto'g'ri konfiguratsiyasi zaifliklar va ruxsatsiz kirishga olib kelishi mumkin bo'lgan xavfsizlik sozlamalari, ruxsatlar va boshqaruv elementlarining noto'g'ri konfiguratsiyasini bildiradi. Xavfsizlik noto'g'ri konfiguratsiyasidan foydalanishi mumkin bo'lgan tahdid manbalarilari maxfiy ma'lumotlarga ruxsatsiz kirishni yoki zararli harakatlarni amalga oshirishni maqsad qilgan foydalanuvchilardir. Tahdid manbalari qurilmaga jismoniy kirish huquqiga ega bo'lgan hujumchi bo'lishi mumkin, bu qurilmadagi zararli dastur bo'lib, u zaif dastur kontekstida ruxsatsiz harakatlarni amalga oshirish uchun xavfsizlik noto'g'ri konfiguratsiyasidan foydalanadi.

**Insecure Data Storage**(Ma'lumotlarni himoyalanmagan tarzda saqlash) – Mobil qurilmada ma'lumotlarni saqlash uchun bizga SQLite ma'lumotlar bazasi kerak, bu mobil ilovada tez-tez ishlatiladigan saqlash turi. Foydalanish juda oson. Ushbu turdagi bizda zaiflik xavfi bor, chunki ishlab chiquvchi ma'lumotlarni keshlash yoki ikkinchi marta ishlatish uchun saqlaydi, bu umuman yaxshi amaliyot emas. Ushbu ma'lumotlar bazasida foydalanuvchi nomi, parol, debet karta raqami va hokazolarni saqlash shart emas. Biz ilovalardan birida zaiflikni topdik, lekin ular foydalanuvchi nomi va parolni saqlagan ilova nomini ayta olmaymiz. Telefon root qilingan bo'lsa, SQLite DB dan ma'lumotlarni olish oson.

**Insufficient Cryptography**(Kriptografik kamchiliklar) – Mobil ilovalarda xavfsiz kriptografiyadan foydalanadigan tahdid manbalari maxfiy ma'lumotlarning maxfiyligi, yaxlitligi va haqiqiyligini buzishi mumkin. Ushbu tahdid agentlari sirli

ma'lumotlar shifrini ochish uchun kriptografik algoritmlar yoki ilovalarni nishonga olgan tajovuzkorlar, kriptografik jarayonlarni manipulyatsiya qiluvchi yoki shifrlash kalitlarini sizib tashlaydigan zararli insayderlar, razvedka maqsadlarida kriptotahlil bilan shug'ullanuvchi davlat tomonidan homiylik qilinadigan hakerlar, zaif shifrlashdan foydalanadigan kiberjinoyatchilar, moliyaviy ma'lumotlarni o'g'irlash yoki o'g'irlash uchun hujum qiluvchi kiberjinoyatchilarni o'z ichiga oladi. kriptografik protokollar yoki kutubxonalardagi zaifliklar.

### **Mobil ilovalarni buzish va teskari muhandislik**

Teskari muhandislik va dasturiy o'zgarishlar kiritish (patching) usullari uzoq yillardan beri krakerlar, modifikatsiya qiluvchilar (modderlar), zararli dasturiy ta'minot tahlilchilari va shu sohadagi boshqa mutaxassislar faoliyatining ajralmas qismi bo'lib kelmoqda. Ilgari "an'anaviy" xavfsizlik tadqiqotchilari uchun teskari muhandislik qo'shimcha bilim va malaka sifatida qaralgan bo'lsa, bugungi kunda bu yondashuv mobil ilovalarni chuqur tahlil qilishda tobora muhim o'rin egallab bormoqda.

Zamon o'zgarmoqda endi mobil ilovalarni "qora quti" sifatida tahlil qilish, ya'ni ularning ichki tuzilmasi va ish faoliyatini manba kodisiz tahlil qilish uchun ilovalarni teskari tuzish, ularga yamoq (patch) qo'llash, hatto jonli ishlayotgan jarayonlarni manipulyatsiya qilish kabi murakkab usullar zarur bo'lmoqda. Shu bilan birga, ko'pgina mobil ilovalar buzg'unchilikka qarshi turli himoya mexanizmlarini joriy qilmoqda, bu esa xavfsizlik sinovchilari ishini anchayin murakkablashtiradi.

Umuman olganda, mobil ilovani teskari muhandislik qilish bu kompilyatsiya qilingan (ya'ni mashina uchun tarjima qilingan) dasturiy kod orqali uning dastlabki tuzilmasi va funksiyalarini anglash, ya'ni asl manba kodi haqida xulosa chiqarish jarayonidir. Teskari muhandislikning asosiy maqsadi kodni tushunish, uning qanday ishlashini anglab yetishdir.

Buzg'unchilik (tampering) esa bu jarayonga yanada faol aralashuvni anglatadi: bu ilovaning o'zini yoki uning ishlash muhitini o'zgartirish orqali dastur xatti-harakatiga ta'sir o'tkazishdir. Masalan, ilova ildiz (root) huquqiga ega bo'lgan qurilmada ishlashdan bosh tortsa, sinovchi uning bu xatti-harakatini chetlab o'tish

yoki o'zgartirish ehtiyojida qoladi. Bunday hollarda ilova ish faoliyatini sun'iy tarzda o'zgartirishga to'g'ri keladi.

Mobil xavfsizlik bo'yicha faoliyat yurituvchi mutaxassislar teskari muhandislikning asosiy tushunchalarini chuqur tushunmog'i lozim. Bundan tashqari, ularga mobil qurilmalarning texnik xususiyatlari – protsessor arxitekturasini, bajariladigan fayl formatlari, platformaga xos dasturlash tillari va boshqa tizimli tafsilotlar ham yaxshi tanish bo'lishi zarur.

Shuni unutmaslik kerakki, teskari muhandislik bu nafaqat texnika, balki san'atdir. Uning har bir jihatini to'liq tushuntirish uchun butun bir kutubxona yozish mumkin. Bu sohada ishlovchi mutaxassislar ba'zan yillar davomida faqat bitta aniq, izolyatsiyalangan texnik muammoni yechishga harakat qilishadi – bu zararli kodni avtomatlashtirilgan tarzda tahlil qilish yoki obfuskatsiya (murakkablashtirish) orqali yashirilgan funksiyalarni ochib berish kabi murakkab vazifalar bo'lishi mumkin. Aksincha, xavfsizlik sinovchilari esa strategik yondashuvga ega generallardir: ular tahlil qilish zarur bo'lgan katta hajmdagi texnik ma'lumotlarni ajratib olish, baholash va amaliy xulosalar chiqarishda samarali bo'lishi kerak.

### **Android ilovalardagi zaifliklari**

Android ilovalarda ko'p uchraydigan zaifliklar quydagilar. Ilovada parollar, tokenlar yoki kriptografiya kalitlari kabi maxfiy ma'lumotlarni kod ichiga hard kod qilib yozib qo'yish (hardcoding) keng tarqalgan zaiflik hisoblanadi. Bunday holatda hujumchi oddiygina APK faylini dekompiletsiya qilib, ichidan maxfiy ma'lumotlarni topib olishi mumkin. Android Developer sahifasida ta'kidlanishicha, dasturchilar ko'pincha aniq paket fayllarda yoki strings.xml kabi resurslarda sirli ma'lumotlarni matn yoki bayt massiv ko'rinishida saqlaydi. Bu Kerchoff prinsipiga zid bo'lib, ilovaning xavfsizlik modelini tubdan buzadi. JADX-GUI ni ochish uchun

siz shunchaki jadx-guiterminalga kirishingiz mumkin. Ushbu vosita bizga APKdan ma'lumot olish va dekompilyatsiya qilingan kodni ko'rish imkonini beradi.

### **Lokal ma'lumotlarni saqlash**

Mobil ilovalarda xavfsizlik masalasi doimo dolzarb bo'lib kelgan. Ayniqsa, autentifikatsiya belgilarini (masalan, tokenlar, parollar), shaxsiy ma'lumotlarni va boshqa nozik axborotlarni himoya qilish mobil xavfsizlikning eng asosiy ustunlaridan biridir. Ushbu bobda Android tizimi lokal ma'lumotlarni saqlash uchun taqdim etadigan vositalar, ularning imkoniyatlari va ularni xavfsiz tarzda qo'llash bo'yicha eng yaxshi amaliyotlar bayon etiladi.

Ma'lumotlarni saqlash borasida quyidagi sodda, ammo muhim qoida mavjud: Umumiy ma'lumotlar bimalol foydalanish uchun ochiq bo'lishi mumkin, biroq maxfiy va shaxsiy ma'lumotlar doimo himoyalangan bo'lishi, hattoki iloji bo'lsa, umuman qurilma xotirasiga yozilmasligi kerak. Quyida Androidda keng tarqalgan ma'lumotlarni saqlash usullari keltirilgan:

- **Umumiy afzalliklar (SharedPreferences)** — oddiy kalit-qiymat juftliklarini saqlash uchun qulay, ayniqsa foydalanuvchi sozlamalarini yozib borishda keng qo'llaniladi.
- **SQLite ma'lumotlar bazalari** — murakkab va tuzilgan ma'lumotlarni saqlash uchun ishlatiladi; SQL sintaksisi asosida ishlaydi.
- **Firebase ma'lumotlar bazalari** — real vaqtli, bulutga asoslangan ma'lumotlar bilan ishlovchi ilovalar uchun samarali tanlov.
- **Realm ma'lumotlar bazalari** — tezkor, yengil va obyektga yo'naltirilgan ma'lumotlar ombori bo'lib, oson integratsiyalanadi.
- **Ichki xotira (Internal Storage)** — faqat ilovaning o'ziga tegishli bo'lgan fayllarni saqlash uchun xavfsiz va yopiq muhit.
- **Tashqi xotira (External Storage)** — foydalanuvchi yoki boshqa ilovalar bilan baham ko'riladigan fayllar uchun.

- **Android Keystore** — kriptografik kalitlar va maxfiy ma'lumotlarni xavfsiz tarzda saqlashga mo'ljallangan tizim.

### **Qo'shimcha Saqlash Nuqtalari**

Bularning ustiga, Android tizimi turli kontekstlarda avtomatik ravishda yoki ilova orqali nozik ma'lumotlarni saqlab qo'yishi mumkin bo'lgan boshqa holatlarni ham o'z ichiga oladi. Bunday holatlar quyidagilardan iborat:

- **Ro'yxatga olish (log) funktsiyalari** — noto'g'ri sozlangan loglar maxfiy ma'lumotlarni ochib qo'yishi mumkin.
- **Android zaxira nusxalari (backups)** — avtomatik saqlanadigan ma'lumotlar orqali nozik axborotlar tarqalib ketishi mumkin.
- **Xotirani qayta ishlash (memory management)** — vaqtinchalik saqlanayotgan ma'lumotlar noto'g'ri boshqarilsa, zaiflik manbaiga aylanishi mumkin.
- **Klaviatura keshlari** — foydalanuvchi kiritgan ma'lumotlar (jumladan parollar) vaqtincha saqlanib qolishi mumkin.
- **Skrinshotlar** — maxfiy ekranlar tasodifan yoki ruxsatsiz suratga olinishi ehtimoli mavjud.

Har bir saqlash mexanizmi o'ziga xos xavfsizlik tahdidlariga ega. Shuning uchun, ularni to'liq tushunish va ular bilan bog'liq sinov holatlarini aniq va to'g'ri bajara olish sinovchilarning asosiy vazifasidir. Bu umumiy ko'rinish har bir saqlash usulining qisqacha tavsifini beradi va kerakli hollarda chuqurroq hujjat yoki manbalarga murojaat qilish uchun boshlang'ich nuqta bo'lib xizmat qiladi.

### **Autentifikatsiya muammolari**

Mobil ilovalar foydalanuvchi xavfsizligini ta'minlashda autentifikatsiya muhim rol o'ynaydi. Bu jarayonda ilova foydalanuvchining qurilmada saqlangan hisob ma'lumotlariga nisbatan tekshiruv o'tkazadi ya'ni, foydalanuvchi o'z PIN-kodi,

paroli yoki biometrik belgilarini (masalan, yuz tanish yoki barmoq izi) taqdim etish orqali ilovani yoki ilova ichidagi muhim funksiyalarni "qulfdan chiqaradi".

Autentifikatsiya, odatda, quyidagi vaziyatlarda qo'llaniladi:

- Ilovaga kirishdan oldin foydalanuvchini tezda tasdiqlash
- Muayyan maxfiy funksiyalar (masalan, to'lov, sozlamalar, shifrlangan fayllar) uchun qo'shimcha xavfsizlik bosqichi
- Foydalanuvchi ilgari autentifikatsiyadan o'tgan bo'lsa ham, mavjud seansni himoyalash yoki davom ettirish

Bu autentifikatsiya turi foydalanuvchiga qulaylik va xavfsizlikni bir vaqtning o'zida taqdim etadi. Ammo bu imkoniyat xavfsiz deb hisoblanishi uchun muhim bir narsa yodda tutilishi kerak: mahalliy autentifikatsiya hech qachon mustaqil xavfsizlik chorasiga aylantirilmasligi kerak. Avval "Mobil ilovalarni autentifikatsiya qilish arxitekturasini" bo'limida ta'kidlanganidek, sinovchilar doimo shuni yodda tutishlari kerak:

Mahalliy autentifikatsiya faqat kriptografik ibtidoiy asosga yoki masofaviy (server) tekshiruvga bog'langandagina ishonchli bo'ladi.

Agar foydalanuvchi autentifikatsiyasidan keyin ilova hech qanday tasdiqlovchi ma'lumotni olmasa, tajovuzkor oddiygina kodni teskari muhandislik orqali tahlil qilib, bu autentifikatsiyani chetlab o'tishi mumkin. Ya'ni, faqat ilova ichidagi if-else shartlari orqali amalga oshirilgan PIN yoki biometrik tekshiruv xavfsizlik kafolati bo'la olmaydi.

### **Androidda mahalliy autentifikatsiyaning mexanizmlari**

Android platformasi mahalliy autentifikatsiya uchun ikkita asosiy mexanizmni taklif etadi.

#### **1. Hisob ma'lumotlarini tasdiqlash oqimi (Device Credential Authentication)**

Bu mexanizm foydalanuvchining qurilma xavfsizlik sozlamalariga tayangan holda ishlaydi. PIN-kod, grafik naqsh yoki parol kabi elementlar orqali autentifikatsiya

amalga oshiriladi. Foydalanuvchi qurilmani qanday qulfdan chiqarishni bilsa u bu autentifikatsiyani ham o‘ta oladi.

## **2. Biometrik autentifikatsiya oqimi (Biometric Authentication Flow)**

Yuzni tanish, barmoq izini aniqlash yoki ko‘z skaneri orqali foydalanuvchini tasdiqlovchi zamonaviy yondashuv. Bu mexanizm foydalanuvchi tajribasini sezilarli darajada yengillashtiradi va xavfsizlikni oshiradi.

Android biometrik autentifikatsiya API’lari quyidagilarni ta’minlaydi:

- BiometricPrompt orqali soddalashtirilgan interfeys
- Biometrik ma’lumotlar hech qachon ilova darajasida ko‘rinmaydi — ular faqat ishonchli apparat va tizim qatlamida saqlanadi.

### **Yakuniy Tavsiyalar**

- Mahalliy autentifikatsiya faqat foydalanuvchini tezkor tekshirish uchun ishlatilishi kerak, ammo asosiy qarorlarni qabul qilish (masalan, token berish, maxfiy ma’lumotlarga kirish) server yoki ishonchli kriptografik modulga tayanishi lozim.
- Sinovchi ilova qanday autentifikatsiya usulidan foydalanayotganini, bu usul tashqi tasdiqlov bilan bog‘langanmi yo‘qmi, kodni tahlil qilish orqali aniqlashi kerak.
- Har qanday holatda ham, foydalanuvchi interfeysi orqali berilgan signal (masalan, “barmoq izi tasdiqlandi”) asosiy xavfsizlik manbasi emas, balki faqat kirish nuqtasi bo‘lishi kerak.

### **Tarmoq Xavfsizligi muammolari**

Bugungi kunda deyarli har bir Android ilovasi biror masofaviy xizmat bilan doimiy aloqada ishlaydi. Bu xizmatlar odatda foydalanuvchi ilovada amalga oshiradigan harakatlarga javob beradigan serverlar bo‘lib, ular tarmoq orqali ko‘pincha ishonchsiz yoki ochiq Wi-Fi tarmoqlari orqali ulanadi.

Aynan shu sababli, mobil ilovalarda tarmoq xavfsizligi muhim xavfsizlik masalalaridan biriga aylanadi. Foydalanuvchining nozik ma'lumotlari (masalan, autentifikatsiya tokenlari, parollar yoki shaxsiy ma'lumotlar) noto'g'ri sozlangan tarmoq konfiguratsiyasi tufayli osongina fosh bo'lishi mumkin.

### **HTTP Protokollar va zamonaviy yondashuvlar**

Ko'pgina Android ilovalari bugungi kunda HTTP(S) asosidagi veb-xizmatlardan foydalanadi. Ushbu protokollar keng qo'llaniladi, yaxshi hujjatlashtirilgan va ko'plab platformalarda qo'llab-quvvatlanadi. Shu bilan birga, ular bilan ishlashda xavfsizlikka oid maxsus ehtiyot choralarini ko'rish zarur.

### **Android Tarmoq xavfsizligi konfiguratsiyasi**

Android 7.0 (API darajasi 24) va undan yuqori versiyalarda, ishlab chiquvchilar uchun ilova tarmoq xavfsizligini nazorat qilish imkonini beruvchi maxsus funksional: Network Security Configuration joriy etilgan.

Bu konfiguratsiya ilovaga quyidagi muhim xavfsizlik sozlamalarini qo'llash imkonini beradi:

#### **1. Aniq matn trafiki (Cleartext traffic)**

- Aniq matnda (ya'ni shifrlanmagan HTTP) yuborilayotgan ma'lumotlar foydalanuvchi uchun xavf tug'diradi.
- Ilova ishlab chiquvchisi ushbu trafika ruxsat berishni yoki uni to'liq taqiqlashni belgilashi mumkin.

#### **2. Maxsus ishonch langarlari (Custom trust anchors)**

- Ilova qanday sertifikat idoralariga (CA – Certificate Authority) ishonishini aniq belgilaydi.
- Bu orqali, masalan, o'z-o'zidan imzolangan sertifikatlarni ishonchli deb belgilash yoki faqat muayyan CA'larni qabul qilish mumkin.

#### **3. Sertifikatni mahkamlash (Certificate pinning)**



- Ilova faqat oldindan belgilangan sertifikat(lar) bilan ulanadi. Bu MITM (Man-In-The-Middle) hujumlariga qarshi samarali himoya chorasi hisoblanadi.

#### 4. Faqat disk raskadrovka uchun bekor qilish (Debug-only overrides)

- Disk raskadrovka (debug) rejimida xavfsizlik qatlamlarini vaqtinchalik chetlab o'tish imkonini beradi.
- Bu ishlab chiqish davrida ishni qulaylashtiradi, ammo ishlab chiqarish (production) versiyada o'chirib qo'yilishi kerak.

#### Tarmoq Xavfsizligi Konfiguratsiyasini Qanday Topish Mumkin?

Agar ilovada network\_security\_config mavjud bo'lsa, uni quyidagicha aniqlash mumkin:

**AndroidManifest.xml** faylida application tegida bunday yozuv mavjud bo'ladi:

```
<application android:networkSecurityConfig="@xml/network_security_config" ... >
```

Bu yerda @xml/network\_security\_config — bu faylning joylashuvi bo'lib, u odatda quyidagi yo'ldan topiladi:

res/xml/network\_security\_config.xml

Ushbu faylni tahlil qilib, siz quyidagi narsalarni bilib olishingiz mumkin:

- Ilova HTTP (aniq matn) trafika ruxsat beradimi?
- Qanday sertifikatlar yoki CA'lar ishonchli deb belgilangan?
- Sertifikat pinlash mavjudmi?
- Disk raskadrovka uchun qanday cheklovlar berilgan?

Bundan tashqari, ilova ishga tushganda tizim loglarida (masalan, Logcat'da) konfiguratsiyaga oid hodisalarni ko'rishingiz mumkin. Bu sinovchilar uchun qo'shimcha signal bo'lib xizmat qiladi.

Dinamik kod yuklash ilova ishga tushgach, istalgan vaqtda tashqi kod (Java/DEX yoki o'zga modul)ni yuklab olib bajara olish imkoniyatidir. Bu oddiy ilova uchun keng tarqalgan xususiyat emas, biroq ko'plab zararli ilovalar buni o'z foydasiga ishlatadi. Masalan, hujumchi o'z dasturini ilova ichidan alohida modullar

shaklida yuklab, DexClassLoader yordamida ishga tushirishi mumkin. Shunday qilib, u dasturga orqa fonda zararli kod qo'shadi. Android Developers manbaiga ko'ra, dinamik yuklangan kodni xakerlar osongina manipulyatsiya qilib, ma'lumotlarni o'g'irlashga yoki ilgari ko'zda tutilmagan amallar bajarishga moyil bo'lishi mumkin.

Ko'p Android ilovalari foydalanuvchi ma'lumotlarini qurilmada, ochiq ko'rinishda, shifrlanmagan holda saqlaydi. Masalan, SQLite bazasi, fayllar, *SharedPreferences* yoki log fayllarda foydalanuvchi ism-sharifi, parollar yoki tokenlar ochiq nusxada yozilishi mumkin. Har qanday ilova (yoki hatto telefonni kompyuterga ulangandan keyin oddiy dastur yordamida) ushbu fayllarni o'qishi mumkin. Root qilingan qurilma esa bu ma'lumotlarni ko'rishi va yuklab olishi mumkin bo'ladi.

SSL/TLS (Secure Sockets Layer / Transport Layer Security) bu internetda xavfsiz aloqani ta'minlovchi kriptografik protokol bo'lib, u ma'lumotlarning shifrlangan holda uzatilishini va serverning haqiqiylikini tekshirishni kafolatlaydi. "SSL certificate validation" bu foydalanuvchi qurilmasi yoki ilovasi serverdan olingan sertifikatni tekshiradi (masalan, domen mosligi, ishonchli CA (Certificate Authority) tomonidan berilganligi va amal qilish muddati). Agar bu tekshiruv ataylab o'chirilsa yoki noto'g'ri sozlansa, bu juda xavfli xavfsizlik zaifligiga olib keladi.

Dastur kodini obfusikatsiya qilmaslik bu Android ilovalarda ilova kodi ochiq yoki tushunarli shaklda qoldirilgan bo'lib, uni oson teskari muhandislik (reverse engineering) qilish mumkinligini bildiradi. Ya'ni, ilovani .apk fayli orqali yuklab olib, jadx, APKTool, yoki MobSF kabi vositalar bilan dekompilyatsiya qilib, kodni o'qib olish mumkin. Agar kodda hech qanday chalkashtirish (obfuscation) qilinmagan bo'lsa, loyihaning ichki logikasi, API kalitlari, autentifikatsiya mexanizmlari va boshqa muhim komponentlar oson ochib olinadi.

## iOS ilovalardagi zaifliklari

**Insecure Data Storage**(Ma'lumotlarni himoyalanmagan tarzda saqlash) - Foydalanuvchi hisob ma'lumotlari kabi maxfiy ma'lumotlarni xavfsiz tarzda saqlash ma'lumotlarning buzilishiga olib kelishi mumkin.

*Yechim* : Maxfiy ma'lumotlarni xavfsiz saqlash uchun Keychain Services API dan foydalaning. Muammo kodi:

```
// UserDefaults da parolni xavfsiz saqlash
UserDefaults.standard.set( " mySecurePassword" , forKey: "password" )

//
Agar parol bo'lsa parolni olish = UserDefaults.standard.string(forKey: "password" ) {
    print ( "Qabul qilingan parol: \(password )" )
}
```

*Yechim kodi*:

```
import Security

func savePassword ( xizmat : String , account : String , password : String ) -> Bool {
    let data = password.data (foydalanish: .utf8) !
    let query = [
        kSecClass: kSecClassGenericPassword, kSecAttrService :
        service,
        kSecAttrAccount: account,
        kSecValueData: data
    ] as CFDictionary

    SecItemDelete (so'rov) // Mavjud bo'lgan narsalarni o'chirish let
    status = SecItemAdd(nicr = return statuser ( query ) } func getPassword ( xizmat : String , hisob :
String ) -> String ? { let query = [ kSecClass: kSecClassGenericPassword,
kSecAttrService: service, kSecAttrAccount: account, kSecReturnData: true ,
kSecMatchLimit: kSecMatchLimitOne ] as CFDictionary var DataTypeRef: 0 ? status =
SecItemCopyMatching (so'rov va dataTypeRef) agar status == errSecSuccess bo'lsa , ma'lumotlar =
dataTypeRef shunday bo'lsin ? Ma'lumotlar { qaytish String (ma'lumotlar: ma'lumotlar, kodlash: .utf8)
} qaytish nil }
```

**Improper Certificate validation**(Sertifikatni yetarlicha tekshirmaslik). SSL/TLS sertifikatlarini to'g'ri tekshirib bo'lmamaslik ilovangizni o'rtadagi odam hujumlariga qarshi himoyasiz qoldirishi mumkin.

*Yechim*: to'g'ri SSL pinlashni amalga oshiring.

Muammo kodi:

```
// Barcha sertifikatlarga noto'g'ri ishonish
seans = URLSession (konfiguratsiya: .default, delegate: nil , delegateQueue: nil ) ruksat
url = URL ( string: "https://example.com" ) !
let task = session.dataTask(with: url) { ma'lumotlar, javob, xato //
    javobni boshqarish
}
task.resume()
```

Yechim kodi:

```
import Foundation

class CustomSessionDelegate: NSObject, URLSessionDelegate {
    func urlSession(_ session: URLSession, didReceive challenge: URLAuthenticationChallenge,
        completionHandler: @escaping (URLSession.AuthChallengeDisposition, URLCredential?) -> Void) {
        if let serverTrust = challenge.protectionSpace.serverTrust,
            SecTrustEvaluateWithError(serverTrust, nil) {
            let credential = URLCredential(trust: serverTrust)
            completionHandler(.useCredential, credential)
        } else {
            completionHandler(.cancelAuthenticationChallenge, nil)
        }
    }
}

let session = URLSession(configuration: .default, delegate: CustomSessionDelegate(), delegateQueue:
    nil)
let url = URL(string: "https://example.com")!
let task = session.dataTask(with: url) { data, response, error in
    // Handle response
}
task.resume()
```

**Insufficient Input Validation**(Kiritilgan ma'lumotlarni yetarli darajada tekshirmaslik). Foydalanuvchi ma'lumotlarini to'g'ri tasdiqlamasdan qabul qilish va qayta ishlash SQL in'ektsiyasi, bufer to'lib ketishi va boshqalar kabi muammolarga olib kelishi mumkin.

*Yechim:* Kirishni to'liq tekshirish va tozalashni amalga oshiring.

Muammo kodi:

```
// Accepting user input without validation
let userInput = "DROP TABLE Users;"
print("User input: \(userInput)")
// Inserting user input directly into an SQL query can lead to SQL injection
let query = "SELECT * FROM Users WHERE username = '\(userInput)'"
print("SQL Query: \(query)")
```

Yechim kodi:

```
func validateInput(input: String) -> Bool {
    let regex = "[a-zA-Z0-9]+"
    let predicate = NSPredicate(format: "SELF MATCHES %@",
        regex)
    return predicate.evaluate(with: input)
}

// Usage
let userInput = "testInput123"
if validateInput(input: userInput) {
    print("Valid input")
} else {
    print("Invalid input")
}
```

**Insecure API Usage**(API dan noto‘g‘ri va zaif tarzda foydalanish). Ishonchsiz APIlardan foydalanish yoki API javoblarini noto‘g‘ri ishlatish ilovangizni turli xil hujumlarga duchor qilishi mumkin.

Yechim : Har doim xavfsiz, autentifikatsiya qilingan APIlardan foydalaning va javoblarni ehtiyotkorlik bilan bajaring.

Muammo kodi:

```
// Making API call without authentication
let url = URL(string: "http://api.example.com/data")!
let task = URLSession.shared.dataTask(with: url) { data, response, error
in if let data = data {
    print("Data received: \(data)")
}
}
task.resume()
```

Yechim kodi:

```
import Foundation

struct APIClient {
    static let baseURL = URL(string: "https://secure-api.example.com")!

    static func fetchSecureData(endpoint: String, completion: @escaping (Data?, URLResponse?, Error?) -
> Void) {
        guard let url = URL(string: endpoint, relativeTo: baseURL) else { return }
        var request = URLRequest(url: url)
        request.addValue("Bearer YOUR_ACCESS_TOKEN", forHTTPHeaderField: "Authorization")

        let task = URLSession.shared.dataTask(with: request) { data, response, error in
            guard let httpResponse = response as? HTTPURLResponse, httpResponse.statusCode == 200 else
            {
                print("Invalid response")
                completion(nil, response, error)
                return
            }
            completion(data, response, error)
        }
        task.resume()
    }
}

// Usage
APIClient.fetchSecureData(endpoint: "/data") { data, response, error in
    if let data = data {
        print("Secure data received: \(data)")
    } else {
        print("Failed to fetch data: \(error?.localizedDescription ?? "Unknown error")")
    }
}
```

**Weak Cryptographic Practices**(Kriptografiyada noto‘g‘ri va zaif usullardan foydalanish): Eskirgan yoki zaif kriptografik algoritmlardan foydalanish ma’lumotlaringiz xavfsizligini buzishi mumkin.

Yechim : Zamonaviy kriptografik kutubxonalar va algoritmlardan foydalaning.

Muammo kodi:

```

// Using outdated and insecure cryptographic algorithms
import CommonCrypto

func encryptWithDES(data: Data, key: Data) -> Data? {
    var numBytesEncrypted: size_t = 0
    var encryptedData = Data(count: data.count + kCCBlockSizeDES)

    let status = encryptedData.withUnsafeMutableBytes { encryptedBytes in
        data.withUnsafeBytes { dataBytes in
            key.withUnsafeBytes { keyBytes in
                CCCrypt(
                    CCOperation(kCCEncrypt),
                    CCAAlgorithm(kCCAlgorithmDES),
                    CCOptions(kCCOptionPKCS7Padding),
                    keyBytes.baseAddress, kCCKeySizeDES,
                    nil,
                    dataBytes.baseAddress, data.count,
                    encryptedBytes.baseAddress, encryptedData.count,
                    &numBytesEncrypted
                )
            }
        }
    }

    if status == kCCSuccess {
        encryptedData.removeSubrange(numBytesEncrypted..)
    }
    return nil
}

```

Yechim kodi:

```

import CommonCrypto

func encrypt(data: Data, key: Data) -> Data? {
    var numBytesEncrypted: size_t = 0
    var encryptedData = Data(count: data.count + kCCBlockSizeAES128)

    let status = encryptedData.withUnsafeMutableBytes { encryptedBytes in
        data.withUnsafeBytes { dataBytes in
            key.withUnsafeBytes { keyBytes in
                CCCrypt(
                    CCOperation(kCCEncrypt),
                    CCAAlgorithm(kCCAlgorithmAES),
                    CCOptions(kCCOptionPKCS7Padding),
                    keyBytes.baseAddress, kCCKeySizeAES128,
                    nil,
                    dataBytes.baseAddress, data.count,
                    encryptedBytes.baseAddress, encryptedData.count,
                    &numBytesEncrypted
                )
            }
        }
    }

    if status == kCCSuccess {
        encryptedData.removeSubrange(numBytesEncrypted..)
    }
    return nil
}

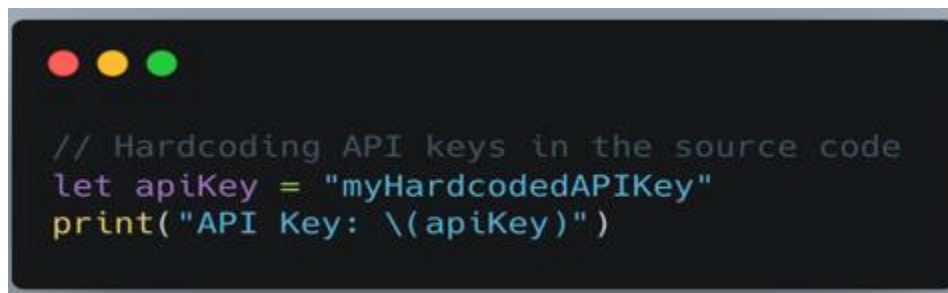
// Usage
let key = "your-encryption-key".data(using: .utf8)!
let data = "Sensitive data".data(using: .utf8)!
if let encryptedData = encrypt(data: data, key: key) {
    print("Encrypted data: \(encryptedData)")
} else {
    print("Encryption failed")
}

```

**Hardcoded Sensitive Information**(Kod ichida bevosita yozilgan maxfiy ma'lumotlar). API kalitlari yoki manba kodingizdagi parollar kabi nozik ma'lumotlarni qattiq kodlash, agar tajovuzkorlar kodga kirish imkoniga ega bo'lsa, ularga oson kirishiga olib kelishi mumkin.

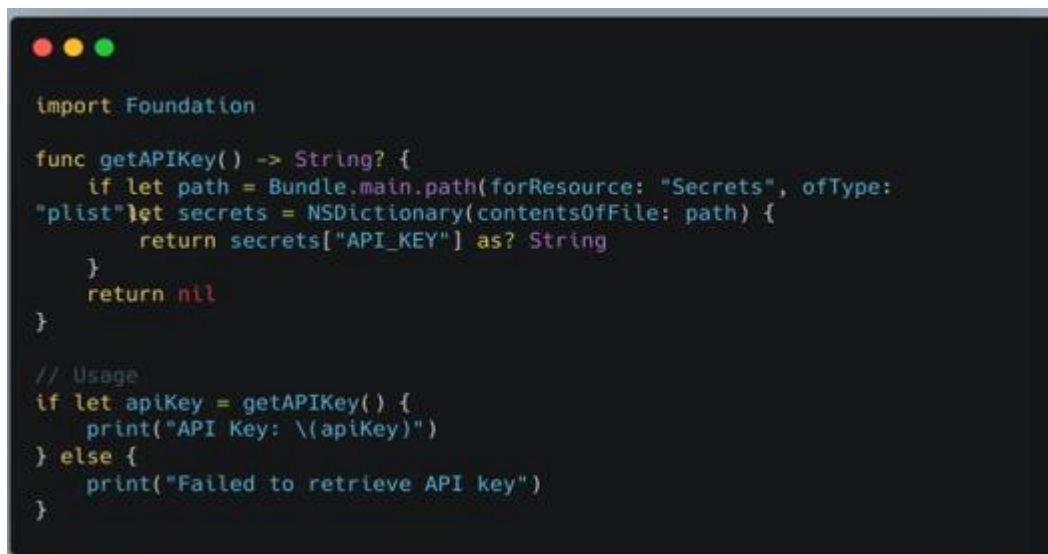
Yechim : Xavfsiz saqlash va xiralashtirish usullaridan foydalaning.

Muammo kodi:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Swift and demonstrates hardcoded sensitive information.

```
// Hardcoding API keys in the source code
let apiKey = "myHardcodedAPIKey"
print("API Key: \(apiKey)")
```

Yechim kodi:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Swift and demonstrates a secure way to retrieve API keys by storing them in a plist file.

```
import Foundation

func getAPIKey() -> String? {
    if let path = Bundle.main.path(forResource: "Secrets", ofType:
    "plist") {
        let secrets = NSDictionary(contentsOfFile: path) {
            return secrets["API_KEY"] as? String
        }
        return nil
    }
}

// Usage
if let apiKey = getAPIKey() {
    print("API Key: \(apiKey)")
} else {
    print("Failed to retrieve API key")
}
```

**Insecure Communication**(Xavfsiz bo'lmagan aloqa). Maxfiy ma'lumotlarni himoyalanmagan kanallar orqali uzatish zararli shaxslar tomonidan tutib olinishiga olib kelishi mumkin.

Yechim: Barcha aloqalar uchun HTTPS dan foydalaning va SSL pinning o'rnatilganligiga ishonch hosil qiling.



*Muammo kodi:*

```
// Using HTTP instead of HTTPS for communication
let url = URL(string: "http://api.example.com/data")!
let task = URLSession.shared.dataTask(with: url) { data, response, error
in if let data = data {
    print("Data received: \(data)")
}
}
task.resume()
```

*Yechim kodi:*

```
import Foundation

let url = URL(string: "https://secure-api.example.com/data")!
var request = URLRequest(url: url)
request.addValue("Bearer YOUR_ACCESS_TOKEN", forHTTPHeaderField: "Authorization")

let task = URLSession.shared.dataTask(with: request) { data, response, error in
    guard let httpResponse = response as? HTTPURLResponse, httpResponse.statusCode == 200 else
    {
        print("Invalid response")
        return
    }
    if let data = data {
        print("Data received: \(data)")
    }
}
task.resume()
```

**Lack of Multi-Factor Authentication**(Ko‘p bosqichli autentifikatsiyaning yo‘qligi). Autentifikatsiya qilish uchun faqat foydalanuvchi nomi va parolga tayanish xavfli bo'lishi mumkin.

Yechim : Ko‘p faktorli autentifikatsiyani (MFA) amalga oshirish.

Muammo kodi:

```
// Using only username and password for authentication
let username = "user"
let password = "password"
print("Logging in with username: \(username) and password: \
(password)")
```

Yechim kodi:



```

import LocalAuthentication

func authenticateUser(completion: @escaping (Bool, Error?) -> Void) {
    let context = LAContext()
    var error: NSError?

    if context.canEvaluatePolicy(.deviceOwnerAuthentication, error: &error) {
        context.evaluatePolicy(.deviceOwnerAuthentication, localizedReason: "Access requires authentication") { success, evaluationError in
            DispatchQueue.main.async {
                completion(success, evaluationError)
            }
        }
    } else {
        completion(false, error)
    }
}

// Usage
authenticateUser { success, error in
    if success {
        print("User authenticated successfully")
    } else {
        print("Authentication failed: \(error?.localizedDescription ?? "Unknown error")")
    }
}

```

**Missing App Transport Security**(Ilovada xavfsiz ma'lumot uzatish himoyasi yo'q).

ATS konfiguratsiyasiz xavfsiz ulanishlarga ruxsat berish ilovangizni tarmoq hujumlariga duchor qilishi mumkin. Yechim : ATS ni to'g'ri sozlang va faqat zarur hollarda istisnolarga ruxsat bering.

Muammo kodi:

```

<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsArbitraryLoads</key>
    <false/>
    <key>NSExceptionDomains</key>
    <dict>
        <key>example.com</key>
        <dict>
            <key>NSExceptionAllowsInsecureHTTPLoads</key>
            <key>NSIncludesSubdomains</key>
            <true/>
            <key>NSExceptionRequiresForwardSecrecy</key>
            <true/>
        </dict>
    </dict>
</dict>
</dict>

```

**Hardcoded Credentials**(Kod ichida bevosita yozilgan login va parollar). Plist faylida iOS mobil ilovasi uchun muhim konfiguratsiya tafsilotlari, jumladan, qoʻllab-quvvatlanadigan iOS versiyalari va qurilma mosligi mavjud. Ushbu maʼlumot operatsion tizimning ilova bilan samarali ishlashi uchun juda muhimdir. Ushbu .plist fayllar qattiq kodlangan hisob maʼlumotlarini ham oʻz ichiga olishi mumkin. Quyidagi buyruq yordamida siz ilova identifikatorini topishingiz mumkin

```
C:\Users\Public>frida-ps -Ua | findstr DVIA
11200 DVIA-v2 com.hightitudehacks.DVIAswiftv2
```

Endi biz .plist fayllarini topish va oʻqish uchun objectiondan foydalanamiz:

```
C:\Users\Public>objection -g com.hightitudehacks.DVIAswiftv2 explore
Checking for a newer version of objection...
Using USB device 'Apple iPhone'
Agent injected and responds ok!

┌───┴───┐ ┌───┴───┐ ┌───┴───┐ ┌───┴───┐
│ . │ . │ . │ . │ . │ . │ . │ . │
└───┴───┘ └───┴───┘ └───┴───┘ └───┴───┘
      (object)inject(ion) v1.11.0

Runtime Mobile Exploration
by: @leonjza from @sensepost

[tab] for command suggestions
....hightitudehacks.DVIAswiftv2 on (iPhone: 15.7.1) [usb] # |

C:\Users\Public>objection -g com.hightitudehacks.DVIAswiftv2 explore
Checking for a newer version of objection...
Using USB device 'Apple iPhone'
Agent injected and responds ok!

┌───┴───┐ ┌───┴───┐ ┌───┴───┐ ┌───┴───┐
│ . │ . │ . │ . │ . │ . │ . │ . │
└───┴───┘ └───┴───┘ └───┴───┘ └───┴───┘
      (object)inject(ion) v1.11.0

Runtime Mobile Exploration
by: @leonjza from @sensepost

[tab] for command suggestions
....hightitudehacks.DVIAswiftv2 on (iPhone: 15.7.1) [usb] # env

Name Path
-----
BundlePath /private/var/containers/Bundle/Application/C6A11852-971C-4D56-B91D-AB9F4EE35C91/DVIA-v2.app
CachesDirectory /var/mobile/Containers/Data/Application/2260A850-63A1-448B-837A-74EFA851E3FF/Library/Caches
DocumentDirectory /var/mobile/Containers/Data/Application/2260A850-63A1-448B-837A-74EFA851E3FF/Documents
LibraryDirectory /var/mobile/Containers/Data/Application/2260A850-63A1-448B-837A-74EFA851E3FF/Library
....hightitudehacks.DVIAswiftv2 on (iPhone: 15.7.1) [usb] # cd /var/mobile/Containers/Data/Application/2260A850-63A1-448B-837A-74EFA851E3FF/Documents
/var/mobile/Containers/Data/Application/2260A850-63A1-448B-837A-74EFA851E3FF/Documents
....hightitudehacks.DVIAswiftv2 on (iPhone: 15.7.1) [usb] # ls
NSFileType Perms NSFileProtection Read Write Owner Group Size Creation Name
-----
Regular 420 CompleteUntilFirstUserAuthentication True True mobile (501) mobile (501) 279.0 B 2024-04-20 15:38:04 +0000 userInfo.plist

Readable: True Writable: True
....hightitudehacks.DVIAswiftv2 on (iPhone: 15.7.1) [usb] # ios plist cat userInfo.plist
{
  password = test123;
  username = test;
}
....hightitudehacks.DVIAswiftv2 on (iPhone: 15.7.1) [usb] # |
```

Endi esa ios plist cat userInfo.plist orqali biz parol va foydalanuvchi nomini koʻrishimiz mumkin.

**Insecure Touch/Face ID Fallbacks**(Touch ID yoki Face ID tizimlarida zaif zaxira (fallback) autentifikatsiya). Touch ID yoki Face ID bu iOS'ning biometrik autentifikatsiya mexanizmlari. Dasturchilar foydalanuvchilarni identifikatsiya qilish uchun LocalAuthentication kutubxonasidan foydalanadilar. Ammo, fallback (ya'ni biometrik autentifikatsiya muvaffaqiyatsiz bo'lsa) noto'g'ri amalga oshirilsa, bu zaif autentifikatsiya yoki auth bypass holatiga olib kelishi mumkin.

Dasturchi LAContext.evaluatePolicy() funksiyasida foydalanuvchi biometrik autentifikatsiyani rad etsa yoki u ishlamasa, uni automat tarzda tizimga kiritadi yoki true deb qabul qiladi.

Yoki noto'g'ri fallback mexanizmini (masalan, UIAlert orqali oddiy "yes" tugmasi) qo'llaydi.

```
import LocalAuthentication
import UIKit

class ViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        authenticateUser()
    }

    func authenticateUser() {
        let context = LAContext()
        var error: NSError?

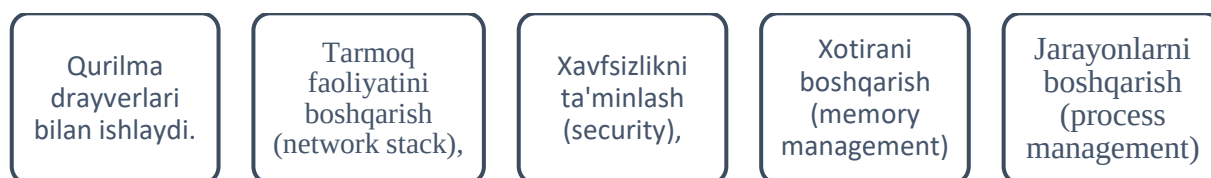
        if context.canEvaluatePolicy(.deviceOwnerAuthenticationWithBiometrics, error: &error) {
            context.evaluatePolicy(.deviceOwnerAuthenticationWithBiometrics, localizedReason: "Please
authenticate") { success, error in
                if success {
                    DispatchQueue.main.async {
                        self.showSecretScreen()
                    }
                } else {
                    // ❌ FALLBACK: Avtomatik kirishga ruxsat
                    DispatchQueue.main.async {
                        self.showSecretScreen()
                    }
                }
            }
        }
    }

    func showSecretScreen() {
        print("🟢 Kirish muvaffaqiyatli! (hacker uchun ham)")
        // maxfiy kontent
    }
}
```

## Android va iOS arxitekturası

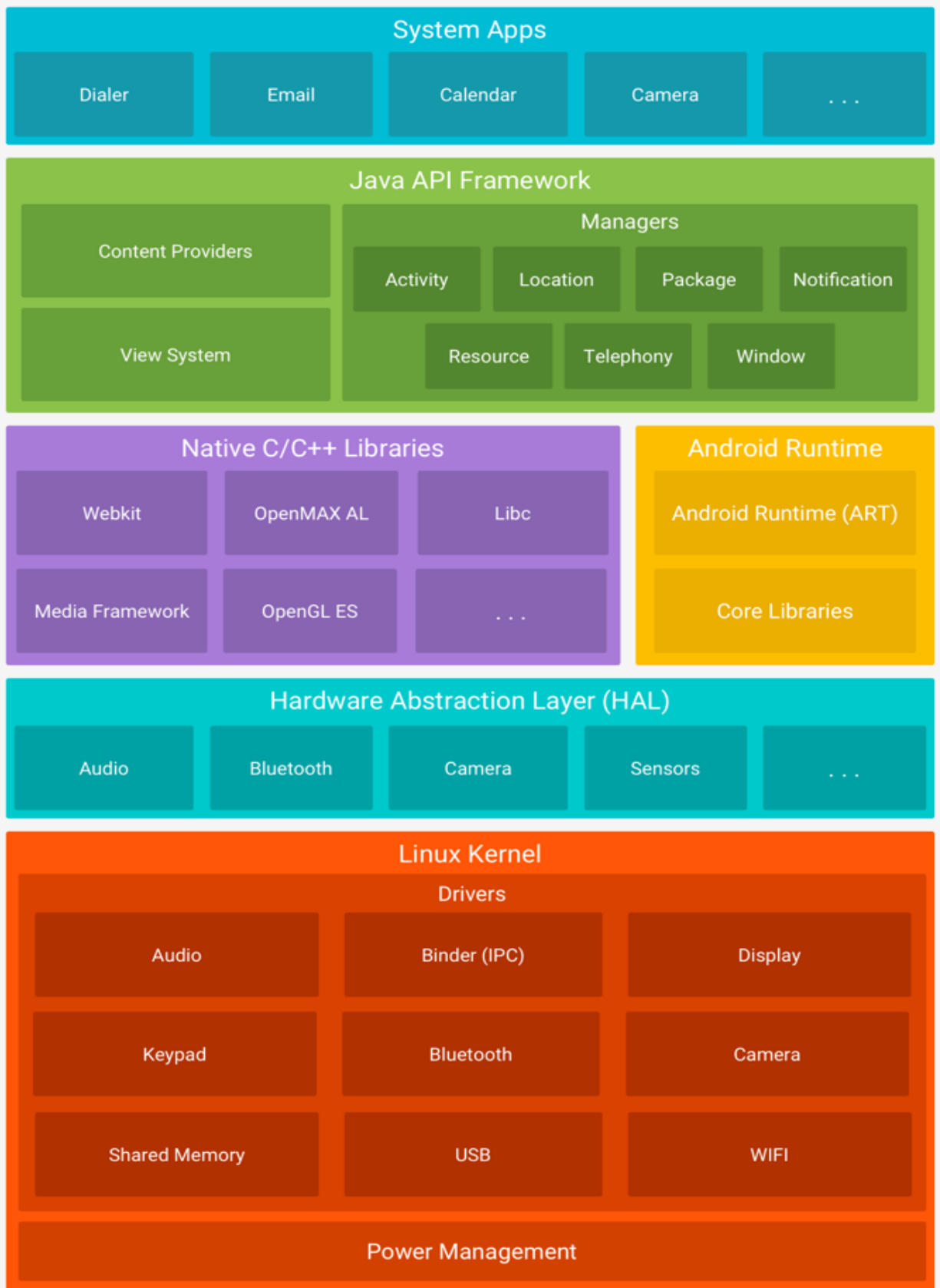
### *Android arxitekturası*

Android operatsion tizimi o'zining asosi sifatida Linux yadrosidan (kernel) foydalanadi. Bu yadro Android tizimining yuragi bo'lib xizmat qiladi. Odatda Android qurilmalari Linuxning 4.x yoki 5.x versiyasi asosida ishlaydi. Linux yadrosi Android tizimida quyidagi asosiy funksiyalarni bajaradi.



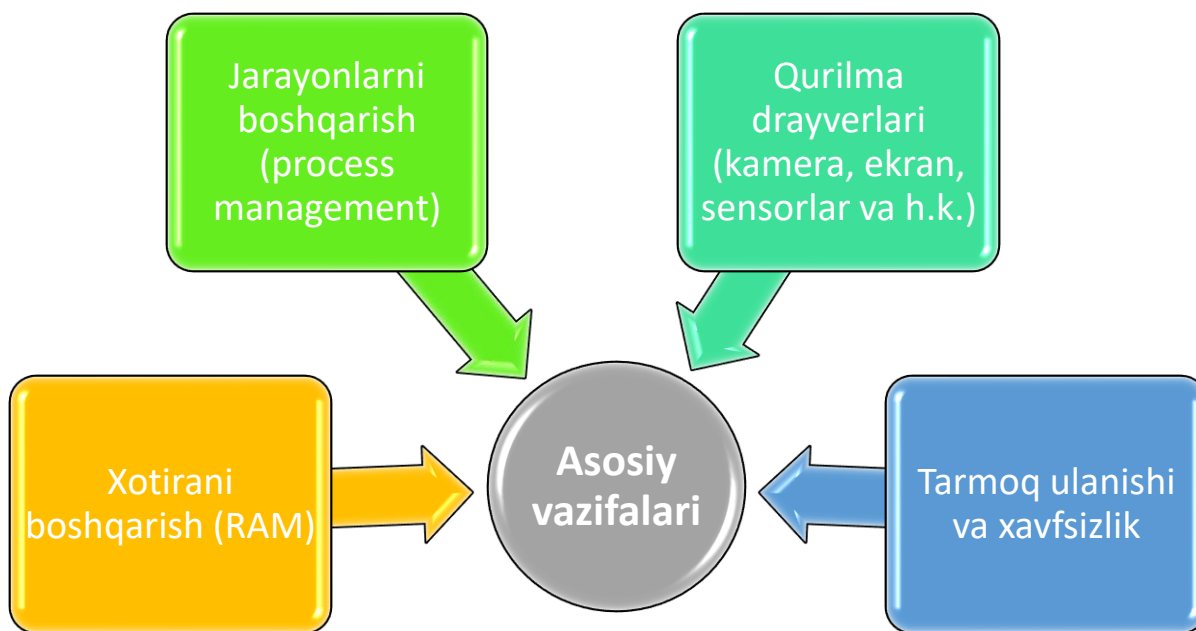
Ya'ni Android mustaqil operatsion tizim bo'lib ko'rinsada, aslida u Linux yadrosiga qurilgan tizimdir. Bu yadro Android ilovalari va qurilma "qattiq qatlam" (hardware) o'rtasidagi ko'priq vazifasini bajaradi hamda tizim barqarorligi va xavfsizligini ta'minlaydi. Android operatsion tizimi qatlamli (layered) tuzilmaga ega bo'lib, har bir qatlam o'ziga xos vazifani bajaradi. Ushbu tuzilma tizimni moslashuvchan, xavfsiz va samarali qiladi. Android arxitekturası 5 asosiy qatlamdan tashkil topgan.





## Linux kernel

Linux Kernel – Android platformasining poydevori Linux yadrosi hisoblanadi. U apparat resurslarini boshqarish (xotira, jarayonlar), qurilma driverlari va asosiy xavfsizlik funksiyalarini (UID asosida izolyatsiya, SELinux) ta'minlaydi. Android uchun yadroning maxsus kengaytmalari ya'ni wakelock(qurilmani uxlash rejimidan uyg'otish), ashmem(ko'p jarayonli xotira almashish), Binder IPC(jarayonlararo aloqalar) kabi komponentlar kiritilgan. Bundan tashqari, Androidning C kutubxonasi Bionic bo'lib, u kichikroq hajm va lisenziya qulayligi tufayli qo'llanadi.



Linux kernel bepul va ochiq manbali, Unixga o'xshash yadrodir va u butun dunyo bo'ylab ko'plab kompyuter tizimlarida qo'llaniladi. Yadro 1991-yilda Linus Torvalds tomonidan yaratilgan va tez orada Unix o'rnini bosuvchi bepul operatsion tizim bo'lishi uchun yaratilgan GNU operatsion tizimi uchun yadro sifatida qabul qilingan. 1990-yillarning oxiridan beri, u ko'plab operatsion tizim distributivlariga kiritilgan bo'lib, ularning ko'pchiligi Linux deb ataladi. Shunday Linux yadrosi

asosida ishlovchi operatsion tizimlardan biri, Android bo'lib, u ko'plab mobil qurilmalarda ishlatiladi.

Yadro kodining aksariyati GNU kompilyator to'plami (GCC) tomonidan qo'llab-quvvatlanadigan, standart C tilidan tashqarida ba'zi kengaytmalarni o'z ichiga olgan C dasturlash tilida yozilgan. Shuningdek, kodda ba'zi arxitekturaga xos funksiyalar uchun masalan, xotira foydalanishini va vazifalarni bajarishni optimallashtirish uchun assembly (assembler) dasturlash tili kodlari ham mavjud. Yadro modulli dizaynga ega, ya'ni modullar alohida dasturiy komponent sifatida birlashtirilishi, hattoki dinamik ravishda yuklab olinishi mumkin. Biroq arxitektura nuqtayi nazaridan yadro monolitik hisoblanadi, chunki butun operatsion tizim yadrosi kernel fazosida ishlaydi. Linux GNU Umumiy Ommaviy Litsenziyasining 2-versiyasi (GPLv2) asosida tarqatiladi, garchi uning tarkibida boshqa mos litsenziyalar ostida bo'lgan fayllar ham mavjud. Linux kernelni rasmiy manbadan quyidagi saytdan yuklab olishingiz mumkin(<https://www.kernel.org>). Bu Linux Foundation tomonidan boshqariladigan va doimo eng so'nggi, barqaror versiyalar e'lon qilinadigan rasmiy sayt.

### **Hardware Abstraction Layer**

Hardware Abstraction Layer (HAL) – Android arxitekturasining dastlabki versiyalaridan beri mavjud bo'lib, birinchi Android qurilmalaridan boshlab ishlab chiquvchilar va apparat yetkazib beruvchilar uchun umumiy interfeys vazifasini bajaradi. Android 8.0 bilan bu qatlam yangi modul tuzilishga o'tkazildi: har bir HAL uchun mustaqil AIDL/HIDL interfeys yaratilishi talab qilina boshlandi. 2015-yildan boshlab HIDL (Hardware Interface Definition Language), keyinchalik 2019-yildan AIDL (Android IDL) orqali HAL interfeyslari belgilandi. Haligacha Android 8.0 gacha eski HAL qatlami bo'lgan, lekin 8.0 dan har bir HAL uchun standart interfeys joriy etilgan. HAL apparat qurilmalariga (kamera, sensorlar, audio, Bluetooth va

boshqalarga) kirish uchun yuqori qatlamda (Java API ramkasi) mavjud bo'lgan abstraktsiya qatlami. HAL modulalari har bir turdagi apparat komponenti uchun alohida bo'lib, Android tizimiga quyi darajadagi qurilma xususiyatlarini o'zgartirmasdan, umumiy interfeys orqali taqdim etish imkonini beradi. Masalan, kamera, Bluetooth, audio, sensorlar uchun alohida HAL modullar mavjud. Ilovalar ramkasidagi tegishli API chaqirig'i HAL moduliga tushganda, Android u modulni yuklaydi va uskunaga murojaat qiladi. HAL amalda C/C++ kutubxonalar bo'lib, ularning interfeyslari Android tomonidan aniq belgilangan. Android 8.0 dan boshlab HAL interfeyslari AIDL/HIDL yordamida tavsiflanadi. *Binderized HAL* tushunchasi bor bu jarayonlararo bog'lanish (IPC) uchun Binder qo'llanib, har bir HAL moduli alohida jarayonda (service) ishlaydi. Android 8.0 dan yuqori versiyada har bir zaruriy HAL modul Android qurilmasida vendor bo'limida (vendor partition) yaratilishi va ro'yxatdan o'tishi kerak. Android 13 bilan HIDL eskirdi va yangisi sifatida AIDL-hal interfeyslari tavsiya etilmoqda. Application Frameworkdagi apparatga oid API (masalan, CameraManager, BluetoothAdapter) HAL interfeysini chaqiradi. HAL esa o'z navbatida Linux yadrosidagi drayver bilan bog'lanadi. Misol uchun, kamera suratga olish: foydalanuvchi yoki ilova camera API'ga so'rov yuboradi. Android tizimi Camera HAL modulini ishga tushiradi; bu modul esa camera nomli noyob modul sifatida Linux yadrosi drayveri bilan o'zaro ishlaydi. Shu tariqa, yuqori qatlamlar apparatga bevosita bog'lanmaydi, balki HAL orqali ko'prik vazifasini bajaradi. Ko'p uchraydigan misollardan biri – Bluetooth, Wi-Fi, GPS modullari bilan ishlash. Masalan, ilova BluetoothAdapter ni yoqish uchun murojat qiladi; ramka bu so'rovni Bluetooth HAL moduliga uzatadi. HAL esa chipsetdagi Bluetooth drayveri bilan aloqa o'rnatib, qurilmani yoqadi. Yoki Sensor API orqali tezlanish o'lchagich (accelerometer) ma'lumotini olish uchun Sensor HAL orqali sensor haydovchisi ishlatiladi. Har bir apparat turi uchun shunga

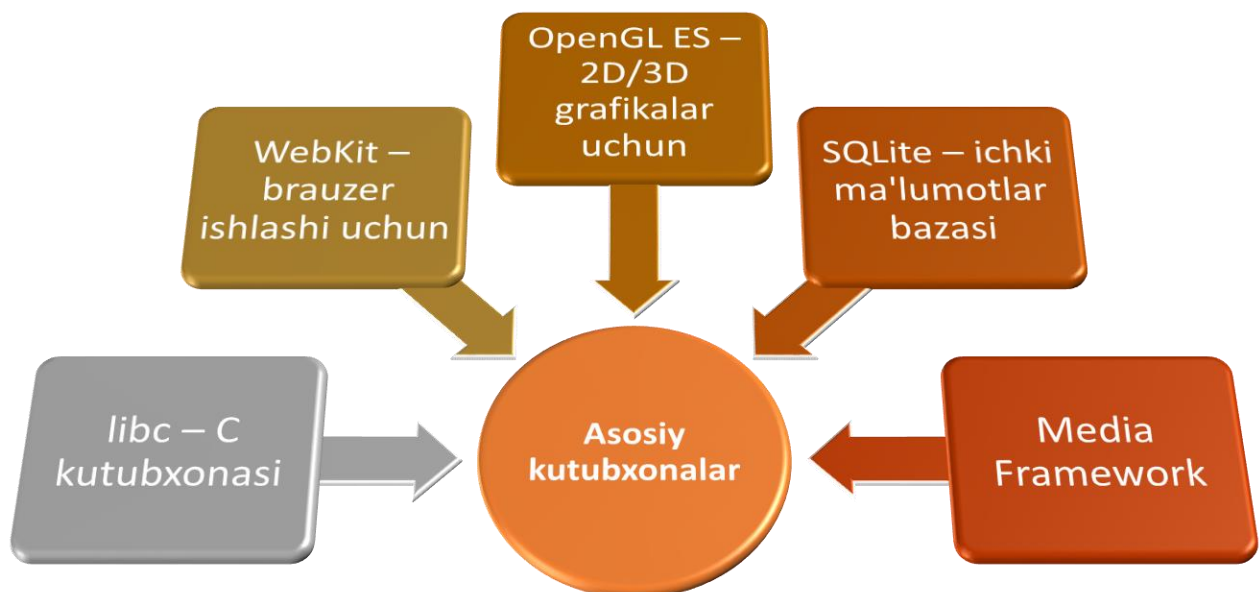


o‘xshash jarayon bo‘lib, HAL va Linux drayverlari o‘rtasida HTTP muloqot sodir bo‘ladi.

### **Android Runtime (Dalvik va ART)**

Android Runtime (Dalvik va ART) – Androidning dastlabki yillari (2008–2014) Dalvik virtual mashinasi ustida ishladi (bu Java bytekodini *just-in-time* kompilyatsiya bilan bajaradigan maxsus JVM edi). 2014-yilda Android 5.0 (Lollipop) da Dalvik o‘rni ART (Android Runtime) egalladi, u ilovalarni *ahead-of-time* va *just-in-time* kompilyatsiya qilib, unumdorlikni oshirdi. Shu qatlamda asosiy C/C++ kutubxonalar ham yaratilgan: masalan, Bionic libc (Android uchun maxsus C kutubxonasi), libm (matematik kutubxona), libz (siqish uchun), OpenSSL (shifrlash), SQLite (ma’lumotlar bazasi) va boshqalar. Android NDK (Native Development Kit) esa 2009-2010-yillarda ishlab chiqilib, dasturchilarga C/C++ kodini yozish imkonini berdi. Android runtime – bu Androidning C/C++ da yozilgan kutubxonalari va ART/Dalvik virtual mashinasidan iborat bo‘lib, tizim xizmatlari va yuqori qatlamlar uchun past darajali ishlov berishni ta’minlaydi. Yuqori qatlamdagi Java kodlarining kerakli qismlarini C/C++ ga o‘tkazib bajaradi (masalan, grafik chizish, multimedia ishlov berish). Shu qatlam libbinder.so orqali inter-protsess muloqotiga (Binder IPC) xizmat qiladi, va ART/Dalvik (virtual mashina) bu yerda ishlaydi, ya’ni Java kodlarini natijaviy mashina kodiga aylantiradi. Shuningdek, Android Runtime (ART) bu qatlamda joylashgan: u C++ da yozilgan bo‘lib, DEX formatidagi bytekodni CPU arxitekturasiga mos mashina kodiga aylantirib bajarguchi muhitdir. ART har bir ilovaga alohida jarayon va VM instance ajratadi, bu esa ko‘p ilova bir vaqtning o‘zida samarali ishlashini ta’minlaydi. Yuqori qatlamdagi Java kodlari muayyan C/C++ funksiyalarini chaqirish uchun JNI (Java Native Interface) orqali kutubxonalarga murojaat qiladi. Masalan, UI chizishda View tizimi libhwui va libskia kabi grafik kutubxonalardan

foydalanadi. SQLite bilan ishlash uchun ContentProvider va ActivityManagerService libsqlite orqali so‘rovlar bajaradi. WebKit/Chromium bilan yasalgan browser yoki WebView komponenti esa C++ da yozilgan veb-dvigatelga suyanadi. Ilova dasturchilari ba’zi hisoblash yoki grafik ishlovlarni NDK yordamida C/C++ da yozishi mumkin. Masalan, 3D o‘yinlar OpenGL ES bilan ishlaydi; WebView browser tarkibi WebKit/Chromium rendering dvigatelida bajariladi. Yoki yuklab olinayotgan faylni saqlash jarayonida SQLite kutubxonasi ishlatiladi. Shu bilan birga, libbinder orqali yuqori qatlamlar Android sistem xizmatlari (system\_server) bilan muloqot qiladi: masalan, ilovalar va ramka xizmatlari o‘zaro xabar almashish uchun binder driverini qo‘llaydi.



### **Application Framework**

Application Framework – Androidning dastlabki versiyalarida (2008-yildan boshlab) Java tilida yozilgan Android Framework qatlamlari bo‘lib, ular ilovalarga bir qator xizmatlar va APIlar taqdim etadi. Android Inc. va keyinchalik Google

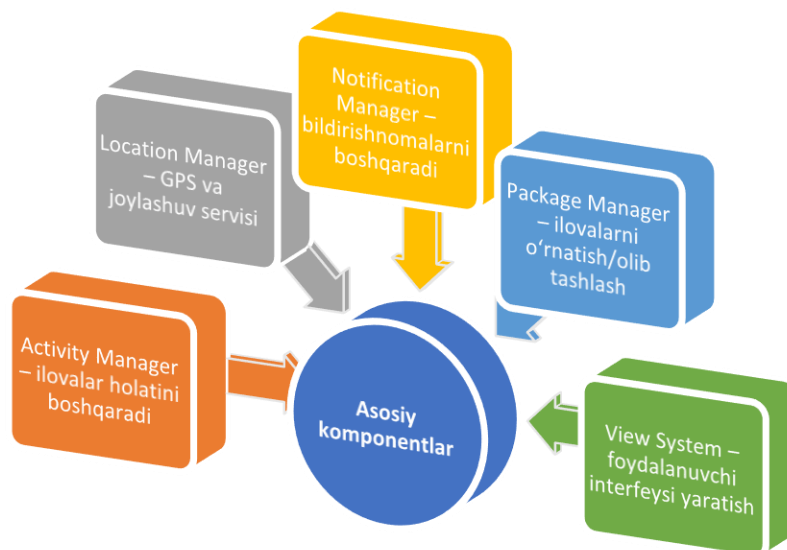
ishlab chiqqan ushbu ramkalar yaratilishi 2007–2008-yillarda, birinchi Android telefon taqdim etilganda boshlangan. Ilovalar ramkasi qatlamining asosiy vazifasi ilovalar va yuqori qatlam komponentlariga (Activity, Service va hokazo) tizimning past darajadagi funktsiyalariga qulay kirishni ta'minlovchi APIlar to'plamini taqdim etish. Ushbu qatlamda ishlab chiquvchiga qulay yuqori darajadagi dasturiy interfeyslar (APIs) to'plami mavjud bo'lib, ular qayta ishlangan modullar va xizmatlardan foydalanib, ilovalarni tezda yaratishga yordam beradi. Masalan:

- View System – Interfeys elementlarini (tugmalar, matn maydonlari, ro'yxatlar, grafikalar) ekranga chiqarish uchun ramka sinflari.
- Resource Manager (Resurs menejeri). Tillangan matnlar, grafikalar, tartib fayllari kabi resurcelarni boshqarish uchun xizmat.
- Notification Manager (Bildirishnoma menejeri). Har xil ilovalar uchun status panelida va bildirishnoma panelida ogohlantirishlar chiqarish imkonini beruvchi xizmat.
- Activity Manager (Activity menejeri). Ilovalar hayot tsiklini (aktiv, fon, to'xtatilgan holatlar) boshqaradi va Back Stack (orqa ketma-ketlik) ni nazorat qiladi.
- Content Provider (Kontent provayder). Ilovalarga o'z ma'lumotlarini boshqa ilovalar bilan bo'lishish yoki boshqa ilovalardan ma'lumot olish imkonini beruvchi modul.

Yuqoridagi ro'yxat Android Developer tomonidan bergan asosiy ramka komponentlari misolidir. Bular bilan bir qatorda Window Manager, Location Manager, Package Manager, Telephony Manager kabi boshqa menejer va xizmatlar ham mavjud bo'lib, ular turli funktsiyalarni bajaradi (masalan, ekran oynalarini boshqarish, geolokatsiya ma'lumotlarini olish, o'rnatilgan ilovalarni boshqarish va tarmoqqa ulanmagan va telefon qo'ng'iroqlarini boshqarish). Android ramkasi ichida chaqiriladigan ko'plab xizmatlar (masalan, ActivityManagerService,

WindowManagerService va boshqalar) system\_server nomli jarayon (daemon) ichida ishlaydi. Bu system services nomli modul tizim komponentlari bo'lib, ular quyi darajadagi apparat va tizim resurslariga kirishish uchun ramka API'lari bilan muloqot qiladi. Masalan, ilovada ActivityManager orqali ilovani ishga tushirish talab etilganida, bu chaqiruv system\_server ichidagi ActivityManagerService ga uzatiladi. *Android Framework* C/C++ darajasida bajaruvchi libandroid\_runtime.so, libdvm.so kabi kutubxonalar yordamida ishlaydi, lekin tashqi ko'rinishda Java sinflaridan tashkil topgan. Dasturchilar Java klasslari orqali kerakli funksiyalarni chaqiradi. Android API'lari orqali ilovalar ramkasi binder IPC orqali pastki qatlamlardagi xizmatlar bilan ham aloqa qilish imkoniga ega. Masalan, NotificationManager orqali bildirishnoma jo'natish ramka ichidagi binder so'rovi sifatida system\_serverdagi NotificationService ga yuboriladi. *Framework* qatlami masalasi sifatida, foydalanuvchi telefon ekranida yangi xabar kelganini ko'rsatuvchi bildirishnoma misol bo'la oladi. Dasturchi ilovasi NotificationManager API'sini chaqiradi; ramka bu so'rovni qabul qilib, ichki xizmatlarni ishga tushiradi va tizim panelida bildirishnoma chiqishini ta'minlaydi. Yoki, Content Provider orqali kontaktlarni o'qish uchun Contacts dasturini qo'zg'atadi va kerakli ma'lumotni oladi. Shu tarzda, ramka qatlamidagi APIlar foydalanuvchi ilovalariga qulay interfeyslar taqdim etadi va ularni tizimning quyi qatlamlari bilan bog'laydi. Android platformasi ko'plab C/C++ kutubxonalarini o'z ichiga oladi. Masalan, media fayllar bilan ishlash uchun Media kutubxonasi, grafik chizish uchun SurfaceFlinger va OpenGL ES bilan bog'liq kutubxonalar ishlatiladi. Ma'lumotlar bazasi uchun SQLite qo'llanadi, veb brauzer mexanizmi sifatida WebKit (keyinchalik Chromium) va internet xavfsizligi uchun SSL/TLS kutubxonalari bor. Shuningdek, Android grafik tasvirlar uchun Skia grafika dvigateli va shriftlar uchun FreeType kutubxonalarini ham o'z ichiga oladi. Tizimdagi asosiy komponentlarning aksariyati natviy kodda yozilgan bo'lib, masalan, SurfaceFlinger, AudioFlinger kabi

xizmatlar C++ da amalga oshiriladi. Tizim kamchiliklarini bartaraf etish uchun bu kutubxonalar Java APIlari orqali ilovalarga ochiladi (masalan, Android Java APIlari yordamida OpenGL ES chaqiriladi).



### Applications (Ilovalar)

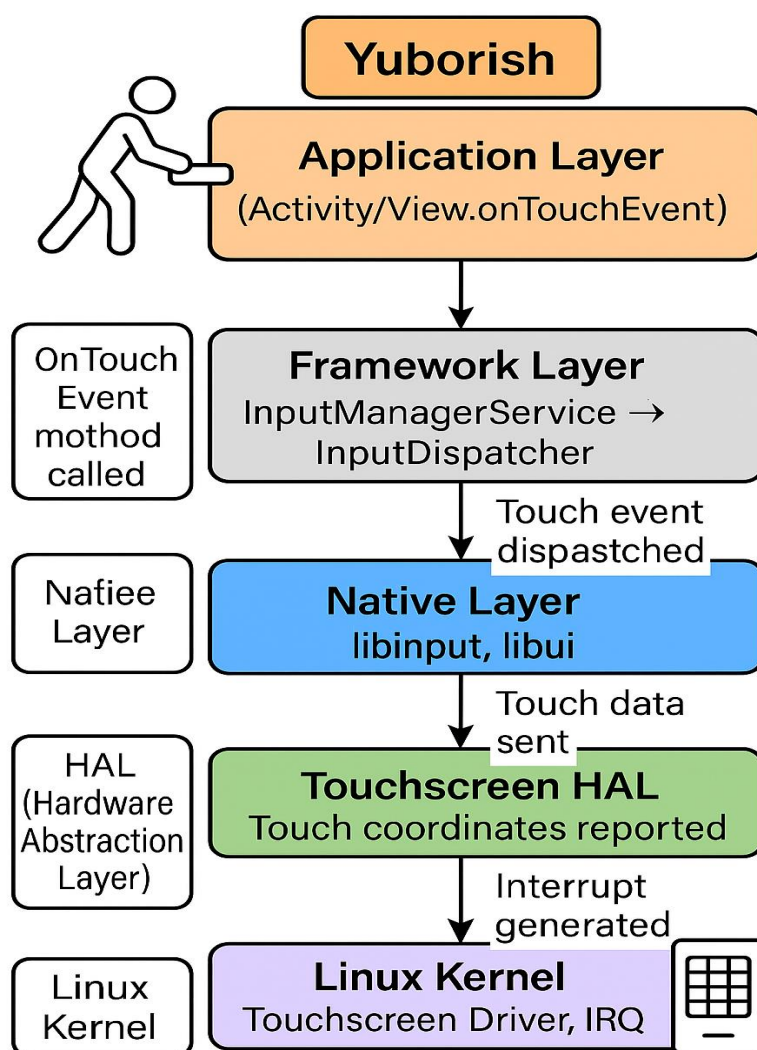
Applications (Ilovalar) – foydalanuvchiga ko‘rinarli va foydali dasturlar (masalan, suhbat, internet-brauzer, o‘yin, foto-video ilovalari) uchun muhit. Har bir Android ilovasi o‘z jarayonida (process) alohida ishlaydi va unga alohida foydalanuvchi identifikatori (UID) ajratiladi. Bu Linux yadrosi asosidagi *sandbox* xususiyati orqali ilovalarni bir-biridan izolyatsiya qilishni (himoya qilishni) ta‘minlaydi. Masalan, bir ilova boshqa ilovaning ma‘lumotlariga ruxsatsiz kira olmaydi yoki telefon funksiyalarini ruxsatsiz chaqira olmaydi. Shu tariqa, Android ilovalari tabiiy o‘zaro ajratilganlikda ishlaydi va tizim xavfsizligi kuchaytirilgan. Ilovalar odatda Java yoki Kotlin tilida yaratiladi va Android SDK yordamida to‘g‘ridan-to‘g‘ri *Android Runtime (ART)* ustida bajariladi. Har bir ilova *AndroidManifest.xml* fayliga ega bo‘lib, u yerda ilovaning tarkibiy qismlari (Activity, Service, ContentProvider,

BroadcastReceiver) va talab qilinadigan ruxsatlar (permissions) ko'rsatiladi. Android ilovalari quyidagi asosiy komponentlardan iborat bo'ladi.

- *Activity*: foydalanuvchi interfeysini tashkil etuvchi ekranning bir sahifasi (masalan, menyu yoki forma).
- *Service*: orqa fonda ma'lum vazifani bajarayotgan xizmat (masalan, musiqa ijro etish, yuklab olish).
- *Broadcast Receiver*: tizim yoki boshqa ilovalar yuborgan *broadcast* xabarlarini (masalan, batareya holati yoki tarmoqqa ulanish o'zgarishi) qabul qiladi.
- *Content Provider*: ilovalarga ma'lumotlarni (kontaktlar, fayllar, ma'lumotlar bazasi) saqlash va boshqa ilovalar bilan ulashish imkonini beruvchi komponent.

Ilovalar ramkasi (Application Framework) orqali taqdim etiladigan API'lar yordamida ilovalar ushbu komponentlarni boshqaradi. Masalan, Notification Manager yordamida status panelda bildirishnoma chiqarish, View System orqali ekranga UI komponentlarini joylashtirish mumkin. Dasturchi ilovalari kodi \*.java yoki \*.kt fayllardan iborat bo'lib, \*.apk paketiga yig'ilib, Android Runtime (ART) tomonidan .dex formatida ijro etiladi. Ilovalar to'g'ridan-to'g'ri quyi qatlamdagi apparat qurilmalariga kira olmaydi. Ilovalar istalgan yuqori darajadagi xususiyatdan foydalanish uchun ramka API'larini chaqiradi. Masalan, kamera bilan ishlovchi ilova Camera API ga murojaat qilib, ramka orqali tegishli HAL modulini ishga tushiradi. Telefon qo'ng'irog'i yoki SMS jo'natish funksiyasi uchun esa ilova Android-ning standart Telefon yoki SMS tizim ilovalarini chaqirishi mumkin; masalan, o'z ilovangizdan SMS jo'natmoqchi bo'lsangiz, alohida SMS jo'natish moduli yozishga hojat yo'q o'rniga tizimga o'rnatilgan SMS ilovasini chaqirish mumkin. Bu misolda ilova (Application qatlam) Intents orqali tizim ilovasini ishga

tushirib, SMS jo‘natadi. Oddiy real misol sifatida, foto-iltimos qiluvchi ilovalarni olaylik. Foydalanuvchi kamerani ochish uchun ilovani ishga tushiradi; bu ilova Camera API chaqiruvini yuboradi. Android ramkasi bu so‘rovni tegishli Camera HAL moduliga uzatadi, HAL esa Linux yadrosidagi camera drayveri orqali haqiqiy kamera sensoridan surat oladi. Olingan surat yana yuqoridan pastga – HAL dan Framework’ga, undan ilovaga qaytariladi va foydalanuvchiga ko‘rsatiladi. Shunday qilib, Ilovalar qatlami foydalanuvchi funksiyalarini taqdim etib, ortida Framework va pastki qatlamlar bilan o‘zaro aloqa qiladi.



## **Java virtual mashina**

Java Virtual Mashinasi (JVM) — bu abstrakt hisoblash mashinasi bo‘lib, u Java baytkodlarini bajaradi. JVM platformadan mustaqil bo‘lib, Java ilovalari “Write Once Run Anywhere” tamoyili bilan har qanday Java muhitida ishlaydi. Javadagi \*.java fayllar javac kompilyatori yordamida sinf (class) formatidagi baytkodga (\*.class) aylantiriladi. Keyin JVM sinf yuklovchi (Class Loader) orqali ushbu .class faylni yuklab oladi va bajara boshlaydi. Bajarish vaqtida Execution Engine bo‘limi baytkod yo‘riqnomalarini bajaradi. JVM dinamik tarzda ishlaydi: u baytkodni talqin qiluvchi (interpretor) yordamida qadam-baqadam bajarishi yoki muhim bo‘laklarni Just-In-Time (JIT) kompilyatori yordamida mashina kodiga o‘girishi mumkin. Masalan, HotSpot VM eng ko‘p ishlatiladigan metodlarni ish vaqtida profillab, ularni natijaviy protsessor buyruqlariga kompilyatsiya qiladi. Shu tariqa, takroriy ishlatiladigan kod uchun qayta talqin qilish o‘rniga tayyor natijaviy kod bajariladi, bu samaradorlikni oshiradi.

## **Dalvik virtual mashina**

Dalvik Virtual Machine (DVM) - bu Android ilovalari uchun taqdim etilgan maxsus dastur. U Java kodini oladi va Dalvik bajariladigan fayl sifatida tanilgan .dex (kengaytma) bilan faylda uning optimallashtirilgan versiyasini yaratadi. Bu format ilovalarga kamroq resurslar, ya’ni mobil telefonlar va xotirasi past, sekinroq qurilmalarda tez ishlashiga imkon beradi. Bu odatiy Java Virtual Mashinasidan ( JVM ) farq qiladi, chunki u Android uchun kamroq xotira ishlatadigan va turli versiyalarda ishlaydigan millionlab qurilmalarga mos keladigan ilovalarni ishga tushirish uchun optimallashtirilgan.



## Android ruxsatlari

Android ilovalari foydalanuvchining shaxsiy ma'lumotlariga yoki qurilmaning ayrim funksiyalariga kirish uchun *ruxsat* (*permission*) so'raydi. Bu foydalanuvchining maxfiylikini himoya qilishga yordam beradi.



Oddiy(Normal) ruxsatlar qurilmaning xavfsizligiga yoki foydalanuvchi maxfiyligiga katta xavf tug'dirmaydi.

| Permission                                  | Vazifasi                                             |
|---------------------------------------------|------------------------------------------------------|
| android.permission.INTERNET                 | internetga ulanish                                   |
| android.permission.ACCESS_NETWORK_STATE     | tarmoq holatini tekshirish                           |
| android.permission.BLUETOOTH                | Bluetoothdan foydalanish                             |
| android.permission.BLUETOOTH_ADMIN          | Bluetooth sozlamalarini boshqarish                   |
| android.permission.BLUETOOTH_CONNECT        | Bluetooth qurilmaga ulanish (Android 12+)            |
| android.permission.BLUETOOTH_SCAN           | Bluetooth qurilmalarni qidirish (Android 12+)        |
| android.permission.BLUETOOTH_ADVERTISE      | Bluetooth orqali reklamalarni yuborish (Android 12+) |
| android.permission.NFC                      | NFC (yaqin aloqa) qurilmasidan foydalanish           |
| android.permission.ACCESS_WIFI_STATE        | Wi-Fi holatini tekshirish                            |
| android.permission.CHANGE_WIFI_STATE        | Wi-Fi sozlamalarini o'zgartirish                     |
| android.permission.VIBRATE                  | Qurilmani titratish                                  |
| android.permission.WAKE_LOCK                | Qurilmaning uxlamasligini ta'minlash                 |
| android.permission.FOREGROUND_SERVICE       | Foreground servis ishga tushirish (Android 9+)       |
| android.permission.REQUEST_INSTALL_PACKAGES | APK fayl o'rnatish uchun ruxsat so'rash              |

|                                               |                                                   |
|-----------------------------------------------|---------------------------------------------------|
| android.permission.RECEIVE_BOOT_COMPLETED     | Qurilma yuklanganda ishga tushish                 |
| android.permission.ACCESS_NOTIFICATION_POLICY | Bildirishnoma sozlamalariga cheklangan kirish     |
| android.permission.USE_FULL_SCREEN_INTENT     | Xabarnomalarni to'liq ekranda ko'rsatish          |
| android.permission.SET_WALLPAPER              | Fonga rasm o'rnatish                              |
| android.permission.SET_WALLPAPER_HINTS        | Fonga rasm pozitsiyasini sozlash                  |
| android.permission.MODIFY_AUDIO_SETTINGS      | Ovoz sozlamalarini o'zgartirish                   |
| android.permission.BROADCAST_STICKY           | Sticky broadcast yuborish (deprecated)            |
| android.permission.CHANGE_NETWORK_STATE       | Tarmoq holatini o'zgartirish                      |
| android.permission.DISABLE_KEYGUARD           | Lock screenni vaqtinchalik o'chirish (deprecated) |
| android.permission.MOUNT_UNMOUNT_FILESYSTEMS  | Fayl tizimini mount/unmount qilish (deprecated)   |
| android.permission.READ_SYNC_SETTINGS         | Sync sozlamalarini o'qish                         |
| android.permission.WRITE_SYNC_SETTINGS        | Sync sozlamalarini o'zgartirish                   |
| android.permission.EXPAND_STATUS_BAR          | Status barni ochish yoki yopish                   |
| android.permission.GET_PACKAGE_SIZE           | Ilovaning o'lchamini aniqlash                     |

Bu ruxsatlar avtomatik beriladi, foydalanuvchi alohida tasdiqlamaydi.

Xavfli (Dangerous) ruxsatlar – Foydalanuvchi shaxsiy ma'lumotlariga yoki qurilma resurslariga kira oladi. Bu ruxsatlar foydalanuvchidan so'raladi.

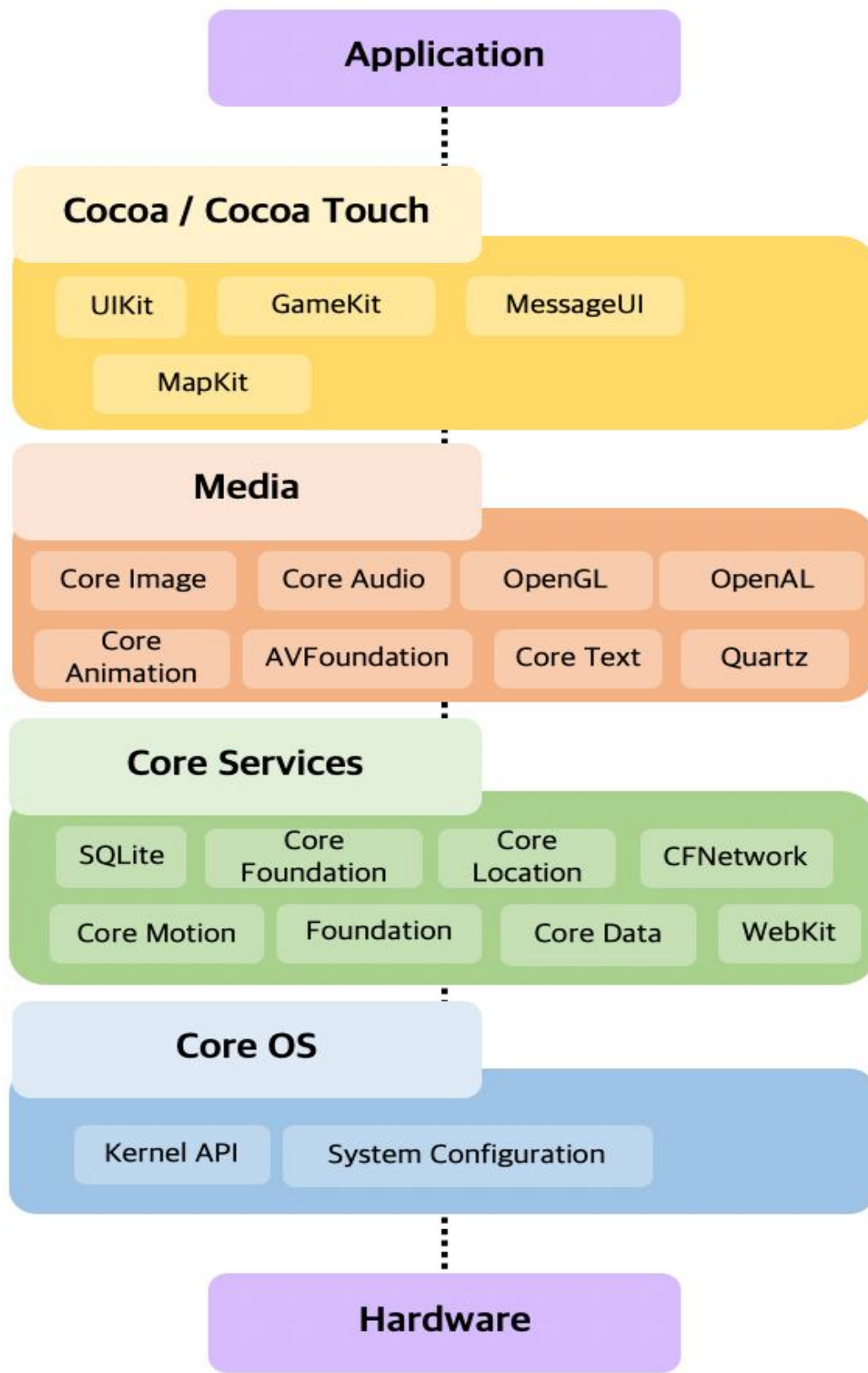
| Permission                                | Vazifasi                                |
|-------------------------------------------|-----------------------------------------|
| android.permission.RECORD_AUDIO           | Mikrofon orqali ovoz yozish             |
| android.permission.CAMERA                 | Kameradan foydalanish                   |
| android.permission.READ_CONTACTS          | Kontaktlarni o'qish                     |
| android.permission.WRITE_CONTACTS         | Kontaktlarga o'zgartirish kiritish      |
| android.permission.GET_ACCOUNTS           | Qurilmadagi akkauntlar ro'yxatini olish |
| android.permission.ACCESS_FINE_LOCATION   | Aniq GPS joylashuvini aniqlash          |
| android.permission.ACCESS_COARSE_LOCATION | Taxminiy joylashuvni olish              |

|                                            |                                                             |
|--------------------------------------------|-------------------------------------------------------------|
| android.permission.READ_CALENDAR           | Kalendar ma'lumotlarini o'qish                              |
| android.permission.WRITE_CALENDAR          | Kalendar ma'lumotlarini o'zgartirish va yozish              |
| android.permission.READ_EXTERNAL_STORAGE   | Tashqi xotiradagi fayllarni o'qish                          |
| android.permission.WRITE_EXTERNAL_STORAGE  | Tashqi xotiraga fayl yozish                                 |
| android.permission.MANAGE_EXTERNAL_STORAGE | Tashqi xotirani to'liq boshqarish (Android 11 va yuqoriroq) |
| android.permission.SEND_SMS                | SMS yuborish                                                |
| android.permission.RECEIVE_SMS             | SMS qabul qilish                                            |
| android.permission.READ_SMS                | SMS xabarlarini o'qish                                      |
| android.permission.RECEIVE_MMS             | MMS xabarlarini qabul qilish                                |
| android.permission.RECEIVE_WAP_PUSH        | WAP push xabarlarini qabul qilish                           |
| android.permission.CALL_PHONE              | Telefon orqali qo'ng'iroq qilish                            |
| android.permission.READ_PHONE_STATE        | Telefon holati va identifikatorlarini o'qish                |
| android.permission.USE_SIP                 | SIP protokoli orqali qo'ng'iroqlarni amalga oshirish        |
| android.permission.PROCESS_OUTGOING_CALLS  | Chiqayotgan qo'ng'iroqlarni kuzatish yoki o'zgartirish      |
| android.permission.ANSWER_PHONE_CALLS      | Kiruvchi qo'ng'iroqlarga javob berish (Android 8.0+)        |
| android.permission.WRITE_CALL_LOG          | Qo'ng'iroqlar tarixini tahrirlash                           |
| android.permission.ADD_VOICEMAIL           | Voicemail qo'shish                                          |
| android.permission.BODY_SENSORS            | Tana sensorlaridan (masalan, yurak urishi) foydalanish      |
| android.permission.ACTIVITY_RECOGNITION    | Foydalanuvchining harakatini (yurish, yugurish) aniqlash    |
| android.permission.READ_MEDIA_IMAGES       | Rasmlarga kirish (Android 13+)                              |
| android.permission.READ_MEDIA_VIDEO        | Videolarga kirish (Android 13+)                             |

|                                                       |                                                                    |
|-------------------------------------------------------|--------------------------------------------------------------------|
| android.permission.READ_MEDIA_AUDIO                   | Audiolarga kirish (Android 13+)                                    |
| android.permission.BIND_INPUT_METHOD                  | Keyboard (input method) servisini ulash                            |
| android.permission.BIND_NOTIFICATION_LISTENER_SERVICE | Xabarnomalarni o'qish va boshqarish servisi                        |
| android.permission.USE_BIOMETRIC                      | Biometrik autentifikatsiyadan foydalanish (barmoq izi, Face ID)    |
| android.permission.USE_FINGERPRINT                    | Barmoq izidan foydalanish (Android 9 va undan past)                |
| android.permission.REBOOT                             | Qurilmani qayta yuklash (root huquqi kerak)                        |
| android.permission.GET_TASKS                          | Foydalanuvchi ishlatayotgan ilovalar ro'yxatini olish (deprecated) |
| android.permission.REORDER_TASKS                      | Ilovalar tasklarini o'zgartirish (Activity Stack)                  |
| android.permission.RESTART_PACKAGES                   | Ilovalarni to'xtatish va qayta ishga tushirish (deprecated)        |
| android.permission.DEVICE_POWER                       | Qurilmani o'chirish yoki yoqish (faqat system apps uchun)          |

### Ios arxitekturası

IOS operatsion tizimining tuzilishi qatlamli (layered) asosda qurilgan. Unda to'g'ridan-to'g'ri aloqa sodir bo'lmaydi. Ilova qatlamidan (Application Layer) apparat qatlamigacha (Hardware Layer) bo'lgan qatlamlar o'zaro aloqa qilishda yordam beradi. Pastki darajadagi qatlamlar barcha ilovalar tayanuvchi asosiy xizmatlarni taqdim etsa, yuqori darajadagi qatlamlar grafik va foydalanuvchi interfeysi bilan bog'liq xizmatlarni taqdim etadi. iOS arxitekturası to'rt asosiy qatlamdan tashkil topgan: *Cocoa Touch*, *Media*, *Core Services* va *Core OS (Darwin)*. Har bir qatlam yuqoridagi qatlamlarga xizmat ko'rsatadi va o'ziga xos komponentlarga ega.



## **Cocoa Touch**

*Cocoa Touch* – bu foydalanuvchi ilovalar interfeysi qatlamidir. Bu qatlamda UIKit, Foundation, Core Animation va boshqa yuqori darajadagi ramkalar mavjud bo‘lib, ular mobil ilovalar uchun tugmalar, oynalar, xabarlar va sensorli kirishni boshqaradi. Cocoa Touch qatlamida Touch ID va Face ID biometrik autentifikatsiyasi ham amalga oshiriladi.

## **Media**

*Media* – audio va grafikani boshqarish qatlamidir. Bu yerga AVFoundation, Core Graphics, Core Animation, Metal/OpenGL, Core Audio kabi ramkalar kiradi. Masalan, video dekodlash va ko‘rsatish, grafik chizish, audio tahrirlash Media qatlamida amalga oshiriladi. Ushbu qatlam multimediya tarkiblarini o‘zgartirish (video, surat, o‘yin grafikasi) bilan bog‘liq xavfsizlik muammolariga duch kelishi mumkin (masalan, noto‘g‘ri faylni qayta ishlashda xatolar).

## **Core services**

*Core Services* – pastroq darajadagi umumiy xizmatlar qatlamidir. Bu yerda ma’lumotlar bazasi (SQLite, Core Data), tarmoq (CFNetwork), joylashuv (Location Services), Core Foundation, Grand Central Dispatch (GCD) va boshqa tizim xizmatlari mavjud. Shuningdek, Keychain, Data Protection kabi komponentlar ham Core Servicesda joylashgan. Bu qatlamda ma’lumotlar bazasi va tarmoq protokollari orqali ma’lumot almashinuvi, shifrlash va saqlash amalga oshiriladi. Zaifliklar esa ko‘pincha bu qatlamdagi ramkalarning xotira bog‘lanishlari yoki noto‘g‘ri konfiguratsiyalarda yuzaga keladi.

## **Core Os**

*Core OS (Darwin yadro)* – eng pastki, apparat va yadro qatlamidir. Bu Darwin (XNU) yadrosi, drayverlar, BSM audit, Apple Filing Protocol, CryptoKit, tizim

kutubxonalari va boshqa past darajadagi komponentlarni o‘z ichiga oladi. Bu qatlam protsessor (ARM) ustida ishlaydi va qurilmaning eng asosiy resurslarini boshqaradi. Tizimni tiklash va yangilashlar bu qatlam yordamida amalga oshiriladi.

### **iOS SDK va Xcode**

iOS SDK (Software Development Kit) bu Apple kompaniyasi tomonidan iOS operatsion tizimi uchun ilovalar yaratish uchun taqdim etilgan dasturiy vositalar to‘plamidir. U dasturchilarga iPhone, iPad va iPod touch qurilmalari uchun ilovalar ishlab chiqish imkonini beradi. SDK tarkibiga turli xil kutubxonalar, APIlar (Application Programming Interface), simulyatorlar va hujjatlar kiradi. iOS SDK odatda Xcode bilan birga ishlatiladi bu Applening rasmiy dasturlash muhiti bo‘lib, unda Swift yoki Objective-C tillarida kod yoziladi.

Xcode — bu Apple tomonidan ishlab chiqilgan rasmiy dasturlash muhiti (IDE — Integrated Development Environment) bo‘lib, iOS, macOS, watchOS va tvOS operatsion tizimlari uchun ilovalar yaratishda ishlatiladi. Xcode dasturchilarga Swift va Objective-C dasturlash tillarida kod yozish, xatoliklarni tuzatish (debug), interfeys dizayni yaratish (Interface Builder), test qilish va App Store’ga ilovani yuborish imkonini beradi. U o‘z ichiga iOS SDK’ni ham oladi, ya’ni Xcode orqali to‘liq iOS ilovalarini yaratish mumkin. Xcode faqat macOS operatsion tizimida ishlaydi.

### **iOS ruxsatlari**

Quyida iOS permission (ruxsatlar) larining asosiylari jadval ko‘rinishida keltirilgan. Har bir permission nomi, Info.plist dagi kaliti va u nima vazifani bajarishi tushuntirilgan. Bu ruxsatlar *Info.plist* faylida majburiy ko‘rsatilishi kerak. Aks holda ilova ruxsatni so‘ray olmaydi va to‘xtashi mumkin.

| <b>Permission nomi</b>            | <b>Kodda yozilishi</b>                                        | <b>Vazifasi</b>                                                    |
|-----------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------|
| Kamera (Camera)                   | NSCameraUsageDescription                                      | Kamera ga kirish uchun ruxsat so‘rash.                             |
| Mikrofon (Microphone)             | NSMicrophoneUsageDescription                                  | Mikrofon orqali audio yozish.                                      |
| Joylashuv (Location)              | NSLocationWhenInUseUsageDescription                           | Ilova ishlaganda joylashuvni aniqlash.                             |
| Joylashuv (Doimiy)                | NSLocationAlwaysUsageDescription                              | Ilova yopiq bo‘lsa ham joylashuvni kuzatish.                       |
| Kontaktlar (Contacts)             | NSContactsUsageDescription                                    | Kontaktlarga kirish, o‘qish/qo‘shish.                              |
| Suratlar (Photos)                 | NSPhotoLibraryUsageDescription                                | Suratlar kutubxonasiga kirish.                                     |
| Surat yozish                      | NSPhotoLibraryAddUsageDescription                             | Faqat surat kutubxonasiga yozish (o‘qimasdan).                     |
| Bluetooth                         | NSBluetoothAlwaysUsageDescription                             | Bluetooth qurilmalar bilan aloqa.                                  |
| Kamera rulosini saqlash           | NSAppleMusicUsageDescription                                  | Media kutubxonaga yozish (ko‘proq audio uchun).                    |
| Kalendarga kirish                 | NSCalendarsUsageDescription                                   | Kalendarga kirish, o‘qish va yozish.                               |
| Reminders (Eslatmalar)            | NSRemindersUsageDescription                                   | Eslatmalar ilovasiga kirish.                                       |
| Sog‘liq ma’lumotlari              | NSHealthShareUsageDescription, NSHealthUpdateUsageDescription | HealthKit orqali sog‘liq ma’lumotlarini o‘qish va yozish.          |
| Harakat va fitness                | NSMotionUsageDescription                                      | Qurilma harakati (accelerometer, gyroscope) haqida ma’lumot olish. |
| Foydalanuvchi kutubxonasi (Media) | NSMediaLibraryUsageDescription                                | Foydalanuvchi musiqa/media kutubxonasiga kirish.                   |
| Face ID / Touch ID                | NSFaceIDUsageDescription                                      | Biometrik autentifikatsiya ishlatish.                              |



## Mobil ilovalar pentestingi

---

### Android Studio va SDK

2013-yil 16-may kuni Google I/O konferensiyasida Katherine Chou tomonidan Apache 2.0 litsenziyasi asosida chiqarilgan Android Studio nomli integratsiyalashgan dasturlash muhiti (IDE) Android platformasi uchun ilovalar yaratish maqsadida taqdim etildi. Bu IDE 2014-yilda beta bosqichiga kirgan va 2014-yil dekabr oyida 1.0-versiyadan boshlab birinchi barqaror (stable) versiyasi chiqarilgan. 2015-yil 15-sentabrda Android Studio rasmiy IDE sifatida e'lon qilindi.

Android Studio va SDK (Software Development Kit) haqida batafsil ma'lumot olish uchun quyidagi havolani ko'ring:

👉 <http://developer.android.com/tools/studio/index.html#build-system>

Android Studio va SDK Java SE Development Kit (JDK) ga juda katta darajada bog'liq.

JDK ni quyidagi manzildan yuklab olish mumkin:

👉 <http://www.oracle.com/technetwork/java/javase/downloads/jdk7downloads-1880260.html>

Ba'zi dasturchilar boshqa IDElarni afzal ko'rishadi, masalan Eclipse va boshqalar. Ular uchun Google SDKning faqat o'zini yuklab olishni ta'minlaydi:

👉 [http://dl.google.com/android/installer\\_r24.4.1-windows.exe](http://dl.google.com/android/installer_r24.4.1-windows.exe)

Android Studio'ni samarali ishlatish uchun minimal tizim talablari mavjud. Quyidagi tartib Windows 10 Professional 64-bit operatsion tizimida Android

Studio'ni o'rnatish jarayonini ko'rsatadi. Bu tizim quyidagi texnik xususiyatlarga ega:

- 4 GB RAM
- Minimum 50 GB bo'sh xotira
- Java Development Kit 7 o'rnatilgan bo'lishi kerak

1. Bu IDE Linux, Windows va Mac OS X uchun mavjud. Android Studio'ni quyidagi manzildan yuklab olish mumkin:

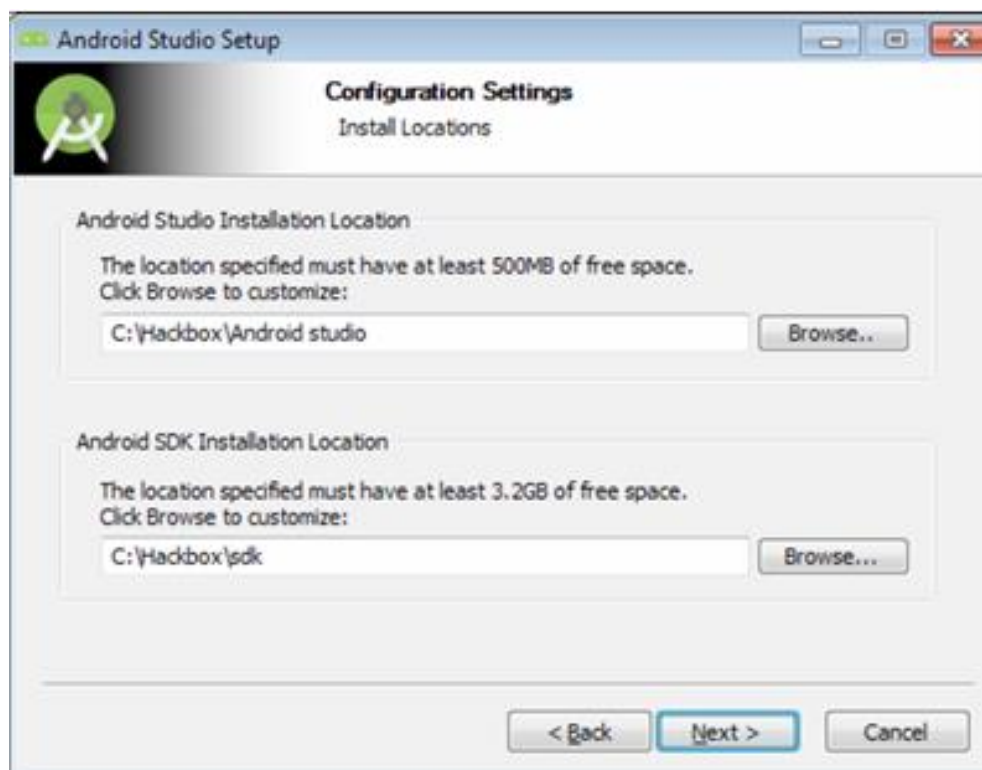
👉 <http://developer.android.com/sdk/index.html>

2. Android Studio yuklab olingach, o'rnatish faylini ishga tushiring. Odatda, quyidagidek o'rnatish oynasi ko'rinadi. Next (Keyingi) tugmasini bosing.



3. Ushbu o'rnatish jarayoni tizimning talablariga javob berishini avtomatik tarzda tekshiradi.

4. Zarur bo‘lgan barcha komponentlarni tanlang va “Next” (Keyingi) tugmasini bosing.
5. Litsenziya shartlarini o‘qib chiqish va qabul qilish tavsiya etiladi, keyin esa “Next” tugmasini bosing.
6. Barcha vositalarni yagona joyda kuzatib borish uchun yangi papka yaratish tavsiya etiladi; bu dalillarni (fayllarni) boshqarishni osonlashtiradi. Masalan, biz C diskida “Hackbox” nomli papka yaratdik bu quyidagi ekranda ko‘rsatilgan:



7. Endi Android tezlashtirilgan muhit (Android-accelerated environment) uchun kerakli joy ajratishimiz mumkin, bu esa tezroq ishlashni ta'minlaydi. Shuning uchun kamida 2 GB joy ajratish tavsiya etiladi.
8. Barcha zarur fayllar C:\Hackbox\ papkasiga ajratiladi (extract qilinadi).
9. O'rnatish tugagach, siz Android Studio'ni ishga tushira olasiz, bu quyidagi ekranda ko'rsatilgan.



## **Android SDK**

Android SDK dasturchilarga Android platformasida ishlaydigan ilovalarni to‘liq yaratish, testdan o‘tkazish va nosozliklarini tuzatish (debug qilish) imkonini beradi. Unda barcha kerakli dasturiy kutubxonalar (libraries), APIlar, emulyatorlar uchun tizim tasvirlari (system images), hujjatlar (documentation) va boshqa foydali vositalar mavjud bo‘lib, Android ilovasini yaratishda yordam beradi.

Biz Android Studioni Android SDK bilan birga o‘rnatdik, va endi iloji boricha SDK ichidagi vositalardan qanday foydalanishni tushunish juda muhim.

Ushbu bo‘limda biz Android ilovasiga hujum qilish (penetratsion test) vaqtida foydalaniladigan asosiy SDK vositalari haqida umumiy ma’lumot beramiz.

### **Emulator, simulatorlar, va real qurilmalar**

Emulator, simulator va real qurilma – dasturiy ta’minotni sinashda (test qilishda) yoki ishlab chiqishda foydalaniladigan muhim vositalar. Quyida ularning farqlari, o‘xshashliklari va har birining afzallik/kamchiliklari keltirilgan.

## **Emulatorlar**

Emulator bu kompyuterda ishlaydigan maxsus dastur bo‘lib, u haqiqiy qurilmaning (telefon, planshet, konsol va boshqalar) ish faoliyatini to‘liq taqlid qiladi. U nafaqat qurilmaning tashqi ko‘rinishini, balki ichki tizimini operatsion tizim, protsessor, xotira, ekran va boshqa apparat qismlarini ham o‘zida aks ettiradi. Dasturchilar yoki testchilar emulatorlardan foydalanib, ilovani haqiqiy qurilmasiz sinab ko‘rishlari mumkin. Masalan, Android dasturchilari Android Studiodagi emulyator orqali turli telefon modellarida ilovani test qiladilar Samsung, Pixel, Xiaomi va boshqalar.

## **Simulatorlar**

Simulator bu kompyuterda ishlaydigan dastur bo‘lib, u real qurilmaning tashqi ko‘rinishi va ishlash muhitini taqlid qiladi, lekin qurilmaning apparat (hardware) qismlarini aniq takrorlamaydi. U faqat qurilmaning xulq-atvori yoki funksional tomonini ko‘rsatadi. Masalan, iOS dasturchilari Xcodedagi iPhone Simulator yordamida iPhone ilovasini haqiqiy iPhonesiz sinab ko‘rishlari mumkin.

## **Real qurilmalar**

Real qurilma bu siz test qilayotgan yoki dastur o‘rnatayotgan haqiqiy fizik telefon, planshet yoki boshqa elektron qurilma. Masalan, siz Android ilova yaratgan bo‘lsangiz, uni Samsung yoki Xiaomi telefonida sinab ko‘rishingiz real qurilmada test hisoblanadi.

Bu test qilishning eng aniq va ishonchli yo‘li, chunki ilova foydalanuvchiga aynan shu qurilmalarda yetib boradi.

## Mobil Pentesting Vositalari

---

### Android xavfsizlik vositalari

Android xavfsizlik vositalari bu foydalanuvchilarning ma'lumotlarini himoya qilish va qurilmaning xavfsiz ishlashini ta'minlash uchun mo'ljallangan dasturiy va apparat vositalaridir. Ular orasida Google Play Protect (ilovalarni avtomatik tekshiradi), biometrik autentifikatsiya (barmoq izi, yuzni tanish), shifrlash (ma'lumotlarni maxfiy saqlash), Permission Manager (ilovalarning ruxsatlarini boshqarish), hamda SafetyNet va App Sandbox kabi tizim darajasidagi himoya vositalari mavjud. Bu vositalar yordamida Android qurilmalari zararli dasturlardan va foydalanuvchi ma'lumotlarining ruxsatsiz tarqalishidan himoyalanaadi.

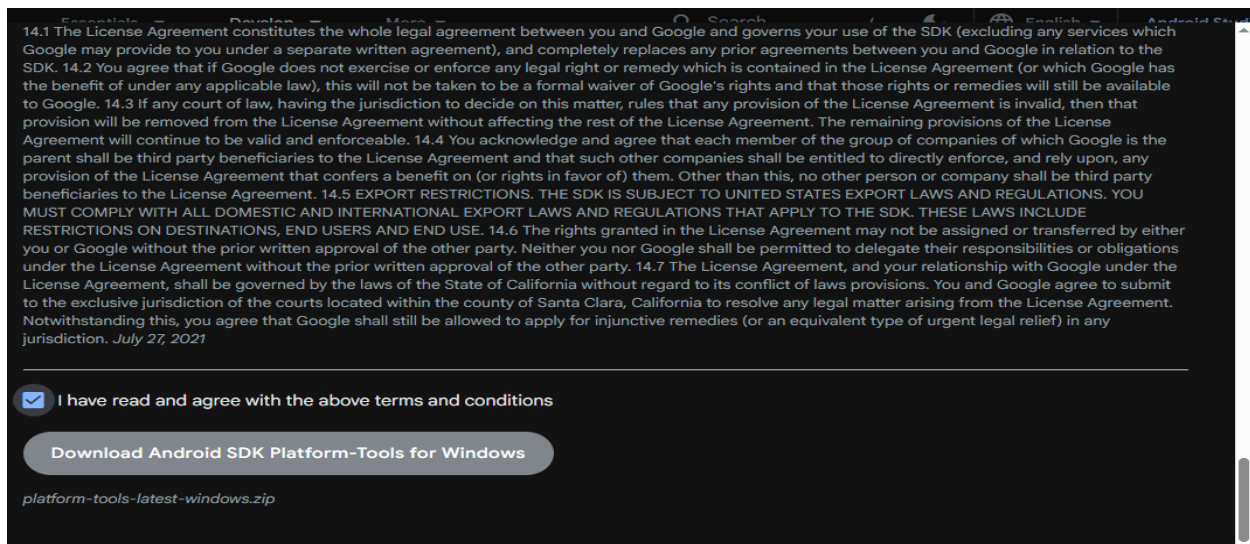
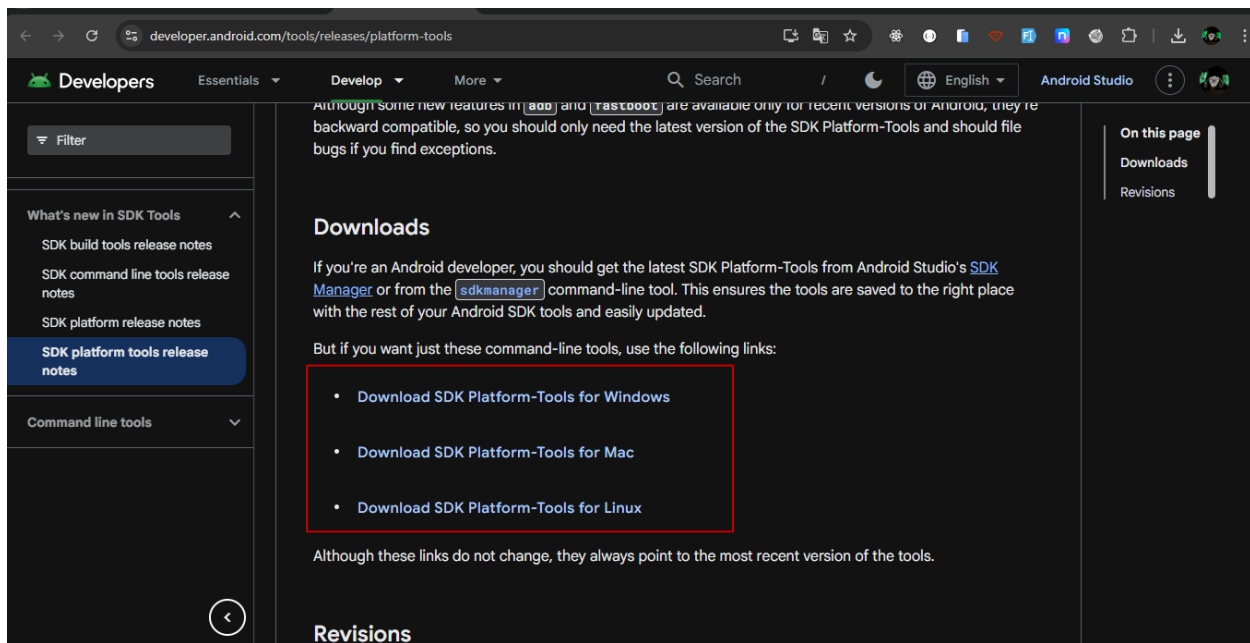
### Android Debug Bridge

ADB (Android Debug Bridge) — bu Android qurilmalarini kompyuter orqali boshqarish va ular bilan aloqa qilish imkonini beruvchi komanda satrli (CLI) vosita. U Android SDK (Software Development Kit) tarkibiga kiradi va ishlab chiquvchilar tomonidan keng qo'llaniladi.

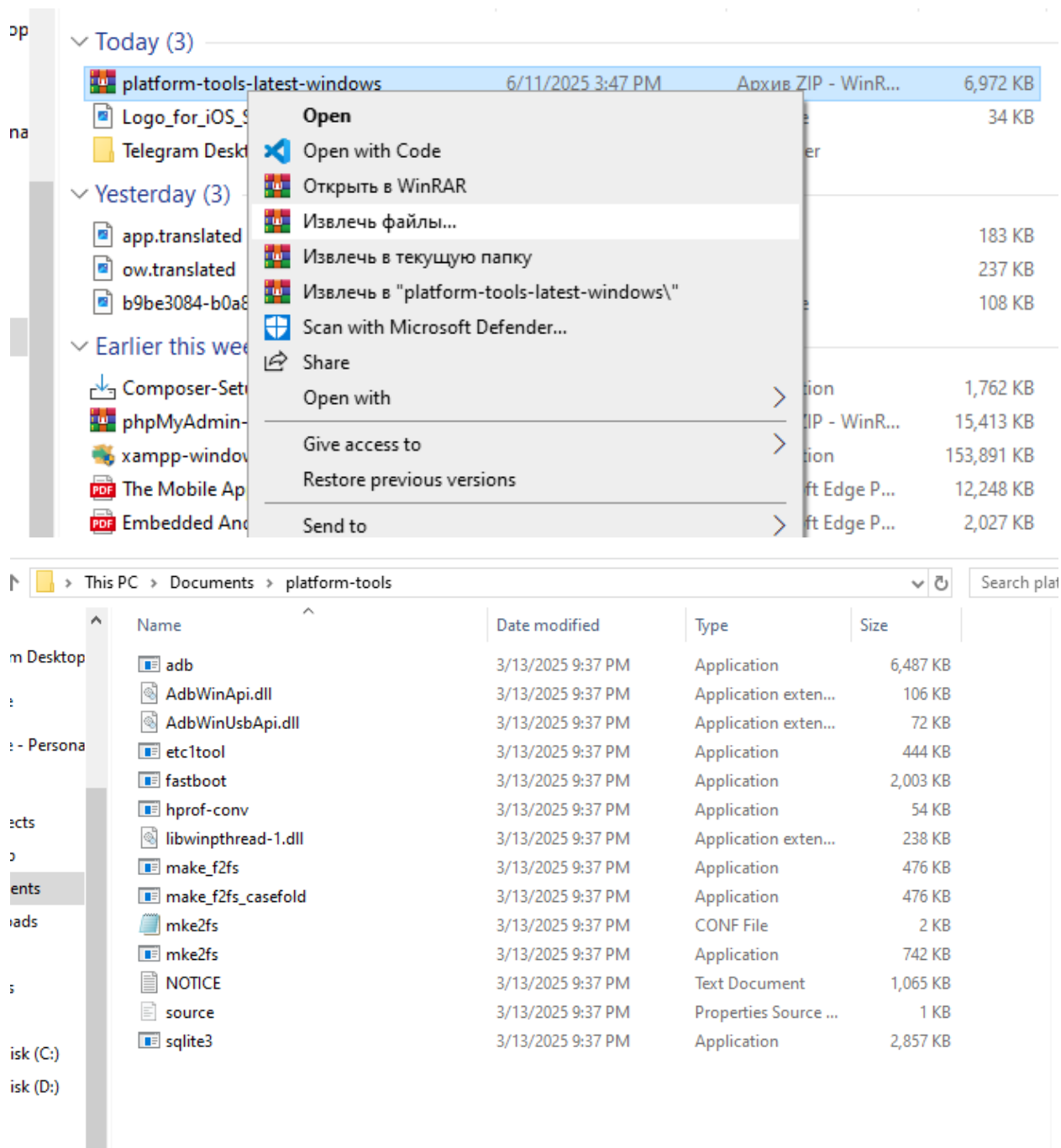
### Adbni o'rnatish

Windows operatsion tizimiga ADB (Android Debug Bridge) ni o'rnatish juda oddiy. Quyidagi bosqichlarni bajarsangiz kifoya. Quyidagi rasmiy saytdan adbni yuklab oling.

👉 <https://developer.android.com/tools/releases/platform-tools>

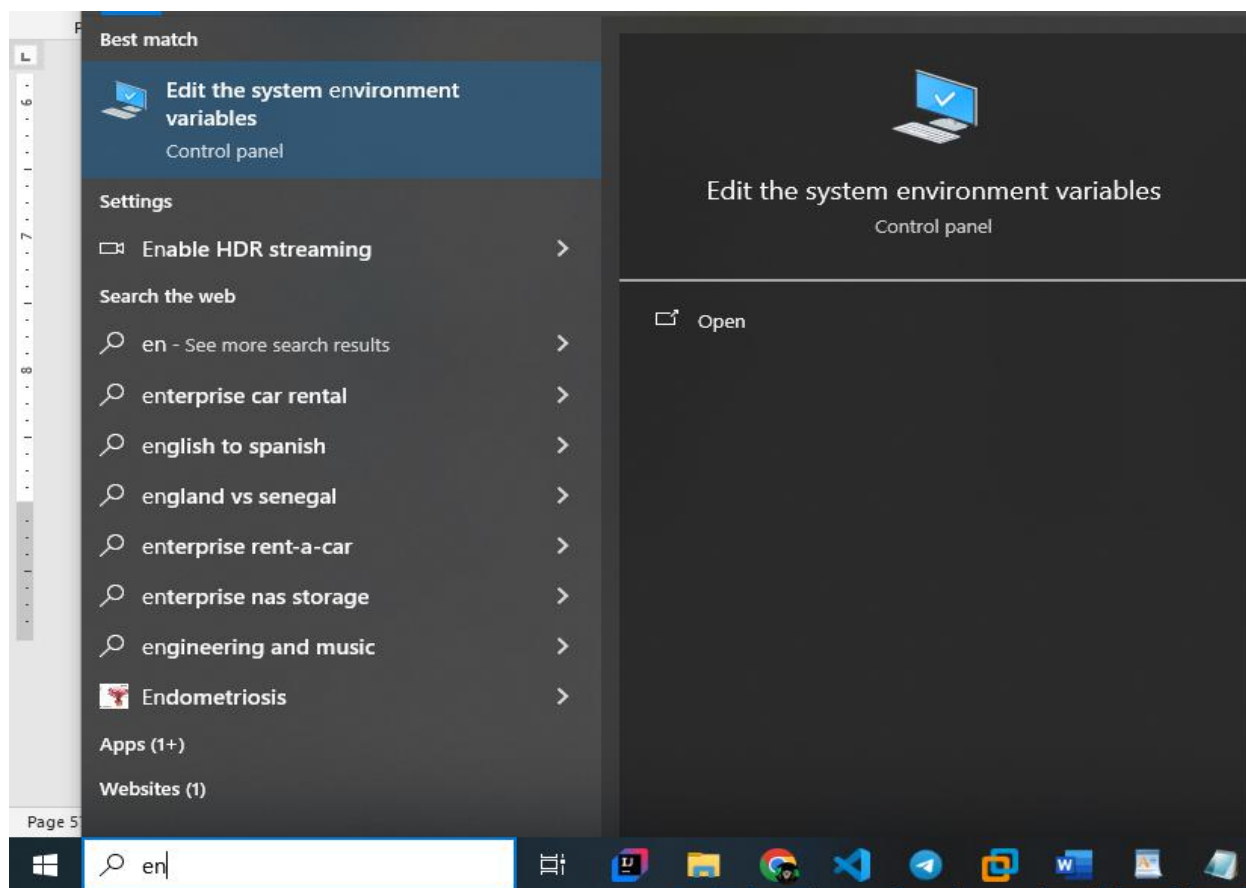


Yuklab bo‘lganingizdan keyin zip faylni rasmdagi kabi zipdan chiqarib oling.

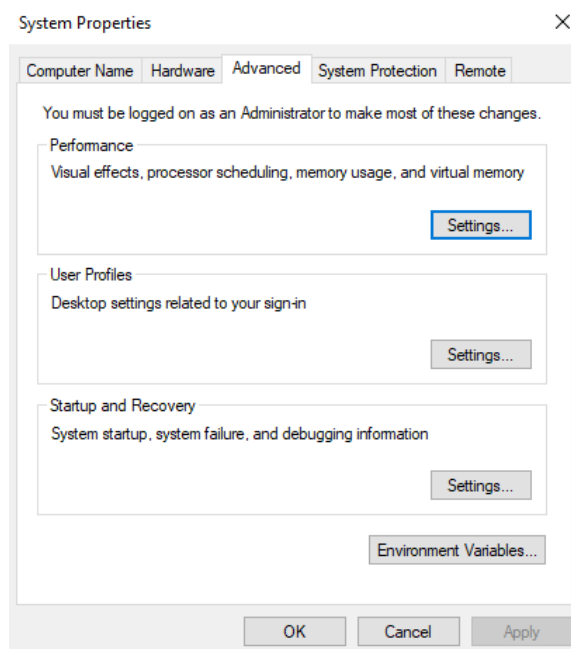


Keyin adb ni kompyuteringiz tanishi uchun uning turgan joyini pathlar ro'yhatiga qo'shib qo'yishingiz kerak bo'ladi.

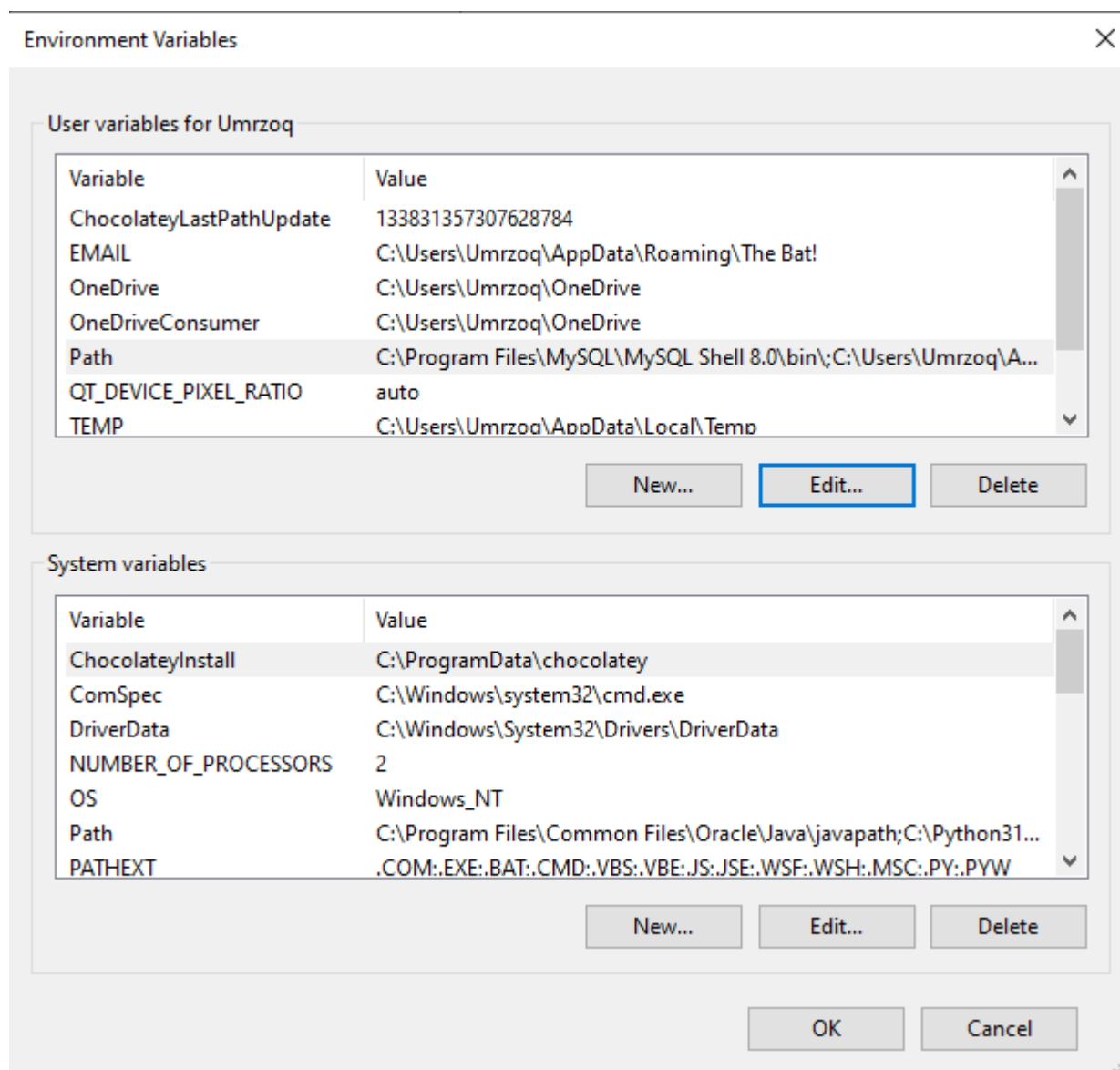




Kampyuteringiz qidiruv bo‘limiga shunday yozing va “*Edit the system environment variables*” ni ustiga bosing. Keyin sizda ham quydagicha oyna ochiladi.



Siz bu yerdan “*Environment Variables*” ni ustiga bosing va yana bir oyna ochiladi.



Shunday ochilgandan keyin siz path qismiga adb turgan joyni yozib qo'yishingiz kerak bo'ladi, va unga qo'shib bo'lganingizdan keyin barcha oynalarga OK ni bosib chiqib keting. Cmdni oching va rasmdagi kabi adb buyrug'ini yozib adb o'rnatilganini tekshirib oling agar rasmdagi kabi chiqsa kamyuteringizga adb dasturi o'rnatilgan bo'ladi va uning imkoniyatlaridan foydalanishingiz mumkin.

```
C:\Windows\system32>adb
Android Debug Bridge version 1.0.41
Version 36.0.0-13206524
Installed as C:\Users\Umrzoq\Documents\platform-tools\adb.exe
Running on Windows 10.0.19045

global options:
-a          listen on all network interfaces, not just localhost
-d          use USB device (error if multiple devices connected)
-e          use TCP/IP device (error if multiple TCP/IP devices available)
-s SERIAL   use device with given serial (overrides $ANDROID_SERIAL)
-t ID       use device with given transport id
-H          name of adb server host [default=localhost]
-P          port of adb server [default=5037]
-L SOCKET   listen on given socket for adb server [default=tcp:localhost]
```

### Qurilmaga ulanish

ADB orqali fizik qurilmaga ulanishingiz uchun USB-Debugging (USB nosozlikni tuzatish) opsiyasini yoqish muhimdir. Google Nexus 5 qurilmasida bu sozlamani quyidagicha topishingiz mumkin:

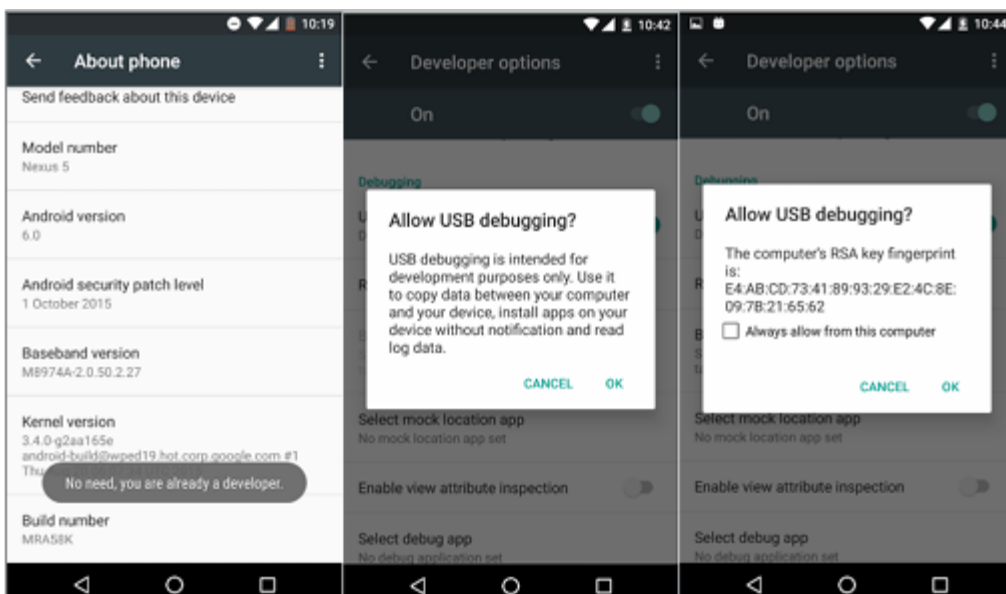
**Sozlamalar (Settings) → Dasturchi parametrlar (Developer options).** Quyidagi ekranda bu ko'rsatilgan:

### Test muhiti yaratish

Agar **Dasturchi parametrlar (Developer options)** menyusi ko'rinmasa, demak u yashiringan. Uni yoqish uchun quyidagi amallarni bajaring:

**Sozlamalar (Settings) → Qurilma haqida (About device) → Build number** bo'limiga kirib, **Build number** ustiga 7 marta ketma-ket bosing.

Shundan so'ng, **Dasturchi parametrlar** menyusi faollashadi va u orqali **USB-Debugging** ni yoqishingiz mumkin bo'ladi.



## Qurilmaga ilova o'rnatish

Birinchi navbatda siz qurilmangiz yoki emulator kampyuterga to'g'ri ulanganligini tekshirib olishingiz kerak bo'ladi buning uchun **adb devices** kamandasini terminalga yozing.

```

Administrator: Command Prompt

C:\Windows\system32>adb devices
List of devices attached
2601294910EA0DPP      device

C:\Windows\system32>_

```

Rasmdagi kabi sizda chiqqan bo'lsa qurilmangiz va kampyuteringiz ulangan bo'ladi. Qurilmaga to'g'ridan to'g'ri ulanish uchun siz terminalga rasmdagi kabi **adb shell** kommandasini yozib ulanishingiz mumkin bo'ladi.

```
Administrator: Command Prompt - adb shell

C:\Windows\system32>adb shell
RED8D1:/ $ whoami
shell
RED8D1:/ $
```

Agar siz qurilmangizga o'rnatilgan barcha ilovalar ro'yhatini ko'rmoqchi bo'lsangiz quydagicha kamandani terminalga yozing.

```
Administrator: Command Prompt

C:\Windows\system32>adb shell pm list packages -3
package:org.telegram.messenger
package:uz.aloqabank.zoomrad
package:uz.zirh.mobil_lib
package:com.google.android.contactkeys
package:uz.kapitalbank.android
package:com.google.android.safetycore

C:\Windows\system32>_
```

Bu yerda foydalanuvchi o'zi o'rnatgan ilovalarni package nemelar ro'yxati chiqadi

Agar siz ilova haqida batafsil ma'lumot (masalan, joylashuvi, versiyasi) ko'rmoqchi bo'lsangiz, bu buyruqdan foydalaning:

```
C:\Windows\system32>adb shell dumpsys package uz.aloqabank.zoomrad
Activity Resolver Table:
  Schemes:
    http:
      702567e uz.aloqabank.zoomrad/.activities.splash.MainActivity filter eb5d6f5
        Action: "android.intent.action.VIEW"
        Category: "android.intent.category.DEFAULT"
        Category: "android.intent.category.BROWSABLE"
        Scheme: "http"
        Authority: "staging-checkout.zoomrad.uz": -1
        Authority: "checkout.zoomrad.uz": -1
        Path: "PatternMatcher{GLOB: /pay/..*}"
        AutoVerify=true
      702567e uz.aloqabank.zoomrad/.activities.splash.MainActivity filter caa2fb
        Action: "android.intent.action.VIEW"
        Category: "android.intent.category.DEFAULT"
```

## Qurilmadan fayllarni olish

Agar siz qurilmadan fayl olishingiz kerak bo'lsa quydagicha kamandani yozasiz.

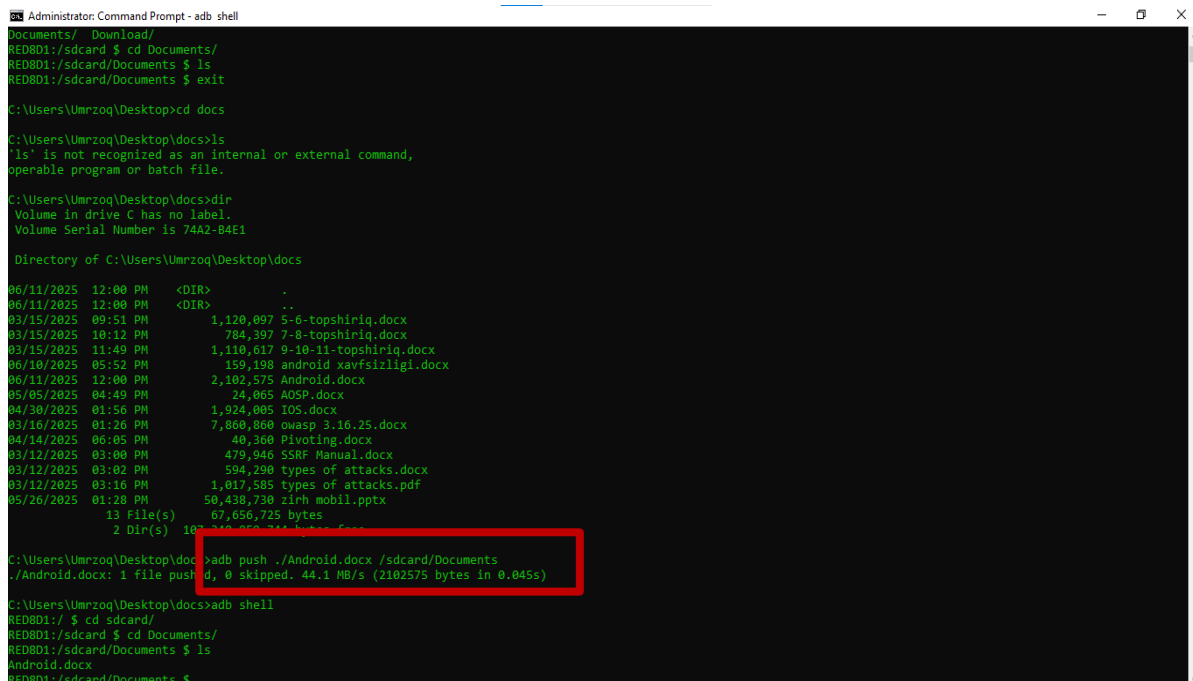
```
C:\Users\Umrzoq\Desktop>adb pull /sdcard/Pictures/Screenshots/Screenshot_20250517-082201.png .adb-photo.png
/sdcard/Pictures/Screenshots/Screenshot_20250517-082201.png: 1 file pulled, 0 skipped. 19.9 MB/s (874758 bytes in 0.042s)
C:\Users\Umrzoq\Desktop>
```

Bu rasmda biz qurilmadagi rasmni yuklab olibganmiz. Agar siz biror bir ilovani yuklab olmoqchi bo'sangiz ham xuddi shunday yuklab olishingiz mumkin bo'ladi buning uchun ilova turgan manzilni topishingiz kerak bo'ladi.

## Fayllarni qurilmaga saqlash

Agar siz kompyuteringizdagi biror faylni qurilmaga joylashtirimoqchi bo'lsangiz **adb push fayl\_manzili faylni\_qurilmaga\_joylashtirish\_manzili** shu kamanda orqali joylashtirishingiz mumkin bo'ladi.

```
adb push ./Android.docx /sdcard/Documents
```



```
Administrator: Command Prompt - adb shell
Documents/ Download/
RED8D1:/sdcard $ cd Documents/
RED8D1:/sdcard/Documents $ ls
RED8D1:/sdcard/Documents $ exit

C:\Users\Umrzoq\Desktop>cd docs
C:\Users\Umrzoq\Desktop\docs>ls
'ls' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Umrzoq\Desktop\docs>dir
Volume in drive C has no label.
Volume Serial Number is 74A2-B4E1

Directory of C:\Users\Umrzoq\Desktop\docs

06/11/2025  12:00 PM    <DIR>          .
06/11/2025  12:00 PM    <DIR>          ..
03/15/2025  09:51 PM             1,120,097 5-6-topshiriq.docx
03/15/2025  10:12 PM             784,397 7-8-topshiriq.docx
03/15/2025  11:49 PM             1,110,617 9-10-11-topshiriq.docx
06/10/2025  05:52 PM             159,198 android xavfsizligi.docx
06/11/2025  12:00 PM             2,102,575 Android.docx
05/05/2025  04:49 PM              24,065 AOSP.docx
04/30/2025  01:50 PM             1,924,085 IOS.docx
03/16/2025  01:26 PM             7,060,060 owasp 3.16.25.docx
04/14/2025  06:05 PM             40,360 Pivoting.docx
03/12/2025  03:00 PM             479,946 SSRF Manual.docx
03/12/2025  03:02 PM             594,290 types of attacks.docx
03/12/2025  03:16 PM             1,017,585 types of attacks.pdf
05/26/2025  01:28 PM             50,438,730 zirh mobil.pptx
               13 File(s)              67,656,725 bytes
               2 Dir(s) 10

C:\Users\Umrzoq\Desktop\docs>adb push ./Android.docx /sdcard/Documents
./Android.docx: 1 file pushed, 0 skipped. 44.1 MB/s (2102575 bytes in 0.045s)

C:\Users\Umrzoq\Desktop\docs>adb shell
RED8D1:/ $ cd /sdcard/
RED8D1:/sdcard $ cd Documents/
RED8D1:/sdcard/Documents $ ls
Android.docx
RED8D1:/sdcard/Documents $
```

## Log ma'lumotlarini ko'rish

Android qurilmaning log ma'lumotlari (logcat) — bu qurilmadagi ilovalar, tizim servislari va boshqa komponentlar tomonidan yozilayotgan xatoliklar, ogohlantirishlar va izohlar hisoblanadi.

Qurilmadagi barcha log yozuvlarini ko'rish uchun terminalga adb logcat buyrug'ini kiritish kifoya. Ushbu buyruq orqali tizimda sodir bo'layotgan barcha jarayonlar, xatoliklar va bildirishnomalar real vaqt rejimida aks etadi.

### adb logcat

```
C:\>adb logcat
----- beginning of crash
00-11 09:34:10.234 16546 16546 F DEBUG : #09 pc 00000000001a8a88 /apex/com.android.art/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, unsigned int*, unsigned int*, art::JValue*, char const*)+228) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #10 pc 000000000054aefc /apex/com.android.art/lib64/libart.so (art::InvokeMethod(art::ScopedObjectAccessAlreadyRunnable const&, _jobject*, _jobject*, _jobject*, unsigned long)+1364) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #11 pc 00000000004cbe0f /apex/com.android.art/lib64/libart.so (art::Method_invoke(_JNIEnv*, _jobject*, _jobject*, _jobjectArray*)+52) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #12 pc 0000000000809f4 /apex/com.android.art/lib64/libart.so (art::Method_invoke(_JNIEnv*, _jobject*, _jobject*, _jobjectArray*)+52) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #13 pc 0000000000809158 /system/framework/arm64/boot-framework.oat (com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run+136) (BuildId: 848064a4f9182aa7085414a5bfb7c9d992501975)
00-11 09:34:10.234 16546 16546 F DEBUG : #14 pc 00000000008a0888 /system/framework/arm64/boot-framework.oat (com.android.internal.os.ZygoteInit.main+2280) (BuildId: 848064a4f9182aa7085414a5bfb7c9d992501975)
00-11 09:34:10.234 16546 16546 F DEBUG : #15 pc 00000000001337e8 /apex/com.android.art/lib64/libart.so (art_quick_invoke_static_stub+568) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #16 pc 00000000001a8a88 /apex/com.android.art/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, unsigned int*, unsigned int*, art::JValue*, char const*)+228) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #17 pc 0000000000549930 /apex/com.android.art/lib64/libart.so (art::JValue art::InvokeWithVarArgs(art::ArtMethod*)+(art::ScopedObjectAccessAlreadyRunnable const&, _jobject*, art::ArtMethod*, std::va_list)+448) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #18 pc 0000000000549dec /apex/com.android.art/lib64/libart.so (art::JValue art::InvokeWithVarArgs(_jmethodID*)+(art::ScopedObjectAccessAlreadyRunnable const&, _jobject*, _jmethodID*, std::va_list)+92) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #19 pc 0000000000432700 /apex/com.android.art/lib64/libart.so (art::JNI<true>::callStaticVoidMethodV(_JNIEnv*, _jclass*, _jmethodID*, std::va_list)+636) (BuildId: 79991d5a5ba026c69376ba7be4e58e38)
00-11 09:34:10.234 16546 16546 F DEBUG : #20 pc 00000000009a428 /system/lib64/libandroid_runtime.so (_JNIEnv::CallStaticVoidMethod(_jclass*, _jmethodID*, ...)+124) (BuildId: 68f91d757c43020ba2b925b1698c1415)
00-11 09:34:10.234 16546 16546 F DEBUG : #21 pc 00000000000a1880 /system/lib64/libandroid_runtime.so (android::AndroidRuntime::start(char const*, android::Vector<android::String8> const&, bool)+832) (BuildId: 68f91d757c43020ba2b925b1698c1415)
00-11 09:34:10.234 16546 16546 F DEBUG : #22 pc 0000000000003574 /system/bin/app_process64 (main+1324) (BuildId: 6198d6633afe419e25cf331b1a39e03c)
00-11 09:34:10.234 16546 16546 F DEBUG : #23 pc 000000000004973c /apex/com.android.runtime/lib64/bionic/libc.so (__libc_init+108) (BuildId: ddedea5a1d071f97c5311841b6be985)
00-11 09:34:40.181 16555 16555 F libc : Fatal signal 11 (SIGSEGV), code 1 (SEGV_MAPERR), fault addr 0x0 in tid 16555 (com.android.nfc), pid 16555 (com.android.nfc)
00-11 09:34:40.379 16689 16689 F DEBUG : *** *** *** *** *** *** *** *** *** ***
00-11 09:34:40.379 16689 16689 F DEBUG : Native Crash TIME: 158410477
00-11 09:34:40.379 16689 16689 F DEBUG : *** *** *** *** *** *** *** *** *** ***
00-11 09:34:40.379 16689 16689 F DEBUG : Build fingerprint: 'realme/RMX3263R0/RED801:11/RP1A.201005.001/1691941785000:user/release-keys'
00-11 09:34:40.379 16689 16689 F DEBUG : Revision: '0'
00-11 09:34:40.380 16689 16689 F DEBUG : ABI: 'arm64'
00-11 09:34:40.380 16689 16689 F DEBUG : Timestamp: 2025-06-11 09:34:40+0500
00-11 09:34:40.380 16689 16689 F DEBUG : pid: 16555, tid: 16555, name: com.android.nfc >>> com.android.nfc <<<
00-11 09:34:40.380 16689 16689 F DEBUG : uid: 1027
```

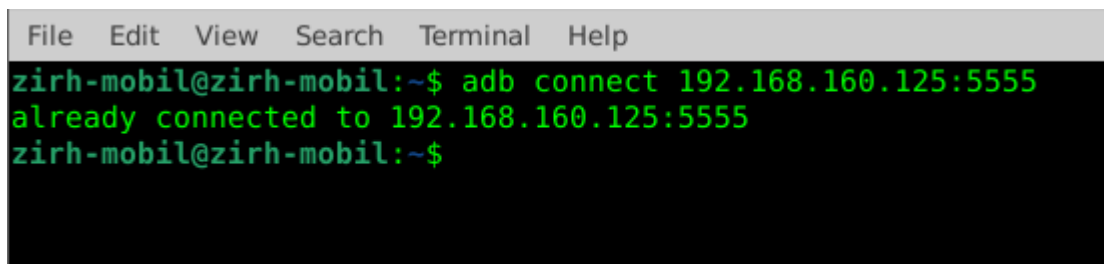
Agar siz faqatgina ma'lum bir ilovaga tegishli log yozuvlarini ko'rmoqchi bo'lsangiz, quyidagi buyruqdan foydalanishingiz mumkin.

### adb logcat | grep uz.anormobile.retail

```
zihr-mobil@zihr-mobil:~$ adb logcat | grep uz.anormobile.retail
06-22 16:19:45.427 479 479 V StorageManagerService: Package uz.anormobile.retail does not have legacy storage
06-22 16:24:40.057 479 551 I ActivityTaskManager: START u0 {act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] flg=0x10200000 cmp=uz.anormobile.retail/uz.anorbank.retail.ui.main.MainActivity bnds=[37,369][238,666]} from uid 10139
06-22 16:24:40.432 479 504 I ActivityManager: Start proc 5725:uz.anormobile.retail/u0a155 for pre-top-activity {uz.anormobile.retail/uz.anorbank.retail.ui.main.MainActivity}
06-22 16:24:47.245 479 1230 I ActivityTaskManager: START u0 {act=android.intent.action.MAIN cmp=uz.anormobile.retail/uz.anorbank.retail.ui.auth.AuthenticationActivity (has extras)} from uid 10155
06-22 16:24:47.406 479 1230 D ConnectivityService: requestNetwork for uid/pid:10155/5725 NetworkRequest [ TRACK DEFAULT id=85, [ Capabilities: INTERNET&NOT_RESTRICTED&TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ], android.os.BinderProxy@2c43afd)
06-22 16:24:48.760 479 1230 D ConnectivityService: requestNetwork for uid/pid:10155/5725 NetworkRequest [ TRACK DEFAULT id=86, [ Capabilities: INTERNET&NOT_RESTRICTED&TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ], android.os.BinderProxy@43463f2)
06-22 16:24:48.879 479 1230 W ActivityManager: Receiver with filter android.content.IntentFilter@45cfa92 already registered for pid 5725, callerPackage is uz.anormobile.retail
06-22 16:44:17.844 479 497 I ActivityManager: Killing 5725:uz.anormobile.retail/u0a155 (adj 905): remove task
06-22 16:44:17.972 479 1327 D ConnectivityService: ConnectivityService NetworkRequestInfo binderDied(NetworkRequest [ TRACK DEFAULT id=86, [ Capabilities: INTERNET&NOT_RESTRICTED&TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ], android.os.BinderProxy@2c43afd)
06-22 16:44:17.972 479 585 D ConnectivityService: releasing NetworkRequest [ TRACK DEFAULT id=86, [ Capabilities: INTERNET&NOT_RESTRICTED&TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ] (release request)
06-22 16:44:17.993 479 1325 D ConnectivityService: ConnectivityService NetworkRequestInfo binderDied(NetworkRequest [ TRACK DEFAULT id=85, [ Capabilities: INTERNET&NOT_RESTRICTED&TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ], android.os.BinderProxy@43463f2)
06-22 16:44:17.994 479 585 D ConnectivityService: releasing NetworkRequest [ TRACK DEFAULT id=85, [ Capabilities: INTERNET&NOT_RESTRICTED&TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ] (release request)
```

Agar siz emulatoreni asosiy operatsion tizimingizdan alohida muhitda, masalan, Windows tizimida VMware orqali ishga tushirilgan Ubuntu ichida foydalansangiz, adb buyruqlarini yuborishdan avval emulator bilan bog‘lanishingiz kerak bo‘ladi. Buning uchun birinchi navbatda qurilmaning IP manzilini aniqlab, so‘ngra quyidagi buyruq yordamida ulanishni amalga oshirasiz.

```
adb connect 192.168.160.125:5555
```

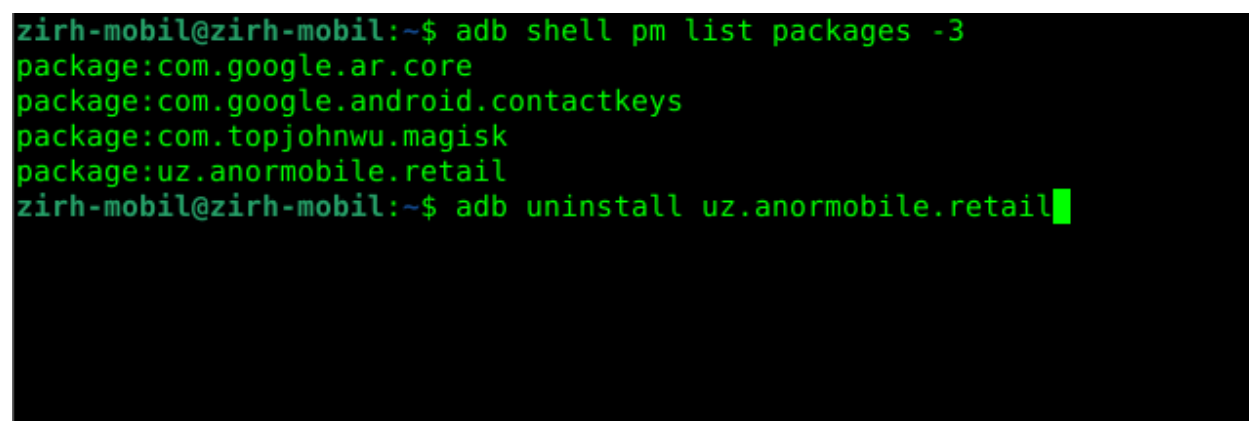


```
File Edit View Search Terminal Help
zirh-mobil@zirh-mobil:~$ adb connect 192.168.160.125:5555
already connected to 192.168.160.125:5555
zirh-mobil@zirh-mobil:~$
```

Ushbu amaldan so‘ng, siz virtual muhit orqali qurilmaga adb buyruqlarini bemaolol yuborishingiz mumkin bo‘ladi.

Agar siz qurilmadan ma’lum bir ilovani butunlay o‘chirib tashlamoqchi bo‘lsangiz, quyidagi buyruqdan foydalanishingiz mumkin.

```
adb shell pm list packages -3
```



```
zirh-mobil@zirh-mobil:~$ adb shell pm list packages -3
package:com.google.ar.core
package:com.google.android.contactkeys
package:com.topjohnwu.magisk
package:uz.anormobile.retail
zirh-mobil@zirh-mobil:~$ adb uninstall uz.anormobile.retail
```

Agar siz qurilmaga biror bir .apk faylini o‘rnatmoqchi bo‘lsangiz, quyidagi buyruq orqali bu jarayonni amalga oshirishingiz mumkin.



```
adb install Android01.apk
```

```
File Edit View Search Terminal Help
zirh-mobil@zirh-mobil:~/Downloads$ adb install Android01.apk
Performing Streamed Install
Success
zirh-mobil@zirh-mobil:~/Downloads$
```

Agar sizda ilova bir nechta .apk fayllardan iborat bo'lsa, masalan, base.apk va arxitekturaga mos qo'shimcha modullar (masalan, arm64\_v8a.apk, x86.apk va boshqalar) ko'rinishida bo'lsa, u holda bu ilovani quyidagi buyruq orqali to'g'ri tarzda o'rnatishingiz mumkin.

```
adb install-multiple base.apk split_config.arm64_v8a.apk split_config.xhdpi.apk
```

```
zirh-mobil@zirh-mobil:~/Downloads$ adb install-multiple base.apk split_config.arm64_v8a.apk split_config.xhdpi.apk
```

Agar siz qurilmada allaqachon o'rnatilgan ilovaning yangilangan (yoki yangi) versiyasini o'rnatmoqchi bo'lsangiz, ilovani butunlay o'chirib yubormasdan, quyidagi buyruq orqali uni qayta o'rnatishingiz mumkin.

```
adb install -r Android01.apk
```

```
app-release.apk.cachef base.apk base_x86.apk
zirh-mobil@zirh-mobil:~/Downloads$ adb install -r Android01.apk
Performing Streamed Install
```

Agar siz qurilmadagi mavjud ilovani ishga tushirmoqchi bo'lsangiz, terminalga quyidagi buyruqni kiriting va Enter tugmasini bosing.

```
adb shell monkey -p uz.anormobile.retail -v 1
```

```

zirh-mobil@zirh-mobil:~/Downloads$ adb shell monkey -p uz.anormobile.retail -v 1
  bash arg: -p
  bash arg: uz.anormobile.retail
  bash arg: -v
  bash arg: 1
args: [-p, uz.anormobile.retail, -v, 1]
arg: "-p"
arg: "uz.anormobile.retail"
arg: "-v"
arg: "1"
data="uz.anormobile.retail"
:Monkey: seed=1750641813933 count=1
:AllowPackage: uz.anormobile.retail
:IncludeCategory: android.intent.category.LAUNCHER
:IncludeCategory: android.intent.category.MONKEY
// Event percentages:
//   0: 15.0%
//   1: 10.0%

```

Agar siz qurilmada Google ilovasini ishga tushirib, unda biror matn yozmoqchi bo'lsangiz, bu jarayonni quyidagi ketma-ket buyruqlar orqali amalga oshirishingiz mumkin.

```
adb shell input text "Salom Google"
```

```

zirh-mobil@zirh-mobil:~/Downloads$ adb shell input text "sSalom-google"
zirh-mobil@zirh-mobil:~/Downloads$ █

```

## Adb buyruqlari

ADB (Android Debug Bridge) ning eng ko'p ishlatiladigan va foydali barcha asosiy buyruqlari (komandalar) kategoriyalar bo'yicha tartiblangan holda taqdim etilgan.

### 1. Qurilma bilan aloqa va holatini tekshirish

| Buyruq                  | Tavsifi                                       |
|-------------------------|-----------------------------------------------|
| adb devices             | Qurilmalar ro'yxatini ko'rsatadi              |
| adb get-serialno        | Qurilmaning serial raqamini chiqaradi         |
| adb get-state           | Qurilma holatini ko'rsatadi (device, offline) |
| adb connect <ip>:<port> | Wi-Fi orqali qurilmaga ulanadi                |
| adb disconnect          | Wi-Fi orqali ulanishni uzadi                  |

### 2. Ilova bilan ishlash

| Buyruq                | Tavsifi              |
|-----------------------|----------------------|
| adb install <app.apk> | APK faylni o'rnatadi |

|                                    |                                                              |
|------------------------------------|--------------------------------------------------------------|
| adb install -r <app.apk>           | APK'ni mavjud versiyasini ustiga qayta o'rnatadi (reinstall) |
| adb uninstall <package>            | Ilovani o'chiradi                                            |
| adb shell pm list packages         | Ilovalar (package) ro'yxati                                  |
| adb shell pm path <package>        | Ilova APK joylashuvini ko'rsatadi                            |
| adb shell monkey -p <package> -v 1 | Ilovani ishga tushiradi (test uchun)                         |

### 3. Fayl uzatish

| Buyruq                    | Tavsifi                                   |
|---------------------------|-------------------------------------------|
| adb push <local> <remote> | Faylni kompyuterdan qurilmaga yuboradi    |
| adb pull <remote> <local> | Faylni qurilmadan kompyuterga yuklaydi    |
| adb shell ls <path>       | Qurilmadagi fayl va papkalarni ko'rsatadi |

### 4. Qurilma nazorati

| Buyruq                | Tavsifi                                     |
|-----------------------|---------------------------------------------|
| adb reboot            | Qurilmani qayta ishga tushiradi             |
| adb reboot bootloader | Qurilmani bootloader holatiga o'tkazadi     |
| adb reboot recovery   | Qurilmani recovery rejimiga o'tkazadi       |
| adb root              | Qurilmani root huquqi bilan ishga tushiradi |
| adb shell             | Qurilmada terminal sessiyasini ochadi       |

### 5. Tizim va loglar bilan ishlash

| Buyruq                  | Tavsifi                                     |
|-------------------------|---------------------------------------------|
| adb logcat              | Qurilmadagi loglarni ko'rsatadi (real-time) |
| adb logcat -d > log.txt | Loglarni faylga yozadi                      |

|                                              |                                   |
|----------------------------------------------|-----------------------------------|
| adb shell dumpsys                            | Tizim servislarining ma'lumotlari |
| adb shell top                                | CPU ishlash statistikasi          |
| adb shell screencap /sdcard/screen.png       | Ekran rasmi olish                 |
| adb shell screenrecord<br>/sdcard/record.mp4 | Ekran yozuvi qilish               |

## 6. Port forwarding / Networking

| Buyruq                       | Tavsifi                           |
|------------------------------|-----------------------------------|
| adb forward <local> <remote> | Port forwarding qiladi            |
| adb reverse <remote> <local> | Qurilmadan kompyuterga port ulash |
| adb shell netstat            | Tarmoq ulanishlarini ko'rsatadi   |

## 7. Foydalanuvchi/ma'lumotlar

| Buyruq                                        | Tavsifi                                                                |
|-----------------------------------------------|------------------------------------------------------------------------|
| adb shell settings list system                | Tizim sozlamalarini ko'rsatadi                                         |
| adb shell am start -n<br><package>/<activity> | Belgilangan activity ni ishga tushiradi                                |
| adb shell input text "Hello"                  | Qurilmaga matn yuboradi                                                |
| adb shell input keyevent 26                   | Qurilmaning tugmalarini emulyatsiya qiladi<br>(masalan, Power tugmasi) |

## Developer va debugging uchun

| Buyruq            | Tavsifi                                             |
|-------------------|-----------------------------------------------------|
| adb bugreport     | Qurilma xatoliklari haqida batafsil hisobot         |
| adb shell getprop | Qurilma xususiyatlari (build info, model va h.k.)   |
| adb tcpip 5555    | Qurilmani Wi-Fi orqali ulanadigan holatga o'tkazadi |

## **APKAnalyser**

ApkAnalyzer — bu Android ilovalarini (APK fayllarini) tahlil qilish uchun mo'ljallangan vosita bo'lib, u Android Studio ichida mavjud bo'lgan grafik interfeysli qulay vositadir. ApkAnalyzer yordamida APK faylning tuzilmasini, resurslarini, kodlarini, ruxsatlarini, foydalangan kutubxonalarini va boshqa ko'plab muhim jihatlarni chuqur tahlil qilish mumkin. Bu vosita ilovaning hajmini optimallashtirish, xavfsizlikni baholash va muhim texnik tafsilotlarni o'rganish uchun foydalidir.

ApkAnalyzer'ning asosiy imkoniyatlari quyidagilardan iborat: APK fayl tuzilmasini ko'rish va undagi barcha papka va fayllarni analiz qilish mumkin. Masalan, `classes.dex`, `res/`, `lib/`, `AndroidManifest.xml` va boshqalar. Aynan `classes.dex` fayli ichida ilovaning kompilyatsiyalangan bytecode'lari saqlanadi, ya'ni bu ilovaning logikasini tushunishga yordam beradi. `AndroidManifest.xml` fayli orqali ilova qanday ruxsatlar (permissions), faoliyatlar (activities), xizmatlar (services) va intent-filtrlar ishlatganini aniqlash mumkin. Bu esa xavfsizlik tahlilida juda muhim.

Shuningdek, ApkAnalyzer ilovaning umumiy hajmini tahlil qilib beradi. Har bir modul, kutubxona yoki resurs qancha joy egallayotganini grafik ko'rinishda ko'rsatadi. Bu esa ilovani optimallashtirish uchun juda qulay. Undan tashqari, ilovada ishlatilgan tashqi kutubxonalarni ko'rish, ular versiyalarini aniqlash va ilova qaysi SDKlar yoki ishlab chiquvchi vositalar bilan yig'ilganini bilish imkonini beradi.

ApkAnalyzer yordamida ilovadagi dex fayllarni ko'rib chiqib, unda nechta klass, metod va maydonlar borligini aniqlash mumkin. Bu esa ilovaning murakkablik darajasini baholashga yordam beradi. Ayrim hollarda, ilovada ortiqcha yoki foydalanilmayotgan resurslar mavjud bo'lishi mumkin — ApkAnalyzer bularni aniqlashda ham qo'l keladi.

ApkAnalyzerni ishlatish uchun Android Studio'ni ochib, Build menyusidan Analyze APK bo'limini tanlash kerak bo'ladi. Ochilgan oynada kerakli .apk faylni tanlaysiz va ilova tahliliga kirishasiz. Bu vosita Android Studio bilan birga o'rnatiladi, alohida yuklab olishga ehtiyoj yo'q.

Xulosa qilib aytganda, ApkAnalyzer bu — Android ilovalari ustida ishlayotgan dasturchilar va xavfsizlik tahlilchilari uchun muhim vosita bo'lib, APK faylni dekomplyatsiya qilmasdan turib, muhim ma'lumotlarni tahlil qilish imkonini beradi. Ayniqsa ilovaning ruxsatlari, resurslari va hajmiga oid chuqur tahlillarni samarali amalga oshirish uchun juda qulay. APKAnalyser – bu Sony Mobile Communications tomonidan ishlab chiqilgan va u ochiq manba sifatida quyidagi [havo](#)da mavjud.

### **APKTool**

APKTool — bu Android ilovalarini (APK fayllarini) teskari muhandislik qilish (reverse engineering) uchun ishlatiladigan kuchli va mashhur konsol (komanda qatori) vositasidir. U orqali APK faylni dekomplyatsiya qilish, ya'ni ilova ichidagi resurslar, AndroidManifest.xml, smali kodlar va boshqa muhim tarkibiy qismlarni ochish mumkin. Bu vosita Android ilovalari xavfsizligini tahlil qilishda, modifikatsiya qilishda yoki resurslarni o'rganishda keng qo'llaniladi.

APKTool vositasi yordamida APK faylni ikki asosiy bosqichda tahlil qilish mumkin: dekomplyatsiya (ya'ni .apk faylni ochish) va qayta yig'ish (recompile). Dekomplyatsiya jarayonida APKTool ilovani alohida papkalarga ajratadi: bunda resurslar (masalan, res/, assets/, drawable/), AndroidManifest.xml, va eng muhimi, ilovaning asosiy logikasi yozilgan .smali fayllar olinadi. .smali fayllar bu — Androidning dex bytecodeining o'qiladigan shaklidir. Bu orqali dasturchi Java kodining qanday ishlayotganini tushunishi mumkin bo'ladi.

APKTool yordamida tahlil qilinayotgan ilova ichidagi matnlar, interfeys elementlari, ruxsatlar, ekranlar (activities), xizmatlar (services) va boshqa ko‘plab konfiguratsiyalarni ko‘rish mumkin. Ayniqsa AndroidManifest.xml fayl to‘liq o‘qiladigan formatda olinadi, bu esa ilovaning xavfsizlik modelini, foydalanuvchi ruxsatlarini va tashqi qurilmalar bilan aloqasini tushunishga yordam beradi.

Ilova tahlil qilinganidan so‘ng, agar kerak bo‘lsa, .smali fayllarda yoki resurslarda o‘zgartirishlar kiritib, APKTool yordamida ilovani qaytadan yig‘ish (recompile) mumkin. Bu qayta yig‘ilgan faylni imzolash orqali yana Android qurilmaga o‘rnatish mumkin bo‘ladi. Shu jihat bilan APKTool, masalan, ilovani test maqsadida modifikatsiya qilish yoki xavfsizlik testlarini o‘tkazishda juda foydalidir. APKToolni ishlatish uchun komanda satrida quyidagi kabi buyruqlardan foydalaniladi.

```
apktool d app.apk
```

Bu ilovani dekompiletsiya qiladi.

```
apktool b app_folder -o modifed.apk
```

Bu kod teskari muhandislik qilingan papkani yana APK faylga yig‘adi.

APKTool Javada yozilgan bo‘lib, kross-platformalidir — ya’ni Windows, Linux va macOS tizimlarida ishlaydi. Uni rasmiy sayt yoki GitHub sahifasi orqali yuklab olish mumkin. Ko‘pincha APKTool dan boshqa vositalar bilan birga, masalan keytool, jarsigner, uber-apk-signer, Frida, yoki Dex2Jar kabi vositalar bilan birgalikda foydalaniladi.

Xulosa qilib aytganda, APKTool — bu Android APK fayllarini tahlil qilish, o‘zgartirish va qayta yig‘ish imkonini beruvchi kuchli teskari muhandislik vositasidir. Bu vosita yordamida dasturchilar va xavfsizlik tahlilchilari ilovaning tuzilmasi, ishlash mexanizmi va zaif joylarini chuqur o‘rganish imkoniga ega bo‘ladilar.

## Androguard

Androguard — bu Android ilovalari (APK fayllari) ustida teskari muhandislik va xavfsizlik tahlilini avtomatlashtirilgan tarzda amalga oshirish imkonini beruvchi kuchli va moslashuvchan Python kutubxonasi va vositalar to‘plamidir. U yordamida APK, DEX va Java class fayllarini o‘qish, tahlil qilish, va ularning ichki tuzilmasini chuqur o‘rganish mumkin. Androguard ayniqsa avtomatik va statik tahlil (ya’ni ilovani ishga tushirmasdan turib kodni o‘rganish) uchun ishlatiladi va xavfsizlik tadqiqotchilari, mobil dastur tahlilchilari orasida keng tarqalgan.

Androguard vositasi yordamida APK faylni ochish va u tarkibidagi .dex fayllarni tahlil qilish mumkin. DEX fayllar Android ilovaning asosiy logikasini o‘z ichiga oladi va ularni Androguard vositasi smali emas, balki Python orqali to‘g‘ridan-to‘g‘ri o‘qiladigan obyektlarga aylantirib beradi. Bu esa tahlilchini katta hajmdagi kodlar orasida avtomatik tarzda qidiruv, filtr, analiz ishlari olib borishga imkon yaratadi. Misol uchun, siz Androguard yordamida ilova qaysi ruxsatlar (permissions) so‘rayotganini, qaysi API chaqiruvlari ishlatilayotganini, qanday ma’lumotlar tarmoqqa yuborilayotganini yoki qanday kutubxonalar ishlatilayotganini aniqlay olasiz. Androguard asosan uch asosiy modulga ega.

- **androxml** — AndroidManifest.xml faylni tahlil qilish va undan kerakli ma’lumotlarni ajratib olish uchun ishlatiladi.
- **androguard.core.bytecodes** — DEX faylni o‘qish va uni Python obyektlariga aylantirish vazifasini bajaradi.
- **androguard.misc** — umumiy foydali funksiyalar, masalan, APK faylni yuklash va analiz qilishni soddalashtiruvchi interfeyslar taqdim etadi.

Androguarddan foydalanish uchun dasturchi Python tilida skript yozadi yoki bu vositaning interaktiv komanda satri interfeysidan foydalanadi. Misol uchun, quyidagicha ishlatilishi mumkin:



```
from androguard.misc import AnalyzeAPK

apk_path = "test.apk"
a, d, dx = AnalyzeAPK(apk_path)

print(a.get_permissions())
print(a.get_activities())
print(dx.get_classes())
```

Bu kod orqali siz APK fayldagi barcha ruxsatlar, faoliyatlar (activities) va klasslar ro'yxatini ko'ra olasiz. Bundan tashqari, Androguard yordamida ilovada noto'g'ri ishlatilgan kriptografik funksiyalar, xavfsiz bo'lmagan URL manzillar yoki ilova ichida joylashgan maxfiy ma'lumotlarni (token, parol, API key) avtomatik tarzda aniqlash mumkin.

Androguard ko'pincha boshqa vositalar bilan birgalikda ishlatiladi. Masalan, siz apktool bilan APK faylni dekomplyatsiya qilib, so'ng Androguard bilan kodni analiz qilishingiz mumkin. Bundan tashqari, Androguardni o'z skriptlaringizga qo'shib, yuzlab ilovalarni avtomatik tarzda tahlil qilish uchun ham foydalanishingiz mumkin.

Xulosa qilib aytganda, Androguard — bu kuchli va script asosida boshqariladigan vosita bo'lib, u orqali APK fayllarni chuqur statik tahlil qilish, xavfsizlik zaifliklarini aniqlash va ilovaning ichki tuzilmasini to'liq o'rganish mumkin. Python dasturchilar uchun bu vosita yuqori darajada moslashuvchanlik va kengaytma imkoniyatlarini taqdim etadi. Yuklab olish uchun havola: (<https://github.com/magaaz/androwarn>)

## MobSF

MobSF (Mobile Security Framework) — bu Android, iOS va Windows mobil ilovalari uchun statik, dinamik va zararli dastur (malware) tahlilini avtomatlashtirilgan tarzda bajaradigan kuchli va ochiq manbali xavfsizlik tahlil vositasidir. U mobil ilovalarning xavfsizlik darajasini aniqlashda, zaif joylarni

topishda, ruxsatlar va API chaqiruvlarini tekshirishda keng qo'llaniladi. MobSF tajribali xavfsizlik mutaxassislari bilan bir qatorda, yangi o'rganuvchilar uchun ham qulay va tushunarli interfeysga ega. MobSF ikki asosiy tahlil turini qo'llab-quvvatlaydi.

Statik tahlil — ilovani ishga tushirmasdan turib, APK yoki IPA fayl ichidagi resurslar, kodlar, AndroidManifest.xml, ruxsatlar, komponentlar, smali yoki java kodlar, URL manzillar, tokenlar, kriptografik xatoliklar va boshqa xavfsizlik zaifliklarini avtomatik tarzda tahlil qiladi. Bu bosqichda ilovaning ichki logikasi va xavfsizlik siyosati chuqur o'rganiladi.

Dinamik tahlil — ilovani real vaqt rejimida virtual Android emulyatorda ishga tushiradi va ilovaning tarmoq trafigi, API so'rovlari, sistemaga murojaatlari, fayl tizimi bilan ishlashi va boshqa faoliyatlarini yozib boradi. Dinamik tahlilda siz Frida bilan integratsiya qilib, SSL Pinning, Root detection, anti-debugging, va boshqa mudofaa mexanizmlarini avtomatik aniqlashingiz mumkin.

MobSFning yana bir kuchli jihati — malware tahlili. Agar ilovada zararli xatti-harakatlar (masalan, SMS yuborish, ruxsatsiz audio yozish, kontaktlarni o'g'irlash, reklama kodlari) bo'lsa, ularni ham tahlil qiladi va xavflilik darajasini baholaydi. Bundan tashqari, MobSF sizga score (ball) beradi, ya'ni ilovaning umumiy xavfsizlik reytingi — qanchalik ishonchli yoki xavfli ekanini foizda ko'rsatib beradi.

MobSFni o'rnatish uchun sizga Python va Docker kerak bo'ladi, lekin siz uni virtual mashina yoki to'g'ridan-to'g'ri tizimga ham o'rnatish olasiz. MobSF veb-interfeysga ega bo'lib, unga kirganingizda .apk, .aab, .ipa, .zip yoki .source faylni yuklash orqali tahlilni boshlashingiz mumkin. Tahlil tugagach, sizga grafik hisobot, JSON yoki PDF ko'rinishida to'liq natijalar taqdim etiladi.

Xulosa qilib aytganda, MobSF — bu mobil ilovalarning xavfsizligini tez, chuqur va aniq tahlil qilish imkonini beruvchi zamonaviy va avtomatlashtirilgan vositadir. U nafaqat tahlil, balki ishlab chiquvchilarga xavfsizlikni qanday yaxshilash bo'yicha aniq tavsiyalar ham beradi. Shu sababli, u pentesterlar, mobil xavfsizlik tahlilchilari, va ishlab chiquvchilar orasida juda mashhur hisoblanadi.

### **APKleaks**

APKLeaks — bu Android ilovalari (APK fayllari) ichidan maxfiy ma'lumotlar va potensial xavfsizlik zaifliklarini aniqlab beruvchi ochiq manbali, avtomatlashtirilgan statik tahlil vositasi hisoblanadi. Asosan APK faylni dekompiletsiya qilib, undan hardcoded ma'lumotlar — ya'ni APK ichida qattiq kodlangan holda saqlangan API kalitlari, tokenlar, parollar, server URL manzillari, yashirin fayllar yo'llari va boshqa nozik ma'lumotlarni qidirib topadi.

APKLeaks Python tilida yozilgan bo'lib, apktool va grep kabi vositalardan foydalanadi. U APK faylni avval apktool yordamida dekompiletsiya qiladi, so'ngra .smali, .xml, va boshqa matnli fayllar ichida oldindan belgilangan regex (muntazam ifodalar) asosida ma'lumot izlaydi. Shu tariqa ilovadagi ehtimoliy zaif joylarni avtomatik aniqlaydi. Masalan, APKLeaks yordamida quyidagi narsalarni topish mumkin.

- Firebase URLlari va API kalitlari
- Google Maps yoki Stripe API kalitlari
- JWT tokenlar
- Base64 kodlangan ma'lumotlar
- AWS kalitlari
- OAuth tokenlar
- Shaxsiy server manzillari (IP, endpointlar)
- Qattiq kodlangan foydalanuvchi ma'lumotlari yoki login-parollar

APKLeaksni ishlatish juda oddiy. Masalan, quyidagicha buyruq orqali tahlilni boshlash mumkin:

```
python3 apkLeaks.py -f app.apk -o results.txt
```

Bu yerda -f — tahlil qilinadigan APK fayl, -o — natijani yozib olinadigan fayl. Shuningdek, foydalanuvchi o'zining maxsus regex qoidalarini .json fayl ko'rinishida sozlashi va APKLeaksga qo'shib tahlilni kengaytirishi mumkin.

APKLeaks asosan statik tahlil vositasi hisoblanadi, ya'ni u ilovani ishga tushirmasdan tahlil qiladi. Ammo u ilova ishlab chiquvchilari xavfsizlik nuqtai nazaridan qattiq kodlangan ma'lumotlarni APK ichida qoldirib ketmaganini tekshirish uchun juda foydali. Chunki APK fayl foydalanuvchiga yetib borganidan so'ng, har qanday tajribali kishi uni ochib, bu ma'lumotlarni osongina ko'rib olishi mumkin.

Xulosa qilib aytganda, APKLeaks — bu Android ilovalarda maxfiy ma'lumotlar sizib chiqish xavfini aniqlashda eng yengil, avtomatlashtirilgan va tezkor vositalardan biridir. U xavfsizlik tahlilchilari, pentesterlar va mobil ilova ishlab chiquvchilar tomonidan, ayniqsa CI/CD jarayonlariga avtomatik tahlil vositasi sifatida keng foydalaniladi.

### **Frida**

Frida bu dynamic instrumentation toolkit, ya'ni harakatdagi (real vaqtli) ilova ichida kodlarni tahlil qilish, o'zgartirish, kuzatish uchun ishlatiladi. Frida yordamida mobil ilovalarga teskari muhandislik qilish, xavfsizlik testlari, hook qilish, API chaqiruvlarini ko'rish, SSL Pinning bypass, va boshqalar amalga oshiriladi. Fridani yuklab olish uchun rasmiy sayt <https://frida.re> yoki python orqali yuklab olsangiz bo'ladi.

```
pip install frida-tools
```

```
pip install frida
```

Frida serverni <https://github.com/frida/frida/releases> orqali yuklab olishingiz mumkin. Frida serverni yuklab olayotganda qurulma masalan emulatoreni arxitekturasiga mos va fridani versiyasiga mos qilib yuklab olish kerak aks holda ishlamaydi(masalan:frida-server-16.1.4-android-arm64.xz). Frida quydagicha qismlardan tashkil topgan.

- frida-server — Android qurilmada ishlaydigan server, bu orqali ilovalarni tahlil qilamiz.
- frida-tools (CLI) — Komandalarni terminal orqali bajarishga yordam beradi.
- frida-python API — Pythonda Frida skriptlar yozish uchun.
- Frida Gadget — Ilova ichiga joylashtiriladigan kutubxona (root kerak emas).
- Frida Repl (Interactive Console) — Dinamik kod injeksiya qilish uchun interaktiv muhit.

Fridani oʻrnatib olganingizdan keyin uni toʻgʻri ishlayotgani va versiyasini tekshirish uchun quydagi buyruqni terminalda bajarib koʻring.

```
C:\Windows\system32>frida --version
17.2.0

C:\Windows\system32>
```

Android qurilmangiz arxitekturasiga mos keladigan Frida serverini yuklab olishingiz kerak. Masalan, qurilmangiz arm, arm64, x86 yoki x86\_64 boʻlishi mumkin. Frida serverini rasmiy GitHub sahifasidan yuklab olish tavsiya etiladi(<https://github.com/frida/frida/releases>). Bu yerda yana siz etibor qaratadigan joyingiz bu kompyuteringizdagi Frida versiyasi bilan siz yuklab olmoqchi boʻlgan Frida server versiyasi bir xil boʻlishi kerak.

Frida serverni quydagicha qilib oʻz qurilmangizga oʻrnatib ishga tushursangiz boʻladi.

```
adb push frida-server /data/local/tmp/
```

```
adb shell
su
chmod 755 /data/local/tmp/frida-server
/data/local/tmp/frida-server
```

Agar siz qurilmadagi barcha preccesalarni ko'rmoqchi bo'lsangiz terminalga quydagicha buyruq yozing va enter tugmasini bosing.

```
root@kali:~# frida-ps -U
PID  Name
-----
123  adbd
10912 android.process.acore
247  batteryd
8832 cn.wps.moffice_eng:gcmpush
12604 com.android.chrome
12688 com.android.chrome:privileged_process0
12662 com.android.chrome:sandboxed_process0
10121 com.android.deskclock
9539 com.android.gallery3d
860  com.android.inputmethod.latin
921  com.android.launcher3
```

Quyidagi buyruqlar yordamida biz tashqi skriptlarni (JavaScript fayllarini) ilovaga yuklay olamiz. Buning uchun **-l** opsiyasi orqali JavaScript faylini ko'rsatamiz. **-f** opsiyasi esa ilovani topish va unga *hook* qilish (ya'ni ulanib, nazorat qilish) uchun ishlatiladi.

```
root@kali:~# frida -U -f owasp.mstg.uncrackable1 -l disableroot.js

/  _  |   Frida 12.4.4 - A world-class dynamic instrumentation toolkit
|  _||   |
>  _||   |   Commands:
/_/_||   |   help      -> Displays the help system
. . . . .   object?    -> Display information about 'object'
. . . . .   exit/quit  -> Exit
. . . . .
. . . . .   More info at http://www.frida.re/docs/home/

Spawned `owasp.mstg.uncrackable1`. Use %resume to let the main thread start executing!
[Genymotion Custom Phone - 6.0.0 - API 23::owasp.mstg.uncrackable1]-> %resume
[Genymotion Custom Phone - 6.0.0 - API 23::owasp.mstg.uncrackable1]-> Exploit Complete
In function A
In function B
In function C
[Genymotion Custom Phone - 6.0.0 - API 23::owasp.mstg.uncrackable1]->
[Genymotion Custom Phone - 6.0.0 - API 23::owasp.mstg.uncrackable1]-> exit

Thank you for using Frida!
```

Diqqat bu ishlashi uchun siz Frida serverni o'z qurilmangizni masalan emulaturingizni data/local/tmp fayliga joylashirgan bo'lishingiz va adb orqali Frida serverni ishlatib qo'yishingiz kerak bo'ladi.

```
C:\Windows\system32>adb shell
generic_x86_arm:/ $ su
generic_x86_arm:/ # cd data/local/tmp
generic_x86_arm:/data/local/tmp # ./frida-server
```

Frida vositasining eng yaxshi jihatlari shundaki, u bepul va ochiq manbali, hamda Windows, Linux va macOS kabi turli platformalarni qo'llab-quvvatlaydi.

Frida skript yozishni qo'llab-quvvatlaydi, ya'ni biz o'z skriptlarimizni ilovaga kiritib (inject qilib), har qanday funksiyalarni, hatto ilova manba kodi bo'lmasa ham, API darajasida hook qilishimiz mumkin.

Sizning kamyuteringizga ulangan barcha qurilmalar ro'yhatini olish uchun quydagi buyruqni terminalga yozing.

```
root@kali:~# frida-ls-devices
```

| Id                  | Type   | Name                                     |
|---------------------|--------|------------------------------------------|
| local               | local  | Local System                             |
| 192.168.59.101:5555 | usb    | Genymotion Custom Phone - 6.0.0 - API 23 |
| tcp                 | remote | Local TCP                                |

Ushbu buyruq barcha ishlaydigan jarayonlarni qaytaradigan jarayonlarni ro'yxatga olish uchun ishlatiladi. USB orqali ulangan qurilmadan jarayonni qaytarish uchun - U opsiyasini qo'shing.

```

root@kali:~# frida-ps -U
PID  Name
-----
123  adbd
10912 android.process.acore
247  batteryd
8832 cn.wps.moffice_eng:gcmpush
12604 com.android.chrome
12688 com.android.chrome:privileged_process0
12662 com.android.chrome:sandboxed_process0
10121 com.android.deskclock
9539 com.android.gallery3d
860  com.android.inputmethod.latin
921  com.android.launcher3

```

| Komanda                                  | Izoh                                            |
|------------------------------------------|-------------------------------------------------|
| frida-ps                                 | Ishlayotgan processlar ro'yxati                 |
| frida-ps -U                              | USB qurilmadagi processlar ro'yxati             |
| frida-ps -R                              | Remote qurilma (TCP orqali)                     |
| frida-trace                              | API yoki function'ni kuzatish (hook qilish)     |
| frida-trace -n <package>                 | Muayyan ilova ichidagi function hook            |
| frida-trace -n com.example -m<br>"*ssl*" | SSL bilan bog'liq barcha metodlarni<br>kuzatish |
| frida -U -n <package>                    | Interaktiv muhitda ilovaga ulanish              |
| frida -U -f <package>                    | Ilovani ishga tushirib ulanish                  |
| frida -U -n <package> -l script.js       | JavaScript faylni hook sifatida yuklash         |

## MAVS

MAVS — bu Mobile Application Vulnerability Scanner so'zining qisqartmasi bo'lib, mobil ilovalarni xavfsizlik nuqtai nazaridan statik va yarim-avtomatik tarzda tahlil qilish uchun mo'ljallangan ochiq manbali vositadir. MAVS, ayniqsa Android ilovalarini tahlil qilishda foydalidir va ishlab chiquvchilarga hamda xavfsizlik tadqiqotchilariga APK fayl ichidagi zaifliklar, noto'g'ri ruxsatlar, xavfsiz bo'lmagan amaliyotlar va boshqa potentsial muammolarni aniqlashda yordam beradi.



MAVS odatda APKTool, MobSF, Qark, Androguard kabi boshqa vositalar bilan birga ishlatiladi yoki ularning ustiga qurilgan. Ya'ni u mustaqil skaner bo'lishi mumkin, yoki mavjud tahlil vositalaridan natijalarni yig'ib, ularni boshqaruvli va aniq formatda birlashtiradi. MAVS avtomatik tarzda ilova ichidagi ma'lumotlarni ajratadi, masalan:

- AndroidManifest.xml faylni o'qiydi va undagi ruxsatlarni tahlil qiladi
- smali yoki .dex kodlarda ishlatilayotgan API chaqiruvlar, xavfli funksiyalar (masalan, WebView, JavaScriptInterface) mavjudligini aniqlaydi
- Kriptografik xatoliklar (masalan, noto'g'ri ishlatilgan Cipher, MessageDigest), tokenlar, URL manzillar va boshqa nozik ma'lumotlarni qidiradi
- Ruxsatlar (permissions) noto'g'ri belgilangan holatlarni aniqlaydi (masalan, READ\_SMS, WRITE\_EXTERNAL\_STORAGE ruxsatlari kerakligidan ortiq bo'lsa)

MAVS natijalarni hisobot shaklida taqdim etadi, odatda HTML yoki JSON formatda, va bu hisobotlar ichida aniqlangan har bir zaiflik haqida izoh, tavsiya, hamda xavflilik darajasi (kritik, o'rtacha, past) ko'rsatiladi. Bu esa ishlab chiquvchilarga zaiflikni nafaqat topish, balki uni tuzatish yo'llarini ham tushunishga yordam beradi. MAVS vositasi juda yengil va soddalashtirilgan CLI (komanda qatori interfeysi) orqali ishlaydi. Asosan Linux yoki macOS tizimlarida ishlatiladi. Quyidagi buyruq orqali oddiy tahlil boshlab yuboriladi.

```
python3 mavs.py -f path/to/app.apk -o output/
```

Bu yerda -f – tahlil qilinadigan APK fayl, -o – hisobot saqlanadigan papka.

MAVS ba'zida mobil ilovalar uchun CI/CD xavfsizlik testingi ga qo'shib qo'yiladi, ya'ni har bir build jarayonidan keyin avtomatik xavfsizlik tekshiruvi o'tkaziladi. Bu dasturchilarni APK ichidagi noxush xatoliklarni dastlabki bosqichdayoq aniqlashga majbur qiladi.

Xulosa qilib aytganda, MAVS — bu mobil ilovalar xavfsizligini avtomatik tarzda tahlil qilish uchun ishlatiladigan, soddalashtirilgan va foydali vosita bo‘lib, u orqali Android ilovalardagi zaif joylar tez va samarali aniqlanadi. U tajribali xavfsizlik mutaxassislari bilan bir qatorda, ishlab chiquvchilar uchun ham foydali, chunki u zaiflikni ko‘rsatibgina qolmay, uni qanday tuzatish kerakligini ham tushuntiradi. Yuklab olish uchun havola(<https://github.com/sho-luv/mavs>).

### **APKHunt**

APKHunt — bu Android ilovalarini (APK fayllarini) xavfsizlik nuqtai nazaridan statik tahlil qilishga mo‘ljallangan ochiq manbali yengil va samarali vosita bo‘lib, u APK fayl ichidan maxfiy ma’lumotlar, tokenlar, API kalitlari, va xavfli kod yozilishlarini avtomatik tarzda aniqlaydi. APKHunt asosan qattiq kodlangan (hardcoded) ma’lumotlarni qidirishga ixtisoslashgan bo‘lib, xavfsizlik tahlilchilari, pentesterlar va ishlab chiquvchilarga ilovani tahlil qilishda qo‘l keladi. APKHunt dastlab APK faylni apktool yordamida dekomplyatsiya qiladi, ya’ni uni ochadi va matnli ko‘rinishga keltiradi. So‘ngra .smali, .xml, .json, .properties, va boshqa resurs fayllar ichida oldindan belgilangan muntazam ifodalar (regex) yordamida qidiruv amalga oshiradi. Ushbu regexlar yordamida quyidagi turdagi ma’lumotlarni aniqlaydi:

- Firebase manzillari (.firebaseio.com)
- Google Maps, Stripe, Twitter, GitHub API kalitlari
- JWT tokenlar, OAuth tokenlar
- AWS Access/Secret kalitlari
- Base64 kodlangan satrlar
- Foydalanuvchi login-parollar
- IP manzillar, URL endpointlar
- Kriptografik funksiya nomlari (AES, MD5, SHA1 va h.k.)

APK Hunt bu topilgan ma'lumotlarni kategoriya bo'yicha ajratib, tahlil natijalarini qulay HTML yoki JSON shaklida hisobot ko'rinishida taqdim etadi. Bu hisobotlar orqali siz ilova ichida nozik yoki xavfli ma'lumotlar bor-yo'qligini aniq ko'ra olasiz. APK Hunt komanda satrida ishlaydi va juda oddiy buyruqlar bilan boshqariladi.

```
python3 apkhunt.py -f app.apk -o results/
```

Bu yerda -f bu tahlil qilinadigan APK fayl, -o esa natijalar saqlanadigan papka. Yuklab olish uchun havola(<https://github.com/Cyber-Buddy/APKHunt>).

### Tapjacker

Tapjacker — bu Android ilovalardagi tapjacking (ya'ni foydalanuvchini aldash orqali noto'g'ri tugmalarni bosdirish) zaifligini aniqlash va sinovdan o'tkazish uchun ishlab chiqilgan ochiq manbali ekspluatatsiya vositasi hisoblanadi. Uning asosiy vazifasi — boshqa ilovalarning ustiga shaffof (invisible) yoki yarim shaffof (semi-transparent) qatlamlar joylashtirib, foydalanuvchining xatti-harakatlarini nazorat qilishga urinadigan holatlarni modellashtirish.

Tapjacker ilova sifatida ishlaydi va u orqali siz sinovdan o'tkazmoqchi bo'lgan haqiqiy ilovaning ustiga Overlay (ustki qatlam) joylashtirasiz. Bu qatlam foydalanuvchiga ko'rinmasligi yoki chalg'ituvchi interfeys orqali uni noto'g'ri tugmani bosishga majburlashi mumkin. Tapjacker aynan shunday sinov uchun ishlatiladi ya'ni u orqali.

- Ilovaning ustiga oynacha (toast, dialog, yoki SYSTEM\_ALERT\_WINDOW ruxsatli overlay) joylashtiriladi
- Tugma, ruxsat olish oynasi, yoki xavfsizlik bilan bog'liq harakatlar ustiga interaktiv qoplama qo'yiladi
- Foydalanuvchi real tugmani bosdim deb o'ylasa ham, aslida boshqa yashirin amal bajariladi

Tapjacker asosan ta'limiy va xavfsizlik test maqsadlarida ishlatiladi. Uni Android Studio orqali yig'ib olish mumkin yoki tayyor APK shaklida yuklab olib test qilish mumkin. Odatda quyidagi bosqichlar amalga oshiriladi:

- Test qilinadigan ilova Android qurilmada o'rnatiladi
- Tapjacker ilovasi ochiladi va unda "target package" tanlanadi (ya'ni hujum qilinadigan ilova)
- Tapjacker oynasi ustki qatlam hosil qiladi va foydalanuvchi uni bosgandek tuyulgan harakatni kuzatadi

Agar sinov davomida foydalanuvchi ilovaning ustiga o'rnatilgan shaffof qatlam orqali noto'g'ri amal bajarayotgan bo'lsa, demak ilova tapjacking zaifligiga ega. Tapjacker bu holatni fosh qilishi va sizga ilova ichida `filterTouchesWhenObscured="true"` yoki `onFilterTouchEventForSecurity()` kabi himoya mexanizmlari yo'qligini ko'rsatishi mumkin. Yuklab olish uchun havola(<https://github.com/dzmitry-savitski/tapjacker>).

### **Logcat**

Mobil ilova xavfsizligini tahlil qilishda ishlatiladigan eng asosiy vositalardan biri bu Logcat. Logcat bu Android tizimining real vaqtda ishlaydigan log yozuvlarini ko'rsatib boradigan jurnal tizimi bo'lib, ilova ishga tushgan vaqtdan boshlab, u qanday metodlarni bajarayotgani, qanday xatoliklar sodir bo'layotgani, qanday ma'lumotlar chiqarilayotgani, hatto tizim darajasida qanday signal va xabarnomalar uzatilayotgani haqida to'liq ma'lumot beradi. Asosan, Logcat yordamida ilova ichida yuz berayotgan barcha hodisalar, ya'ni `Log.d`, `Log.e`, `Log.i`, `Log.w`, `System.out.println`, hatto native darajadagi xatoliklar ham tahlil qilinadi. Ilovaning qanday so'rov yuborayotgani, foydalanuvchidan olingan ma'lumotlar, o'zgartirilayotgan qiymatlar, ishga tushayotgan komponentlar, bajarilayotgan funksiyalar va ularning ketma-ketligi, Logcat orqali aniqlanadi.

Xavfsizlik tahlilchisi Logcat yordamida ilova qanday API endpointlarga murojaat qilayotganini, tokenlar yoki session IDlar qanday hosil bo'layotganini, ularning qiymatlari va bu qiymatlar logga tushib qolganmi-yo'qmi, bularni kuzatadi. Chunki ko'plab noto'g'ri ishlab chiqilgan ilovalarda maxfiy ma'lumotlar, masalan parollar, JWT tokenlar yoki OAuth kalitlar Log.d() orqali logga chiqarib yuborilgan bo'ladi va foydalanuvchi qurilmasidagi log fayllar orqali buni tahlil qiluvchi kishi ko'rib olishi mumkin. Bu esa xavfsizlikka katta tahdid tug'diradi. Shu sababli Logcat nafaqat ishlab chiquvchilar uchun test va debug vositasi, balki xavfsizlik tahlilchilari uchun ham real vaqtda ilovaning ichki ishini tahlil qiluvchi ko'zdir.

Logcatni ishlatish uchun adb logcat buyrug'i ishlatiladi. Bu buyruq orqali siz qurilmaga ulanib, real vaqt rejimida loglar oqimini kuzatishingiz mumkin. Istasangiz, loglarni filtr qilib faqat kerakli tag bilan chiqarishingiz yoki faqat bir ilovaga tegishli loglarni ko'rishingiz mumkin. Misol uchun, siz ilovani ishga tushirganingizda qanday ruxsatlar so'rayapti, qaysi activity yoki service qanday ketma-ketlikda ishga tushmoqda, bu ma'lumotlarning barchasini Logcat aniq ko'rsatadi. Shu jihatdan olganda, Logcat orqali siz ilova ichidagi harakatlarning "ichki ovozi"ni eshitgandek bo'lasiz.

Logcat yordamida dinamik tahlil ham amalga oshiriladi. Masalan, siz foydalanuvchi login qilganida token logga chiqyaptimi yo'qmi, API so'rovlar yuborilganda qanday javoblar qaytmoqda, ilova nimani yodda saqlayapti — bularni aniq ko'rib olasiz. Shuningdek, ba'zida ilovada try-catch bloklar noto'g'ri yozilgani sababli xatoliklar logda ochiq-oydin chiqarilgan bo'ladi. Bu esa tajribali hujumchi yoki xavfsizlik tahlilchisi uchun juda katta manba hisoblanadi. Chunki bu orqali ilovaning ichki tuzilmasini, ishlash mantig'ini va himoya darajasini tushunib olish mumkin bo'ladi.

## APKDeepLens

APKDeepLens — bu Android ilovalarini chuqur xavfsizlik tahlilidan o'tkazish uchun mo'ljallangan ilg'or statik tahlil vositasi bo'lib, uning asosiy vazifasi ilova ichidagi potentsial zaifliklarni avtomatik tarzda aniqlash va ularni semantik darajada tahlil qilishdir. U an'anaviy qidiruv vositalaridan farqli o'laroq, faqatgina kalit so'zlar yoki regex asosida emas, balki kodning ishlash mantig'i, funksiyalar orasidagi aloqalar, chaqiruvlar ketma-ketligi, va ma'lumotlar oqimi (data flow) orqali zaiflikni aniqlashga harakat qiladi. APKDeepLens aynan “deep” — ya'ni chuqur analizga tayanadi. Bu esa uni boshqa yengil vositalardan ajratib turadi.

Bu vosita ilovaning .apk faylini ochib, avval dex faylni analiz qiladi, so'ngra uni semantik darajada parchalab, undan ma'lumot oqimi, ruxsatlar, API chaqiruvlar, xavfsizlikka oid anomaliyalar va noto'g'ri ishlatilgan kod namunalarini aniqlaydi. Masalan, ilovada WebView komponenti ishlatilgan bo'lsa, APKDeepLens uni qanday sozlanganini, `setJavaScriptEnabled(true)` chaqiruvlari bilan birga `addJavascriptInterface` ishlatilganmi-yo'qmi, foydalanuvchi ma'lumotlari shifrlanmasdan tarmoqqa yuborilayotganmi, yoki ilovada noto'g'ri ishlatilgan Cipher, MD5, SHA1 kabi kriptografik funksiya mavjudmi, bularni to'liq semantik kontekstda ko'rib chiqadi.

APKDeepLensning eng kuchli jihatlaridan biri — bu u yordamida aniqlangan zaifliklar faqatgina sathiy ko'rinish emas, balki mantiqiy bog'lanish asosida topilgan bo'ladi. Ya'ni, oddiy string orqali topilmaydigan, lekin kodning ishlash jarayonida sodir bo'ladigan zaifliklarni ham aniqlashga urinadi. Bu esa uni real xavfsizlik tahlilida juda foydali vositaga aylantiradi. Ayniqsa korporativ darajadagi ilovalar, katta hajmdagi APK fayllar yoki xavfsizlikka sezgir sohalarda ishlab chiqilgan ilovalarni tekshirishda APKDeepLensdan foydalanish katta afzallik beradi. APKDeepLens ishlashi uchun sizda Python muhiti, ba'zi kerakli tahlil kutubxonalari va dekompileyatorlar o'rnatilgan bo'lishi kerak. Ilova ishlatilgan kutubxonalarni,

manba koddan qoldirilgan maxfiy ma'lumotlarni, noto'g'ri sozlangan Intent, Broadcast, Service, Provider kabi komponentlarni ham aniqlaydi. Tahlil natijalari esa ko'pincha JSON yoki HTML ko'rinishida chiqariladi va bu hisobotlar ichida aniqlangan zaifliklar, xavflilik darajasi, ta'siri va uni bartaraf etish bo'yicha tavsiyalar beriladi. Yuklab olish uchun [havola](#) ustiga bosing.

### **Reqable**

Reqable — bu Android va iOS mobil ilovalari uchun real vaqt rejimida yuborilayotgan barcha tarmoq so'rovlarini to'liq ko'rish, tahlil qilish va manipulyatsiya qilish imkonini beruvchi zamonaviy va qulay trafik tahlil vositasidir. U mitmproxy (Man-In-The-Middle) texnologiyasi asosida ishlaydi va mobil ilova bilan server o'rtasidagi barcha HTTP va HTTPS so'rovlarini ushlab qoladi. Bu orqali foydalanuvchi nafaqat ilova qanday so'rov yuborayotganini, balki qaysi ma'lumotni yuborayotganini, qanday javob olayotganini, qanday headerlar bilan ishlayotganini, cookie, token, session ID kabi sezgir ma'lumotlar qanday shaklda ketayotganini ham to'liq kuzatishi mumkin bo'ladi.

Reqable vositasi asosan GUI — ya'ni grafik interfeysga ega bo'lib, o'zining intuitiv va qulay dizayni orqali xavfsizlik tahlilchisiga yoki dasturchiga mobil ilova tarmoq trafigini to'liq tahlil qilish imkonini beradi. U ilovalar tomonidan yuborilgan so'rovlarni satrma-satr ko'rsatadi, ularni saqlab qo'yadi, filtrlaydi, hamda xohlagan so'rovni edit qilib, qayta yuborish imkonini beradi. Bu, ayniqsa, ilovadagi autentifikatsiya jarayonlarini, API endpoint zaifliklarini, tokenni buzish va test qilish, yoki JWT, OAuth, va session cookie'larni sinovdan o'tkazishda juda foydali hisoblanadi.

Reqable yordamida foydalanuvchi SSL Pinning yoki boshqa trafikni yashirishga urinishlarga qaramay, sertifikatni import qilish orqali HTTPS so'rovlarni ham to'liq tahlil qilishi mumkin. Bu holat ko'plab xavfsizlik testlarida muhim ahamiyat kasb

etadi. Masalan, siz ilova login qilayotganda qanday token olayotgani, bu token JSON Web Token (JWT) bo'lsa, uning payloadi qanday shaklda ekanini, u o'zgaruvchanmi yoki doimiy, qanday xatolik javob qaytaryapti — bularning barchasini Reqable oynasi orqali ko'rib, tahlil qilishingiz mumkin bo'ladi.

Bundan tashqari, Reqable vositasi orqali foydalanuvchi so'rovni o'z xohishiga ko'ra buzib ko'rishi, ya'ni POSTni GETga aylantirishi, Authorization headerga noto'g'ri token qo'yib yuborishi yoki mavjud tokenni boshqa foydalanuvchi tokeni bilan almashtirib yuborishi mumkin. Agar server bunga ham javob qaytaraversa yoki autentifikatsiya mexanizmi bu farqni sezmasa — demak ilovada authorization bypass yoki session hijacking xavfi mavjud bo'lishi mumkin.

Reqable mobil qurilmaga ulanish uchun kompyuterdagi Reqable dasturi va telefonni bir tarmoqda bog'lash orqali ishlaydi. Telefonda maxsus foydalanuvchi tomonidan o'rnatilgan CA Certificate yordamida qurilma Reqable'ni ishonchli server deb qabul qiladi va barcha trafik Reqable orqali o'tadi. Shundan so'ng siz telefoningizdagi istalgan ilovani ishga tushirishingiz bilan uning barcha tarmoq so'rovlari Reqable oynasida paydo bo'ladi. Shuningdek, Reqable tarmoq trafigini .har fayl ko'rinishida eksport qilish, uni JSON formatda saqlash, yoki qidiruv, saralash, statistik tahlil qilish kabi funksiyalarni ham taklif etadi.

### **Objection**

Objection — bu mobil ilovalarni, xususan Android va iOS ilovalarini teskari muhandislik va xavfsizlik tahlili nuqtai nazaridan tezkor va interaktiv tarzda sinash imkonini beradigan zamonaviy, ochiq manbali vositadir. Bu vosita Frida dinamika instrumentatsiya kutubxonasiga tayanadi va xavfsizlik tahlilchisiga ilovaning ish jarayonida ichki obyektlar, funksiyalar, kontekstlar va ruxsatlar ustidan real vaqtda to'liq nazorat o'rnatish imkonini beradi. Objection vositasining eng katta afzalligi shundaki, siz ilovani oldindan teskari muhandislik qilmasdan, ya'ni uni



dekompilyatsiya qilmasdan turib ham, uni ishlayotgan holatida ichkarisiga “kirib”, u yerdagi ma’lumotlarni o’rganishingiz, o’zgartirishingiz va tahlil qilishingiz mumkin.

Objection ko‘pincha “ilovaga root huquqisiz penetration test qilish” vositasi deb ataladi. Chunki u sizga ilovaning ish faoliyatini ko‘rib chiqish, xavfsizlik cheklovlarini chetlab o‘tish, masalan, SSL Pinning, root detection, debug detection, biometric check bypass, yoki flag\_secure kabi himoyalarni real vaqt rejimida bekor qilish imkonini beradi. Ilova ichidagi funksiyalar qanday chaqirilayotganini, ma’lumotlar qanday shaklda saqlanayotganini, qanday ruxsatlar ishlatilayotganini, qanday foydalanuvchi ma’lumotlari uzatilayotganini — bularning barchasini Objection yordamida aniq ko‘rish va tahlil qilish mumkin.

Objection’ni ishlatish juda oddiy. Siz ilovani Frida yordamida ishga tushirasiz, so‘ngra Objection orqali terminal oynasida interaktiv buyruqlar yordamida unga ulanib, ilovani “hujum ostida” sinab ko‘rasiz. Masalan, quyidagi buyruq bilan siz ilovani ishga tushirishingiz va unga ulanishingiz mumkin:

```
objection -g com.example.app explore
```

Bu buyruqdan so‘ng sizga qulay CLI interfeys ochiladi va u orqali quyidagilarni amalga oshirishingiz mumkin. env — ilova ichki muhitidagi o‘zgaruvchilarni ko‘rish.

memory search — ilova xotirasida maxfiy ma’lumotlarni (token, parol, API key) qidirish. android sslpinning disable — SSL Pinning himoyasini real vaqt rejimida o‘chirib tashlash. android root disable — ilova ichida root tekshiruvini bekor qilish. android intent launch — istalgan Intentni sun’iy ravishda ishga tushirish. android hooking watch — ma’lum funksiyani kuzatib borish yoki logga yozib borish. android heap search — xotirada saqlanayotgan obyektlar va ularning qiymatlarini qidirish.

Bu imkoniyatlar yordamida siz ilova qanday ma'lumotni qayerga yuborayotganini, qayerda qanday funksiya chaqirayotganini, qanday SharedPreferences, SQLite, KeyStore, yoki boshqa joylarda ma'lumot saqlayotganini aniqlab, xavfsizlik nuqtai nazaridan ularni chuqur tahlil qilishingiz mumkin.

Shuningdek, Objection yordamida siz ilovaning UI elementlariga ham ta'sir o'tkazishingiz, faoliyatdagi Activity, Service, yoki BroadcastReceiver obyektlarini aniqlashingiz va ularga xabar jo'natishingiz mumkin. Bu esa ilovaning ichki tuzilmasini sinchiklab o'rganish, noto'g'ri konfiguratsiya qilingan komponentlarni topish, yoki ilovaning mantiqiy xatolarini fosh qilish uchun ayni muddao.

### **HTTP Toolkit**

HTTP Toolkit — bu mobil va veb ilovalarning yuborayotgan tarmoq so'rovlarini real vaqt rejimida tahlil qilish, interaktiv tarzda o'zgartirish va trafigini manipulyatsiya qilish imkonini beruvchi kuchli, zamonaviy va ochiq manbali vositadir. Bu vosita, ayniqsa, Android ilovalari bilan ishlaganda, ulardan yuborilayotgan HTTP va HTTPS so'rovlarni to'liq ushlab qoladi, ularni ko'rsatadi, tahlil qiladi va kerak bo'lsa, istalgan joyini o'zgartirib, serverga yuboradi. Bu esa ilova bilan server o'rtasida qanday axborot almashilayotganini to'liq tushunish, foydalanuvchidan yuborilayotgan ma'lumotlar, tokenlar, cookie'lar, session ID'lar va boshqa sezgir qiymatlarni aniqlash uchun juda muhim imkoniyat hisoblanadi.

HTTP Toolkit klassik intercepting proxy vositalarining zamonaviy, yengil va interaktiv shaklidir. U foydalanuvchi uchun maxsus, intuitiv grafik interfeys taqdim etadi. Bu interfeysda ilova yuborgan har bir so'rov va serverdan olingan har bir javob satrma-satr, strukturaviy ko'rinishda ko'rsatiladi. Siz bu so'rovlarning method, URL, headers, body qismlarini osongina tahrirlashingiz, so'rovni to'xtatib turishingiz, uni xohlagancha o'zgartirishingiz, yoki qayta yuborishingiz mumkin. Bu

ayniqsa autentifikatsiya, ruxsat, token almashinuvi, API zaifliklarini aniqlash va test qilishda juda foydali bo‘ladi.

HTTP Toolkit vositasi HTTPS so‘rovlarni ham bemalol ochib tahlil qila oladi. Buning uchun foydalanuvchi qurilmasiga HTTP Toolkit’ning CA sertifikatini o‘rnatiladi. Shu tariqa, foydalanuvchi ilovadan yuborilgan HTTPS so‘rovlarni ham shifrlanmagan holda ko‘ra oladi. Bu imkoniyat SSL Pinning mavjud bo‘lmagan ilovalarda to‘liq trafikni kuzatish imkonini beradi. Agar ilovada SSL Pinning bo‘lsa, HTTP Toolkit u bilan kurasha olmaydi, ammo siz Frida yoki Objection yordamida pinningni o‘chirib, trafikni HTTP Toolkit orqali ko‘ra olasiz.

HTTP Toolkit o‘zining kuchli “auto-interception” funksiyasi orqali sizga telefoningizda o‘rnatilgan barcha ilovalar trafikini tarmoq orqali ushlab olishni taklif etadi. Sizga faqat mobil qurilmangizni HTTP Toolkit o‘rnatilgan kompyuter bilan bir Wi-Fi tarmog‘iga ulash kifoya. HTTP Toolkit qurilmangizni aniqlaydi, unga kerakli proksi va sertifikatni o‘rnatadi, so‘ngra har bir ilova yuborgan so‘rov oynada ko‘rina boshlaydi. Har bir so‘rov bo‘yicha siz statistik ma’lumotlar, so‘rovning o‘rtacha kechikishi, javobning o‘lchami, status kodi kabi metrikalarni ham kuzatishingiz mumkin.

Shuningdek, HTTP Toolkit orqali siz tarmoq qoidalari o‘rnatishingiz mumkin: masalan, har doim Authorization headerni o‘zgartirish, ma’lum URL’ni bloklash, yoki biror ma’lumotni avtomatik tarzda almashtirib yuborish. Bu tahlil jarayonini avtomatlashtirish imkonini beradi va sizga zaifliklarni tezroq aniqlashga yordam beradi.

### **Burpsuite**

Burp Suite — bu veb va mobil ilovalarning tarmoq xavfsizligini chuqur tahlil qilish, zaifliklarni aniqlash va ekspluatatsiya qilish uchun dunyo miqyosida keng

qoʻllaniladigan, kuchli va professional xavfsizlik test vositasi hisoblanadi. U ayniqsa pentesterlar, xavfsizlik tahlilchilari va teskari muhandislar tomonidan HTTP/HTTPS trafikni ushlash, manipulyatsiya qilish, va unga chuqur tahlil oʻtkazish uchun ishlatiladi. Burp Suite aynan shu sohadagi sanoat standarti boʻlib, unda barcha zaruriy vositalar: proxy, repeater, intruder, scanner, decoder, va boshqa koʻplab modul va funksiyalar birlashtirilgan.

Burp Suite yordamida siz ilovadan yuborilgan har bir HTTP yoki HTTPS soʻrovni real vaqt rejimida toʻxtatishingiz, uni istagancha oʻzgartirishingiz va qayta yuborishingiz mumkin. Bu jarayon orqali siz ilovada yuborilayotgan tokenlar, cookie'lar, foydalanuvchi identifikatorlari, API endpointlar, va boshqa sezgir ma'lumotlarni sinchiklab tahlil qilishingiz mumkin. Ayniqsa mobil ilovalarda ishlatilayotgan serverlar bilan aloqani toʻliq ochib, ilovaning ichki muloqot mexanizmini anglashda Burp Suite vositasi juda foydali hisoblanadi.

Burp Suite — bu oddiy intercepting proxy vositasi emas. U oʻzining kuchli modullari yordamida sizga tarmoqdagi zaifliklarni avtomatik ravishda aniqlash (Scanner), aniq parametrlar asosida brutforce yoki fuzzing hujumlarini tashkil etish (Intruder), va JWT yoki boshqa murakkab tokenlarni tahlil qilish (Decoder) imkonini beradi. Masalan, siz ilovadan yuborilgan JWT tokenni Decoder oynasiga olib oʻtasiz va u sizga uning header, payload, va signature qismlarini ochib beradi. Siz uni oʻzgartirib, soʻngra Repeater oynasi orqali yangi token bilan qayta soʻrov yuborishingiz mumkin. Agar server bunga ham javob bersa — bu imzo tekshirilmayotganidan dalolat beradi va zaiflik mavjudligini bildiradi.

Burp Suite, mobil ilovalarda ham xuddi shu printsipta ishlaydi. Siz telefoningizni Burp Suite oʻrnatilgan kompyuter bilan bir Wi-Fi tarmogʻiga ulangach, telefoningizdagi proksi sozlamalariga Burp'ning IP manzili va portini kiritasiz. Soʻngra Burp sertifikatini telefoningizga oʻrnatib qoʻyasiz. Bu orqali siz mobil

ilovadan yuborilgan HTTPS so‘rovlarni ham to‘liq ochiq holda ko‘ra olasiz. SSL Pinning bo‘lsa — bu himoyani Frida, Objection, yoki boshqa bypass vositalari orqali o‘chirib, yana Burp orqali trafikni tahlil qilish mumkin.

Burp Suite yordamida siz quyidagi holatlarni sinab ko‘rishingiz mumkin: noto‘g‘ri sozlangan autentifikatsiya, zaif tokenlar, cookie-based sessiyalar, authorizatsiya zaifliklari, IDOR (Insecure Direct Object Reference), CSRF (Cross Site Request Forgery), XSS, SSRF, va boshqa ko‘plab tarmoq asosidagi xavflar. Ayniqsa, Intruder modulida siz foydalanuvchining login formasi ustida brutforce sinovini o‘tkazishingiz, yoki biror API parametriga fuzzing yuborib server qanday javob berishini kuzatishingiz mumkin.

Burp Suite’ning Professional versiyasi — pullik, va u avtomatik skanerlash, kengaytirilgan reporting, hamda ko‘plab boshqa ilg‘or imkoniyatlarni taklif etadi. Biroq uning Community versiyasi ham asosiy tahlil va intercept vazifalarini to‘liq bajaradi. Shuningdek, Burp Suite extensible — ya’ni siz unga qo‘shimcha modullar, ssenariylar (BApp Store orqali), Python yoki Java’da yozilgan pluginlar ulab, uni o‘zingizga moslashtirishingiz mumkin.

### **Drozer**

Drozer — bu Android ilovalari va ularning tizimdagi komponentlarini xavfsizlik nuqtai nazaridan chuqur tahlil qilish uchun ishlab chiqilgan, professional darajadagi, kuchli va interaktiv ekspluatatsiya ramkasi hisoblanadi. U mobil xavfsizlik tahlilchilari uchun ayniqsa foydali bo‘lib, ilovaning ichki komponentlari — masalan, Activity, Service, BroadcastReceiver, va ContentProvider kabi qismlar ustida sinovlar o‘tkazish, zaifliklarni aniqlash va ularni ekspluatatsiya qilish imkonini beradi. Drozer — bu Android ilova muhitining ichkarisiga kirib, u yerdagi har bir ochiq yoki noto‘g‘ri sozlangan interfeysni chuqur o‘rganish uchun yaratilgan vosita.

Drozer ikki asosiy qismdan iborat: agent (ya'ni Android qurilmaga o'rnatiladigan drozer agent.apk) va desktop client (kompyuterda ishlovchi terminal interfeys). Siz qurilmada agentni ishga tushirasiz, kompyuter orqali unga ulanib, istalgan buyruqlarni yuborasiz. Bu orqali siz ilovaning ichki tuzilmasini, ruxsatlarini, foydalanuvchidan yashirilgan komponentlarini, va ularning qanday funksiyalarni bajarishini aniq tahlil qilishingiz mumkin.

Misol uchun, siz `app.activity.info` buyrug'i yordamida ilovaning barcha faoliyatdagi Activity komponentlarini ko'rishingiz, `app.provider.query` buyrug'i orqali ContentProvider orqali o'qilishi mumkin bo'lgan maxfiy ma'lumotlarni chiqarishingiz, yoki `app.broadcast.send` orqali ilovaga sun'iy Broadcast yuborib, uning qanday javob berishini tahlil qilishingiz mumkin. Bu sizga ilovada komponentlararo kommunikatsiya (Inter-Component Communication) qanday ishlayotganini tushunishga yordam beradi.

Drozer ayniqsa, ilovada mavjud bo'lgan komponentlar himoyasizligiga asoslangan zaifliklarni aniqlashda juda foydali hisoblanadi. Ko'plab mobil ilovalarda Activity, Service yoki Provider komponentlari noto'g'ri sozlangan bo'ladi — ya'ni ular `exported=true` holatda, lekin hech qanday ruxsat bilan himoyalanmagan. Bu holatda boshqa ilovalar (yoki Drozer orqali yuborilgan buyruq) bu komponentlarga kirish huquqiga ega bo'lib qoladi. Shu orqali siz maxfiy ma'lumotlarni o'qib olishingiz, noto'g'ri funksiyalarni ishga tushirishingiz yoki ilovani buzishingiz mumkin.

Drozer tomonidan taklif etiladigan buyruqlar soni ko'p va ular modullar orqali taqdim etiladi. Siz o'zingizga kerakli modulni chaqirasiz, masalan `scanner.provider.finduris` yordamida ochiq ContentProvider manzillarini aniqlaysiz, yoki `scanner.misc.native` yordamida native kutubxonalarda zaifliklar mavjudmi, shuni tekshirasiz. Bundan tashqari, siz o'z Drozer modullaringizni yozishingiz ham

mumkin, bu esa tahlilni chuqurroq, avtomatizatsiya qilingan va moslashtirilgan shaklga keltirishga imkon beradi.

Drozer'ni ishlatish uchun qurilmada adb orqali ulanish, drozer agent ilovasini o'rnatish va unga tegishli ruxsatlarni berish talab etiladi. Agent ishga tushgach, siz kompyuterdagi terminal orqali drozer console connect buyrug'i bilan ulanasiz va undan keyin interaktiv sessiya ochiladi. Shu sessiya davomida siz ilovaga test buyruqlar yuborasiz, javoblarni kuzatasiz, va aniqlangan zaifliklar ustida ishlaysiz.

### **Genymotion**

Genymotion — bu Android tizimiga ega qurilmalarni virtual muhitda yaratish, sinovdan o'tkazish va ularda turli xil ilovalarni ishga tushirib, tahlil qilish imkonini beruvchi kuchli, tezkor va kengaytirilgan imkoniyatlarga ega emulyator platformasidir. U ayniqsa dasturchilar, testchilar va xavfsizlik tahlilchilari tomonidan keng qo'llaniladi. Genymotion yordamida foydalanuvchi Android operatsion tizimining istalgan versiyasida, turli xil ekran o'lchamlariga ega bo'lgan telefon yoki planshet modellarini bir necha soniyada yaratib, ularda real qurilmadek ishlovchi muhitda testlar o'tkazishi mumkin.

Genymotion — bu oddiy emulyator emas. U zamonaviy virtualizatsiya texnologiyalariga, xususan VirtualBox asosida qurilgan bo'lib, undan foydalanish orqali Android qurilmaning to'liq holatini — GPS, kamera, sensorlar, internet ulanishi, SIM karta holati, batareya darajasi va boshqa ko'plab tizim komponentlarini soxta tarzda sozlash mumkin. Bu imkoniyatlar orqali siz, masalan, ilovaning faqat GPS mavjud bo'lgan holatda qanday ishlashini, yoki internet ulanmagan holatda qanday xatolik berishini sinab ko'rishingiz mumkin.

Genymotion xavfsizlik tahlilida ham juda keng foydali. Chunki real qurilmalarda test qilish xavfli yoki vaqt talab etadigan jarayon bo'lishi mumkin, emulyatorda esa

bu juda tez, xavfsiz va qayta-qayta sinab ko‘rish mumkin bo‘lgan muhitda amalga oshiriladi. Masalan, siz APK faylni Genymotion ichiga o‘rnatib, uni tarmoq vositalari — masalan, Burp Suite, HTTP Toolkit, yoki Reqable bilan bog‘lab, ilovaning yuborayotgan va olayotgan trafigini to‘liq tahlil qilishingiz mumkin. Shuningdek, siz bu emulyatorga Frida, Objection, Drozer, yoki boshqa dinamika tahlil vositalarini o‘rnatib, real vaqt rejimida ilovani sinashingiz ham mumkin bo‘ladi.

Genymotion foydalanuvchiga Android qurilmasini to‘liq nazorat qilish imkonini beradi. Ilova qanday ruxsatlar so‘rayapti, qanday faolliklar ishga tushyapti, qanday komponentlar chaqirilyapti — bularning barchasini siz Genymotion emulyatori orqali kuzatishingiz mumkin. U engil ishlaydi, ko‘pchilik real qurilmalarga nisbatan tezroq harakat qiladi va aynan xavfsizlik testlari uchun zaruriy sharoitni tezda yaratadi.

Yana bir katta afzalligi — Genymotion bulutdagi (cloud-based) versiyasiga ham ega. Bu degani, siz kuchli kompyuterga ega bo‘lmasangiz ham, Genymotion Cloud orqali o‘zingiz xohlagan Android muhitini masofaviy serverda yaratib, unga web brauzer orqali ulanishingiz va testlar o‘tkazishingiz mumkin. Bu ayniqsa ko‘p foydalanuvchili yoki murakkab infrastrukturali ilovalarni test qilishda qulaylik yaratadi.

## **Nmap**

Nmap (Network Mapper) — bu tarmoq infratuzilmasini skanerlash, xizmatlar va portlarni aniqlash, hamda ularning xavfsizlik holatini tahlil qilish uchun dunyoda eng ko‘p qo‘llaniladigan, kuchli va moslashuvchan ochiq manbali tarmoq skanerlash vositasidir. Asosan tizim administratorlari, xavfsizlik tahlilchilari va pentesterlar tomonidan ishlatiladi. Mobil xavfsizlik tahlilida esa Nmap orqali mobil ilova ulanishga harakat qilayotgan serverlar, API backend’lar, va ular ishlayotgan xizmatlarning zaifliklari aniqlanadi.



Nmap vositasi TCP/IP protokollari asosida ishlaydi. U istalgan maqsadli IP manzil yoki domen nomiga ulanib, u yerdagi ochiq portlar, faol xizmatlar (services), ular ishlayotgan versiyalar, va hatto operatsion tizim turlari haqida ham ma'lumot olishga harakat qiladi. Bu ma'lumotlar orqali sizga ilova ulanayotgan server qanday texnologiyalardan foydalanayotgani, qaysi portlar himoyalanmagan, va qaysi xizmatlar zaif bo'lishi mumkinligini aniqlash imkonini beradi.

Misol uchun, siz quyidagi kabi oddiy buyruq orqali maqsadli serverda ochiq portlarni tekshirishingiz mumkin:

```
nmap api.example.com
```

Agar siz xizmat versiyalarini ham aniqlamoqchi bo'lsangiz:

```
nmap -sV api.example.com
```

Bu holatda sizga API serverda ochiq bo'lgan portlar (masalan, 80, 443, 22, 3306) va ular orqali ishlayotgan xizmatlar (Apache, nginx, SSH, MySQL, va h.k.) haqida aniq ma'lumot chiqadi.

Nmap'ning kuchli tomonlaridan biri — bu skriptlash tizimi, ya'ni NSE (Nmap Scripting Engine). Bu tizim orqali siz avtomatik tarzda zaifliklarni aniqlovchi skriptlarni ishga tushirishingiz mumkin. Masalan, siz `--script=vuln` bayrog'i orqali serverdagi mashhur zaifliklarga (SQL injection, Heartbleed, Shellshock, va boshqalar) tekshiruv o'tkazishingiz mumkin:

```
nmap --script vuln api.example.com
```

Mobil xavfsizlik kontekstida Nmap quyidagi ishlarda qo'l keladi:

Ilova ulanishga harakat qilayotgan API serverlar tahlili.

Ichki tarmoqlarda (masalan, Wi-Fi orqali ulanayotgan mobil qurilma) boshqa qurilmalarni aniqlash va ularni skanerlash.

Serverda noto'g'ri sozlangan xizmatlarni aniqlash (masalan, himoyalanmagan MongoDB yoki Redis).

VPN yoki proxy orqali ishlayotgan trafikni tahlil qilish va orqa tarafdagi xizmatlarni aniqlash.

Nmap ko‘plab platformalarda ishlaydi (Linux, Windows, macOS), va undan grafik interfeysda foydalanmoqchi bo‘lganlar uchun Zenmap nomli GUI versiyasi ham mavjud. Biroq aksariyat professional foydalanuvchilar komandali satr interfeysi orqali undan foydalanadi, chunki bu ko‘proq moslashuvchanlik va avtomatlashtirish imkonini beradi.

Xulosa qilib aytganda, Nmap — bu tarmoq darajasidagi xavfsizlik tahlilining yuragi bo‘lib, mobil ilovalar ulanish qilayotgan infratuzilmaning qanday holatda ekanini, qaysi xizmatlar ochiq, qaysi portlar zaif, va qaysi komponentlar ekspluatatsiya uchun ochiq ekanini aniqlashda muhim rol o‘ynaydi. Shu sababli, har bir xavfsizlik tahlilchisi yoki mobil ilova tahlilchisi o‘z ishi davomida Nmap vositasini bilishi, undan samarali foydalanishi va uni avtomatlashtirish ssenariylarida qo‘llashi zarur.

### **Dirb / Dirbuster / FFUF**

Dirb, DirBuster, va FFUF — bular veb ilovalar va API serverlarida yashirin yoki noto‘g‘ri himoyalangan katalog va fayllarni aniqlash uchun ishlatiladigan katalog brutforce va kontent aniqlash vositalaridir. Ular xavfsizlik tahlilchilari va pentesterlar tomonidan veb-serverning ko‘zga ko‘rinmaydigan joylarida yashiringan resurslarni ochish, foydalanuvchi uchun mo‘ljallanmagan admin panel, zahira fayllar, API endpointlar yoki konfiguratsiya fayllarini aniqlash uchun keng qo‘llaniladi.

Dirb — bu terminalda ishlaydigan eng oddiy va eng qadimgi vositalardan biri bo‘lib, u URL manzilga nisbatan berilgan wordlistdagi katalog nomlarini birma-bir sinab ko‘radi. Har bir sinov so‘rov yuboradi va agar serverdan 200 OK, 301/302 redirect yoki 403 forbidden kabi ijobiy javob kelsa — bu katalog yoki fayl mavjud degani. Dirb juda sodda ishlaydi, lekin kuchli recursive (rekursiv) qidiruvga ega emas, shuning uchun u faqat birinchi darajali kataloglarni yaxshi ochadi.

```
dirb https://example.com /usr/share/wordlists/dirb/common.txt
```

Bu buyruq orqali Dirb common.txt faylidagi katalog nomlarini https://example.com saytiga birma-bir so‘rov qilib sinaydi.

DirBuster esa — bu Java asosidagi grafik interfeysli vosita bo‘lib, OWASP tomonidan ishlab chiqilgan. U Dirb’ga qaraganda ancha qulayroq interfeysga ega va ko‘proq sozlama variantlarini taklif etadi. DirBuster sizga katalog qidiruvini depth (chuqurlik) bo‘yicha sozlash, parallel ip so‘rovlar sonini belgilash, maxsus fayl

kengaytmalarini sinash (masalan, .php, .bak, .zip, .old) va turli javob kodlarini qanday baholashni tanlash imkonini beradi. Bu vosita foydalanuvchiga jarayonni vizual ko‘rinishda kuzatish imkonini beradi, shu sababli GUI foydalanuvchilari uchun ayni muddao hisoblanadi.

DirBuster yordamida siz nafaqat kataloglar, balki .php, .html, .js, .sql, .conf kabi potentsial xavfli fayllarni aniqlashingiz mumkin. Bu ayniqsa noto‘g‘ri joylashtirilgan .env, .git, yoki zahira konfiguratsiya fayllarini topishda foyda beradi.

FFUF (Fuzz Faster U Fool) esa zamonaviy, C-style sintaksisga ega, eng tez ishlaydigan va ko‘p funksiyali katalog va kontent fuzzing vositasidir. U Go tilida yozilgan va parallel ishga tushirishda juda samarali. FFUF orqali siz nafaqat katalog va fayl nomlarini qidirishingiz, balki URL parametrlarini, POST body qismlarini, yoki hatto cookie va header’lardagi qiymatlarni ham brutforce qilishingiz mumkin. Bu vosita ko‘proq zamonaviy tahlilchilarga mo‘ljallangan bo‘lib, juda ko‘p sozlamalarni qo‘llab-quvvatlaydi.

```
ffuf -u https://example.com/FUZZ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

Bu yerda FUZZ — brutforce qilinadigan joy. FFUF ushbu URL’da FUZZ o‘rniga so‘zlar ro‘yxatidagi har bir qiymatni qo‘yadi va javob holatini tahlil qiladi. Siz status code bo‘yicha filtr qo‘yishingiz, javob uzunligini yoki vaqtini nazorat qilishingiz ham mumkin.

FFUF’ning yana bir kuchli tomoni — u JSON formatida natijalarni saqlab qoladi va ular ustida avtomatik tahlil yoki eksport qilish imkoniyatini beradi. Shu sababli uni skriptlash, CI/CD jarayonlariga integratsiya qilish oson bo‘ladi.

## WAFW00F

WAFW00F — bu veb ilovalar oldida turgan WAF (Web Application Firewall) tizimini aniqlash va uning turini aniqlab berish uchun mo‘ljallangan kuchli, avtomatik va ochiq manbali vositadir. U xavfsizlik tahlilchilari, pentesterlar, va bug bounty mutaxassislari tomonidan server oldida qanday himoya tizimi o‘rnatilganini tushunish va u orqali qanday filtrlar ishlatilayotganini bilish uchun ishlatiladi.

WAF (Web Application Firewall) — bu veb ilovalarning trafikini himoya qilish uchun ishlatiladigan maxsus qatlam bo‘lib, u kiruvchi HTTP/HTTPS so‘rovlarni filtrlaydi, analiz qiladi va xavfli deb topilganlarini to‘sib qo‘yadi. Masalan, SQL

injection, XSS, SSRF kabi hujumlarni avval WAF to'sadi, keyin trafik ilovaga yetib boradi. Shuning uchun, biron bir veb ilovani tahlil qilishdan oldin uning WAF bilan himoyalangan yoki himoyalanganmaganini aniqlab olish — bu muhim bosqichlardan biridir. Aynan shu vazifani WAFW00F bajaradi.

WAFW00F foydalanuvchidan birgina veb manzilni qabul qiladi va avtomatik tarzda WAF mavjudligini aniqlash uchun bir nechta test so'rovlar yuboradi. Ular orasida noto'g'ri so'rovlar, zaiflikka o'xshagan URL parametrlar, yoki maxsus HTTP header'lar mavjud. WAF bu so'rovlarni qanday qabul qilishiga qarab, WAFW00F javobni tahlil qiladi va u orqali WAF tizimi mavjudligini, uning ishlab chiqaruvchisini (vendor), va ba'zida versiyasini ham aniqlab beradi.

Misol uchun, siz quyidagi buyruqni ishga tushirishingiz mumkin:

```
wafw00f https://example.com
```

Natijada WAFW00F sizga quyidagi kabi ma'lumotlarni beradi:

The site https://example.com is behind a Web Application Firewall:

Cloudflare (Cloudflare Inc.)

Yoki boshqa mashhur WAF'lar: Akamai, F5 BIG-IP, AWS WAF, Imperva Incapsula, Barracuda, ModSecurity, DenyAll, NetScaler, Fortinet, va boshqalar aniqlanishi mumkin.

WAFW00F Python tilida yozilgan va terminal orqali ishlaydi. U avtomatik ravishda yuboradigan test so'rovlar serverning WAF himoyasiga qanday javob berishini tahlil qiladi: masalan, so'rov bloklanadimi, 403 qaytaradimi, redirect qilinadimi yoki boshqa maxsus header'lar paydo bo'ladimi — shular asosida WAF mavjudmi yo'qmi degan xulosa chiqaradi.

Bu vosita ayniqsa testdan oldingi "reconnaissance" (razvedka) bosqichida foydali bo'ladi. Chunki, agar siz hujum qilishni rejalashtirayotgan sayt WAF bilan himoyalangan bo'lsa, siz hujum texnikangizni WAF ni chetlab o'tishga yo'naltirishingiz kerak bo'ladi. Ba'zida esa noto'g'ri konfiguratsiya qilingan WAF'lar orqali zaifliklar ochiladi — bu esa hujum imkoniyatini kengaytiradi.

Xulosa qilib aytganda, WAFW00F — bu veb ilova oldida qanday WAF himoyasi turganini aniqlab beruvchi, oddiy, tezkor va foydali vosita bo'lib, siz tahlil

qilayotgan server qanday xavfsizlik tizimlari bilan himoyalanganini bilish orqali test strategiyangizni aniqroq belgilashingiz mumkin. Shu sababli, har bir xavfsizlik tahlilchisi yoki pentester ushbu vositani o'zining razvedka bosqichida ishlatishi lozim.

### **Janus Vulnerability Tester**

Janus Vulnerability Tester — bu Android ilovalaridagi Janus zaifligini aniqlash uchun mo'ljallangan maxsus test vositasidir. Janus zaifligi ilk bor 2017-yilda Google tomonidan e'lon qilingan bo'lib, bu zaiflik Android 5.0 dan 7.0 gacha bo'lgan versiyalardagi APK va DEX fayllarni qayta tuzmasdan, asl imzo o'zgarmagan holda ilovaga zararli kod qo'shish imkonini bergan juda xavfli dizayn xatoligidir. Aynan shu zaiflikni aniqlash va undan himoyalangan ilovalarni test qilish uchun Janus Vulnerability Tester ishlab chiqilgan.

Janus zaifligi shundan iboratki: APK fayl ZIP formatida saqlanadi va Android tizimi APK fayl ichidagi fayllarning imzosini faqat DEX faylga qarab tekshiradi. Biroq ZIP strukturasining o'ziga xosligi sababli, zararli kiritilgan DEX yoki soxta fayl APK oxiriga qo'shilsa ham, tizim uni noto'g'ri tahlil qilib, asl imzo o'zgarmagan deb o'ylaydi. Bu holatda ilovaga zararli kod qo'shish mumkin, va u foydalanuvchining ruxsatisiz ishlay boshlaydi — bu esa real tahdid hisoblanadi.

Janus Vulnerability Tester ilovaning APK faylini yoki qurilmaga o'rnatilgan ilovani tekshiradi va:

- APK faylining strukturasini tahlil qiladi
- Fayl ichidagi imzo turini aniqlaydi (v1 yoki v2)
- Imzo bilan real kontent o'rtasidagi tafovutlarni tekshiradi
- Ilovaning Janus zaifligiga ta'sirchanligini bildiradi

Bu vosita orqali siz ilovaning APK fayli noto'g'ri tuzilganmi, unga zararli kod qo'shish imkoni bormi, va tizim uni aniqlay oladimi — bularning barchasini tahlil qilishingiz mumkin.

Janus zaifligi faqat Android 5.0–7.0 versiyalariga taalluqli bo‘lib, undan keyingi Android versiyalarda APK Signature Scheme v2 (va v3) orqali bu zaiflik bartaraf etilgan. Shunga qaramay, bugungi kunda ham ko‘plab foydalanuvchilar eski Android qurilmalaridan foydalanayotgani sababli, ushbu zaiflik hali ham aktual bo‘lib qolmoqda.

Janus Vulnerability Tester yordamida siz APK faylni offline tarzda yoki qurilmadagi ilovani to‘g‘ridan-to‘g‘ri tekshirishingiz mumkin. Bu ayniqsa .apk fayllarni “mod” qilgan, qayta imzolagan yoki zararli yuklama sifatida tarqatilayotgan versiyalarni tahlil qilishda foydalidir.

Xulosa qilib aytganda, Janus Vulnerability Tester — bu APK fayl tuzilmasi va imzo tizimi asosida Janus zaifligi mavjud yoki mavjud emasligini aniqlashga xizmat qiladigan yengil, foydali va maxsus vositadir. Har bir mobil xavfsizlik tahlilchisi ilovalarni tekshirayotganida aynan shu zaiflikni ham nazardan chetda qoldirmasligi lozim, ayniqsa agar maqsadli foydalanuvchilar eski Android versiyalaridan foydalanayotgan bo‘lsa. Janus zaifligi — foydalanuvchi hech narsa sezmasdan ilova yangilanishi orqali zararli kodni tizimga kiritish imkonini beruvchi xavfli mexanizm bo‘lgan, va buni aniqlashda bu vosita eng aniq yechim hisoblanadi. Yuklab olish uchun havola(<https://github.com/h0tak88r/jnx>).

## **iOS xavfsizlik vositalari**

### **oTool**

oTool — bu iOS ilovalari (IPA fayllari) va ularning binari fayllari ustida statik tahlil o‘tkazish uchun mo‘ljallangan, kuchli va samarali terminal vositasi bo‘lib, ayniqsa iOS reverse engineering, binary inspection, va xavfsizlik tahlilida keng qo‘llaniladi. U Apple'ning o‘zining otool nomli sistemaviy vositasi bo‘lib, Mach-O formatidagi fayllarni (ya’ni iOS va macOS ilovalarining binarilari) chuqur tahlil qilishga mo‘ljallangan. Android dunyosida qanday qilib readelf yoki objdump muhim rol o‘ynasa, iOS ekotizimida ham oTool xuddi shunday darajadagi asosiy vositadir.

oTool yordamida siz quyidagi ma’lumotlarni ko‘rishingiz mumkin:

- Binari fayl ishlatadigan kutubxonalar (libraries)
- Ilova qanday arxitekturani qo'llab-quvvatlaydi (arm64, armv7, h.k.)
- Binarida qanday segmentlar va seksiyalar mavjud
- Class, selector, symbol va method lar
- Ilovaning entitlements, ya'ni tizim ruxsatnomalari
- Binarida ASLR, PIE, stack canary, va boshqa himoya mexanizmlari mavjudmi

Misol uchun, siz quyidagi buyruq orqali biror ilovaning binarida ishlatilayotgan kutubxonalarni ko'rishingiz mumkin:

```
otool -L MyApp
```

Bu sizga ilova ishlatayotgan barcha .dylib (Dynamic Library) fayllarni ro'yxat qilib beradi. Bu juda muhim, chunki ba'zida zararli yoki nojo'ya kutubxonalar orqali ilovaga ekspluatatsiya yo'llari ochilishi mumkin.

```
otool -Vt MyApp
```

Bu orqali siz ilovadagi barcha function symbol va ularning xotira manzillarini ko'rishingiz mumkin, ya'ni bu kodni disassembling darajasida tahlil qilish imkonini beradi. Shuningdek, quyidagilar ham tahlil qilish mumkin:

```
otool -arch arm64 -Iv MyApp
```

Bu esa ilovaning Info.plist ichidagi entitlements (masalan: com.apple.security.application-groups, get-task-allow, aps-environment) haqida ma'lumot beradi. Aynan get-task-allow qiymati true bo'lsa, bu ilovada debuggable (ya'ni Frida orqali bog'lanish mumkin) ekanligini bildiradi.

iOS xavfsizlik tahlilchilari oTool vositasidan ilovaning qanday kutubxonalar bilan bog'langanini, qanday tizimga kirish ruxsatlari mavjudligini va qanday arxitektura ishlatilayotganini aniqlash uchun foydalanishadi. Ayniqsa, ilovaning PIE (Position Independent Executable) yoki ASLR (Address Space Layout Randomization) kabi himoyaga ega yoki yo'qligini tekshirish orqali siz u qanchalik ekspluatatsiyaga tayyorligini baholashingiz mumkin.

oTool — bu statik tahlil vositasi bo'lib, siz .ipa faylni ochib, uning ichidagi Payload/MyApp.app/MyApp binar faylni alohida tahlil qilishingiz mumkin. Bu kod

darajasiga tushmasdan, lekin chuqur struktura va himoya mexanizmlarini ko‘rish imkonini beradi.

Xulosa qilib aytganda, oTool — bu iOS ilovalarning binar tuzilmasini chuqur tahlil qilish, ularning xavfsizlik konfiguratsiyasini baholash va ilova qanday kutubxonalarga, arxitekturaga va ruxsatlarga ega ekanini aniqlashda ishlatiladigan muhim vosita. Har bir iOS tahlilchisi, ayniqsa jailbreaksiz tahlil olib borayotganlar, oTool vositasini yaxshi bilishi va undan samarali foydalana olishi kerak. Bu vosita sizga kodni disassemble qilmasdan turib, zaifliklarni aniqlash uchun eshik ochib beradi.

### **SSL Kill Switch**

SSL Kill Switch — bu iOS va ba’zida Android ilovalari uchun yaratilgan, SSL Pinning mexanizmini chetlab o‘tishga mo‘ljallangan, kuchli va samarali tahlil va bypass vositasidir. U xavfsizlik tahlilchilari, pentesterlar, va tarmoq trafigini tahlil qiluvchilar tomonidan HTTPS trafigini tahlil qilish imkonini berish uchun ishlatiladi. Aynan SSL Pinning mavjud bo‘lgan ilovalarda oddiy MITM (man-in-the-middle) vositalar, masalan, Burp Suite yoki HTTP Toolkit yordamida trafikni kuzatishning imkoni bo‘lmaydi — chunki ilova faqat o‘zining ishonchli sertifikatiga ishonadi. SSL Kill Switch esa bu ishonch mexanizmini yo‘q qiladi.

SSL Kill Switch ilova ichida ishlatilayotgan SSL/TLS bog‘lanish funksiyalarini hook qilish orqali ularni o‘zgartiradi. Bu vosita odatda Frida, Cydia Substrate, yoki Theos kabi tizimlar yordamida iOS tizimida dynamic library sifatida qo‘shiladi va ilova ishga tushganda SSL Pinning kodini bekor qiladi. Ilova bu holatda endi foydalanuvchining ishonchli sertifikatlar do‘koniga asoslanadi, ya’ni MITM proksi vositalar orqali HTTPS trafigi tahlil qilinadigan bo‘ladi.

Masalan, ilova NSURLSession, NSURLConnection, CFNetwork, yoki SecureTransport kabi Apple’ning rasmiy SSL kutubxonalaridan foydalansa, SSL Kill Switch ularni aniqlab, SSL certificate validation qismini o‘chirib qo‘yadi. Natijada ilova trafikni tahlil qilishga to‘sqinlik qilmaydi.

SSL Kill Switch asosan quyidagi holatlarda ishlatiladi:

- Ilova HTTPS orqali ma’lumot yuborayotganini ko‘rish kerak bo‘lsa
- JWT, cookie yoki session ID kabi maxfiy tokenlarni tarmoqqa chiqarib olish zarur bo‘lsa



- API endpointlar qanday ishlashini aniqlash va test qilish kerak bo‘lsa
- Ilovaning real vaqtli server bilan muloqotini tahlil qilish uchun

SSL Kill Switch 2 — bu ushbu vositaning eng mashhur versiyasi bo‘lib, u iOS 12 gacha bo‘lgan versiyalarni qo‘llab-quvvatlaydi. U Cydia Substrate orqali ishlaydi va iOS qurilmaga jailbreak qilingan bo‘lishi kerak. Tizimga .dylib faylni joylashtirish orqali siz ilovani qayta imzolamasdan, uning SSL himoyasini yo‘q qilishingiz mumkin.

Qanday ishlatiladi (jailbroken iOS’da misol):

- SSL Kill Switch .dylib faylni /Library/MobileSubstrate/DynamicLibraries/ papkaga nusxa ko‘chirasiz
- Qurilmani qayta ishga tushirasiz yoki Substrate’ni respring qilasiz
- SSL Pinning mavjud bo‘lgan ilova endi trafikni tahlil qilishga ruxsat beradi
- Burp Suite yoki boshqa proksi vosita orqali HTTPS so‘rovlar ko‘rinadigan bo‘ladi

Shuningdek, Android versiyalari uchun Frida orqali o‘xshash hook-lar ishlatiladi (ssl\_unpinning.js, yoki Frida-ssl-pinning-bypass skriptlari), ammo "SSL Kill Switch" nomi odatda iOS platformasi bilan bog‘liq.

Xavfsizlik tahlilchilari uchun SSL Kill Switch shunchaki bypass vositasi emas — u ilovaning "real dunyoda" yuborayotgan ma’lumotlarini tahlil qilishga eshik ochuvchi vositadir. Ko‘plab bank ilovalari, sog‘liqni saqlash ilovalari yoki shaxsiy ma’lumotlar bilan ishlovchi tizimlar HTTPS bilan yaxshi himoyalangan bo‘lsa ham, aynan SSL Pinning mavjud bo‘lsa, ular tahlil qilib bo‘lmaydi. Shuning uchun, SSL Kill Switch vositasi aynan bunday holatlarni o‘rganish, zaifliklarni aniqlash va xavfsizlikni baholashda zarur vositadir.

Xulosa qilib aytganda, SSL Kill Switch — bu iOS ilovalardagi SSL Pinning mexanizmini avtomatik tarzda o‘chirib qo‘yishga mo‘ljallangan kuchli bypass vositasi bo‘lib, mobil trafikni to‘liq tahlil qilishni imkonini beradi. U Frida, jailbreak va dynamic hooking texnologiyalarini birlashtirib, ilovani qayta tuzmasdan ichki logikani o‘zgartiradi. Har bir mobil xavfsizlik tahlilchisi ushbu vositani bilishi va undan ehtiyotkorlik bilan foydalanishni o‘ziga majbur deb bilishi kerak.

## Keychain dumper

Keychain Dumper — bu iOS ilovalarida va qurilmalarida saqlanayotgan Keychain ma'lumotlarini ko'rib chiqish va tahlil qilish imkonini beruvchi ochiq manbali vositadir. iOS tizimida Keychain — bu ilovalar, tizim xizmatlari va foydalanuvchilar uchun maxfiy ma'lumotlar (parollar, tokenlar, sertifikatlar, Wi-Fi kalitlari) saqlanadigan xavfsiz joy hisoblanadi. Apple ushbu Keychain'ni kuchli kriptografik asosda himoya qiladi, ammo noto'g'ri konfiguratsiya yoki ruxsatlar tufayli bu ma'lumotlarga kirish imkoni paydo bo'lishi mumkin. Keychain Dumper aynan shunday zaifliklarni aniqlash uchun ishlatiladi.

Bu vosita jailbreak qilingan iOS qurilmalarda ishlaydi. Uni terminal (CLI) orqali ishga tushirasiz va u qurilmadagi barcha Keychain yozuvlarni o'qib beradi. Agar ilova Keychain'dan foydalanayotgan bo'lsa, masalan:

- Kirish tokenlarini (JWT, access\_token)
- Parollarni
- Sertifikat private keylarini
- VPN profillarini
- Wi-Fi parollarini

saqlayotgan bo'lsa, Keychain Dumper ularni o'qib berishi mumkin — agar ular noto'g'ri ruxsatlar bilan saqlangan bo'lsa.

Misol uchun, ilova Keychain yozuvini `kSecAttrAccessibleAlways` yoki `kSecAttrAccessibleAfterFirstUnlock` sifatida saqlasa — bu yozuv boshqa ilovalar (yoki tahlil vositalari) tomonidan o'qilishi mumkin. Keychain Dumper aynan shu zaifliklarni ko'rsatadi va sizga qanday yozuvlar zaif turibdi, ularda qanday ruxsatlar (Access Groups) borligini ko'rsatib beradi. Keychain Dumper foydali bo'lgan holatlar:

- Ilova tokenlarni Keychain'da qanday ruxsat bilan saqlayotganini tekshirish
- Maxfiy ma'lumotlar boshqa ilovalar orqali o'qilishi mumkinligini aniqlash
- Mobil ilovaning shaxsiy parollarni Keychain'da noto'g'ri saqlayotganini aniqlash
- iOS tizimidagi umumiy xavfsizlik darajasini tahlil qilish

Oddiy foydalanish:

```
./keychain_dumper
```

Natijada sizga quyidagilar haqida to'liq ma'lumot chiqadi:

- Account
- Service
- Access Group
- Entitlements
- Yozuv turi (Generic password, Internet password)
- Saqlangan qiymat (agar o'qish mumkin bo'lsa)

Masalan, siz quyidagi yozuvni ko'rishingiz mumkin:

Service: com.example.app.token

Account: access\_token

Entitlements: <missing>

Group: com.example.app

Value: eyJhbGciOi...

Bu — JWT token Keychainda noto'g'ri ruxsat bilan saqlanayotganini bildiradi va bu zaiflik hisoblanadi.

Shuningdek, Keychain Dumper sizga yozuvlar orasida inter-app communication orqali o'qilishi mumkin bo'lgan ma'lumotlarni ham ko'rsatadi, bu esa xavfli holat bo'lishi mumkin (masalan, boshqa ilova yoki zararli komponent bu ma'lumotlarni o'g'irlab oladi).

Xulosa qilib aytganda, Keychain Dumper — bu iOS tizimida Keychain xavfsizligini tahlil qilish, noto'g'ri saqlangan maxfiy ma'lumotlarni aniqlash va ilovaning ruxsat konfiguratsiyasini tekshirish uchun eng muhim vositalardan biridir. Har qanday iOS xavfsizlik testida, ayniqsa maxfiy ma'lumotlar bilan ishlaydigan ilovalarda, bu vosita orqali Keychain yozuvlarni to'liq tahlil qilish lozim. Dasturchilar esa Keychain'ga yozuv qo'yayotganda ruxsatlarni to'g'ri belgilaganligini doim tekshirishlari kerak.

## LLDB

**LLDB** — bu iOS va macOS ilovalari uchun mo'ljallangan kuchli va zamonaviy debugger vositasi bo'lib, u dastur bajarilishi jarayonida kodni to'xtatish, o'zgaruvchilarni ko'rish, xotirani tahlil qilish, va funksiyalarni real vaqt rejimida kuzatish imkonini beradi. U Apple'ning rasmiy Xcode muhitiga to'liq integratsiyalashgan va aynan iOS ilovalarni chuqur tahlil qilishda muhim rol o'ynaydi.

LLDB yordamida ilovadagi break point'lar qo'yish, o'zgaruvchilar qiymatini o'zgartirish, runtime'da metod chaqirish, funksiyalarni kuzatish, registrlar va stack qiymatlarini tahlil qilish mumkin. Ayniqsa, xavfsizlik tahlilchilari uni anti-debug texnikalarini tahlil qilish, ilova himoyasini chetlab o'tish yoki dinamik ekspluatatsiya qilish uchun keng qo'llashadi. LLDB orqali siz Ilovaga breakpoint qo'yib to'xtatish Funksiya ichidagi kodni real vaqt kuzatish, Xotira (memory)dagi ma'lumotlarni o'qish va o'zgartirish, Stack va registr qiymatlarini ko'rish, Funksiya va obyektlar ustida tahlil qilish imkoniyatiga ega bo'lasiz

Masalan, `expr` buyrug'i yordamida runtime'da biror o'zgaruvchining qiymatini ko'rish yoki o'zgartirish mumkin:

```
expr myVar = 123
```

Yoki ilova ishga tushganida `viewDidLoad` funksiyasida to'xtash uchun:

```
breakpoint set --name viewDidLoad
```

LLDB terminal asosida ishlaydi va odatda Xcode, debugserver, yoki Frida kabi vositalar bilan birga qo'llaniladi. iOS ilovalarda tahlil olib borilayotganda LLDB orqali ilovaning xavfsizlik darajasi, himoya mexanizmlari, va zaifliklari chuqur o'rganiladi. Shu sababli LLDB har bir mobil xavfsizlik tahlilchisining asboblari qutisida bo'lishi shart.

## Clutch

**Clutch** — bu jailbreak qilingan iOS qurilmalarda o'rnatilgan ilovalarni decrypted (shifri ochilgan) holatda olish imkonini beradigan kuchli vosita hisoblanadi. App Store orqali yuklab olingan iOS ilovalar Apple tomonidan FairPlay DRM bilan shifrlanadi, va bu shifrlangan .ipa fayllar tahlilchi uchun befoyda bo'ladi. Clutch

aynan shu muammoni hal qiladi — u ilova ishga tushgan va xotiraga decrypt bo‘lgan holatini olib, undan toza .ipa fayl yaratadi.

Oddiy decompile yoki unpack orqali App Store dan yuklab olingan ilovaning kodini tahlil qilib bo‘lmaydi, chunki binary fayl shifrlangan bo‘ladi. Clutch esa ilovani ishga tushiradi, u RAM’da ochiladi va shu paytda undan to‘liq ishlaydigan, decrypted .ipa fayl hosil qiladi. Bu jarayon orqali siz ilovani reverse engineering (teskari tahlil) qilish, kodni tahlil qilish, class va metodlarni ko‘rish imkoniyatiga ega bo‘lasiz.

Clutch vositasidan foydalanish uchun qurilmada jailbreak bo‘lishi kerak va undan odatda terminal orqali foydalaniladi.

Masalan, quyidagi buyruq bilan qurilmadagi barcha ilovalarni ko‘rish mumkin:

```
clutch -i
```

Va ma’lum bir ilovani decrypted qilish uchun:

```
clutch -d com.example.app
```

Natijada decrypted bo‘lgan .ipa fayl /var/tmp/clutch ichida hosil bo‘ladi va siz uni Ghidra, Hopper, class-dump, yoki oTool orqali tahlil qilishingiz mumkin.

Clutch asosan iOS tahlilchilari, xavfsizlik mutaxassislari, va eksploatatsiya tahlilchilari tomonidan quyidagi maqsadlarda qo‘llaniladi. App Store dagi ilovaning ichki logikasini tushunish. Obfuskatsiyalangan kodlarni tahlil qilish. Ilova qanday kutubxonalar va APIlardan foydalanayotganini aniqlash. Parollar, tokenlar, API endpointlar yoki zaifliklarni aniqlash

Clutch 2.x va undan keyingi versiyalari iOS 11–14 versiyalargacha ishlaydi, lekin ba’zi holatlarda yangi DRM mexanizmlari qo‘llangan bo‘lsa, qo‘shimcha bypass usullari talab qilinishi mumkin.

Xulosa qilib aytganda, Clutch — bu iOS ilovalarning shifrlangan binari fayllarini ochib, ular ustida to‘liq tahlil qilish imkonini beradigan muhim vosita. U dasturchilar

yoki xavfsizlik mutaxassislari uchun iOS ilova xavfsizligini chuqur o'rganishda zaruriy vositalardan biridir.

### **Class-dump-z**

class-dump-z — bu iOS ilovalarning binar fayllaridan Objective-C'da yozilgan class, method, protocol, property kabi ma'lumotlarni ajratib olishga mo'ljallangan kuchli teskari muhandislik (reverse engineering) vositasidir. U ayniqsa decrypted (shifri ochilgan) ilovalar ustida ishlaydi va sizga ilovaning ichki tuzilmasini to'liq tushunishga yordam beradi. class-dump-z — bu mashhur class-dump vositasining takomillashtirilgan va optimallashtirilgan varianti bo'lib, u yangi iOS versiyalarida, ASLR bilan, va obfuskatsiyalangan ilovalarda ham samarali ishlaydi.

Ilova .ipa faylidagi asosiy binari fayl (odatda MyApp) aslida Mach-O formatda bo'ladi. class-dump-z ushbu binarni tahlil qiladi va ilova ichida ishlatilgan barcha class va method imzolarini .h fayl ko'rinishida chiqarib beradi. Bu tahlilchi uchun ilova qanday tuzilganini, qaysi funksiya nimaga xizmat qilishini va nimani hook qilish yoki kuzatish kerakligini tushunishga yordam beradi.

Masalan, quyidagi buyruq orqali siz ilovaning barcha classlarini chiqarib olishingiz mumkin:

```
class-dump-z -H MyApp -o headers/
```

Natijada siz headers/ papkasida ilovadagi barcha .h fayllarni olasiz. Bu fayllar ilovaning interface qismini ko'rsatadi va siz kodni aslida ko'rmagan bo'lsangiz ham, u qanday tuzilganini bilib olasiz. class-dump-z orqali siz quyidagi ma'lumotlarni ko'rishingiz mumkin.

- Objective-C klass nomlari
- Klasslar qanday superclassdan meros olgan
- Har bir klassdagi metodlar va propertylar
- Protocollar va ularning implementatsiyalari
- Kutilayotgan method parametrlari va turlari

Bu vosita ayniqsa Frida, LLDB, Ghidra, yoki Hopper bilan birgalikda ishlatilganda kuchli natija beradi. Siz ilova ichida `-[LoginViewController submitPressed:]` degan method borligini bilib olasiz va Frida orqali uni hook qilasiz yoki LLDB orqali breakpoint qo'yishingiz mumkin.

`class-dump-z` odatda quyidagi holatlarda ishlatiladi. Ilovaning qanday klass va metodlardan iboratligini aniqlash. Frida script yozishdan oldin kerakli method nomlarini toppish. Ilovadagi obfuskatsiyalangan kod tuzilmasini tahlil qilish. Sensitive funksiyalar joylashgan joylarni aniqlash. API endpointlar, auth, token, login kabi funksiyalarni ajratib olish.

Shuni unutmaslik kerakki, `class-dump-z` faqat decrypted binari fayllarda ishlaydi. Ya'ni, App Store'dan yuklab olingan original .ipa faylda ishlamaydi — siz avval uni Clutch yoki Frida orqali decrypt qilib olishingiz kerak bo'ladi.

Xulosa qilib aytganda, `class-dump-z` — bu iOS ilovaning ichki arxitekturasini chuqur o'rganish, uning class va method tuzilmasini ochib berish, va xavfsizlik tahlilida foydali joylarni tezda aniqlash uchun muhim vositadir. Har bir mobil xavfsizlik tahlilchisi va teskari muhandislik bilan shug'ullanuvchi mutaxassis uchun bu vosita zaruriy hisoblanadi.

## Mobil Ilovalar Pentesting Bosqichlari

---

Android ilovasini pentest qilish bu ilovaning zaifliklarini aniqlash va tuzatish orqali xavfsizlikni mustahkamlash jarayonidir. Uning maqsadi yomon niyatli hujumlardan oldin zaifliklarni topib, ularni bartaraf etishdan iborat. Pentest ilovaning kodini, konfiguratsiyasini va ish jarayonini sinovdan o'tkazib, ma'lumotlar, ruxsatnomalar va tarmoq muloqotidagi xatoliklarni aniqlashga yordam beradi.

### Statik tahlil

Statik tahlil — bu mobil ilovaning ishga tushirilmasdan turib, uning kod tuzilmasi, konfiguratsiyasi va xavfsizlik jihatlarini tahlil qilish jarayonidir. Ushbu bosqichni amalga oshirish uchun eng avvalo tahlil qilinadigan ilovaning o'zi kerak bo'ladi. Ilova bizga uni ishlab chiqqan tashkilot tomonidan taqdim etiladi. Android ilovalari odatda ikkita shaklda taqdim etiladi:

- .apk (Android Package) fayli ko'rinishida
- yoki Play Market havolasi orqali

Agar ilova Play Market havolasi orqali berilgan bo'lsa, bu holatda u test rejimidagi ilova bo'lib, uni yuklab olish uchun tahlilchini test foydalanuvchi sifatida ro'yxatga olish talab etiladi. Buning uchun biz o'zimizning Gmail manzilimizni ishlab chiquvchi tashkilotga taqdim etamiz. Tashkilot esa mazkur Gmail manzilni o'zlarining Google Play Developer Console tizimiga test foydalanuvchi sifatida qo'shadi. Shundan so'ng, biz ushbu test ilovani Google Play orqali yuklab olish imkoniyatiga ega bo'lamiz.



Ilovani to'liq tahlil qilish uchun, odatda, test foydalanuvchi akkaunti va, agar zarur bo'lsa, admin huquqiga ega akkaunt ham taqdim etilishi lozim. Bu akkauntlar orqali ilovaning barcha funksiyalarini ko'rib chiqish va ularni xavfsizlik nuqtai nazaridan baholash mumkin bo'ladi.

iOS ilovalari esa odatda .ipa (iOS App Store Package) fayli shaklida beriladi. Ba'zida esa ilova yuklab olinadigan maxsus havola taqdim etiladi. Bunday holatda, ilovani o'rnatish va ishga tushirish uchun Apple Developer hisobiga ega bo'lish talab etiladi. Tahlilchi ushbu hisob orqali test rejimidagi ilovani o'z qurilmasiga o'rnatadi.

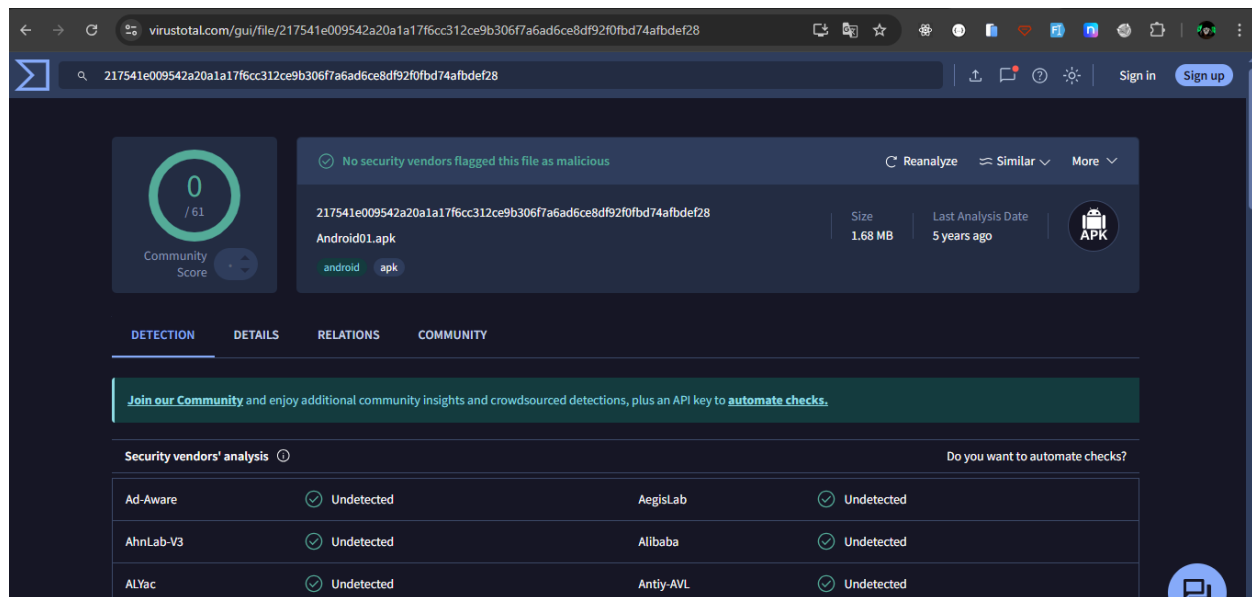
Statik tahlil davomida ilova ishga tushirilmasdan, uning ichki tarkibi — ya'ni kodlar, konfiguratsiya fayllari, ruxsatnomalar, tashqi kutubxonalar, va boshqa komponentlar o'rganiladi. Dastlabki tahlil bosqichlaridan biri sifatida, ilova faylida zararli kod yoki virus mavjud emasligini aniqlash maqsadida uni bir nechta (kamida uchta) turli antivirus skaner vositalari orqali tekshirib chiqish tavsiya etiladi. Bu vositalar ilovadagi zararli faoliyat yoki nomaqbul kutubxonalarni aniqlashda yordam beradi.

### **Ilovani virustotal orqali tekshirish**

Statik tahlilni VirusTotal servisidan boshlash tavsiya qilinadi. Bu orqali ilovada zararli dasturlar yoki kodlar bor yoki yo'qligini aniqlash mumkin. VirusTotal havolasi: <https://www.virustotal.com/gui/home/upload>



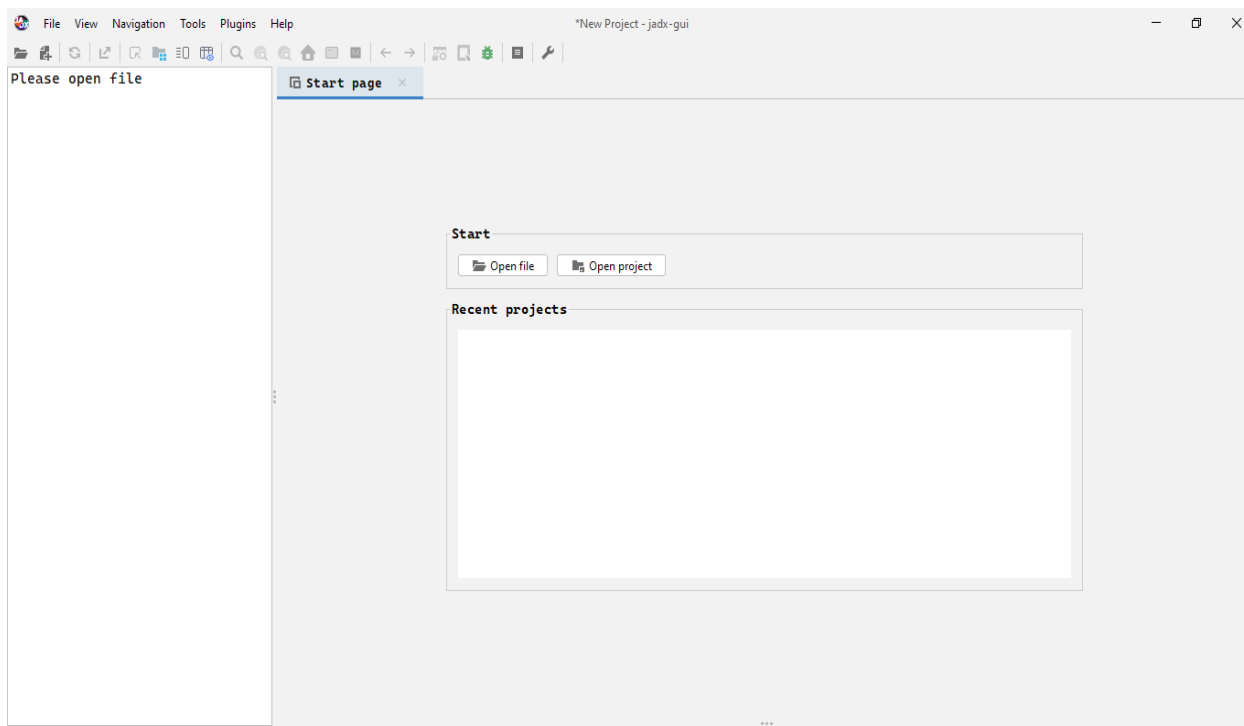
Ilovani tekshirish uchun ilovani kompyuterga ko'chirib oling va virustotal saytiga .apk faylni yuboring bir necha soniyadan keyin sizga rasmdagi kabi natija qaytarish kerak bo'ladi.



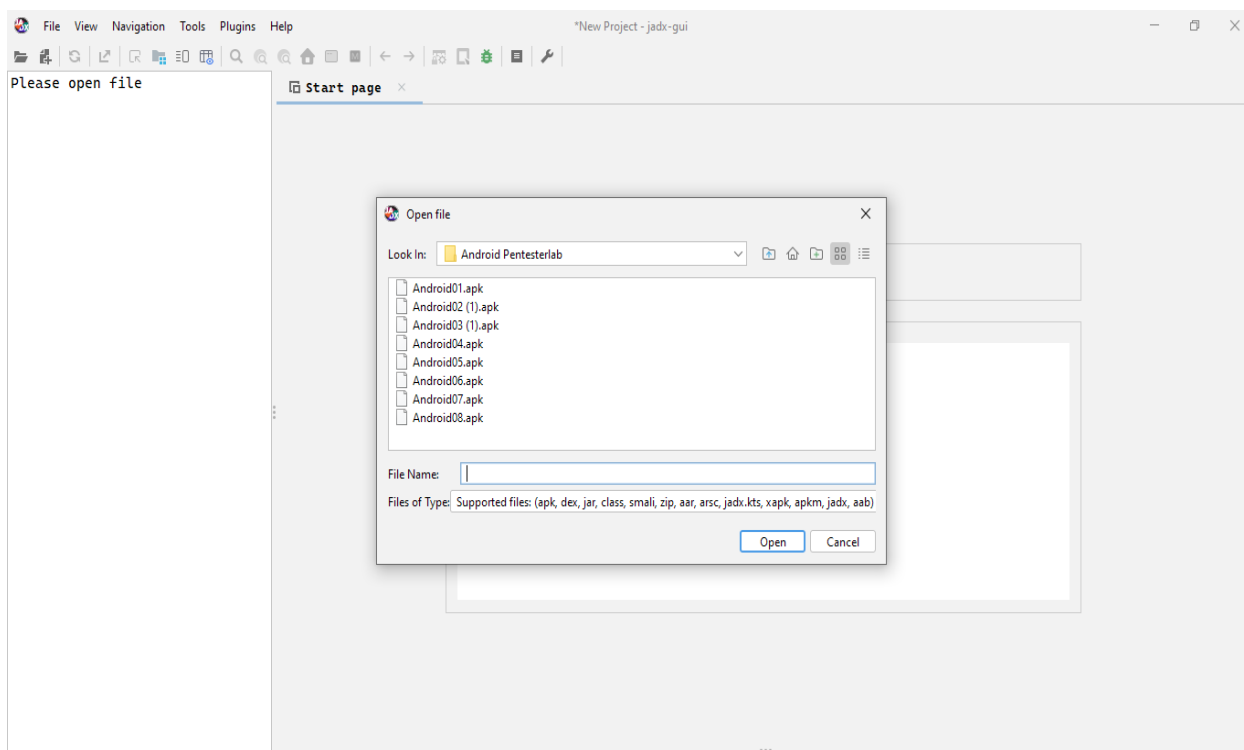
Ilovada hech qanday zararli kod aniqlanmaganligini chiqardi. Sizda buni aksi bo'lishi ham mumkin.

### **Ilovani dekomplyatsiya qilish (apktool, jadx-gui)**

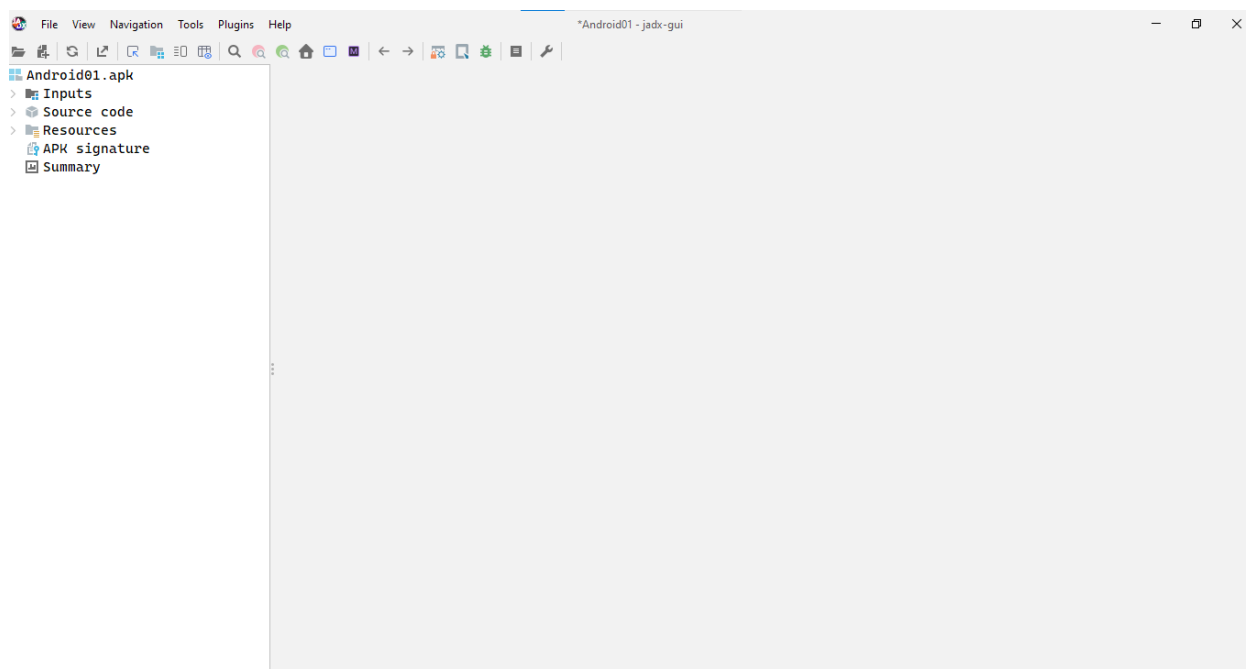
Ilovani (APK faylini) dekomplyatsiya qilish bu Android dasturining ichki tuzilmasi va kodini tahlil qilish jarayonidir. Quyida apktool va jadx vositalaridan foydalanib ilovani dekomplyatsiya qilish bo'yicha asosiy ma'lumot va amaliy ko'rsatmalar berilgan. Jadx dasturi orqali siz bilan ilovani dekomplyatsiya qilish ni ko'rib chiqamiz. Dastlab siz jadx dasturini o'rnatib oling jadx dasturini quydagi manzildan yuklab oling(<https://github.com/skylot/jadx/releases>). Jadx dasturini o'zingizni operatsion tizimingizga mos versiyasini yuklab oling windows uchun windows.zip formatdagiisini yuklab oling va uni zipdan chiqarib ichidagi .exe faylni ishga tushuring. Jadx dasturi sizda dastlab ishga tushganda quydagicha oyna ochiladi.



Ushbu oynadan Open file qilib apk faylni tanlab oling, rasmdagi kabi apk faylni tanlab oling.



Ilova apk faylni tanlab olganingizda sizda rasmdagi kabi bu faylni dekomplatsiya qilib beriladi.



Jadx-gui dasturida .apk fayl ochilgach, chap tomonda quyidagi asosiy bo‘limlar (katta menyular) ko‘rinadi. Ularning har biri APK fayl tarkibini turli tomondan tahlil qilishga yordam beradi. Quyida ularning har biri nima vazifa bajarishini tushuntirib beraman:

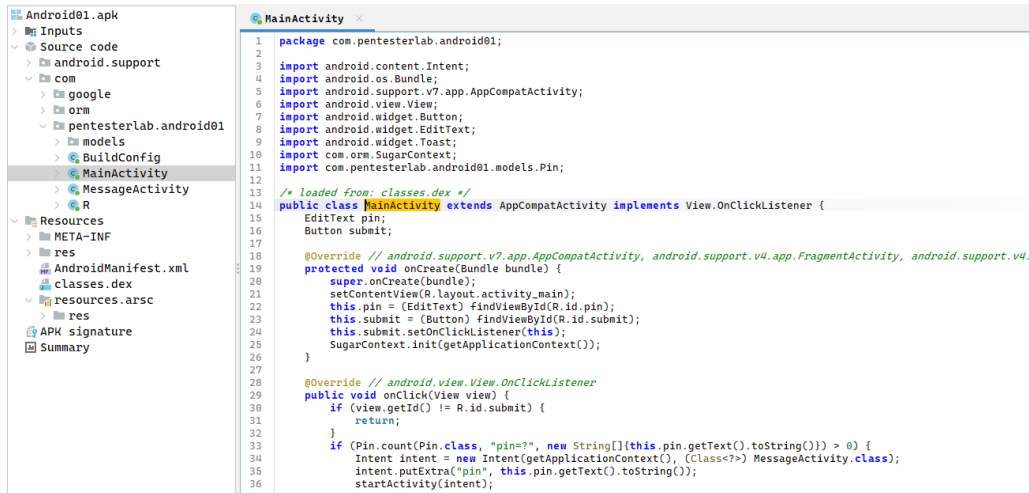
**Inputs** - bu bo‘limda siz yuklagan .apk fayllari ko‘rsatiladi.

- Bir nechta .apk fayl ochgan bo‘lsangiz, barchasi shu yerda ro‘yxat bo‘lib chiqadi.
- Har bir faylga tegishli tarkibiy qismlar (kod, resurslar, imzo va h.k.) shu fayl ostida guruhlanadi.

**Source code** - bu dekomplyatsiya qilingan Java kod joylashgan bo‘lim.

- APK ichidagi .dex fayllar dekomplyatsiya qilinib, o‘qiladigan shakldagi Java kodga aylantirilgan bo‘ladi.
- Har bir package (paket) va klasslar (classlar) struktura holatida ko‘rsatiladi.

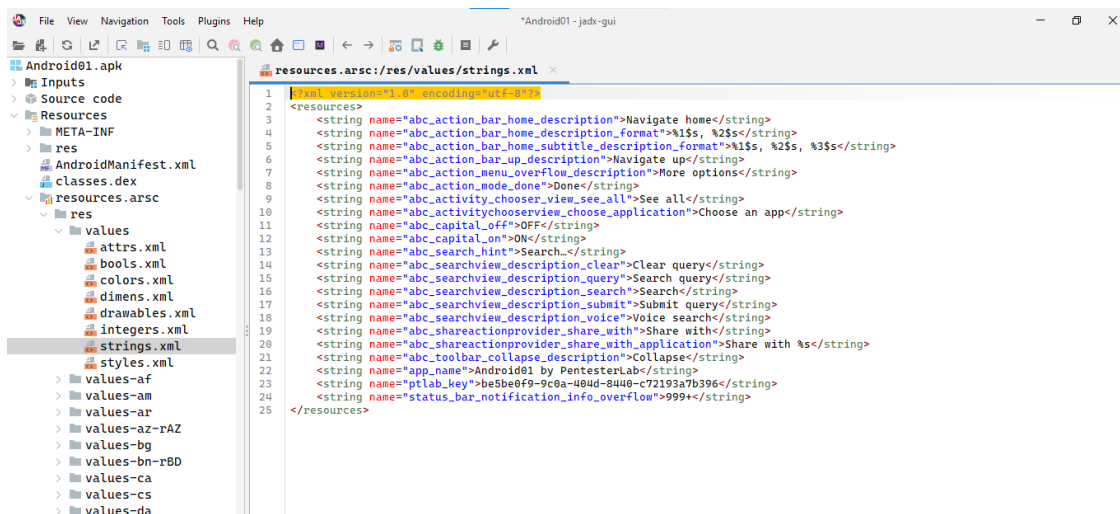
- Bu yerda MainActivity.java, LoginActivity.java kabi asosiy faoliyat fayllarni ko‘rib chiqasiz.



Asosiy joy dasturchining logikasi, API chaqiriqlari, tokenlar, kalitlar shu yerda bo‘ladi.

**Resources** - bu bo‘lim APK ichidagi resurs fayllarini o‘z ichiga oladi:

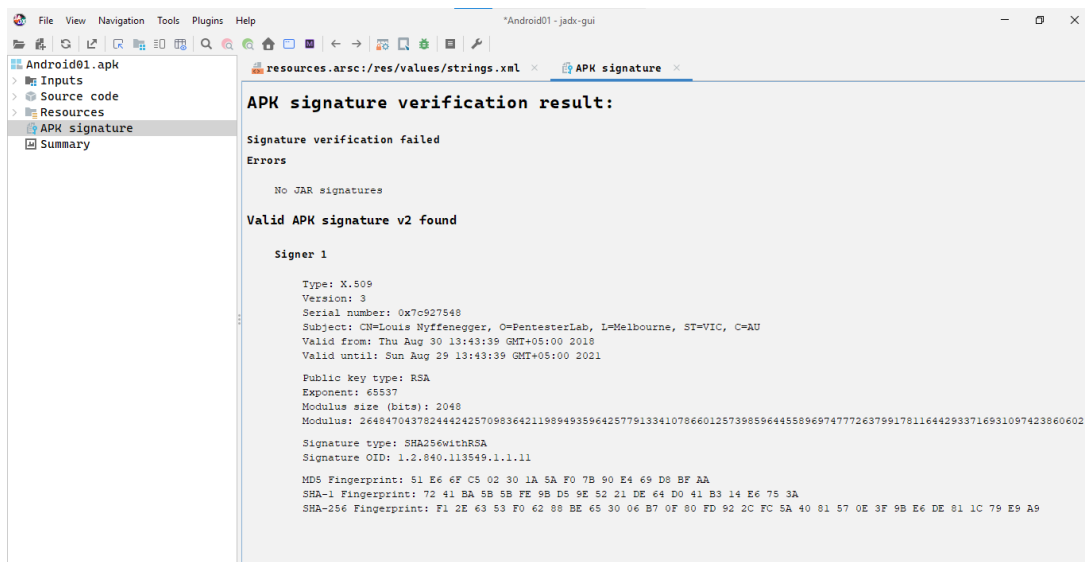
- res/ papkasidagi layout, drawable, values, strings.xml, colors.xml, styles.xml va boshqa XML fayllar shu yerda bo‘ladi.
- Dastur interfeysining qanday ko‘rinishini, ranglar, matnlar, tasvirlar qayerda ishlatilganini shu yerda bilib olish mumkin.



UI dizayn va interfeysga oid ma’lumotlar bu yerda bo‘ladi.

**APK signature** - bu bo‘lim APK faylning imzosi (digital signature) haqida ma’lumot beradi:

- APK fayl kim tomonidan imzolangan (sertifikat ma’lumotlari).
- RSA, SHA algoritmlari orqali tekshirilgan imzo haqida tafsilotlar.
- Agar siz modifikatsiya qilgan bo‘lsangiz, bu yerda imzo noto‘g‘ri yoki “Signed: No” ko‘rsatiladi.

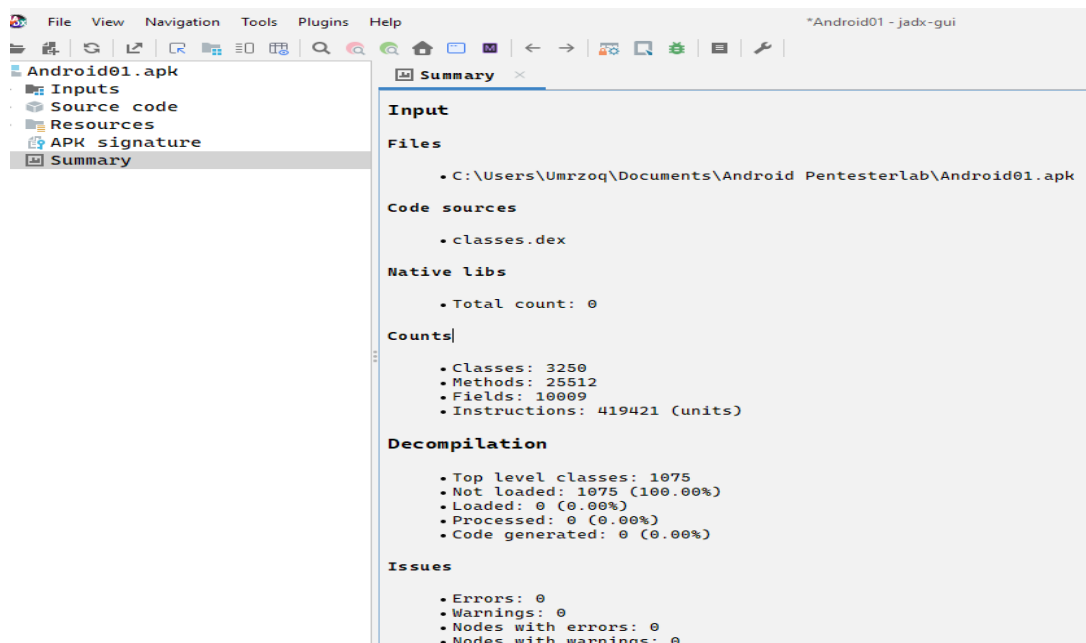


Bu yer APK originalmi yoki yo‘qligini tekshirish uchun muhim.

**Summary** - bu bo‘lim APK fayl haqida umumiy xulosa va texnik ma’lumotlarni beradi:

- Package name (com.example.app)
- Min SDK, Target SDK
- Versiya raqami
- Dex fayllar soni
- Permissiyalar (INTERNET, CAMERA, READ\_SMS va h.k.)

Tez tahlil uchun foydali bo‘lim. Dastur nima qiladi, qaysi ruksatlarni so‘raydi – shularni ko‘rsatadi.



## Apktool

Apktool – bu .apk fayllarni dekomplyatsiya va qayta yig‘ish uchun ishlatiladigan ochiq manbali Java vosita. Apktool dasturini siz quydagi havola orqali yuklab olishingiz mumkin bo‘ladi(<https://github.com/iBotPeaches/Apktool/releases>). Apktoolni o‘rnatib olganingizdan keyin terminalni ochib apk faylingizi turgan manzilga borib oling va terminalga apktool d your\_apk\_name.apk kamandasini yozing bu kod bajarilishi natijasida sizning apk faylingiz yonida apk fayl nomi bilan bir xil papka paydo bo‘ladi, bu papka ichida rasmdagi kabi fayllar paydo bo‘ladi bua pk faylingizni dekomplatsiya ko‘rinishidir.

| Name            | Date modified     | Type                | Size |
|-----------------|-------------------|---------------------|------|
| original        | 6/12/2025 3:06 PM | File folder         |      |
| res             | 6/12/2025 3:06 PM | File folder         |      |
| smali           | 6/12/2025 3:06 PM | File folder         |      |
| AndroidManifest | 6/12/2025 3:06 PM | Microsoft Edge H... | 2 KB |
| apktool         | 6/12/2025 3:06 PM | Yaml Source File    | 1 KB |

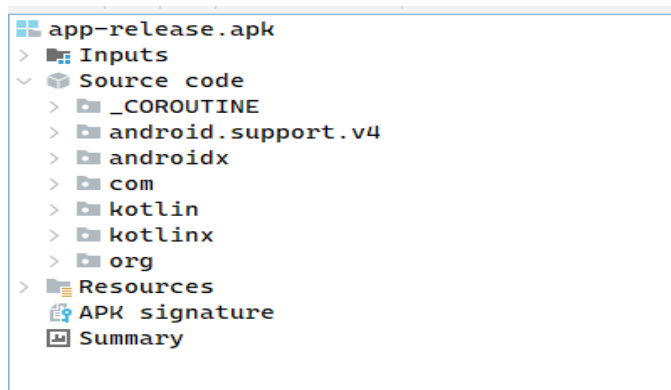
## Obfusikatsiya

Ilova kodi obfuskatsiya qilinganligini aniqlash bu tahlil jarayonida dasturchilar tomonidan dastur kodi ongli ravishda chalkashtirilganini ko'rsatib beruvchi belgilarning mavjudligini aniqlashdir. Bunday holatlarda koddagi klass, metod va o'zgaruvchilar nomlari odatda ma'nosiz yoki juda qisqa (masalan, a, b, c1, d2) bo'lib, ularning asl vazifasini tushunish qiyinlashadi. Dastur tuzilmasi murakkab va tushunarsiz shaklga keltiriladi, smali fayllarda esa izohlar bo'lmaydi yoki juda kam bo'ladi, nomlar esa aniq semantik ma'noga ega emas.

Shuningdek, strings.xml yoki boshqa resurs fayllarda noto'g'ri tartiblangan, chalkash yoki hatto shifrlangan matnlar uchraydi. Ma'lumotlar bazasi jadval nomlari, API endpoint manzillari ham ko'pincha yashirilgan yoki be'mani nomlar bilan almashtirilgan bo'ladi. Bularning barchasi dastur kodining tahlildan, ya'ni teskari muhandislik (reverse engineering)dan himoyalanganini bildiradi.

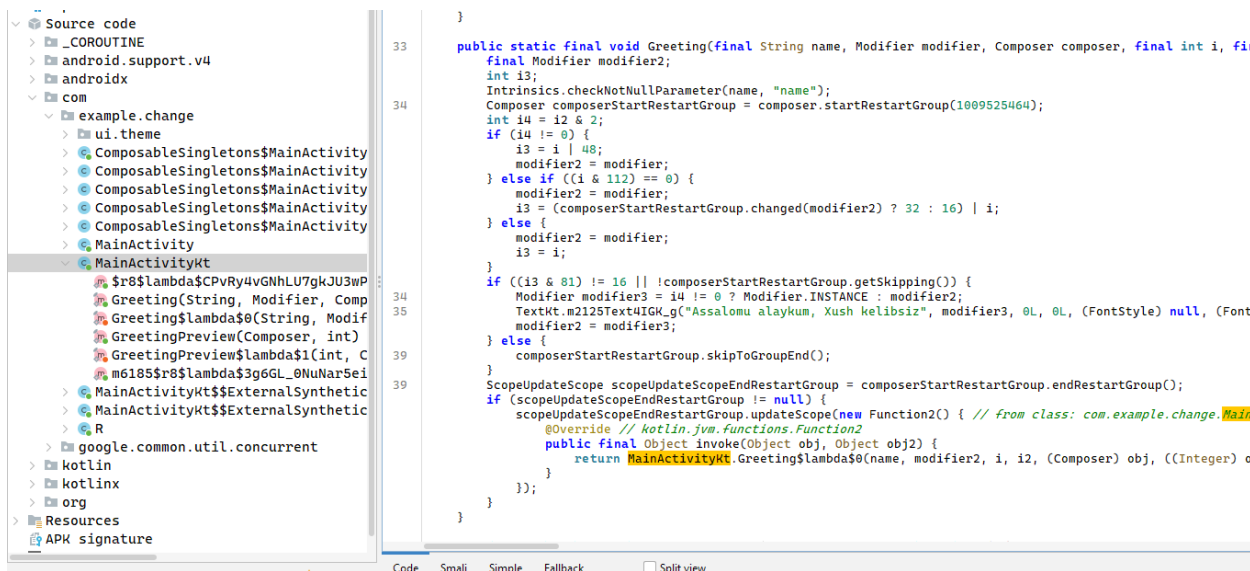
Obfuskatsiya texnikasi ko'pincha ilovaning funksional jihatlarini yovuz niyatli shaxslar tomonidan o'rganilishining oldini olish, foydalanuvchi xavfsizligini ta'minlash, intellektual mulkni muhofaza qilish maqsadida qo'llaniladi.

Keling endi, obfuskatsiya qilinmagan dastur kodi bilan obfuskatsiya qilingan kod o'rtasidagi farqlarni solishtirib, bu jarayonning dastur tuzilmasiga qanday ta'sir qilganini ko'rib chiqamiz.



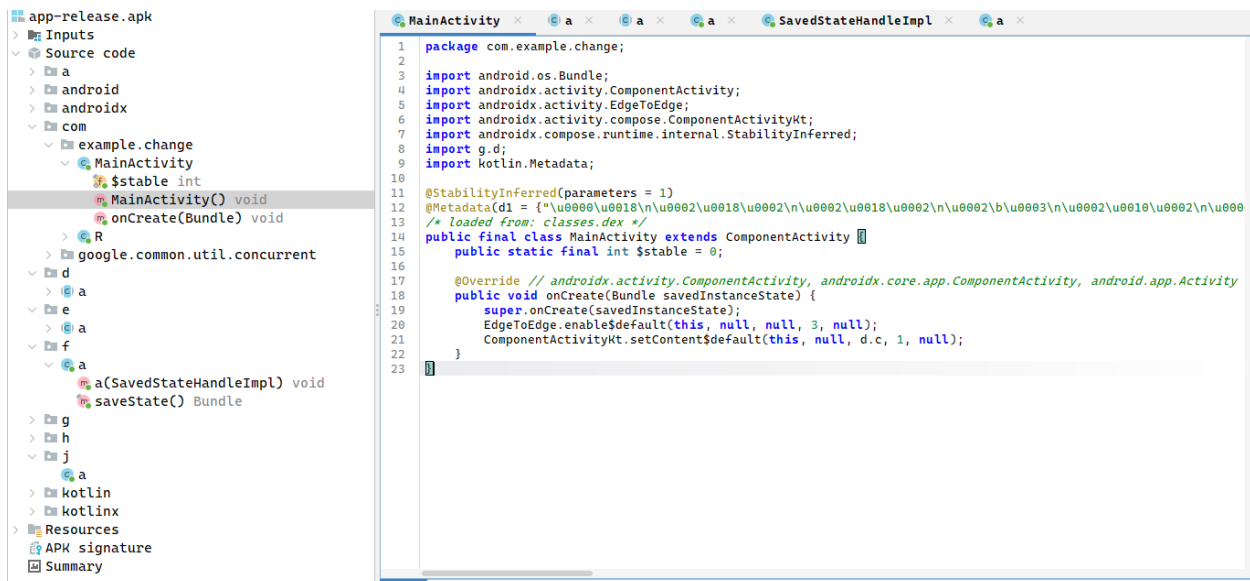
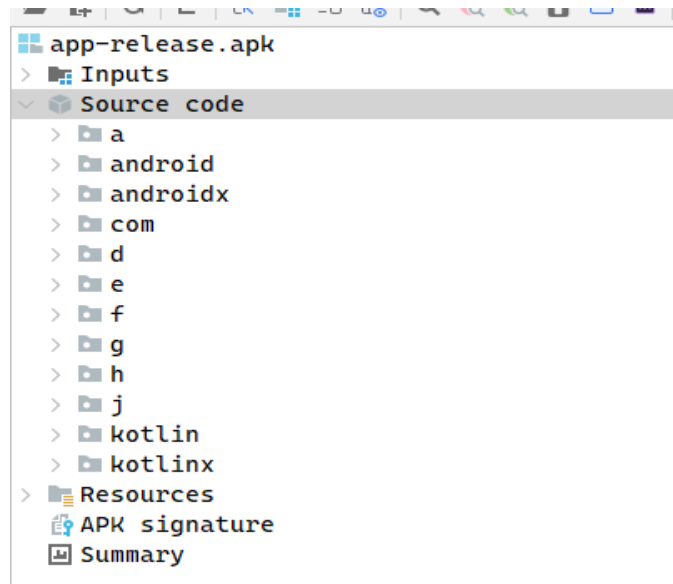


JADX dasturi orqali obfuskatsiya qilinmagan ilovani tahlil qilganimizda, kod tuzilmasi aniq, tartibli va o‘qilishi oson holatda bo‘ladi. Ilovaning har bir komponenti o‘z nomi bilan ochiq-oydin ajratilgan bo‘lib, klasslar, metodlar va o‘zgaruvchilar semantik ma’noga ega nomlar bilan nomlangan.



Yuqoridagi rasmda biz obfuskatsiya qilinmagan ilovaning kodi bilan tanishdik. Unda hamma narsa aniq, tartibli va tushunarli ko‘rinishda bo‘lib, kodni o‘qish hamda tahlil qilish juda oson edi. Klass nomlari, metodlar va o‘zgaruvchilar o‘z vazifasini ochiq-oydin ifodalab turibdi. Ilova arxitekturasini mantiqiy tarzda tuzilgan bo‘lib, uni tushunish mutaxassis uchun hech qanday murakkablik tug‘dirmaydi.

Endi esa, shu ilovani obfuskatsiya qilamiz va uni qayta build qilib, hosil bo‘lgan APK faylni yana JADX dasturi orqali tahlil qilamiz. Obfuskatsiyadan so‘ng ilovaning ko‘rinishi keskin o‘zgaradi. Klass va metod nomlari ma’nosiz yoki qisqa belgilarga aylangan (a, b1, x3 va hokazo), o‘zgaruvchilar chalkash va kontekstdan uzilgan holatda bo‘ladi. Kodlar o‘zaro qanday bog‘langanini anglash qiyinlashadi. Ko‘plab joylarda funksiyalar o‘z vazifasini yashirish uchun murakkab tuzilmaga aylantirilgan bo‘ladi.



Rasmda ko‘rib turganingizdek, obfuskatsiya jarayonidan so‘ng ilova kodi mutlaqo tushunarsiz holga keltirilgan. Klass va metod nomlari endi hech qanday semantik ma‘noga ega emas, o‘zgaruvchilar esa qisqa va chalkash belgilar bilan ifodalangan. Dastur tuzilmasi sun‘iy murakkablik bilan to‘ldirilgan va kod bloklari o‘zaro qanday bog‘langanini anglash ancha qiyinlashgan.

Ayniqsa, ilovada foydalanuvchiga ko‘rsatiladigan matnlarni aniqlash ya‘ni strings.xml faylidagi yozuvlar yoki kod ichidagi satrlarni topish endi ancha

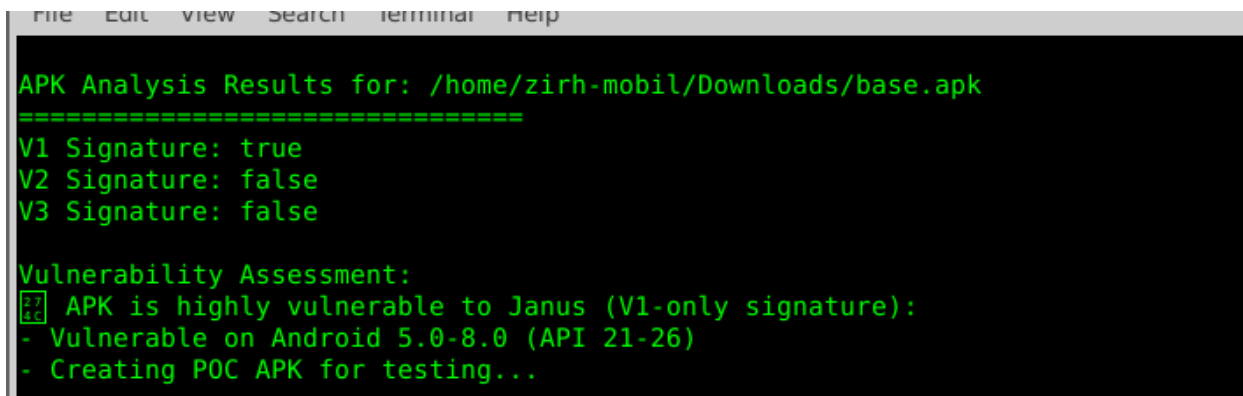
murakkab vazifaga aylangan. Matnlar shifrlangan, obfuskatsiya qilingan yoki ba'zida fragmentlarga bo'lingan holda berilgan bo'lishi mumkin. Ba'zi hollarda esa ular butunlay resurslardan chiqarilib, kod ichiga ko'milib qo'yiladi, bu esa avtomatik analiz vositalari orqali ularni aniqlashni deyarli imkonsiz qiladi.

Bu holat ilovaning tahlildan himoyalanganini yana bir bor tasdiqlaydi. Obfuskatsiya faqat kod nomlarini emas, balki ilovaning matnli interfeysini ham yashirish orqali teskari muhandislikning oldini oladi.

Xulosa qilib aytganda, obfuskatsiyadan so'ng nafaqat kodni o'qish, balki oddiy foydalanuvchi matnlarini topish ham professional tahlilchini ko'p vaqt va kuch sarflashga majbur qiladi. Bu esa ilova xavfsizligini ancha yuqori darajaga olib chiqadi.

### **Ilova imzosini tekshirish.**

Ilovaning imzo yaxlitligini tekshirish va xavfsizligini ta'minlash maqsadida, Janus zaifligi (Janus Vulnerability) bo'yicha tahlil o'tkazish tavsiya etiladi. Buning uchun Janus zaiflik skaneri yoki Mobile Security Framework (MobSF) kabi zamonaviy xavfsizlik vositalaridan foydalanish lozim. Ushbu dasturlar ilova faylining raqamli imzosi buzilgan-buzilmaganini aniqlash, zararli o'zgarishlar kiritilganini tekshirish va umumiy xavfsizlik darajasini baholash imkonini beradi.



```
File Edit View Search Terminal Help
APK Analysis Results for: /home/zirh-mobil/Downloads/base.apk
=====
V1 Signature: true
V2 Signature: false
V3 Signature: false

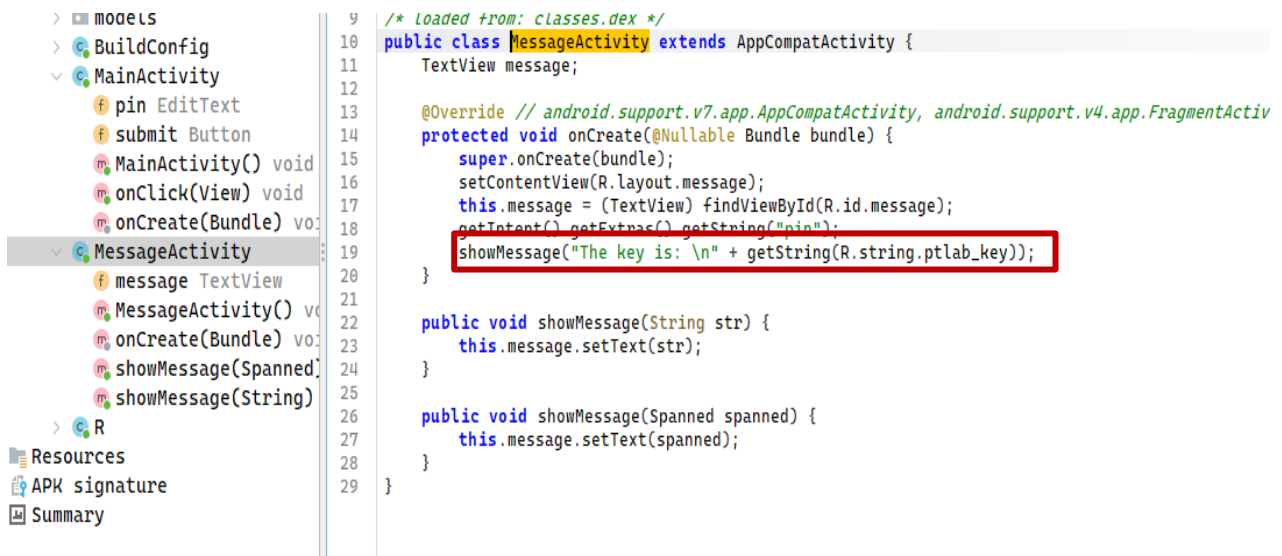
Vulnerability Assessment:
[27] APK is highly vulnerable to Janus (V1-only signature):
- Vulnerable on Android 5.0-8.0 (API 21-26)
- Creating POC APK for testing...
```

Agar ilova faqat V1 imzo sxemasi asosida imzolangan bo'lsa, bu katta xavfsizlik zaifligi hisoblanadi. Chunki bunday imzolangan ilovalar Android 5 (Lollipop) va Android 6 (Marshmallow) versiyalarida ham to'liq ishlay oladi. Mazkur operatsion tizim versiyalarida esa bir qator jiddiy zaifliklar mavjud. Ayniqsa, "Janus" zaifligi orqali, hujumchi APK fayliga zararli kodlar kiritgan taqdirda ham, ilovaning raqamli imzosi o'zgarmaydi. Bu esa foydalanuvchi yoki tizim tomonidan ilova xavfsiz deb qabul qilinishiga olib keladi. Natijada, modifikatsiyalangan va zararli kod bilan boyitilgan ilova hech qanday ogohlantirishsiz ishga tushishi mumkin.

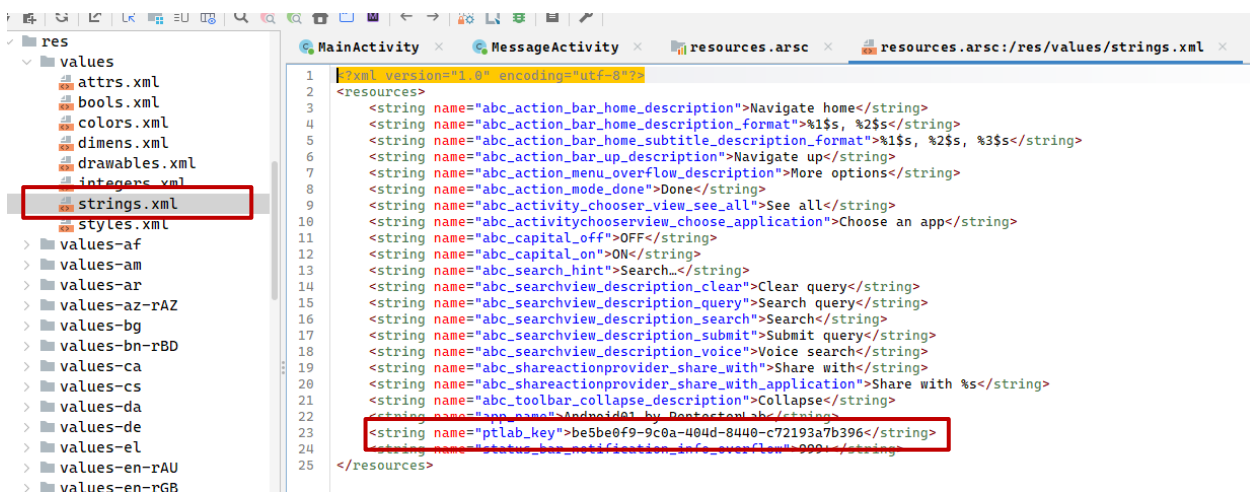
### **Ilova kodini tahlil qilish va o'zgartirish**

Ilova kodini tahlil qilish orqali biz uning ichki logikasini, xavfsizlik darajasini hamda ishlash mexanizmlarini chuqur anglab yetamiz. Bunday tahlil nafaqat zaifliklarni aniqlash, balki ilovani pentest qilish, modifikatsiyalash yoki uning funksiyalarini chuqur o'rganish imkonini beradi. Bu borada Apktool va Jadx kabi vositalar ayniqsa samarali sanaladi. To'g'ri yondashuv bilan olib borilgan dekomplyatsiya jarayoni mobil ilovalardagi zaif joylarni ochib berib, ularga qarshi samarali himoya choralarini ishlab chiqishda muhim ahamiyat kasb etadi.

Yuqoridagi misolda biz dekomplyatsiya qilingan ilovani tahlil qilamiz. Bu ilova Android Pentester laboratoriyasiga tegishli APK fayl bo'lib, biz undan yashiringan "flag"ni topishimiz kerak. Flag'lar odatda ma'lum bir andozaga ega bo'ladi – bu holatda ular ptlab so'zi bilan boshlanishi aytilgan. Shuning uchun biz ilova kodini Jadx orqali ochamiz va kod ichida ptlab iborasi mavjud bo'lgan qatorni izlaymiz. Masalan, MessageActivity sinfida rasmda ko'rsatilgan tarzda kerakli ma'lumotlar joylashganini ko'rish mumkin. Bu tahlil flagni aniqlashda asosiy qadam hisoblanadi.



Kodni tahlil qilish jarayonida ko‘rinib turibdiki, ma’lumot ptlab\_key nomli kalit orqali ekranga chiqarilmoqda. Bu esa shuni anglatadiki, kerakli satr ma’lumoti ilovaning resurslar qismida, aniqrog‘i strings.xml faylida saqlangan. Shundan kelib chiqib, biz res papkasidagi values papkasiga o‘tamiz va uning ichida joylashgan strings.xml faylini ko‘zdan kechirishimiz zarur bo‘ladi. Faylni ochganimizda, kutilgan natija aynan shu yerda ptlab\_key ga mos yozuv shaklida joylashganini ko‘rishimiz mumkin. Bu bosqich flagni aniqlash yo‘lida hal qiluvchi ahamiyatga ega bo‘lib, rasmda keltirilgan ko‘rinishda aniq va ravshan tarzda aks etgan.



Bizga kerakli flag *be5be0f9-9c0a-404d-8440-c72193a7b396* qiymati bo‘lib, u ilovaning strings.xml faylida ptlab\_key kaliti ostida joylashgan. Shu orqali biz ilova kodi ustida muvaffaqiyatli tahlil o‘tkazdik va flagni aniqladik.

Endi navbat ilova kodini o‘zgartirib, uni qayta yig‘ish bosqichiga keldi. Bu jarayonda biz apktool vositasidan foydalanamiz u Android ilovalarini dekompiletsiya va qayta kompilyatsiya qilishda eng ishonchli va qulay vositalardan biridir.

```
apktool d original.apk -o app-release
```

| Name            | Date modified     | Type                | Size |
|-----------------|-------------------|---------------------|------|
| original        | 6/12/2025 3:06 PM | File folder         |      |
| res             | 6/12/2025 3:06 PM | File folder         |      |
| smali           | 6/12/2025 3:06 PM | File folder         |      |
| AndroidManifest | 6/12/2025 3:06 PM | Microsoft Edge H... | 2 KB |
| apktool         | 6/12/2025 3:06 PM | Yaml Source File    | 1 KB |

Endi esa ilovani amalda o‘zgartirish va qayta yig‘ish jarayoniga o‘tamiz. Dastlab, .apk faylni dekompiletsiya qilganingizdan so‘ng, smali papkasiga o‘ting. Bu yerda ilovaning bajariluvchi logikasi smali fayllar ko‘rinishida joylashgan bo‘ladi. Har qanday .smali faylni oching (masalan, MainActivity.smali), so‘ng unga kichik o‘zgarish kiriting bu matn, log yozuvi yoki shart operatori bo‘lishi mumkin.

```
const-string v0, "You Have Hacked Been"
const/4 v1, 0x1
invoke-static {p0, v0, v1}, Landroid/widget/Toast;
>makeText(Landroid/content/Context;Ljava/lang/CharSequence;I)Landroid/widget/Toast;
move-result-object v0
invoke-virtual {v0}, Landroid/widget/Toast;->show()V
```

Masalan yuqoridagi kodni MainActivity faylga joylashtiring bu kod Ilovada Toast chiqishini taminlaydi va o‘zgartirishni kiritgach, faylni saqlang. Endi esa ilovani qayta kompilyatsiya qilish uchun quyidagi buyruqdan foydalaning.

```
apktool b app-release -o app-modded.apk
```

Qayta kompilyatsiya jarayonida buyruq berilayotgan papka nomi, ya’ni app-release, bu ilgari apktool vositasi orqali dekompilyatsiya qilingan APK faylining papka nomi bo‘lishi kerak.

Ilovani o‘zgartirgach, ya’ni app-modded.apk faylini yaratganimizdan so‘ng, uni Android tizimida o‘rnatishdan avval imzolab olish zarur bo‘ladi. Aks holda, tizim ilovani “noma’lum yoki buzilgan” deb topib, o‘rnatishga ruxsat bermaydi.

Imzolash jarayoni uchun bizga .jks (Java KeyStore) formatidagi imzo sertifikat kerak bo‘ladi. Agar sizda mavjud bo‘lmasa, uni yaratish uchun keytool vositasidan foydalanamiz. Quyidagi buyruq orqali yangi sertifikat yaratishingiz mumkin.

```
keytool -genkey -v -keystore my-key.jks -alias myalias -keyalg RSA -keysize 2048 -validity 10000
```

```
zlrh-mobil@zlrh-mobil:~/Documents$ keytool -genkey -v \
-keystore my-key.jks \
-alias myalias \
-keyalg RSA -keysize 2048 -validity 10000
Enter keystore password:
Re-enter new password:
What is your first and last name?
[Unknown]: John Doe
What is the name of your organizational unit?
[Unknown]: Hacked
What is the name of your organization?
[Unknown]: Hack
What is the name of your City or Locality?
[Unknown]: New York
What is the name of your State or Province?
[Unknown]: New York
What is the two-letter country code for this unit?
[Unknown]: KZ
Is CN=John Doe, OU=Hacked, O=Hack, L=New York, ST=New York, C=KZ correct?
[no]: yes

Generating 2,048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 10,000 days
for: CN=John Doe, OU=Hacked, O=Hack, L=New York, ST=New York, C=KZ
[Storing my-key.jks]
zlrh-mobil@zlrh-mobil:~/Documents$
```

Siz yaratgan my-key.jks sertifikat yordamida endi ilovani imzolash mumkin bo‘ladi. Biroq imzolashdan oldin muhim bosqichlardan biri zipalign vositasi orqali APK faylni optimallashtirishdir.

zipalign Android SDK tarkibidagi vosita bo'lib, u APK fayldagi barcha ma'lumotlarni 4 baytli chegaralarga to'g'rilab joylashtiradi. Bu esa ilovaning samarali ishlashini ta'minlaydi va Google Play kabi platformalarda tekshiruvdan muammosiz o'tishiga yordam beradi. Quyidagi buyruq orqali app-modded.apk faylni zipalign qilamiz.

```
zipalign -p -f 4 app-modded.apk app-aligned.apk
```

```
zirh-mobil@zirh-mobil:~/Documents$ ls
app-modded.apk  app-release  app-release.apk  Expertiza  my-release-key.jks
zirh-mobil@zirh-mobil:~/Documents$ zipalign -p -f 4 app-modded.apk app-aligned.apk
```

Zipalign jarayoni muvaffaqiyatli yakunlangach, endi biz ilovani imzolash bosqichiga o'tamiz. Bu jarayon ilovaning yaxlitligini va ishonchliligini kafolatlaydi hamda Android tizimi tomonidan tan olinishi uchun zarur hisoblanadi.

Imzolash uchun biz apksigner vositasidan foydalanamiz. Quyidagi buyruq orqali siz app-aligned.apk faylni avval yaratgan my-key.jks sertifikatida yordamida imzolab olishingiz mumkin.

```
apksigner sign --ks my-release-key.jks --ks-key-alias myalias --out app-signed.apk app-aligned.apk
```

```
zirh-mobil@zirh-mobil:~/Documents$ apksigner sign \
--ks my-release-key.jks \
--ks-key-alias myalias \
--out app-signed.apk \
app-aligned.apk
Keystore password for signer #1:
```

Yuqorida bayon etilgan barcha bosqichlar dekompiletsiya, o'zgartirish, zipalign va imzolash muvaffaqiyatli yakunlangach, endi ilovaning to'g'ri va haqiqiy tarzda imzolanganligini tekshirib ko'rishimiz mumkin. Bu ilovani tizim tomonidan ishonchli deb tan olinishi uchun muhim bosqich hisoblanadi.

Imzo holatini tekshirish uchun terminalga quyidagi buyruqni kiritish kifoya.

```
keytool -list -v -keystore my-release-key.jks
```



```
zirh-mobil@zirh-mobil:~/Documents$ keytool -list -v -keystore my-release-key.jks
Enter keystore password:
Keystore type: PKCS12
Keystore provider: SUN

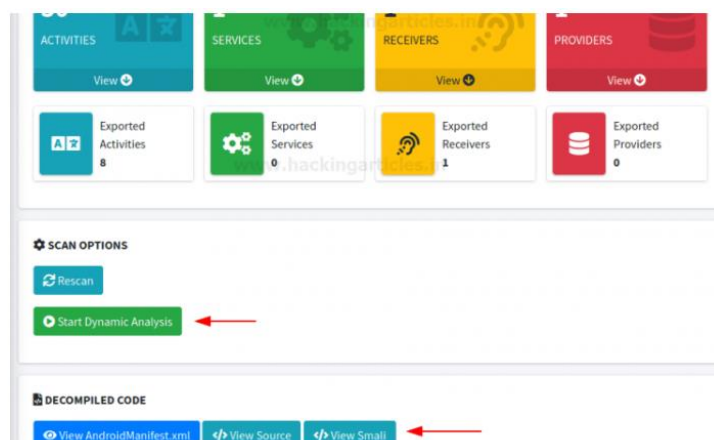
Your keystore contains 1 entry

Alias name: myalias
Creation date: Jun 24, 2025
Entry type: PrivateKeyEntry
Certificate chain length: 1
Certificate[1]:
```

Agar sizda ham apksigner verify buyrug'i hech qanday xatolik qaytarmasa va terminal jim tursa bu ilovaning to'g'ri va muvaffaqiyatli imzolanganligini anglatadi. Endilikda siz ushbu app-signed.apk faylni Android qurilmaga bimalol o'rnatib, uning funksiyalarini to'liq sinab ko'rishingiz mumkin bo'ladi.

### **Eskirgan va zaif kutubxonalarni aniqlash**

Ilovada eskirgan (deprecated yoki zaif) kutubxonalar bor-yo'qligini aniqlash Android ilovani statik tahlil qilishda juda muhim bosqich. Bu kutubxonalar orqali ilovada xavfsizlikka oid zaifliklar yoki ishlamay qolgan APIlar mavjud bo'lishi mumkin. Qanday qilib kutubxonalar ro'yhatini va versiyalarini ko'rish bo'yicha ketma-ketligini ko'rsatib o'tilgan. Eskirgan kutubxonalarni aniqlash uchun eng yaxshi va sinalgan va tushunarli vosita bu mobsf dasturi bo'ladi qurilmangizda mobsf dasturini ochib oling agar sizda hali mobsf dasturi bo'lmasa siz dastlab bu dasturni o'rnatib oling. Mobsf dasturini o'rnatib bo'lganingizdan keyin uni ochib ichiga .apk faylni yuboring va bir necha Soniya kuting va natija quydagicha bo'ladi.



### **AndroidManifest.xml faylini tahlil qilish**

AndroidManifest.xml bu har bir Android ilovada mavjud bo'lgan asosiy konfiguratsiya fayli bo'lib, quyidagilarni belgilaydi:

- Ilovaning paket nomi
- Komponentlar: activity, service, broadcast receiver, content provider
- Ruxsatlar (permissions)
- Intent-filterlar (ya'ni tashqi chaqirishlar)
- Minimal SDK, target SDK versiyasi
- Ilovaning entry point (ya'ni MAIN activity)

Yuqoridagi barcha narsalarni statik tahlil jarayonida aniqlash kerak bo'ladi. Manifest faylida faqat asosiy birinchi ishga tushadigan activity exported true bo'lishi kerak. Agar bunday bo'lmasa hujumchilar tizim resurslaridan foydalanish imkoniyati paydo bo'lib qoladi.

### **Hard kod qilib yozilgan malumotlar**

Hard kod qilib yozilgan ma'lumotlar — bu ilova kodiga to'g'ridan-to'g'ri yozib qo'yilgan (o'zgartirib bo'lmaydigan) ma'lumotlar bo'lib, xavfsizlik nuqtai nazaridan jiddiy zaiflik hisoblanadi. Agar ilova ichida quyidagi ma'lumotlar hardcode qilingan bo'lsa, bu muammo hisoblanadi. API kalitlari (masalan, `const val API_KEY = "sk_test_123..."`). Tokenlar (masalan, Bearer token123456). Server URL manzillari

(<https://api.secretserver.com>). Parollar yoki login ma'lumotlari. Firebase, Stripe, PayPal, Google API konfiguratsiyalari

### Resurs fayllarni ko'rish (res/, assets/)

Android ilovalari tarkibida res/ va assets/ kataloglari mavjud bo'lib, ular ilovaning statik fayllari (matnlar, rasm, sozlamalar, konfiguratsiya fayllari) saqlanadigan joy hisoblanadi. Ilovani statik tahlil qilishda bu papkalarni ko'zdan kechirish xavfsizlik zaifliklarini aniqlash uchun muhim bosqichdir.

**res** — ilovaga birlashtirilgan statik resurslar saqlanadigan joy.

| Papka     | Tahlil ahamiyati                                                                              |
|-----------|-----------------------------------------------------------------------------------------------|
| layout/   | UI tuzilmalari (.xml) — maxfiy tugmalar, yashirin funksiya                                    |
| values/   | strings.xml, config.xml — API kalitlar, server URLlar bo'lishi mumkin                         |
| drawable/ | Ilovadagi rasm fayllari — phishing belgilar yoki logotiplar borligini tekshirish mumkin       |
| menu/     | Ilova menyulari — yashirin funktsiyalar mavjudligini aniqlash mumkin                          |
| xml/      | Qo'shimcha sozlamalar, network_security_config.xml — HTTPS cheklovlari mavjudmi, tekshiriladi |

**assets** papkasi — ilovaga to'liq kiritilgan foydalanuvchi tomonidan belgilangan fayllar saqlanadi. Bu fayllar ilovaning o'zida to'liq ko'rinishda mavjud bo'ladi va APK ichida bexavotir ochiladi.

| Fayl turi    | Xavfsizlikdagi roli                                                         |
|--------------|-----------------------------------------------------------------------------|
| .json, .xml  | API konfiguratsiyasi, URL, port, tarmoqqa ulanish sozlamalari               |
| .js, .html   | Ilova ichida ishlatiladigan WebView hujumlari (XSS, LFI) uchun tekshiriladi |
| .db, .sqlite | Mahalliy ma'lumotlar ombori — parollar, tokenlar, foydalanuvchi malumotlari |
| .pem, .crt   | TLS sertifikatlar — qonuniylik va amal qilish muddatini tekshirish mumkin   |

Ko'pincha bu katalogda noto'g'ri joylashtirilgan API kalitlar, parollar, serverga bog'lanish nuqtalari bo'ladi. Bu esa tahlilchi yoki hujumchiga ilovaning backend tizimiga hujum qilish imkonini beradi.

### Log malumotlarni tekshirish

Ilovalarda statik tahlilda log ma'lumotlarni ko'rish bu ilova ishlab chiqish jarayonida ilova ishlab chiqaruvchilari tomonidan yozib qoldirilgan Log malumotlar orqali

maxfiy ma'lumotlar logga yozib qolish oqibatida ilovaga tegishli maxfiy ma'lumotlarni ko'rish mumkin. Log chaqiriqlarini aniqlab, u orqali ilovaning ichki ishlash mexanizmini tushunishdir. Adb orqali ilova log ma'lumotlarini ko'rish mumkin. Quyidagi buyruqni o'z terminalingizga kiriting va enter buyrug'uni bosib.

```
adb logcat | grep com.example.app
```

```
jirh-mobil@jirh-mobil:~$ adb logcat | grep uz.anormobile.retail
06-22 16:19:45.427 479 479 V StorageManagerService: Package uz.anormobile.retail does not have legacy storage
06-22 16:24:40.052 479 551 I ActivityTaskManager: START u0 (act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] tlg=0x16200000
cnp=uz.anormobile.retail/uz.anorbank.retail.ui.main.MainActivity bnds=[37,369][238,466]) from uid 10139
06-22 16:24:40.432 479 504 I ActivityManager: Start proc 5725:uz.anormobile.retail/u0a155 for pre-top-activity {uz.anormobile.retail/uz.anorbank
.retail.ui.main.MainActivity}
06-22 16:24:47.745 479 1230 I ActivityTaskManager: START u0 (act=android.intent.action.MAIN cnp=uz.anormobile.retail/uz.anorbank.retail.ui.auth.Au
thenticationActivity [has extras]) from uid 10155
06-22 16:24:47.406 479 1230 D ConnectivityService: requestNetwork for uid/pid:10155/5725 NetworkRequest [ TRACK DEFAULT id=85, [ Capabilities: INT
ERNET|NOT_RESTRICTED|TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ]
06-22 16:24:48.760 479 1230 D ConnectivityService: requestNetwork for uid/pid:10155/5725 NetworkRequest [ TRACK DEFAULT id=86, [ Capabilities: INT
ERNET|NOT_RESTRICTED|TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ]
06-22 16:24:48.879 479 1230 W ActivityManager: Receiver with filter android.content.IntentFilter@45cfu92 already registered for pid 5725, callerPa
ckage is uz.anormobile.retail
06-22 16:44:17.844 479 497 I ActivityManager: Killing 5725:uz.anormobile.retail/u0a155 (ad) 985): remove task
06-22 16:44:17.972 479 1327 D ConnectivityService: ConnectivityService NetworkRequestInfo binderDied(NetworkRequest [ TRACK DEFAULT id=86, [ Capab
ilities: INTERNET|NOT_RESTRICTED|TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ), android
.os.BinderProxy@2c43a7d)
06-22 16:44:17.972 479 585 D ConnectivityService: releasing NetworkRequest [ TRACK DEFAULT id=86, [ Capabilities: INTERNET|NOT_RESTRICTED|TRUSTED
Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ] (release request)
06-22 16:44:17.993 479 1325 D ConnectivityService: ConnectivityService NetworkRequestInfo binderDied(NetworkRequest [ TRACK DEFAULT id=85, [ Capab
ilities: INTERNET|NOT_RESTRICTED|TRUSTED Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ), android
.os.BinderProxy@43463723)
06-22 16:44:17.994 479 585 D ConnectivityService: releasing NetworkRequest [ TRACK DEFAULT id=85, [ Capabilities: INTERNET|NOT_RESTRICTED|TRUSTED
Uid: 10155 AdministratorUids: [] RequestorUid: 10155 RequestorPackageName: uz.anormobile.retail ] ] (release request)
jirh-mobil
```

## Debug modni tekshirish

Statik tahlilda ilovaning debug rejimda tuzilganligini tekshirish — bu ilova xavfsizlik darajasini baholashda muhim qadamdir. Agar ilova debug rejimda ishlab chiqilgan bo'lsa, u holda Ilova imzolashda debug.keystore ishlatilgan bo'ladi. Proguard obfuskatsiyasi o'chirilgan yoki zaif bo'ladi. Foydalanuvchiga mo'ljallanmagan loglar, toastlar, debug bayroqlari saqlanib qolgan bo'ladi. Dasturchi tomonidan test uchun ochiq qoldirilgan developer funksiyalar mavjud bo'ladi.

## Zaif WebViewdan foydalanish

Ilovalarda zaif (xavfsizligi past) WebView ishlatilishi — bu mobil xavfsizlik nuqtai nazaridan jiddiy zaiflik hisoblanadi. WebView — bu ilova ichida veb sahifalarni

ko'rsatishga mo'ljallangan Android komponenti bo'lib, noto'g'ri sozlangan taqdirda foydalanuvchi, tizim va server xavfsizligiga tahdid tug'diradi.

### **Zaif hashlash va shifrlash algorithmarlari**

Mobil ilovalarda zaif hashlash va shifrlash algoritmlaridan foydalanish — bu keng tarqalgan xavfsizlik zaifliklaridan biri bo'lib, maxfiy ma'lumotlar (parollar, tokenlar, sozlamalar) osonlik bilan buzilishiga olib keladi. Bunday algoritmlar kriptanalizga bardosh bera olmaydi va ilova foydalanuvchilari uchun jiddiy xavf tug'diradi.

### **Backup olishga ruxsat berish oqibatida**

Ilovadagi /data/data/com.example.app katalogidagi.

- Kirish tokenlari
- Foydalanuvchi parollari
- Session IDlar
- Caching/qurilmaga yozilgan maxfiy fayllar
- Sozlamalar (SharedPreferences)

hammasi .ab faylga shifrlanmagan holatda saqlanadi va bu malumotlarni ko'rish mumkin:

Agar siz SharedPreferences yoki SQLite bazada quyidagilarni plaintext (ochiq) saqlagan bo'lsa:

- Kirish tokeni
- Email/parol
- API kalitlari
- Mahfiy sozlamalar

ular ham backupga kiradi va ochiladi.

Zararli foydalanuvchi yoki tester backup faylni tahrirlab, keyin uni tiklasa:

- Ilova noto'g'ri ishlaydi
- Token spoofing sodir bo'ladi
- Login bypass bo'lishi mumkin

## **Statik zaifliklar**

Mobil ilovalarni tahlil qilishda statik tahlil (Static Analysis) dastur kodini ishga tushirmasdan turib xavfsizlik zaifliklarini aniqlash imkonini beradi. Quyida mobil ilovalarda eng ko‘p uchraydigan statik zaifliklar sanab o‘tilgan.

### **Hard kodlangan maxfiy ma’lumotlar (Hardcoded Secrets)**

Ko‘plab dasturchilar test vaqtida API kalitlari, parollar yoki tokenlarni bevosita kod ichida yozib qo‘yishadi. Bu esa ilovani dekompiletsiya qilgan har qanday hujumchiga ushbu ma’lumotlarga erkin kirish imkonini beradi. Maxfiy ma’lumotlarni hech qachon kod ichida saqlamang. Ularni xavfsiz saqlash uchun ‘KeyStore’, ‘Secure Enclave’ yoki server tomonli saqlashni qo‘llang.

### **Zaif fayl saqlash (Insecure Data Storage)**

Statik tahlil orqali ilovaning foydalanuvchi ma’lumotlarini (parollar, tokenlar, shaxsiy ma’lumotlar) shifrlanmagan holda saqlayotgani aniqlanishi mumkin. Ayniqsa, *SharedPreferences*, *SQLite*, *Log* fayllarida bu tez-tez uchraydi. Ma’lumotlarni har doim shifrlab saqlang va iloji boricha minimal darajada mahalliy saqlashdan foydalaning.

### **Noto‘g‘ri ruxsatlar (Improper Permissions)**

Ilova ortiqcha tizim ruxsatlarini talab qilishi yoki noto‘g‘ri sozlangan ruxsatlar orqali boshqa ilovalar bilan ma’lumot almashishini ta’minlashi mumkin.

Foydalanuvchidan faqat kerakli ruxsatlarni so‘rang va manifest faylini doimiy tekshirib boring.

### **Zaif kod obfuskatsiyasi (Weak Obfuscation)**

Ilova kodining to‘liq yoki yetarli darajada obfuskatsiya qilinmaganligi, uni oson dekomplyatsiya qilish va tahlil qilishga olib keladi. Android uchun ProGuard yoki R8, iOS uchun Bitcode va boshqa obfuskatsiya vositalaridan foydalaning.

### **Insecureniya uchinchi tomon kutubxonalari**

Ilovada ishlatilayotgan kutubxonalar va SDKlar eski yoki zaif versiyada bo‘lishi mumkin, bu esa zaiflik manbai bo‘lib xizmat qiladi. Uchinchi tomon kutubxonalarini doimiy yangilab boring va ularning changeloglarini kuzatib boring.

### **Zaif kriptografik algoritmlar**

Statik tahlil yordamida ilovaning zaif yoki eskirgan kriptografik algoritmlardan (masalan, MD5, SHA-1) foydalanayotgani aniqlanishi mumkin. Zamonaviy kriptografik standartlardan foydalaning (masalan, SHA-256, AES-256).

### **Logga maxfiy ma’lumotlarni chiqarish**

Ba’zan ishlab chiquvchilar noto‘g‘ri ravishda foydalanuvchi ma’lumotlarini yoki so‘rov javoblarini `Log` ga chiqarib qo‘yishadi. Bu esa ilovani tahlil qilayotgan hujumchi uchun katta imkoniyat yaratadi.

Statik zaifliklar dastur ishga tushmagan holatda aniqlanishi mumkinligi sababli, ularni erta aniqlab bartaraf etish – ilovaning umumiy xavfsizligi uchun juda muhimdir. Har doim secure coding amaliyotlariga amal qilish va kodni avtomatlashtirilgan tahlil vositalari (masalan, MobSF, SonarQube, Fortify) orqali tekshirib borish tavsiya etiladi.

### **Dinamik tahlil**

Mobil ilovalarda dinamik tahlil (dynamic analysis) bu ilovani real vaqtda ishga tushirib, uning xatti-harakatlarini kuzatish orqali xavfsizlik va funksionallikni tekshirish jarayonidir. Dinamik tahlilda ilova qurilmada yoki emulyatorda ishlatib ko‘riladi.

## Ilovani qurilmaga yoki emulyatorda o'rnatish

Ilovani qurilmaga yoki emulyatorda o'rnatish bu dinamik tahlilning birinchi bosqichi hisoblanadi. Dastlab kompyuterga Genymotion dasturi o'rnatib olish kerak bo'ladi. Genymotion saytiga kirib, hisob yaratishimiz kerak. Genymotionni yuklab olish va undan foydalanish uchun sizga ushbu hisob kerak (aniq), shuning uchun avval buni qilganingizga ishonch hosil qiling. Buni qilganingizdan so'ng, quyidagi havolaga(<https://www.genymotion.com/product-desktop/download>) o'ting va Linux (64-bit) versiyasini yuklab oling.

```
File Edit View Search Terminal Help
ubuntu@ubuntu:~/Downloads$ ls
genymotion-3.9.0-linux_x64.run
ubuntu@ubuntu:~/Downloads$
```

Natijada rasmdagi kabi genymotionni linux uchun versiyasi yuklanmalar jildida chiqish kerak. Keyin esa genymotionga `chmod +x genymotion-x.y.z-linux_64.run` deb bu faylga kerakli ruxsatlarni berib olishimiz kerak.

```
File Edit View Search Terminal Help
ubuntu@ubuntu:~/Downloads$ chmod +x genymotion-3.9.0-linux_x64.run
ubuntu@ubuntu:~/Downloads$
```

Yuqoridagi buyruq bajarilganidan keyin quyidagi rasmdagi buyruqni yozib olishimiz kerak bu orqali genymotion dasturi o'rnatilishni boshlaydi.

```
ubuntu@ubuntu:~/Downloads$ ./genymotion-3.9.0-linux_x64.run -d .
Installing for current user only. To install for all users, restart this installer as root.

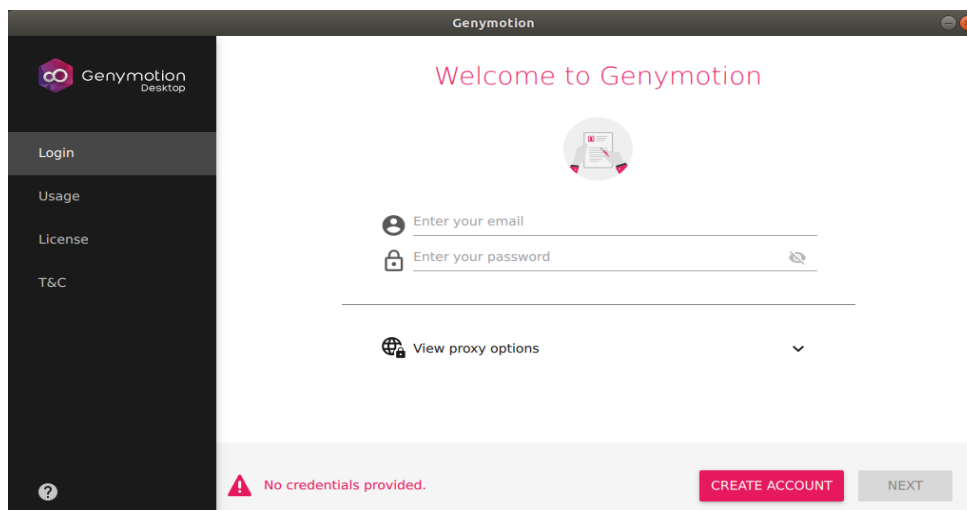
Installing to folder [/home/ubuntu/Downloads/genymotion]. Are you sure [y/n] ?
Installing to folder [/home/ubuntu/Downloads/genymotion]. Are you sure [y/n] ? y

- Extracting files .....
```



Genymotion dasturini ishga tushurish uchun siz quydagi rasmdagi kabi qilib `./genymotion` buyrug'ni yozing va genymotion dasturi ishga tushadi.

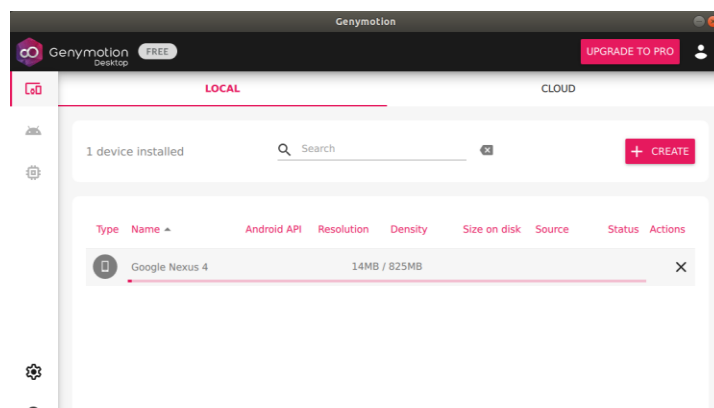
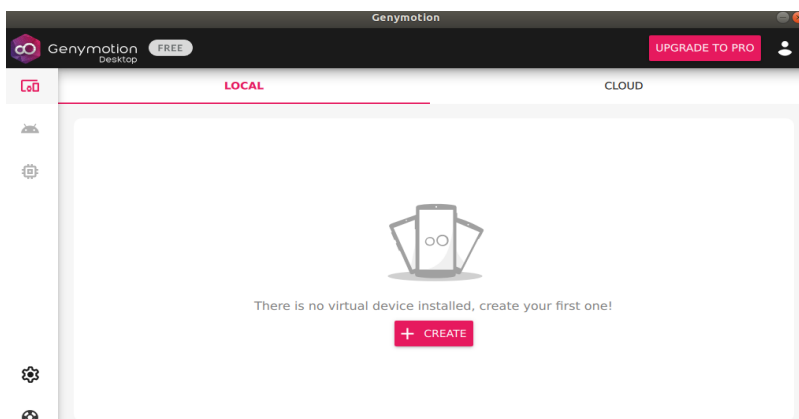
```
ubuntu@ubuntu:~/Downloads/genymotion$ ./g
gamepads/          genymotion          genymotion-shell  genyshe
ces/               gmtool
ubuntu@ubuntu:~/Downloads/genymotion$ ./genymotion
Logging activities to file: /home/ubuntu/.Genymobile/genymotion.log
```



Bu yerga yuqorida ro'yhatdan o'tgan ma'lumotlaringizni yozing. Keyingi kerak bo'ladigan vositalardan virtualboxni o'rnatib olishingiz kerak bo'ladi buning uchun terminalga `sudo apt install virtualbox` buyrug'ini yozishni o'zi yetarli.

```
root@ubuntu:/home/ubuntu/Downloads# sudo apt install virtualbox
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  dkms libdouble-conversion1 libgsoap-2.8.60 libqt5core5a libqt5dbus5 libqt5gui5 libqt5
  libqt5widgets5 libqt5xml5 libstdl1.2debian libvncserver1 libxcb-xinerama0 qt5-gt
  virtualbox-qt
Suggested packages:
  menu qt5-image-formats-plugins qwayland5 vde2 virtualbox-guest-additions-iso
The following NEW packages will be installed:
  dkms libdouble-conversion1 libgsoap-2.8.60 libqt5core5a libqt5dbus5 libqt5gui5 libqt5
  libqt5widgets5 libqt5xml5 libstdl1.2debian libvncserver1 libxcb-xinerama0 qt5-gt
  virtualbox-dkms virtualbox-qt
0 upgraded, 20 newly installed, 0 to remove and 0 not upgraded.
Need to get 36.9 MB of archives.
After this operation, 159 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

Virtualbox dasturi to'liq o'rnatilib olingandan keyin Genymotion dasturiga qaytib olamiz va bu yerda create tugmasini bosib yangi device qo'shib olamiz.



Yuklab olish to'liq tugaganidan so'ng start kamandasini bosib va sizda ham quydagicha bo'lib emulator ishga tushadi.



Emulatorni ishga tushgandan keyin terminalda *adb devices* buyrug‘uni bajarib ko‘rish orqali emulator ishga tushgani va uni adb tanib olganini ko‘rish mumkin.

```
PS C:\Program Files\platform-tools> .\adb devices
List of devices attached
1C161FDF600KTM device

PS C:\Program Files\platform-tools> |
```

Emulatorga sinovdan o‘tkizmoqchi bo‘lgan ilovangizni Play Marketdan yuklash yoki *adb install your\_app.apk* buyrug‘uni terminalda bajarish orqali emulatorga o‘rnatsangiz bo‘ladi.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.22621.1105]
(c) Microsoft Corporation. All rights reserved.

C:\Users\...> adb install file.apk
Performing Streamed Install
Success

C:\Users\...> APK Files>
```

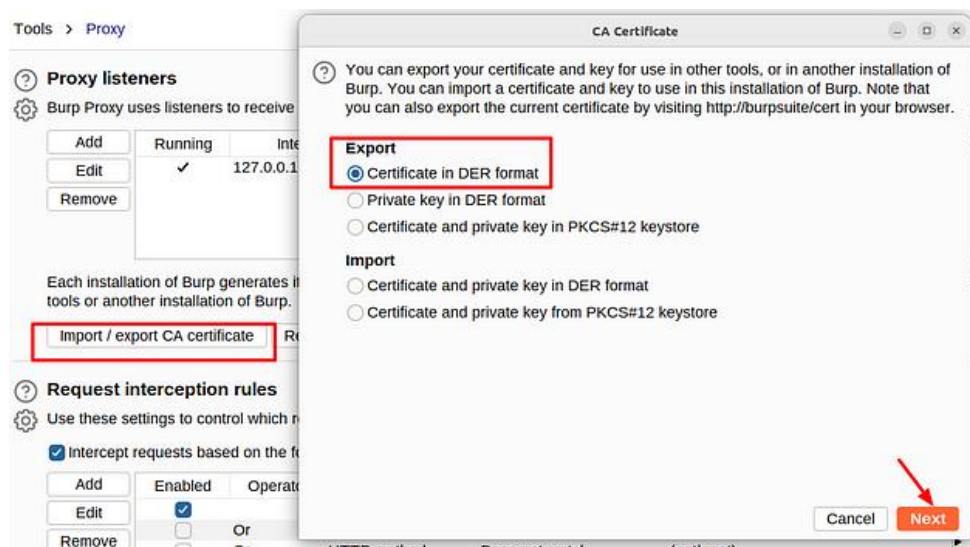
### **Tarmoq trafikni kuzatish (burp suite, mitmproxy)**

Pentesting jarayonida muhim bosqichlardan biri bu ilovaning tarmoq orqali yuborayotgan va olayotgan ma’lumotlarini kuzatishdir. Buni amalga oshirish uchun proxy vositalar (masalan, Burp Suite yoki mitmproxy) keng qo‘llaniladi. Tarmoq trafikni kuzatishdan maqsad, ilova yuborayotgan maxfiy ma’lumotlarni (token, login, parol, GPS, kiritilgan ma’lumotlar) tahlil qilish va tarmoq orqali yuborilgan API so‘rovlarni tahlil qilish (HTTP request yoki response). Burpsuite va Genymotiondagi emulator orqali ilovalardan chiqayotgan tarmoq trafigini kuzatib ko‘ramiz. Buning uchun bizga burpsuite dasturi kerak bu dasturni yuklab olish

havolasi(<https://portswigger.net/burp/communitydownload>). Quydagicha bu yerda Community versiyani yuklab olish mumkin. Burpsuite dasturini ishga tushuramiz.



Burpsuite ilovasi ochilganda Proxy bo'limiga o'tib proxy sozlamalariga 8081 portni yozib All Interface bo'limini tanlab qo'yish kerak yoki boshqa portni ham yozish mumkin va shu yerdan burpsuite ssl sertifikatni yuklab oling yoki <https://burp> sahifasi orqali yuklab olish kerak bo'ladi. Bu orqali https ma'lumotlarni ko'rishimiz mumkin bo'ladi.



Bu muhim bosqichda biz sertifikatimiz formatini o'zgartirib olishimiz kerak bo'ladi buni amalga oshirishimiz uchun OpenSSL orqali amalga oshiramiz.

```
openssl x509 -inform DER -in cacert.der -out cacert.pem
```

Endi esa bizga sertifikatning issuer (beruvchi tashkilot) hash qiymatini olish kerak bo'ladi. Quyidagi buyruq bu ishda bizga yordam beradi.

```
openssl x509 -inform PEM -subject_hash_old -in cacert.pem |head -1
```

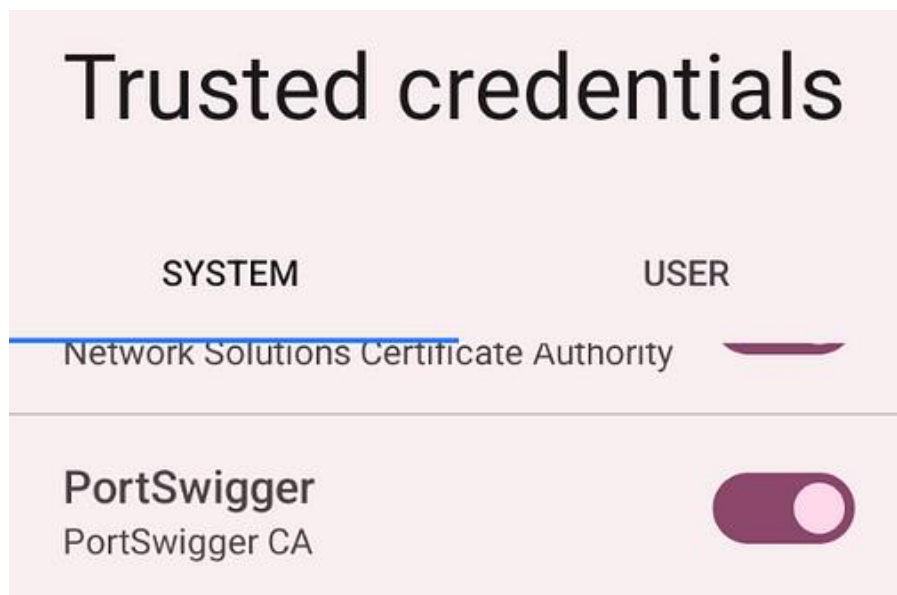
Keling, avvalgi buyruq natijasi 9a5ba575 deb faraz qilaylik. Endi siz sertifikat faylini aynan shu qiymatga qayta nomlashingiz va oxiriga .0 kengaytmasini qo'shishingiz kerak bo'ladi.

```
mv cacert.pem 9a5ba575.0
```

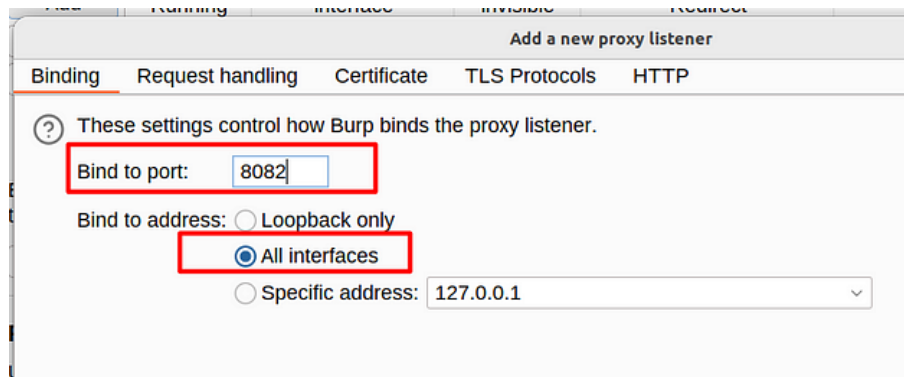
Keyin bu sertifikatni adb orqali qurilmaga o'rnatish kerak bo'ladi bu qurulma root qurulma bo'lishi kerak agar root qurulma bo'lmasa xatolik beradi.

```
adb push 9a5ba575.0 /system/etc/security/cacerts/  
9a5ba575.0: 1 file pushed. 0.1 MB/s (1326 bytes in 0.013s)
```

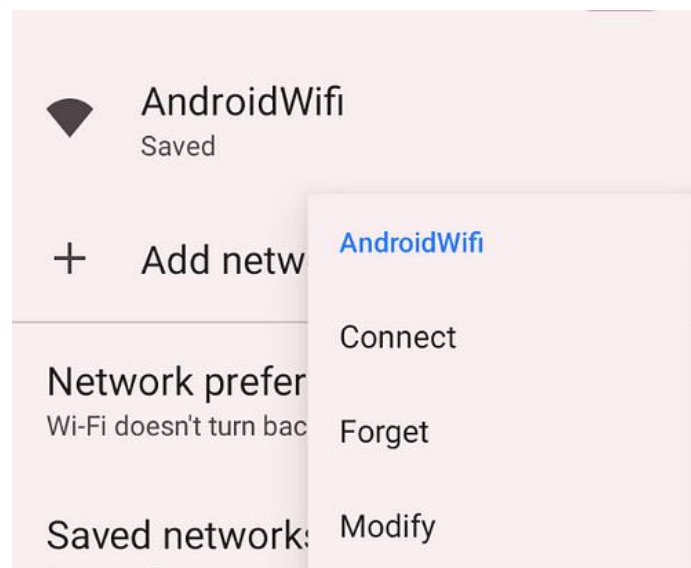
Qurulmani qayta ishga tushuring va Sertifikat o'rnatilganligini tekshirish uchun qurulmadan *Settings -> Security -> Device Security->Encryption & Credentials->Trusted Credentials* ga kiring quydagicha natija chiqishi kerak bo'ladi.



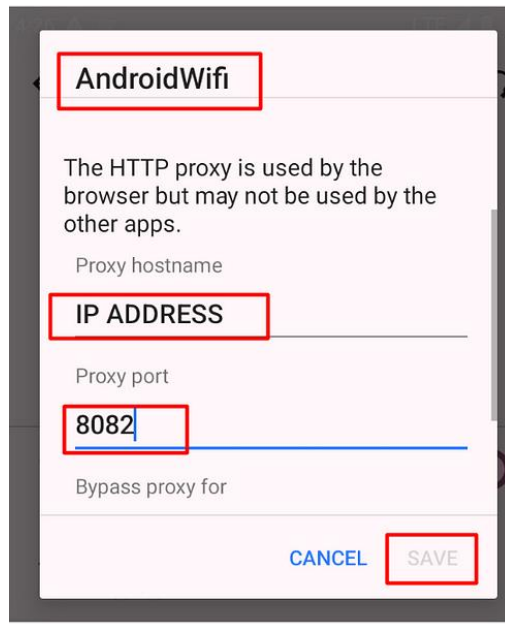
BurpSuiteda port raqamini kiriting va *bind to all interfaces* qatorini tanlang .



Yangi yaratgan Android qurilmangizda quyidagicha amallar ketma ketligini bajaring: Sozlamalar (Settings) → Tarmoq va Internet (Network & Internet) → AndroidWifi bo‘limiga o‘ting. Ulab qo‘yilgan tarmoqqa bosing va “O‘zgartirish” (“Modify”) qatorini tanlang.



“Modify” (O‘zgartirish) qatorini bosganingizdan so‘ng bir nechta parametrlar ko‘rinadi. IP manzilini kompyuteringizning IP manziliga, port raqamini esa Burp Suiteda hozirgina qo‘shgan port raqamiga o‘zgartiring.



Android qurulma orqali biror bir ilovaga kiring va burpsuitedan malumotlarni ko‘rishingiz mumkin.



## **Xavfsizlik cheklovlarini sinash (root, emulator, vpn, zararli kod deteksiya, SSL pinning)**

Android ilovalari foydalanuvchilarning tizimni manipulyatsiya qilishining oldini olish uchun turli xavfsizlik cheklovlarini joriy qiladi. Pentesting jarayonida bu cheklovlarning to‘g‘ri ishlashini sinash muhim hisoblanadi. Quyida Android ilovalarida uchraydigan asosiy xavfsizlik cheklovlari va ularni sinash usullari keltirilgan.

### *Root Aniqlash (Root Detection)*

Ilova root-huquqiga ega bo'lgan qurilmalarda ishlamasligi yoki xavfsizlik choralari kuchaytirishi mumkin. Root qurilmani aniqlash uchun.

- su binar faylining mavjudligi tekshiriladi.
- build.prop faylga yozish ruxsati aniqlanadi.
- Magisk, SuperSU kabi root vositalari aniqlanadi.

Ilovada root qurilmalarni aniqlash funksiyasi bor yuqligini tekshirish uchun quydagi amallarni bajaring.

- Root qilingan qurilmada ilovani ishga tushiring.
- Logcat orqali kuzating yoki ilovaning xatti-harakatini tekshiring (masalan, ilova yopiladimi yoki xatolik beradi).

Bu funktsiyani chetlab o'tish uchun quydagi usullar keng qo'llaniladi.

- Magisk Hide yoki Shamiko modulidan foydalanish.
- Xposed moduli: RootCloak orqali rootni yashirish.
- Frida yordamida isRooted() funksiyalarini falsega o'zgartirish.

### *Emulator Aniqlash (Emulator Detection)*

Ilova faqat real qurilmada ishlashi kerak, emulyatorda esa testdan o'tmasligi mumkin. Emulator qurilmalarni aniqlashda quydagilardan foydalaniladi.

- Qurilma modeli sdk bo'lsa.
- IMEI yoki SIM karta yo'qligi.
- qemu o'ldirish fayllari (/dev/qemu\_pipe, ro.kernel.qemu).

Ilovada emulator qurilmalarni aniqlash funksiyasi bor yuqligini tekshirish uchun quydagi amallarni bajaring.

- Android Studio emulatorida ilovani ishga tushiring.
- Ilova xatti-harakatini kuzating.



Bu funksiyani chetlab o'tish uchun quydagi usullar keng qo'llaniladi.

- Frida bilan isEmulator() funksiyalarini falsega o'zgartirish.
- Emulator identifikatorlarini yashirish (build.prop o'zgartirish).

### *VPN Aniqlash (VPN Detection)*

Ilova tarmoq trafigi VPN orqali yuborilayotganini aniqlasa, uni to'xtatishi mumkin.

Ilova ishga tushgan qurilmada vpn aniqlashda quydagilardan foydalaniladi.

- Ilova ConnectivityManager orqali VPN mavjudligini aniqlaydi.
- Trafikni tekshirib, proxy vositalar orqali ketayotganini sezadi.

Ilovada vpn aniqlash funksiyasi bor yo'qligini tekshirish uchun quydagi amallarni bajaring.

- Qurilmada VPN (masalan, Burp Suite yoki mitmproxy) yoqing.
- Ilovaning API so'rovlariga javob berishiga qarang.

Bu funksiyani chetlab o'tish uchun quydagi usullar keng qo'llaniladi.

- Frida orqali isVpnActive() funksiyasini falsega o'zgartirish.
- Tarmoq tahlil metodlarini patch qilish (dekompilyatsiya orqali).

### *Zararli Kod, Debugger yoki Frida Aniqlash*

Ilova zararli kodlar, debugger yoki reverserlar ishlatilayotganini aniqlab, o'zini himoya qiladi. Zararli kodlarni aniqlashda quydagilardan foydalaniladi.

- Debug rejim (android.os.Debug.isDebuggerConnected())
- Frida, Xposed, Magisk modullarining mavjudligi.
- TracerPid orqali debugging aniqlanadi.

Ilovada zararli kodlarni aniqlash funksiyasi bor yuqligini tekshirish uchun quydagi amallarni bajaring.

- Frida, Xposed yoki debugger ishlatib, ilovani kuzating.
- Logcat orqali xatolik va ogohlantirishlarni ko'ring.

Bu funksiyani chetlab o'tish uchun quydagi usullar keng qo'llaniladi.

- Frida yordamida isDebuggerConnected() yoki detectFrida() funksiyalarini falsega o'zgartirish.

- Antidebug kodni ilovadan olib tashlash (smali patch).

### *SSL Pinning*

Ilova faqat o‘zining ishonchli serveri bilan bog‘lanishini xohlaydi. Burp Suite yoki mitmproxy orqali trafikni ko‘rishga ruxsat bermaydi.

- HTTPS trafikni Burp Suite orqali tutib bo‘lmaydi.
- SSLHandshakeException, TrustManager xatolari chiqadi.

Ilovada ssl pinning funksiyasi bor yo‘qligini tekshirish uchun quydagi amallarni bajaring.

- Burp Suiteni yoqing, ilovani ishga tushiring.
- Agar trafikni ko‘ra olmasangiz, demak SSL pinning mavjud.

Bu funktsiyani chetlab o‘tish uchun quydagi usullar keng qo‘llaniladi.

- Frida skript bilan:
- `frida -U -n com.example.app -l frida-ssl-pinning-bypass.js`
- Ilovani dekompiletsiya qilish.

Xavfsizlik cheklovlarini tahlil qilish – Android pentestingda muhim bosqichlardan biridir. Ilovalar bu cheklovlar orqali tahlil qilishni qiyinlashtiradi. Shuning uchun, har bir cheklovni alohida sinab, kerak bo‘lsa, teskari muhandislik (reverse engineering), Frida, yoki smali patch orqali aylanib o‘tish mumkin.

### **Faoliyatlarni sinash (Activity, Service, BroadcastReceiver)**

Mobil ilovalar bir nechta asosiy komponentlardan iborat: Activity, Service, BroadcastReceiver va Content Provider. Pentesting jarayonida aynan Activity, Service va BroadcastReceiver orqali ilova ichki funksiyalariga noqonuniy kirish yoki ekspluatatsiya qilish mumkin. Quyida ularni sinash usullari va xavflari keltirilgan.

**Activity** — foydalanuvchi interfeysini taqdim etuvchi ekran. Agar Activity noto‘g‘ri sozlangan bo‘lsa, boshqa ilovalar undan foydalana olishi mumkin. Zaiflikni kelib chiqishi sababi quydagilar.

- exported="true" bo‘lib, hech qanday ruxsat (permission) so‘ralmasa
- Kirishda autentifikatsiya tekshirilmasa

Zaiflikdan foydalanish uchun terminalga quydagi kodni yozing.

```
adb shell am start -n com.example.app/.SecretActivity
```

Zaiflik oqibatida quydagi xavflar sodir bo‘ladi.

- Foydalanuvchi autentifikatsiyasini chetlab o‘tish
- Maxfiy ma’lumotlarga kirish

**Service** — orqa fonda (background) ishlovchi komponent. Agar exported yoki intent-filter noto‘g‘ri belgilangan bo‘lsa, boshqa ilovalar u bilan muloqot qilishi mumkin. Zaiflikni kelib chiqishi sababi Service ochiq va hech qanday tekshiruvsiz ishlov beruvchi bo‘lsa quydagi kodni bajarib zaiflikni tekshirish mumkin.

```
adb shell am startservice -n com.example.app/.MyService
```

Zaiflikdan foydalanish uchun terminalga quydagi kodni yozing.

- Ilovaning noto‘g‘ri ishlashiga sabab bo‘lish
- Noqonuniy amallarni bajarishga majbur qilish (DoS, o‘g‘irlik)

**BroadcastReceiver** — tizim yoki ilovadan kelayotgan xabarlarni qabul qiladi. Agar noto‘g‘ri sozlangan bo‘lsa, xaker maxsus xabar yuborib uni ekspluatatsiya qilishi mumkin. Zaiflikni kelib chiqishi sababi quydagilar.

- Receiver exported va hech qanday tekshiruvsiz bo‘lsa
- intent-filter orqali ommaga ochiq bo‘lsa

Zaiflikdan foydalanish uchun terminalga quydagi kodni yozing.

```
adb shell am broadcast -a com.example.app.CUSTOM_BROADCAST
```

Agar extra parametr kerak bo‘lsa:

```
adb shell am broadcast -a com.example.app.CUSTOM_BROADCAST --es key value
```

## Ma'lumotlar oqimini kuzatish (runtime debugging)

Ma'lumotlar oqimini kuzatish, ya'ni runtime debugging mobil ilovalarning ish jarayonida (real vaqt rejimida) qanday ma'lumotlar kirib-chiqayotgani, qanday funksiyalar bajarilayotgani va qanday ob'ektlar bilan ishlanayotganini tahlil qilishdir. Bu bosqich, ayniqsa, quyidagilarni aniqlashda muhim:

- Kirish (login) ma'lumotlari qanday ishlanmoqda?
- Auth tokenlar qayerda saqlanmoqda?
- API so'rovlariga qanday ma'lumotlar yuborilmoqda?
- Ilova ichida qanday xavfsizlik tekshiruvlari mavjud?

Ma'lumotlar oqimini kuzatish uchun quydagi vositalar kerak bo'ladi.

**Frida** — ilovaga injector orqali ulanib, ish jarayonida funktsiyalarni tutib, ularning argument va natijalarini ko'rish imkonini beradi.

```
frida -U -n com.example.app -l intercept.js
```

```
//intercept.js

Java.perform(function () {
  var LoginClass = Java.use("com.example.app.auth.LoginManager");
  LoginClass.authenticate.implementation = function (username, password) {
    console.log("[+] Username: " + username);
    console.log("[+] Password: " + password);
    return this.authenticate(username, password);
  };
});
```

Bu kod bajarilishi natijasida ilova ishga tushganda foydalanuvchi login parolini real vaqt rejimida terminalda ko'rasiz.

**Objection** – Frida ustiga qurilgan qulay CLI vosita. Interaktiv tarzda runtime debugging va bypass qilish imkonini beradi. Root talab qilinmaydi.

```
objection -g com.example.app explore
```

- env – muhit o'zgaruvchilarini ko'rish
- memory search --string "token" – xotiradan tokenlarni qidirish
- android hooking list classes – barcha class'larni ko'rish

- android hooking watch class\_method – specific metodni kuzatish

**Logcat** – Ilova ish vaqtida chiqarayotgan loglar orqali ma'lumot oqimini passiv ravishda kuzatish.

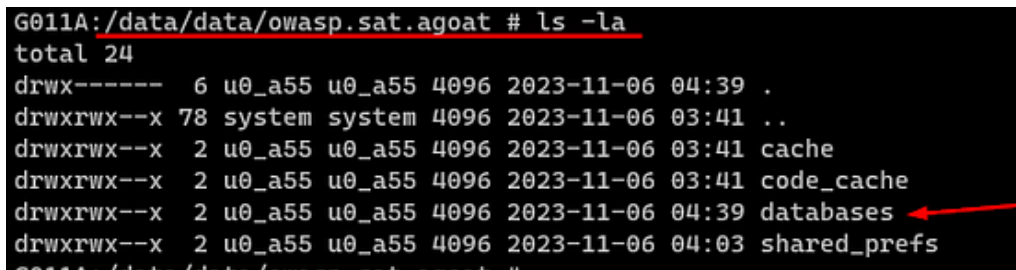
```
adb logcat | grep com.example.app
```

### Umumiy xotira malumotlarini ko'rish

Mobil (Android) pentesting jarayonida umumiy xotira ma'lumotlarini ko'rish ilova qanday ma'lumotlarni xotirada saqlayotganini, qanday darajadagi ma'lumotlar himoyasiz turganini aniqlashda muhim bosqich hisoblanadi. Quyida umumiy xotira (RAM va doimiy saqlash) bilan bog'liq asosiy tekshiruv usullarini tartibli va tushunarli tarzda bayon qilib o'tilgan.

`/data/data/<package_name>` katalogini tekshirish. Bu katalogda ilovaning xususiy fayllari (internal storage) saqlanadi. Faqat rootli qurilmada kirish mumkin. Terminal orqali quydagi buyruqlar ketma-ketligini yozish kerak bo'ladi.

```
adb shell
su
cd /data/data/com.example.app/
ls -la
```



```
G011A:/data/data/owasp.sat.agoat # ls -la
total 24
drwx----- 6 u0_a55 u0_a55 4096 2023-11-06 04:39 .
drwxrwx--x 78 system system 4096 2023-11-06 03:41 ..
drwxrwx--x 2 u0_a55 u0_a55 4096 2023-11-06 03:41 cache
drwxrwx--x 2 u0_a55 u0_a55 4096 2023-11-06 03:41 code_cache
drwxrwx--x 2 u0_a55 u0_a55 4096 2023-11-06 04:39 databases
drwxrwx--x 2 u0_a55 u0_a55 4096 2023-11-06 04:03 shared_prefs
```

Bu buyruqlar bajarilishi natijasida quydagi kataloglarda, `shared_prefs/` – foydalanuvchi sozlamalari (.xml fayllar), ko'pincha tokenlar, login, auth ma'lumotlari bo'lishi mumkin, `databases/` – SQLite ma'lumotlar bazalari, `files/`, `cache/`, `code_cache/` – boshqa saqlangan fayllar.

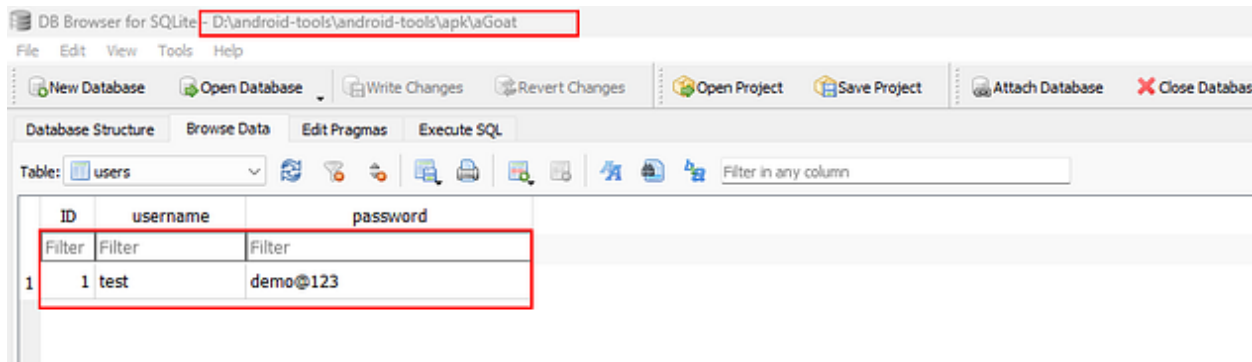
Database (Ma'lumotlar bazasi) katalogi ichini tekshirib ko'rdik. U yerda aGoat nomli fayl mavjudligini ko'rishimiz mumkin, biroq cat buyrug'idan foydalanganda hech qanday ma'lumot chiqmayapti.

```
G011A:/data/data/owasp.sat.agoat/databases # cat aGoat
aGoat-journal
G011A:/data/data/owasp.sat.agoat/databases # cat aGoat
**(*P**vtablessqlite_sequencesqlite_sequenceCREATE TABLE sqlite_sequence(name,seq)**vtableusersusersCREATE TABLE users (ID INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT, username VARCHAR, password VARCHAR)W--ctableandroid_metadataandroid_metadata**testdeusersG011A:/data/data/owasp.sat.agoat/databases
```

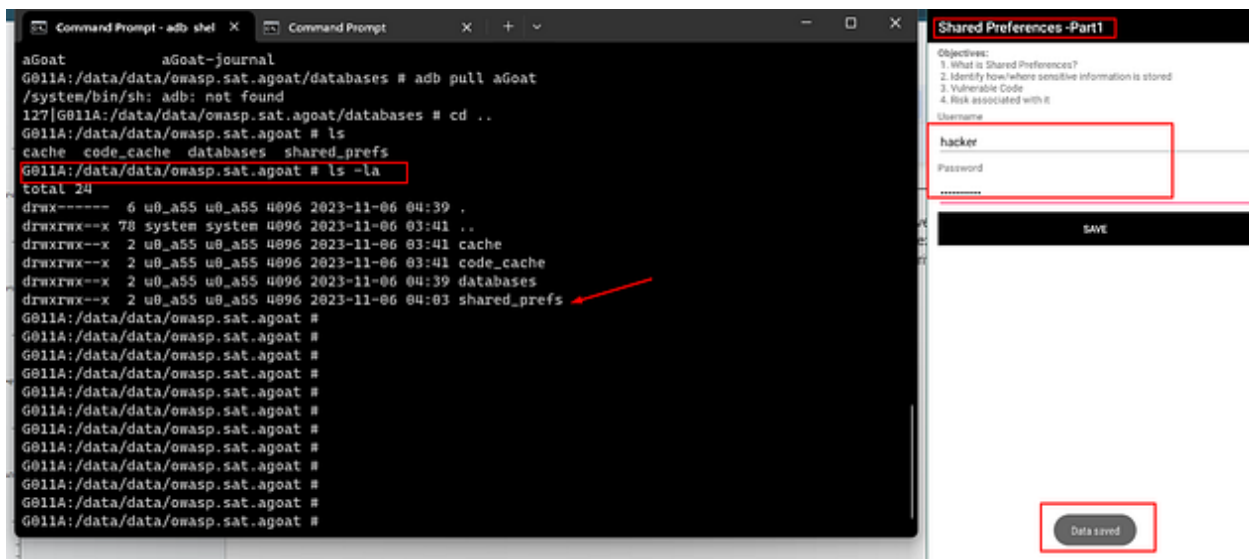
Shuning uchun biz ushbu faylni adb pull buyrug'i yordamida qurilmadan nusxalab olsak bo'ladi.

```
C:\Users\c...p>adb pull /data/data/owasp.sat.agoat/databases/aGoat
/data/data/owasp.sat.agoat/databases/aGoat: 1 file pulled, 0 skipped. 0.4 MB/s (20480 bytes in 0.048s)
```

aGoat faylini SQLite Browser dasturida oching. Fayl ichida foydalanuvchi nomi (username) va parol (password) ochiq matn (plain text) ko'rinishida saqlanganini ko'rishimiz mumkin.



SharedPreferences API odatda kichik hajmdagi kalit-qiymat (key-value) juftliklarini doimiy saqlash uchun ishlatiladi. SharedPreferences obyektida saqlangan ma'lumotlar oddiy matn ko'rinishidagi XML faylga yoziladi. Bu obyekt barcha ilovalar uchun ochiq (world-readable) yoki faqat shu ilovaning o'ziga xos (private) tarzda e'lon qilinishi mumkin. Biz sharedPreferences ichida ba'zi ma'lumotlarni saqladik.



adb yordamida katalogni toping va users.xml faylini tekshiring — unda foydalanuvchi ma'lumotlari (credentials) ko'rinadi.

```
G011A:/data/data/owasp.sat.agoat # cd shared_prefs/
G011A:/data/data/owasp.sat.agoat/shared_prefs # ls -la
total 16
drwxrwx--x 2 u0_a55 u0_a55 4096 2023-11-06 04:57 .
drwx----- 6 u0_a55 u0_a55 4096 2023-11-06 04:39 ..
-rw-rw---- 1 u0_a55 u0_a55 178 2023-11-06 04:03 pinDetails.xml
-rw-rw---- 1 u0_a55 u0_a55 162 2023-11-06 04:57 users.xml
G011A:/data/data/owasp.sat.agoat/shared_prefs # cat users.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="username">hacker</string>
  <string name="password">hacker@123</string>
</map>
G011A:/data/data/owasp.sat.agoat/shared_prefs #
```

Agar ilova Android keystore yoki credential storagedan foydalansa, ma'lumotlar tizim darajasida shifrlangan bo'ladi. Biroq noto'g'ri sozlamalarda ularga ham kirish mumkin.

```
adb shell
su
strings /data/misc/keystore/user_0/*
```

Ba'zi ilovalar xatoliklar yoki log yozuvlar orqali sezilarli darajadagi ma'lumotlarni ochib yuborishi mumkin.

### **API so‘rovlarini kuzatish**

Mobil ilovalarda dinamik tahlil orqali API so‘rovlarni kuzatish – bu ilova ishga tushirilganda u yuborayotgan yoki qabul qilayotgan HTTP yoki HTTPS so‘rovlarni real vaqt rejimida tahlil qilish jarayonidir. Bu usul orqali ilovaning server bilan qanday muloqot qilayotgani, qanday ma’lumotlar uzatilayotgani aniqlanadi. Tahlil davomida ilovaning API endpointlari, uzatilayotgan tokenlar, cookie va session IDlar, foydalanuvchi ma’lumotlari va boshqa maxfiy axborotlarni aniqlash mumkin bo‘ladi. Dinamik kuzatuv orqali ayniqsa xavfsizlik zaifliklarini, ya’ni autentifikatsiya jarayonidagi kamchiliklar, ma’lumotlarning shifrlanmagan holda uzatilishi yoki noto‘g‘ri sozlangan server javoblarini aniqlash mumkin.

Ilova ishga tushgandan so‘ng dastlab autentifikatsiya jarayonini tahlil qilinadi. Agar foydalanuvchi faqat login va parol kiritish orqali tizimga kira olsa, bu bir faktorli autentifikatsiya hisoblanadi. Agar login va paroldan so‘ng telefon raqamga yuborilgan tasdiqlash kodini kiritish talab qilinsa, bu ikki faktorli autentifikatsiya deb ataladi. Agarda bu jarayondan o‘tgach yana qo‘shimcha ma’lumot, masalan biometrik identifikatsiya (barmoq izi yoki yuzni aniqlash) talab qilinsa, bu ko‘p faktorli autentifikatsiya hisoblanadi.

Bu bosqichlar aniqlab olingandan so‘ng login formasi bruteforce hujumiga chidamliligi tekshiriladi. Ya’ni foydalanuvchi parolini taxmin qilish orqali tizimga kirish mumkinmi yoki yo‘qligi tahlil qilinadi. Agar ilova ketma-ket noto‘g‘ri parollar kiritilganda ham hech qanday blokirovka, captcha yoki kutish vaqti (rate limiting) qo‘llamas, bu bruteforce hujumlarga ochiqlik borligini bildiradi. Agar autentifikatsiyadan o‘tgandan so‘ng foydalanuvchidan SMS yoki boshqa tasdiqlovchi kod kiritish so‘raladigan bo‘lsa, bu bosqich ham bruteforce orqali

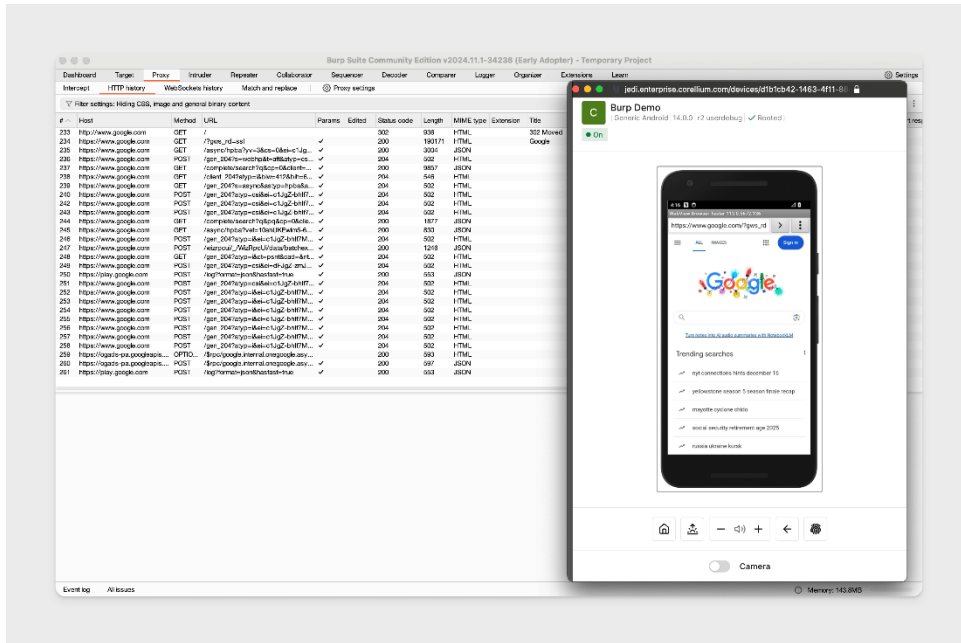


tekshiriladi. Ya'ni kodlar avtomatik ravishda sinab ko'riladi va ilova bunga qanday munosabat bildirayotgani o'rganiladi.

Tizimga muvaffaqiyatli kirilgach, ilova qanday tokenlardan foydalanayotganini aniqlash kerak bo'ladi. Ko'p hollarda JWT — JSON Web Token qo'llaniladi. JWT bu foydalanuvchi autentifikatsiyadan o'tganidan so'ng server tomonidan yaratiladigan, foydalanuvchi haqidagi ma'lumotlarni o'z ichiga olgan raqamli imzo bilan tasdiqlangan matnli token. U odatda uchta qismdan iborat bo'ladi: header, payload va signature. Tahlil davomida bu token server tomonidan qanday tekshirilayotgani o'rganiladi. Agar imzoni o'zgartirib, tokenni serverga yuborganimizda server uni hali ham qabul qilsa, bu juda jiddiy xavfsizlik zaifligini bildiradi. Bu holatda server token imzosini tekshirmayapti yoki noto'g'ri tekshirmoqda.

Agar JWT o'rniga session tokenlar qo'llanilayotgan bo'lsa, bu yerda ham xavfsizlik zaifliklari bo'lishi mumkin. Masalan, session token URL orqali yuborilsa yoki shifrlanmagan holatda uzatilsa, u tarmoq orqali oson tutib olinishi mumkin. Session token muddati tugagandan keyin ham amal qilaversa yoki boshqa foydalanuvchining session tokeni orqali tizimga kira olsak, bu ham katta zaiflik sanaladi. Shuningdek, ilova har bir so'rovda session tokeni talab qilmayotgan bo'lsa, ya'ni foydalanuvchini noto'g'ri identifikatsiyalayotgan bo'lsa, bu ham tahlil davomida aniqlanishi kerak.

Yakuniy tahlil bosqichida butun autentifikatsiya va sesiyalarni boshqarish jarayoni qanday ishlayotgani, server javoblari va ilova qanday xavfsizlik choralarini ko'rayotgani sinchiklab o'rganiladi. Bu ma'lumotlar asosida ilovaning umumiy xavfsizlik darajasi aniqlanadi va zarur bo'lsa, zaif joylar bo'yicha tavsiyalar beriladi.



**Mitmproxy** – CLI asosida ishlaydigan, kuchli va skriptlab bo‘ladigan intercept vosita. Kam resurs talab qiladi. Foydalanish uchun terminalga quydagi kodni yozish kerak.

```
mitmproxy --mode transparent --listen-port 8080
```

- Qurilmada proksi: kompyuter\_ip:8080
- Android qurilmaga mitmproxy sertifikatini o‘rnating
- So‘rovlarni real vaqt rejimida kuzating

```
1: sudo docker run -rm -it -v ~/.mitmproxy/home/mitmproxy/.mitmproxy-p
Flows
16:26:34 HTTPS GET    approve.auth0.com /authorize?audience=https%3A%2F%2Fapprove.auth0.com%2Fuserinfo&auth0Client... 302    text/html 1.29k 678ms
16:26:35 HTTPS GET    approve.auth0.com /login?state=HKFo2S82Umh2dkBTHTVrazFNTmsxRE1BRDZ1RGtyMEtUYxtcaFupxv221u... 200    text/html 1.36k 1.02s
16:26:36 HTTPS POST    approve.auth0.com /usernamepassword/challenge 200    _plication/json 18b 367ms
16:26:36 HTTPS GET    approve.io /images/approov_largeologo.png 404    text/html ...42k 532ms
16:26:37 HTTPS GET    approve.auth0.com /user/ssodata 200    _plication/json 13b 406ms
16:26:44 HTTPS GET    approve.auth0.com /authorize?client_id=y0qjrArkqM8vuRtj1Bv4KrhdfZvnr9kI&response_type=code&... 302    text/html 1.55k 443ms
16:26:45 HTTPS GET    login.auth0.com /approve/authorize?client_id=y0qjrArkqM8vuRtj1Bv4KrhdfZvnr9kI&response_ty... 302    [no content] 455ms
16:26:56 HTTPS POST    approve.auth0.com /oauth/token 200    _plication/json 645b 428ms
16:27:25 HTTPS GET    -st.deno.approov.io /v1/shpments/active 200    _plication/json 2b 528ms
16:27:27 HTTPS GET    -st.deno.approov.io /v1/shpments/nearest_shipment 200    _plication/json 317b 163ms
16:27:33 HTTPS POST    -st.deno.approov.io /v1/shpments/update_state/1 200    [no content] 177ms
16:27:35 HTTPS GET    -st.deno.approov.io /v1/shpments/1 200    _plication/json 317b 275ms
16:27:39 HTTPS POST    -st.deno.approov.io /v1/shpments/update_state/1 200    [no content] 196ms
16:27:41 HTTPS GET    -st.deno.approov.io /v1/shpments/1 200    _plication/json 317b 176ms
16:27:55 HTTPS POST    -st.deno.approov.io /v1/shpments/update_state/1 200    [no content] 179ms
16:27:57 HTTPS GET    -st.deno.approov.io /v1/shpments/1 200    _plication/json 317b 267ms
>>16:28:00 HTTPS GET    -st.deno.approov.io /v1/shpments/delivered 200    _plication/json 641b 176ms

[17/17] [f:approve][showhost] [*:8080]
```

## **Tapjacking hujumiga tekshirish**

Mobil ilovalarda xavfsizlik tahlilining muhim yo‘nalishlaridan biri bu — tapjacking hujumlariga qarshi himoya mavjudligini tekshirishdir. Tapjacking bu foydalanuvchini aldash orqali noto‘g‘ri harakatlar qilishga majbur qiladigan hujum turi bo‘lib, foydalanuvchi ekranidagi haqiqiy tugmalarni ko‘rmay, ular ustida joylashgan shaffof yoki boshqa interfeys elementlarini bosgan deb o‘ylab, aslida fon orqasida turgan xavfli tugmaga bosishiga sabab bo‘ladi. Bunday holatda foydalanuvchi, masalan, tasdiqlash tugmasi o‘rniga muhim ruxsatnomani tasdiqlab qo‘yishi, o‘z ixtiyorisiz ilovaga ruxsat berishi yoki zararli amalni faollashtirishi mumkin.

Tapjacking hujumining ishlash mexanizmi oddiy: hujumchi boshqa ilova oynasi yoki HTML WebView orqali foydalanuvchi ko‘rayotgan interfeys ustiga shaffof qatlam qo‘yadi va shu orqali orqa fondagi tugmalarni "tutadi". Bu ayniqsa Android qurilmalarda sezilarli xavf tug‘diradi. Ilova foydalanuvchidan ruxsatlar so‘raydigan sahifalarda yoki muhim amal bajariladigan joylarda ushbu qatlamlar orqali aldashga uchrashi mumkin.

Tapjacking zaifligini tekshirish uchun odatda test dastur yaratiladi yoki tayyor tapjacking test ilovalardan foydalaniladi. Masalan, OWASP tomonidan tavsiya etilgan maxsus test apk fayllar mavjud bo‘lib, ular ilovaning ustiga joylashtiriladigan shaffof tugmalar orqali foydalanuvchini aldashga urinadi. Bundan tashqari, Android Studio orqali sinov ilova yaratiladi, u foydalanuvchi ustida boshqa ilovalar qatlamini yaratishga harakat qiladi va bu qatlam orqali siz tekshirmoqchi bo‘lgan ilovaning interaktiv tugmalarini "bosishga" urinish amalga oshiriladi.

Tapjacking zaifligini aniqlashda asosiy narsa — ilovaning o‘zida bu kabi hujumlardan himoyalovchi choralar mavjud yoki mavjud emasligini tekshirishdir. Android ilovalarda tapjackingdan himoalanish uchun

`android:filterTouchesWhenObscured="true"` atributi ishlatiladi yoki `onFilterTouchEventForSecurity()` funksiyasi orqali foydalanuvchi interfeysiga boshqa ilova tomonidan aralashuv bo'lsa, bunday harakatlar rad etiladi. Agar ilova ushbu choralarni ko'rmagan bo'lsa, foydalanuvchi tasdiqlash sahifalarida o'z ixtiyorisiz tugmalar bosib yuborilishi xavfi mavjud bo'ladi.

Tekshiruv davomida siz Frida yoki boshqa hook vositalaridan foydalanmaysiz, chunki bu yerda gap foydalanuvchi interfeysga qanday aralashuv bo'lishini aniqlash haqida ketmoqda. Buning o'rniga siz Android qurilmada test ilovani o'rnatib, u orqali sinov qilmoqchi bo'lgan asosiy ilovangiz ustiga shaffof oynani joylashtirasiz va foydalanuvchi bu holatda muhim tugmani bosganida nimalar sodir bo'lishini kuzatasiz. Agar ilova bunga ruxsat bersa, ya'ni hech qanday ogohlantirish yoki harakatni rad etmasa, bu uning tapjacking zaifligiga ega ekanligini bildiradi.

Agar ilova ochiq ruxsat so'rovlarini (masalan, kamera, mikrofon, fayl tizimiga kirish) biron bir interaktiv sahifa orqali berayotgan bo'lsa va shu vaqtda boshqa ilova ustiga joylashishi mumkin bo'lsa, bu holatlar alohida e'tibor bilan tekshiriladi. Chunki aynan shu paytda tapjacking orqali foydalanuvchini aldatish xavfi eng yuqori bo'ladi.

Yakuniy tahlilda esa siz ilovaning foydalanuvchi tajribasini buzmasdan, uni ekran ustida boshqa oynalar orqali boshqarish imkoniyatlari bor yoki yo'qligini aniqlaysiz. Agar mavjud bo'lsa, ishlab chiquvchilarga tavsiya qilinadi: muhim tugmalar ishlatilgan interfeyslarda yuqorida aytib o'tilgan `filterTouchesWhenObscured` kabi xavfsizlik choralarini yoqish, shuningdek, foydalanuvchi tugmalarni bosayotganida ularning harakati ochiq va tushunarli bo'lishini ta'minlash.

Shunday qilib, tapjacking zaifligini aniqlash – bu foydalanuvchi ustidan boshqaruvni o'g'irlab olish mumkin yoki yo'qligini real sinov orqali aniqlashdir. Bu

tahlil mobil ilovaning haqiqiy dunyo tahdidlariga qay darajada tayyorligini ko'rsatadi.

### **Dinamik zaifliklar**

Dinamik zaifliklar — bu ilova ishga tushgan vaqtda, ya'ni real vaqt rejimida yuzaga keladigan xavfsizlik kamchiliklaridir. Bu zaifliklar odatda ma'lumotlar oqimi, autentifikatsiya, trafik yoki xotira ishlovlari bilan bog'liq bo'ladi. Quyida keng uchraydigan dinamik zaifliklar, ularning sabablari va oqibatlari keltirilgan.

*1. Ma'lumotlarning shifrlanmagan uzatilishi* – API so'rovlarida parol, token kabi maxfiy ma'lumotlar ochiq yuboriladi.

- Kelib chiqishi: HTTPS o'rniga HTTP ishlatilishi yoki noto'g'ri konfiguratsiya.
- Oqibati: Trafikni tuta olgan hujumchi ma'lumotlarni o'g'irlashi mumkin (MITM).

*2. SSL pinning mavjud emas yoki chetlab o'tish mumkin* – Ilova server bilan aloqani haqiqiylikiga ishonmaydi.

- Kelib chiqishi: SSL sertifikat tekshiruvining yo'qligi yoki noto'g'ri bajarilishi.
- Oqibati: Trafikni oson intercept qilib, soxta javoblar bilan foydalanuvchini aldash mumkin.

*3. Session hijacking (sessiyani egallab olish)* – Hujumchi foydalanuvchining sessiyasini qo'lga kiritadi.

- Kelib chiqishi: Session tokenlar brauzerda, loglarda yoki ochiq havoda yuborilishi.
- Oqibati: Hujumchi foydalanuvchi nomidan tizimga kira oladi.

*4. Ma'lumotlarni xotirada ochiq saqlash* – Parol, token, karta raqamlari operativ yoki doimiy xotirada shifrlanmagan holda turgan bo'ladi.

- Kelib chiqishi: Dasturchilar xavfsiz saqlash mexanizmlaridan foydalanmasligi.

- Oqibati: Rootli qurilmada yoki dumplar orqali maxfiy ma'lumotlar o'g'irlanadi.

5. *Runtime bypass imkoniyatlari* – Debug, root yoki emulator tekshiruvlari bo'sh joy qoldirgan.

- Kelib chiqishi: Ilova tekshiruvlarni noto'g'ri yoki yuzaki bajaradi.
- Oqibati: Ilova himoyasiz rejimda ishlashi, zararli injektsiyalar oson o'tadi.

6. *Kodning teskari muhandislikka ochiqqligi* – Ilovaning ishga tushgan kodi (runtime) ustida manipulyatsiya qilish mumkin.

- Kelib chiqishi: Obfuskatsiya yo'qligi yoki zaif APK himoyasi.
- Oqibati: Hujumchi kodni tahrirlab, funksiyalarni o'zgartiradi, pin bypass qiladi.

7. *Autentifikatsiyaning zaifligi* – Ilova foydalanuvchini noto'g'ri yoki soddalik bilan tekshiradi.

- Kelib chiqishi: Tokenni tekshirmaslik, har bir so'rovda user ID'ni o'zi yuborish.
- Oqibati: Foydalanuvchilarning akkauntlari oson buziladi.

8. *API cheklovlarining yo'qligi (rate limiting, auth)* – APIga cheksiz so'rov yuborish mumkin.

- Kelib chiqishi: So'rovlar soni va huquqlar nazorat qilinmaydi.
- Oqibati: Brute-force, ma'lumotlar o'g'irlanishi, DDoS mumkin.

## Hisobot shakllantirish

---

Mazkur hisobot mobil ilovalarning xavfsizlik holatini baholash, mavjud zaifliklarni aniqlash va ularni bartaraf etish bo'yicha tavsiyalar berish maqsadida tuzilgan. Tahlil davomida Android va iOS platformalaridagi ilovalar hamda ular bilan bog'langan backend API xizmatlari chuqur o'rganiladi. Har bir platforma uchun tahlil alohida keltirilib, yakunda umumiy xulosa taqdim etiladi.

### Android

Android ilovasi tahlili ilovaning umumiy ma'lumotlari bilan boshlanadi. Bu yerda ilova nomi, versiyasi, paket nomi, SDK darajalari va ishlatilgan texnologiyalar haqida ma'lumot beriladi. Statik tahlil bosqichida APK fayli tahlil qilinib, undagi muhim xavfsizlik belgilariga e'tibor qaratiladi. Manifest fayli orqali ilovada ruxsatlar qanday berilganligi, debug holatining yoqligi, WebView elementlarining xavfsizligi, shifrlash algoritmalarining to'g'ri qo'llanilishi, hardcoded ma'lumotlar mavjudligi kabi jihatlar tekshiriladi.

Dinamik tahlil bosqichida ilovaning ish holatidagi xatti-harakatlari kuzatiladi. Logcat orqali nozik ma'lumotlar chiqayotgan-chiqmayotgani, ilova ichidagi input maydonlar qanday ishlashi, inyeksiya hujumlariga ochiqlik mavjudligi, hamda Frida yoki boshqa hooking vositalari bilan cheklovlarni aylanib o'tish mumkinmi — ana shu masalalar o'rganiladi. Har bir aniqlangan zaiflik OWASP Mobile Top 10 mezonlariga asoslanib tavsiflanadi, xavf darajasi belgilanadi va aniq yechimlar bilan birga taqdim etiladi.

### iOS

iOS ilovasi tahlili ham shunga o'xshash tartibda olib boriladi. Ilovaning nomi, versiyasi, Bundle ID, deployment target kabi texnik ma'lumotlar ko'rsatiladi. Statik tahlilda entitlement'lar, ATS (App Transport Security) sozlamalari, kalitlar yoki foydalanuvchi ma'lumotlari noto'g'ri joyda saqlanayotgan bo'lsa aniqlanadi. Jailbreak holatini aniqlash uchun qo'llanilgan himoya choralari baholanadi.

Dinamik tahlilda esa ilovaning runtime xatti-harakatlari, foydalanuvchi ma'lumotlarining shifrlanmasdan ishlatilishi, hooking yoki bypass qilish imkoniyatlari chuqur tahlil qilinadi. Har bir zaiflik izchil yoritilib, tegishli xavf darajasi va bartaraf etish bo'yicha aniq tavsiyalar beriladi.

### **API**

API tahlili esa ilovaning orqa fonidagi xizmatlar xavfsizligini o'rganishga qaratilgan. Hisobotda API URL manzili, autentifikatsiya va avtorizatsiya mexanizmlari, foydalanuvchi ma'lumotlarini himoya qilish usullari ko'rib chiqiladi. Endpointlar input tekshiruviga ochiqligi, noto'g'ri autentifikatsiya, IDOR, SQL injection, XSS, rate limiting yo'qligi, tokenlar noto'g'ri ishlatilganligi va boshqa muhim jihatlar bo'yicha tahlil qilinadi. Har bir zaiflik uchun isbot (proof-of-concept) keltiriladi, xavf darajasi aniqlanadi va aniq tavsiyalar beriladi.

Hisobot yakunida umumiy xulosa beriladi. Bu qismda ilovaning xavfsizlik darajasi umumlashtirilib, eng dolzarb zaifliklar alohida ko'rsatib o'tiladi. Texnik va strategik tavsiyalar orqali ilovaning xavfsizlik holatini qanday yaxshilash mumkinligi haqida yo'l-yo'riqlar beriladi. Agar kerak bo'lsa, qayta test o'tkazish zarurati ham qayd etiladi.

Qo'shimcha ilovalar sifatida esa test davomida olingan skrinshotlar, ishlatilgan vositalar ro'yxati, tahlil davomida yozilgan POC (isbot) fayllar keltiriladi.